RateGate Build Journal

2026-11-02 · @meimei

Contents

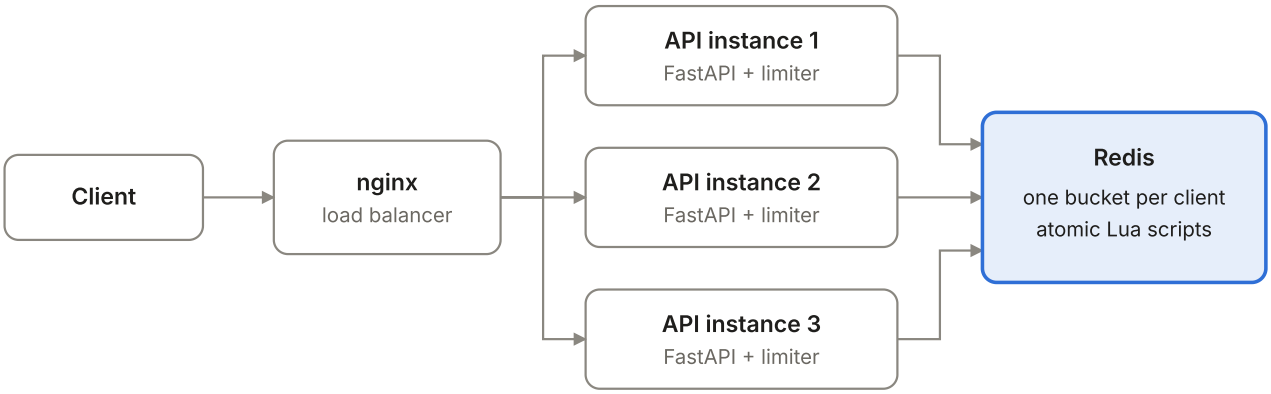
1. Overview
2. How we work
3. Phase 0: Setup (done 2026-11-02)
4. Phase 1: Token bucket rate limiter (done 2026-11-02)
5. Phase 2: Three algorithms behind one interface (done 2026-11-02)
6. Phase 3: Correctness under concurrency (done 2026-11-02)
7. Phase 4: Failure handling, fail-open or fail-closed (done 2026-11-02)
8. Phase 5: Horizontal scaling, 3 copies behind nginx (done 2026-11-02)
9. Phase 6: Continuous integration with GitHub Actions (done 2026-11-03)
10. Phase 7: Load testing with Locust (done 2026-11-03)
11. Phase 8: Polish: tiers, metrics, architecture and design decisions (done 2026-11-03)
12. What's next
13. Real-world context
14. Glossary
15. Progress log and decisions
16. Sources

Overview

RateGate caps how many requests each client can make to an API, and the cap holds even when the API runs as several copies, because every copy counts in one shared Redis database.

Without a shared counter, a client limited to 10 requests could get 10 from each of 3 servers, so 30 in total. RateGate's whole design exists to stop that. The finished system is a client calling nginx, which spreads requests over 3 FastAPI instances, all of which check limits in the same Redis.

Every API copy checks the same Redis, so a client limit is global

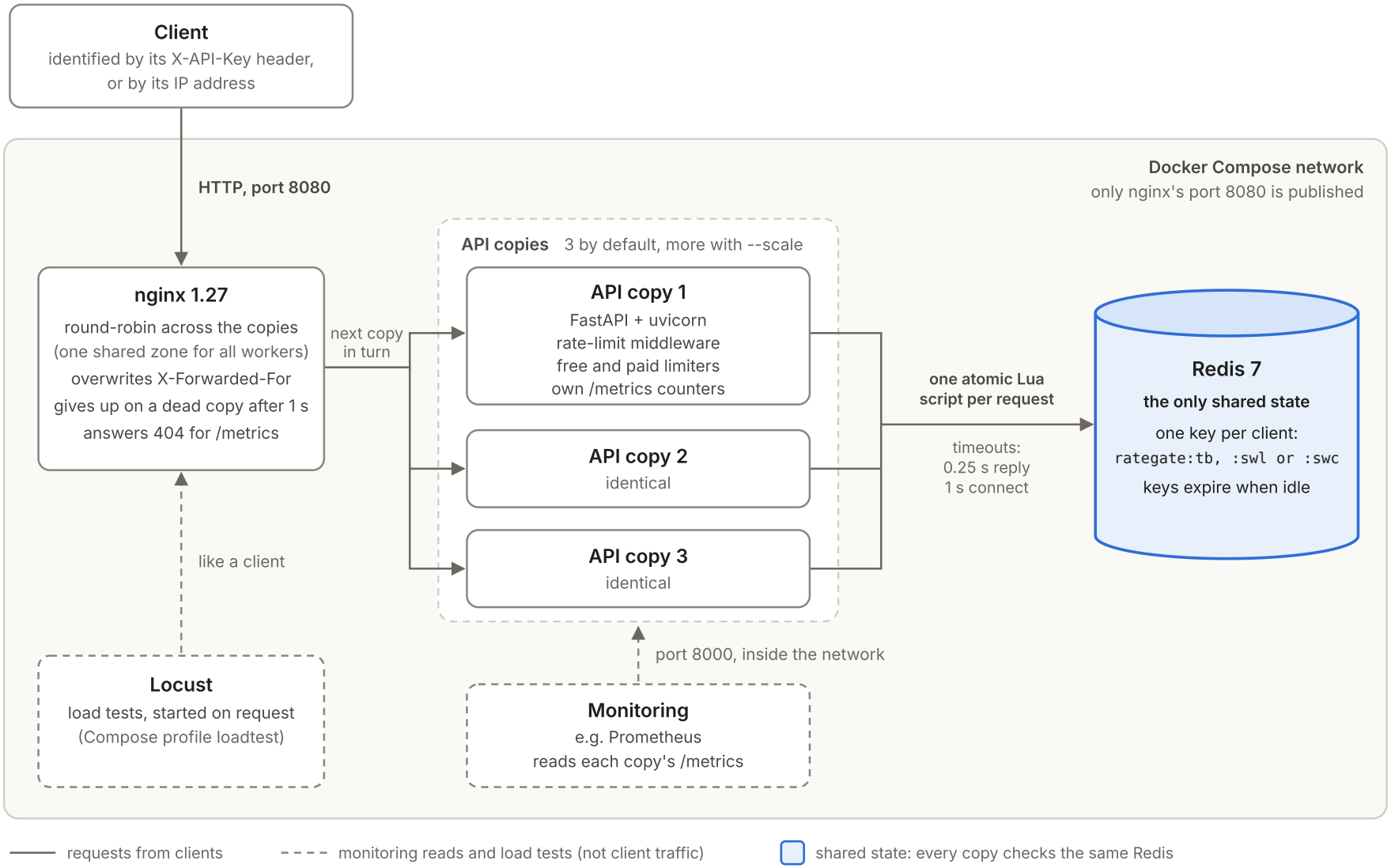


Phase 1 runs one API instance against Redis; Phase 5 adds nginx and instances 2 and 3.

RateGate architecture · final shape after Phase 5

Whichever instance nginx picks, it reads and updates the same bucket in Redis, so the client's limit counts once.

The full system after Phase 8



RateGate architecture · redrawn from the README's "The system" diagram

The copies keep no state of their own, so any copy can answer any request, and adding copies adds capacity. Redis runs each script on its own, so no two copies can interleave a client's read and write.

Why build it

Almost every public API needs a rate limiter, and the hard part is making the limit hold across several servers. RateGate builds exactly that hard part.

Why rate limiting exists. An API has limited capacity, and some clients send far more than their share. A rate limiter answers "slow down" (429) before a request reaches your code. It protects against:

- **Abuse:** scrapers, bots and brute-force login attempts.
- **Accidents:** a buggy script stuck in a retry loop.
- **Unfairness:** one heavy client slowing the API for everyone else.
- **Cost:** wasted server and database time on traffic you don't want.

Why the multi-server case is hard. Real APIs run as several copies behind a load balancer. Counting on one server is easy, but per-server counters multiply the limit. Doing it correctly means solving three problems: one shared counter, no race conditions, and one clock.

Why it's a good portfolio project. It's small enough to finish, it's a classic system-design interview topic, and it involves real trade-offs: concurrency, scaling, failure handling and testing. Each phase proves one claim with evidence: a test, a demo, a benchmark.

Why keep this journal. The code shows what was built. This journal records why each choice was made and where you left off. That reasoning is easy to forget, and it's what interviewers ask about.

How the copies stay in sync

The API copies never sync with each other. They keep nothing important in memory, and every client's bucket lives in one place outside them: Redis.

Why run several copies. One Python process mostly uses one CPU core. Three copies handle roughly three times the traffic. If one copy crashes or is being redeployed, the other two keep serving.

What nginx does. Clients only know one address, nginx's. For each request, nginx picks a copy and forwards the request to it. By default it takes them in turn (1, 2, 3, 1, 2, 3...), called round robin, and it stops sending to a copy that stops responding.

Shared state, not syncing. Think of a bank with 3 cashiers and one ledger. The cashiers never phone each other; they all read and write the same ledger. In RateGate the copies are stateless, and each client's bucket is a Redis key such as rategate:tb:apikey:abc. One request goes like this:

1. nginx sends the request to copy 2.
2. Copy 2 works out who the client is (API key, otherwise IP address) and asks Redis to run the token-bucket script on that key.
3. Redis refills the bucket, takes a token if there is one, saves it, and replies "allowed, 4 left" or "blocked, retry in 1s".
4. Copy 2 passes the request to the route, or returns 429.

The client's next request may go to copy 3, which reads the bucket exactly as copy 2 left it.

Two copies at the same instant. Redis executes commands one at a time, and a Lua script runs as one uninterruptible unit. If copies 1 and 3 send the script for the same client together, Redis runs one fully, then the other. The second sees the first's result, so the last token is never spent twice. The Python code needs no locks.

What would quietly break it.

- **Counting in each server's memory:** a Python dict per copy brings back the "10 per server, 30 in total" bug.
- **Using each server's own clock:** clocks drift, so refill maths would disagree between copies. The script uses Redis's TIME instead.
- **Identifying clients by connection IP behind nginx:** every request would look like it came from nginx. Phase 5 reads the real IP from X-Forwarded-For.

The cost. Every request makes one extra trip to Redis, usually well under a millisecond. Redis also becomes the one thing every copy depends on, which is why Phase 4 exists.

Tool stack

Every tool is free. The order below is roughly the order you meet them.

Tool	Its job in RateGate	Why this one
uv	Installs Python and libraries for this project only	Fast, and its lockfile makes installs identical on every machine, including CI
Python 3.12	The language	Your system has 3.15, which is too new for some libraries; 3.12 is stable and widely supported
FastAPI	Web framework: routes, requests, responses	Async (handles many requests at once), auto-generated docs at /docs, common in industry
Uvicorn	The server process that runs the FastAPI app	The standard server for FastAPI
Redis 7	Shared, in-memory store holding every client's counter	Sub-millisecond speed, and it runs Lua scripts atomically, which removes race conditions
redis-py	Python library that talks to Redis	Supports async and caches Lua scripts on the server
pydantic-settings	Reads settings (limits, Redis address) from environment variables	Change behaviour without editing code, which Docker relies on
Docker Desktop	Runs Redis now; later 3 API copies + nginx	Redis has no native Windows build, and Docker makes the multi-server setup one command
pytest + httpx	Automated tests	The standard Python test tool; httpx lets tests call the API without a real network
ruff	Linter and formatter	Catches mistakes and keeps style consistent, in under a second
nginx (Phase 5)	Load balancer in front of the 3 API copies	Free, simple, and what many companies use
GitHub Actions (Phase 6)	Runs tests on every push	Free for public repositories
Locust (Phase 7)	Load testing	Write the simulated users in Python

How we work

We build in 9 phases, and each phase must work and be committed before the next one starts, so a bug always lives in the newest, smallest piece of code.

Every phase follows the same loop:

1. **Understand**: read what the phase builds and why, before writing code.
2. **Build the smallest working piece**: one feature, not five at once.
3. **Test it**: run it and check the phase's "done when" condition.
4. **Commit**: save a snapshot in git with a clear message. Your git history then shows your progress, which reviewers and interviewers look at.
5. **Log it**: add one line to the progress log at the end of this journal.

Three files guide the work:

Where	What it holds
This journal	What we did, step by step, and why. Updated as each phase finishes
ROADMAP.md (project root)	The checklist of tasks per phase
docs/PHASE_1_GUIDE.md	The code for Phase 1 with line-by-line explanations; later phases get their own guide

Each time you sit down to work: open Docker Desktop, run `docker start redis`, and open the project folder in VS Code.

Phase 0: Setup (done 2026-11-02)

Phase 0 gave the project a private Python 3.12 environment with all libraries installed, and a running Redis that answered `PONG`.

The goal is a setup that rebuilds the same way on any machine. Each step below shows what was run, what it did, and why.

Step 1: Check what is already installed

Ran: `python --version`, `git --version`, `docker --version`, `uv --version`, `wsl --status`

Found: Python 3.15.0, Git 2.56.0, Docker 29.9.2 with WSL2, and no uv. The project folder was empty.

Why: a plan should start from the machine you actually have. This check found two things that shaped later steps: uv was missing, and Python 3.15 is very new.

Step 2: Install uv

Ran: `powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`

Did: installed uv 0.13.6 into `C:\Users\diyal\.local\bin`.

Why: uv does three jobs that would otherwise need three tools (pip, venv, pyenv). It downloads the right Python version, creates an isolated environment, and installs libraries. Terminals opened before the install don't know about uv yet, so restart the terminal once.

Step 3: Create the project

Ran: `uv init --app --python 3.12 --name rategate`, then `uv python pin 3.12`

Did: created `pyproject.toml`, `.python-version`, `.gitignore`, `README.md` and a git repository.

Why:

- `pyproject.toml` is the project's ID card: its name, Python version and libraries. Every modern Python tool reads it.
- `.python-version` says 3.12, so uv always uses 3.12 here, even though the system has 3.15. New Python releases often lack ready-made builds of libraries like pydantic-core for a while, so a stable version avoids install errors.
- `.gitignore` lists files git should never save, such as `.venv` and `__pycache__`.

Step 4: Simplify the layout

Did: deleted the `src/rategate/` folder uv generated, removed the build-system section from `pyproject.toml`, and added a pytest setting `pythonpath = ["."]`.

Why: this version of uv set the project up as an installable library. RateGate is a web app that runs straight from its folder, so that packaging adds steps without benefit. Code will live in a plain `app/` folder. The pytest setting lets tests `import app` from the project root.

Step 5: Install the libraries

Ran: `uv add fastapi "uvicorn[standard]" redis pydantic-settings`, then `uv add --dev pytest httpx ruff`

Did: created `.venv/` (the private environment) and `uv.lock`, then installed FastAPI 0.162.0, Uvicorn 0.54.0, redis-py 8.1.0, pydantic-settings 2.16.2, pytest 9.2.0, httpx 0.28.1 and ruff 0.17.3.

Why:

- Runtime libraries are what the app needs to run. `--dev` libraries are only for development, so a production image can leave them out.
- `uv.lock` records the exact version of every library, including the ones they depend on. Anyone (or CI) running `uv sync` gets the identical set. Commit it.
- A "Failed to hardlink files" warning appeared. It's harmless: uv's cache is on drive C: and the project is on D:, so it copies files instead of linking them.

Step 6: Start Redis in Docker

Ran: started Docker Desktop, then `docker run -d -p 6379:6379 --name redis redis:7` and `docker exec redis redis-cli ping`

Did: downloaded the official Redis 7 image and started a container named `redis`. It replied `PONG`.

Why:

- Redis doesn't run natively on Windows. Docker runs it in a small Linux container instead.
- `-d` runs it in the background. `-p 6379:6379` connects port 6379 on your PC to Redis inside the container. `--name redis` lets later commands refer to it by name.
- `PONG` is Redis's reply to `ping`, which proves it's alive and reachable.

Step 7: Verify

Ran: `uv run python -c "import fastapi, redis, pydantic_settings, uvicorn"`

Result: Python 3.12.15 loaded every library.

Why: `uv run` runs a command inside the project's private environment. A clean import proves the environment is complete before any code depends on it.

Left for you

- [x] Restart the VS Code terminal and check `uv --version`
- [x] First commit: `git add .` then `git commit -m "Phase 0: project setup with uv, FastAPI, Redis"`
- [x] Create a public GitHub repo named RateGate and push to it (`git remote add origin …`, `git push -u origin main`)

Phase 1: Token bucket rate limiter (done 2026-11-02)

Phase 1 works: each client gets a burst of 10 requests, then about 1 per second, and every extra request is answered with `429 Too Many Requests`. The counting lives in Redis, so later phases can add more API servers without changing it.

It was built as 5 files, about 130 lines in total, and committed as `d7687fa`. The checks run on 2026-11-02 gave these results:

Check	Result we saw	Meaning
15 rapid requests	10 × `200`, then 5 × `429`	The limit of 10 holds
12 requests, wait 5 s, 12 more	10 allowed, then 5 allowed	Tokens refill at 1 per second
Your IP, then API key "alice"	Both got their own 10	Every client has a separate bucket
Redis right after a burst	`tokens 0.22`, expiry 9,602 ms	State is stored in Redis and cleans itself up
Headers on a blocked request	`x-ratelimit-remaining: 0`, `retry-after: 1`	Clients are told when to retry

How this section is organised

Read it top to bottom the first time. It moves from ideas to code to proof:

1. **Theory (4 parts).** How the web works, the token bucket, Redis, and the Python and FastAPI features used. Nothing is assumed.
2. **The request path diagram.** One picture of what happens to every request.
3. **Build order.** Which file comes first, and why.
4. **The 5 build steps.** The full code of each file, explained line by line.
5. **Running and verifying.** How to start the server, every check in order, and what each proves.
6. **Bonus test, troubleshooting, and review questions.**

The same code also lives in the project at `docs/PHASE_1_GUIDE.md`. This journal adds the theory and the results.

Theory 1: How the web works

Every interaction with RateGate is one HTTP request from a client and one HTTP response from a server. A rate limiter decides which response to send.

Client and server. A server is a program that waits for messages and answers them. A client is anything that sends those messages: a browser, a phone app, a script, or the `curl` command. In Phase 1, the server is your FastAPI app and the client is curl or your browser.

API. An Application Programming Interface is a server meant for programs, not people. It answers with data, usually JSON (text shaped like `{"message": "You got through!"}`), instead of a web page.

HTTP. HyperText Transfer Protocol is the agreed format of those messages. A request names a **method** (`GET` means "give me something") and a **path** (`/api/resource`). It can also carry **headers**, which are extra labelled facts such as `X-API-Key: alice`. A real request is plain text:

```
GET /api/resource HTTP/1.1
Host: 127.0.0.1:8000
X-API-Key: alice
```

The response starts with a **status code**, then headers, then a body:

```
HTTP/1.1 429 Too Many Requests
retry-after: 1
content-type: application/json

{"detail":"Too many requests. Slow down."}
```

Status codes are 3-digit numbers grouped by their first digit:

Code	Name	Meaning	Where you meet it
200	OK	It worked	Allowed requests
404	Not Found	Nothing exists at that path	Opening `/` in the browser
429	Too Many Requests	Over your limit; slow down	Blocked requests
500	Internal Server Error	The server's code crashed	Bugs
503	Service Unavailable	The server can't serve right now	Phase 4, fail-closed

2xx means success, 4xx means the client did something wrong (including asking too often), and 5xx means the server failed.

Headers matter for rate limiting. RateGate reads one request header, `X-API-Key`, to know who is calling. It writes four response headers so the client knows where it stands:

Header	Example	Tells the client
`X-RateLimit-Limit`	10	The bucket's size
`X-RateLimit-Remaining`	7	Requests left right now
`X-RateLimit-Reset`	3	Seconds until the bucket is full again
`Retry-After` (only on 429)	1	Seconds to wait before trying again

The `X-` prefix marks a header that's a widely used convention rather than an official standard. `Retry-After` is official.

IP address, port and localhost. An IP address identifies a computer on a network. `127.0.0.1`, also called **localhost**, always means "this same computer". A **port** is a numbered door on that computer, so several programs can listen at once: your API listens on port **8000** and Redis on port **6379**. So `http://127.0.0.1:8000/api/resource` means: this computer, door 8000, path `/api/resource`.

curl. A command-line client that sends one HTTP request and prints the answer. On Windows PowerShell, always type `curl.exe`, because plain `curl` is an alias for a different PowerShell command with other options.

Theory 2: Rate limiting and the token bucket

A token bucket gives each client a short burst allowance plus a steady long-run rate, using just two stored numbers per client.

The picture. Each client owns a bucket of tokens:

- It holds at most **capacity** tokens (ours: 10) and starts full.
- Every request must take **1 token** (the request's **cost**). No token means the request is rejected with 429.
- Tokens drip back at the **refill rate** (ours: 1 per second), never above capacity.

Two behaviours follow. **Burst:** an idle client can fire 10 requests at once. **Sustained rate:** after that, it gets about 1 per second, however hard it tries. Over any stretch of T seconds, a client can make at most capacity + T × refill_rate requests. Over 60 seconds that's 10 + 60 = 70.

Why not just count requests per minute? The simplest limiter, a **fixed window**, counts requests in each calendar minute and resets at :00. Its flaw is the boundary: a client can send 100 at 12:00:59 and another 100 at 12:01:00, which is 200 in two seconds against a limit of 100 per minute. A token bucket has no boundary to exploit, because tokens return smoothly. Phase 2 adds two sliding-window algorithms that fix this in other ways.

Lazy refill: no timers. Running a timer per client to add tokens every second would mean millions of timers. Instead, the bucket stores the token count and the time it was last updated (`ts`). When the next request arrives, the refill for all the time in between is worked out in one step:

$$\text{tokens}_{\text{now}} = \min\left(\text{capacity},\ \text{tokens}_{\text{saved}} + (\text{now} - \text{ts}) \times \text{refill_rate}\right)$$

The `min` is the bucket's rim: tokens can't overflow capacity. Nothing happens between requests. The bucket is brought up to date only when someone looks at it.

The other three numbers are derived the same way, all in milliseconds inside the script:

- **Retry after** (when blocked): how long until the missing part of a token drips in: (cost − tokens) ÷ refill_rate. With 0.22 tokens: (1 − 0.22) ÷ 1 = 0.78 s, rounded up to 1 s.
- **Reset**: how long until the bucket is completely full: (capacity − tokens) ÷ refill_rate.
- **Expiry (TTL)**: capacity ÷ refill_rate = 10 s. After 10 idle seconds any bucket is full again, and a full bucket is exactly what a brand-new client gets. So Redis can delete the key without changing any answer.

Worked example with capacity 10 and refill 1 per second:

Time	Event	Tokens before	Refill since last	Decision	Tokens after
0.0 s	First request ever	none (new client)	starts full: 10	allow	9
0.0–0.1 s	9 more requests	9 → 1	about 0.1	allow all	about 0.1
0.1 s	11th request	0.1	~0	**block**, retry after 1 s	0.1
5.1 s	Next request	0.1	+5.0	allow	4.1
5.1 s	4 more quickly	4.1 → 1.1	~0	allow all	0.1
5.1 s	One more	0.1	~0	**block**	0.1
15.1 s	(no request)			key expired, deleted	
20.0 s	New request	none	starts full: 10	allow	9

This is exactly the "10 allowed, wait 5 seconds, 5 allowed" pattern you saw in the verification run.

Theory 3: Redis

Redis is a database that keeps everything in memory and executes one command at a time. Those two facts make it fast enough to consult on every request and safe enough to share between servers.

What Redis is. A **key-value store**: you save a value under a name (the key) and get it back by that name, like a giant Python dictionary that lives in a separate program. Because the data sits in RAM rather than on disk, a lookup takes well under a millisecond. Your Redis runs inside a Docker container named `redis`, listening on port 6379.

Why not a Python variable? A dictionary inside your app would work for one server. But Phase 5 runs 3 copies of the app, each with its own memory. Each would count separately, and a client could get 10 requests from each copy: 30 in total. Redis is one store outside all of them, so there is one bucket per client no matter which copy answers.

Key naming. Keys are plain strings. By convention, parts are separated with colons, from general to specific: `rategate:tb:ip:127.0.0.1` reads as "project RateGate, token bucket, client identified by IP 127.0.0.1". The prefix keeps our keys apart from anything else in the same Redis, and lets you list them all with the pattern `rategate:*`.

Hashes. A value can be a simple string, or a **hash**: a small dictionary of fields stored under one key. Each bucket is a hash with two fields:

```
rategate:tb:ip:127.0.0.1  →  { tokens: "0.22", ts: "1793618507825" }
```

`HSET` writes fields, `HMGET` reads several, and `HGETALL` shows them all. Redis stores everything as text, which is why the script converts with `tonumber`.

Expiry (TTL). Any key can be given a time to live. `PEXPIRE key 10000` means "delete this key in 10,000 milliseconds", and `PTTL key` shows the milliseconds left. Each request resets the countdown. That's why the bucket vanished when you looked more than 10 seconds after your burst.

The race condition. Imagine the check written in Python as three separate steps: read the tokens, decide, write the new count. Two requests for the same client arrive together with 1 token left:

Moment	Request A (server 1)	Request B (server 2)	Tokens in Redis
1	reads 1		1
2		reads 1	1
3	1 ≥ 1, allow	1 ≥ 1, allow	1
4	writes 0		0
5		writes 0	0

Both were allowed with only one token. Under heavy traffic this happens constantly, and the limit leaks. A bug whose outcome depends on timing like this is a **race condition**.

Atomicity through Lua. Redis runs one command at a time on a single thread. It also accepts small programs written in **Lua**, a tiny scripting language, and runs a whole script as if it were one command. While the script runs, nothing else touches Redis. Read, decide and write therefore happen as one **atomic** (indivisible) step. In the table above, request B would only start after A finished, would read 0, and would be blocked. No locks are needed in Python.

How a script is called. `EVAL` sends a script's full text plus its inputs. Inputs come in two lists: `KEYS` (the key names the script touches) and `ARGV` (every other value). Redis wants key names declared separately so it knows which data a script uses. Resending the text every time wastes bandwidth, so Redis also stores scripts by a fingerprint, their **SHA1 hash**: a 40-character code computed from the text. `EVALSHA <hash>` runs a stored script by fingerprint. The redis-py library handles this for us.

One clock. Every computer's clock drifts slightly. If server 1 saved `ts` with its clock and server 2 measured elapsed time with a clock 300 ms ahead, it would hand out tokens that never dripped in. A clock behind could even produce negative time. The script therefore asks Redis itself for the time (`TIME`), so every server's request is measured by the same clock.

Why the browser shows nothing for port 6379. Redis speaks its own protocol, called RESP, not HTTP. When a browser speaks HTTP to it, Redis treats it as a possible attack and closes the connection. To look inside Redis, use `redis-cli`, its command-line client, for example `docker exec redis redis-cli PING`.

Theory 4: Python and FastAPI features used

Phase 1 uses eight Python and FastAPI ideas. Each is explained here once, so the code walk-throughs can refer back to them.

1. Modules, packages and `__init__.py`. Every `.py` file is a **module**. A folder containing an `__init__.py` file is a **package**, a module that holds other modules. That's what makes `from app.limiter.token_bucket import TokenBucket` work: package `app`, sub-package `limiter`, module `token_bucket`, name `TokenBucket`. The `__init__.py` files can stay empty, since their existence is what counts. Imports are resolved from the folder you run commands in, which is why you always run from `D:\RateGate`.

2. Classes, objects and type hints. A **class** is a blueprint and an **object** is one thing built from it. `TokenBucket` is a class; `TokenBucket(redis, 10, 1.0)` creates an object. `__init__` is the method that runs when the object is created, and `self` is the object itself. Annotations like `capacity: int` and `-> Decision` are **type hints**. They document what a value should be, and tools like Pydantic and editors read them.

3. Dataclasses. `@dataclass` writes a class's boring parts for you, such as its `__init__`, from the field list. `frozen=True` makes objects read-only after creation, so a `Decision` can't be accidentally changed after Redis produced it.

4. Settings from the environment. An **environment variable** is a named value handed to a program by the shell, outside its code. In PowerShell: `$env:RATEGATE_CAPACITY = "3"`. **Pydantic** is a library that checks data against type hints. `pydantic-settings` builds on it: a `BaseSettings` class fills each field from an environment variable with the same name, converts `"3"` to the integer 3, and refuses invalid values. It falls back to the default when nothing is set. This is the standard way to configure apps that run in containers.

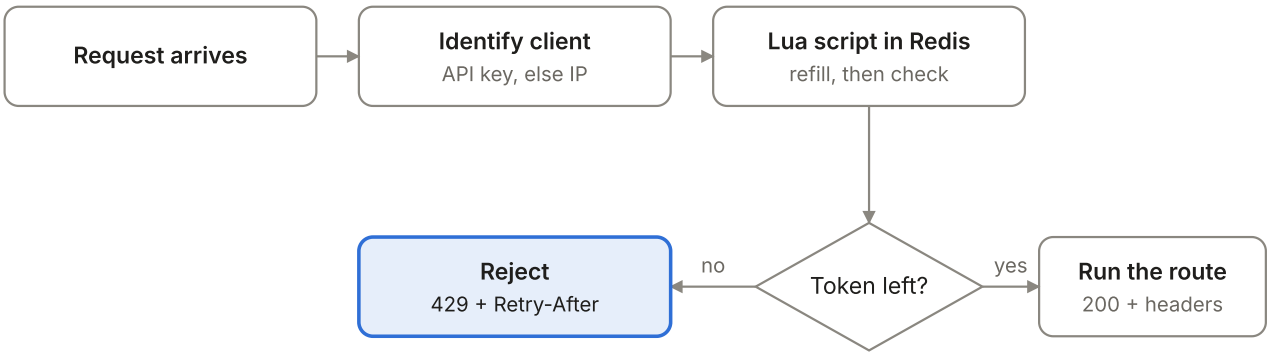
5. `async` and `await`. Most of a request's time is spent waiting: for the network or for Redis. A normal function blocks while it waits. An `async def` function can pause at each `await` and let the same Python process serve other requests meanwhile, then resume when the answer arrives. One process can therefore juggle hundreds of requests. The rule is that an `await` only works inside an `async def`, and anything slow inside one must be awaited, which is why we use the async version of the Redis client, `redis.asyncio`.

6. ASGI, FastAPI and Uvicorn. ASGI (Asynchronous Server Gateway Interface) is the standard contract between a Python web app and the server that runs it. **Uvicorn** is the server: it opens port 8000, reads raw HTTP, and hands each request to the app. **FastAPI** is the framework you write the app with. It's built on a smaller framework, **Starlette**, which is why the middleware imports come from `starlette`. A **route** is a function tied to a method and path, declared with a decorator such as `@app.get("/health")`. A **decorator** (the `@` line) wraps or registers the function below it.

7. Middleware. Middleware is code that sits between the server and every route, like a security guard at a building's entrance. Each request passes through it on the way in, and each response on the way out. It can also refuse entry and answer by itself. Rate limiting belongs here, because one guard then protects every route and the routes need no rate-limit code at all.

8. Lifespan, `yield`, and `app.state`. Some things should happen once, not per request: opening the Redis connection pool at startup and closing it at shutdown. A **lifespan** function does that. Everything before its `yield` runs at startup, the app serves requests while paused at `yield`, and everything after runs at shutdown. `@asynccontextmanager` is the decorator that turns such a function into the start-and-stop pair FastAPI expects. A **connection pool** is a set of open network connections reused across requests, because opening a new connection for each request is slow. `app.state` is an empty container on the app where shared objects can be stored, and middleware reaches it through `request.app.state`.

A request without a token stops at 429 before reaching the route



Phase 1 request path · one decision in Redis

The only decision happens inside Redis, in one atomic step; the route never runs for a rejected request.

Build order: foundations first, wiring last

The five files are written in dependency order, so every file only uses things that already exist when you write it.

First, the folder structure (created empty, then filled):

```
RateGate/
  app/
    __init__.py             (empty: makes app a package)
    config.py               Step 1
    main.py                 Step 5
    middleware.py           Step 4
    limiter/
      __init__.py           (empty: makes limiter a package)
      token_bucket.py       Step 3
      scripts/
        token_bucket.lua    Step 2
  pyproject.toml            (from Phase 0)
```

Order	File	Uses	Used by	Why it sits here
1	`app/config.py`	nothing from our code	`main.py`	Pure settings with no logic. It fixes the numbers (10 tokens, 1 per second) that every later step talks about

Order	File	Uses	Used by	Why it sits here
2	`app/limiter/scripts/token_bucket.lua`	nothing (runs inside Redis)	`token_bucket.py`	The heart of the project: all the decision logic. Writing it early means the hardest idea is understood before anything is built on it
3	`app/limiter/token_bucket.py`	the Lua file, redis-py	`middleware.py` (indirectly), `main.py`	Wraps the script in a Python object and defines `Decision`, the result every later file reads
4	`app/middleware.py`	a limiter with a `check` method	`main.py`	Turns a `Decision` into HTTP: 429 or pass-through, plus headers
5	`app/main.py`	all of the above	Uvicorn	The wiring: creates the app and connects the pieces. It imports everything, so it can only be written last

Why this order works for learning:

- **No forward references.** You never type a name you haven't written yet, so each file makes sense on its own.
- **Inside out.** You start at the core (the algorithm) and move outward to the edges (HTTP, then the app). Later layers only translate the core's answer.
- **Fewer moving parts per step.** If a file has a mistake, it's in the newest file, because everything below it already existed.
- **Separation of concerns.** Each file has one job: settings, decision, Redis access, HTTP translation, wiring. That's why Phase 2 can add algorithms by touching only the `limiter/` folder.

The opposite order (start from `main.py` and fill in the pieces) is also used by professionals, but it means writing imports of files that don't exist yet. For a first build, bottom-up is easier to follow.

Step 1: `app/config.py`, the settings

This file defines the three numbers the whole app depends on, and lets you change any of them without editing code.

```
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="RATEGATE_", env_file=".env")

    redis_url: str = "redis://localhost:6379/0"
    capacity: int = 10  # max tokens in a bucket (burst size)
    refill_rate: float = 1.0  # tokens added per second


settings = Settings()
```

Line by line:

- `from pydantic_settings import ...`: brings in two tools from the pydantic-settings library. `BaseSettings` is the parent class that knows how to read environment variables. `SettingsConfigDict` describes how it should read them.

- `class Settings(BaseSettings):` declares our own settings class. The `(BaseSettings)` means it **inherits** from it: it gets all of `BaseSettings`'s abilities for free.
- `model_config = SettingsConfigDict(env_prefix="RATEGATE_", env_file=".env")` configures the reading:
  - `env_prefix="RATEGATE_"`: field `capacity` is read from the variable `RATEGATE_CAPACITY`. The prefix stops our settings clashing with other programs' variables (plenty of tools use a variable called `CAPACITY` or `REDIS_URL`).
  - `env_file=".env"`: if a file named `.env` exists in the project folder, values are also read from it, one `NAME=value` per line. It's optional; there's none yet.
- `redis_url: str = "redis://localhost:6379/0"`: where Redis lives. Reading it piece by piece: `redis://` is the protocol, `localhost` is this computer, `6379` is the port the Docker container exposes, and `/0` is database number 0 (Redis has 16 numbered databases; 0 is the default). In Phase 5, inside Docker, this becomes `redis://redis:6379/0`, set by an environment variable, with no code change.
- `capacity: int = 10`: bucket size, so also the largest burst. The `int` hint makes Pydantic convert `"3"` from the environment into the number 3, and reject `"abc"` with a clear error at startup.
- `refill_rate: float = 1.0`: tokens per second. It's a `float` (decimal number) so you could set 0.5, meaning one token every 2 seconds.
- `settings = Settings()`: creates the one settings object when the file is first imported. At that moment Pydantic reads the environment and fills each field. Other files import this ready-made object rather than creating their own.

Why settings live in one file. The limits are policy, not logic. Keeping them in one place, overridable from outside, means the same code can run with 10 tokens on your laptop, 3 in a test, and 1,000 in production. This principle, configuration kept outside the code, is one of the "twelve-factor app" rules most cloud deployments follow.

Try it. Precedence is: environment variable, then `.env` file, then the default in the code. `$env:RATEGATE_CAPACITY = "3"` before starting the server changes the burst to 3 (Check 7 below).

Step 2: `app/limiter/scripts/token_bucket.lua`, the algorithm

This script is the only place a request is allowed or blocked. Redis runs it atomically, so its read, decision and write can never interleave with another request's.

```
-- Token bucket, executed atomically inside Redis.
-- KEYS[1] = the bucket key for one client
-- ARGV[1] = capacity, ARGV[2] = refill rate (tokens/sec), ARGV[3] = cost of this request

local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local cost = tonumber(ARGV[3])

-- Redis server time: {seconds, microseconds}. Convert to milliseconds.
local t = redis.call('TIME')
local now = tonumber(t[1]) * 1000 + math.floor(tonumber(t[2]) / 1000)

-- Load the saved state. A brand new client gets a full bucket.
local state = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(state[1])
local ts = tonumber(state[2])
if tokens == nil then
  tokens = capacity
  ts = now
```

```
end

-- Lazy refill: add tokens for the time that passed, never above capacity.
local elapsed = math.max(0, now - ts)
tokens = math.min(capacity, tokens + elapsed * refill_rate / 1000)

local allowed = 0
local retry_after_ms = 0
if tokens >= cost then
  tokens = tokens - cost
  allowed = 1
else
  -- How long until enough tokens drip in?
  retry_after_ms = math.ceil((cost - tokens) * 1000 / refill_rate)
end

-- Save state. Expire the key once the bucket would be full again:
-- a missing key and a full bucket mean the same thing, so idle clients cost no memory.
redis.call('HSET', key, 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', key, math.ceil(capacity * 1000 / refill_rate))

local reset_ms = math.ceil((capacity - tokens) * 1000 / refill_rate)
return {allowed, math.floor(tokens), retry_after_ms, reset_ms}
```

Lua in 60 seconds. `--` starts a comment. `local x = 5` creates a variable. Lists are called tables and **count from 1, not 0**, so `KEYS[1]` is the first key. `nil` means "no value" (Python's `None`). `if ... then ... else ... end` is an if-statement. `redis.call('COMMAND', ...)` runs a Redis command from inside the script. `math.floor` rounds down, `math.ceil` rounds up, `math.min` and `math.max` pick the smaller or larger.

Block 1: inputs (lines 5–8). Python passes one key and three arguments. Everything arrives as text, so `tonumber` converts "10" to 10. `cost` is 1 for every request today. Keeping it as an input means a future expensive route could cost 5 tokens with no script change.

Block 2: the time (lines 11–12). `TIME` returns two numbers: whole seconds since 1 January 1970 (the **Unix epoch**, the usual starting point for computer time), and the microseconds into the current second. The line combines them into milliseconds: seconds × 1000, plus microseconds ÷ 1000. Example: `{1793618507, 825431}` becomes 1793618507000 + 825 = 1793618507825, exactly the `ts` value you saw in Redis.

Block 3: load state (lines 15–21). `HMGET` reads both fields of the hash in one go. If the key doesn't exist (a new client, or one whose key expired), both come back empty, `tonumber` gives `nil`, and the `if` gives the client a full bucket stamped with the current time.

Block 4: lazy refill (lines 24–25). `elapsed` is the milliseconds since the last update. `math.max(0, ...)` guards against negative time, as a safety net. Multiplying by `refill_rate / 1000` converts tokens-per-second into tokens-per-millisecond. `math.min(capacity, ...)` is the bucket's rim.

Block 5: decide (lines 27–35). If at least `cost` tokens are present, take them and allow. Otherwise leave the tokens alone and compute how many milliseconds until the missing amount drips in. A blocked request costs nothing, which is fair: a client that waits will get through.

Block 6: save and expire (lines 39–40). `HSET` stores the new count and time. Tokens are saved as a decimal such as 0.22, so partial tokens keep accumulating. `PEXPIRE` sets the key to vanish after capacity ÷ refill_rate seconds (10,000 ms), the time after which the bucket is full anyway. Without it, every client who ever called would occupy memory forever.

Block 7: answer (lines 42–43). `reset_ms` is the time until the bucket is full. The script returns four numbers in a table, which Python receives as a list. Redis converts Lua decimal numbers to whole numbers when returning them, so the script rounds deliberately: `math.floor(tokens)` reports 0.22 tokens

as 0 remaining, because 0.22 of a token can't buy a request.

Traced example matching the real run. Suppose the bucket held 0.02 tokens, last updated 200 ms before the 11th request arrived:

1. `elapsed` = 200, so refill = 200 × 1 ÷ 1000 = 0.2. New tokens = min(10, 0.02 + 0.2) = 0.22.
2. 0.22 ≥ 1 is false, so the request is blocked. `retry_after_ms` = ceil((1 − 0.22) × 1000 ÷ 1) = 780.
3. Saved: `tokens 0.22`, `ts` = now. Expiry reset to 10,000 ms.
4. `reset_ms` = ceil((10 − 0.22) × 1000) = 9,780.
5. Returned: `{0, 0, 780, 9780}`, meaning blocked, 0 remaining, retry in 0.78 s, full in 9.78 s.

The `0.22000000000000203` you saw in Redis is the same 0.22. Computers store decimals in binary, so most can't be stored exactly, and tiny rounding noise appears. This is **floating-point precision**, and it's harmless here because the comparison and the reported number are both rounded.

Step 3: `app/limiter/token_bucket.py`, the Python wrapper

This file hides Redis and Lua behind one simple call, `await bucket.check(client_id)`, which returns a `Decision`.

```
from dataclasses import dataclass
from pathlib import Path

from redis.asyncio import Redis

LUA_SCRIPT = (Path(__file__).parent / "scripts" / "token_bucket.lua").read_text(encoding="utf-8")


@dataclass(frozen=True)
class Decision:
    allowed: bool
    limit: int
    remaining: int
    retry_after: float  # seconds until the next request would be allowed (0 if allowed)
    reset_after: float  # seconds until the bucket is completely full again


class TokenBucket:
    def __init__(self, redis: Redis, capacity: int, refill_rate: float) -> None:
        self.capacity = capacity
        self.refill_rate = refill_rate
        self._script = redis.register_script(LUA_SCRIPT)

    async def check(self, client_id: str, cost: int = 1) -> Decision:
        key = f"rategate:tb:{client_id}"
        allowed, remaining, retry_ms, reset_ms = await self._script(
            keys=[key], args=[self.capacity, self.refill_rate, cost]
        )
        return Decision(
            allowed=bool(allowed),
            limit=self.capacity,
            remaining=int(remaining),
            retry_after=retry_ms / 1000,
            reset_after=reset_ms / 1000,
        )
```

The imports. `dataclass` builds the `Decision` class. `Path` handles file paths. `Redis` (from `redis.asyncio`) is the async Redis client type, used here as a type hint.

Loading the script (`LUA_SCRIPT = ...`). `__file__` is the path of this Python file itself. `Path(__file__).parent` is its folder, `app/limiter`, and `/ "scripts" / "token_bucket.lua"` joins on the rest. Building the path this way finds the script no matter which folder you start Python from. `.read_text(encoding="utf-8")` reads it as text. The encoding is stated because Windows otherwise guesses an older encoding. This runs once, when the module is first imported.

Why the Lua lives in its own file. It could be a long Python string. As a `.lua` file, your editor highlights it as Lua, and Python and Lua stay cleanly apart.

`Decision`, the result object. Five fields describe one verdict:

Field	Type	Example (blocked)	Used for
`allowed`	bool (True/False)	False	429 or pass-through
`limit`	int	10	`X-RateLimit-Limit`
`remaining`	int	0	`X-RateLimit-Remaining`
`retry_after`	float, seconds	0.78	`Retry-After`
`reset_after`	float, seconds	9.78	`X-RateLimit-Reset`

`frozen=True` makes it read-only. Most importantly, `Decision` is the **contract** between the limiter and the rest of the app. The middleware only reads these fields and never sees Lua or Redis. In Phase 2, all three algorithms return this same object, so the middleware won't change.

`__init__`, run once at startup. It stores the capacity and refill rate on the object (`self.capacity`) and calls `redis.register_script(LUA_SCRIPT)`. That doesn't contact Redis yet; it prepares a callable script object. On the first call, the library sends `EVALSHA` with the script's fingerprint. If Redis says it doesn't know that script (true the first time, or after a Redis restart), the library loads the text and retries, automatically. Every later call sends only the 40-character fingerprint. The leading underscore in `_script` is a Python convention for "internal; don't use from outside".

`check`, run for every request. `async def` because it waits on Redis.

1. `key = f"rategate:tb:{client_id}"` builds the key. An **f-string** puts values into text: client `ip:127.0.0.1` gives `rategate:tb:ip:127.0.0.1`.
2. `await self._script(keys=[key], args=[...])` sends the call to Redis and waits, letting other requests run meanwhile. The two lists become `KEYS` and `ARGV` inside Lua.
3. `allowed, remaining, retry_ms, reset_ms = ...` is **unpacking**: the four returned numbers go into four variables, in order.
4. `return Decision(...)` converts units for Python: `bool(1)` becomes True, and milliseconds are divided by 1000 to give seconds.

Why the TokenBucket receives the Redis client instead of creating it. This is called **dependency injection**: an object is handed what it needs rather than building it itself. One connection pool is shared by everything, and a test can hand in a different Redis without editing this file.

Step 4: `app/middleware.py`, the gatekeeper

The middleware runs before every route. It decides who the client is, asks the limiter, and either answers 429 itself or lets the request through with rate-limit headers attached.

```
import math

from starlette.middleware.base import BaseHTTPMiddleware
```

```
from starlette.requests import Request
from starlette.responses import JSONResponse

EXEMPT_PATHS = {"/health", "/docs", "/openapi.json"}


def identify_client(request: Request) -> str:
    api_key = request.headers.get("X-API-Key")
    if api_key:
        return f"key:{api_key}"
    host = request.client.host if request.client else "unknown"
    return f"ip:{host}"


class RateLimitMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        if request.url.path in EXEMPT_PATHS:
            return await call_next(request)

        limiter = request.app.state.limiter
        decision = await limiter.check(identify_client(request))

        headers = {
            "X-RateLimit-Limit": str(decision.limit),
            "X-RateLimit-Remaining": str(decision.remaining),
            "X-RateLimit-Reset": str(math.ceil(decision.reset_after)),
        }

        if not decision.allowed:
            headers["Retry-After"] = str(max(1, math.ceil(decision.retry_after)))
            return JSONResponse(
                status_code=429,
                content={"detail": "Too many requests. Slow down."},
                headers=headers,
            )

        response = await call_next(request)
        response.headers.update(headers)
        return response
```

The imports. `math` gives `ceil` for rounding up. `BaseHTTPMiddleware` is Starlette's ready-made middleware base class: you only write one method, `dispatch`. `Request` is the incoming request object. `JSONResponse` builds a response whose body is JSON.

`EXEMPT_PATHS`. A **set** (curly braces, no duplicates, fast membership checks) of paths that skip the limit. `/health` is for monitoring tools that ask "are you alive?" every few seconds; limiting them would make a healthy server look dead. `/docs` and `/openapi.json` are FastAPI's documentation page and the data it's drawn from. The capital-letter name is a Python convention for a constant.

`identify_client`: who is calling? Rate limits are per client, so the first question is identity.

- `request.headers.get("X-API-Key")` reads the header, or gives `None` if it's missing. `.get` never crashes on a missing header. Header names are case-insensitive, so `x-api-key` works too.
- With a key, the client is `key:alice`. API keys are how real APIs identify a customer, and they follow the customer across networks.
- Without one, the client is identified by IP address: `ip:127.0.0.1`. `request.client` can in rare cases be missing (with some test tools), hence the `if ... else "unknown"` guard.
- The `key:` and `ip:` prefixes keep the two kinds apart: someone sending the API key `127.0.0.1` can't share an IP's bucket.

IP-based identity is weak: everyone behind one office or mobile network shares an IP, and in Phase 5 every request will appear to come from nginx. Phase 5 fixes the second problem with the `X-Forwarded-For` header.

`dispatch`: runs for every request. `call_next` is a function that passes the request on to the next layer (eventually the route) and returns its response. Calling it or not is the whole power of middleware.

1. **Exempt?** If the path is in `EXEMPT_PATHS`, pass straight through, with no Redis call at all.
2. **Find the limiter.** `request.app.state.limiter` is the `TokenBucket` that `main.py` stored at startup.
3. **Ask.** `await limiter.check(...)` runs the Lua script and returns a `Decision`.
4. **Build headers** for both outcomes. Header values must be text, hence `str(...)`. `math.ceil` rounds the reset time up to whole seconds, because rounding down would tell clients to come back too early.
5. **Blocked?** Add `Retry-After`, rounded up and at least 1, since "retry after 0 seconds" would invite an instant retry. Then return a 429 `JSONResponse` **without calling `call_next`**. The route never runs, so a flood costs your real code nothing.
6. **Allowed?** `await call_next(request)` runs the route, then `response.headers.update(headers)` attaches the headers to its response on the way out.

Notice that the middleware works the same whatever the algorithm is. It depends only on `Decision`'s five fields.

Step 5: `app/main.py`, the wiring

`main.py` creates the FastAPI app, connects to Redis once at startup, plugs in the middleware, and defines the two routes.

```
from contextlib import asynccontextmanager

from fastapi import FastAPI
from redis.asyncio import Redis

from app.config import settings
from app.limiter.token_bucket import TokenBucket
from app.middleware import RateLimitMiddleware


@asynccontextmanager
async def lifespan(app: FastAPI):
    redis = Redis.from_url(settings.redis_url, decode_responses=True)
    app.state.limiter = TokenBucket(redis, settings.capacity, settings.refill_rate)
    yield
    await redis.aclose()


app = FastAPI(title="RateGate", lifespan=lifespan)
app.add_middleware(RateLimitMiddleware)


@app.get("/health")
async def health():
    return {"status": "ok"}


@app.get("/api/resource")
async def resource():
    return {"message": "You got through!"}
```

The imports come in three groups, separated by blank lines: the standard library (`contextlib`), installed libraries (`fastapi`, `redis`), then our own code (`app.*`). This grouping is a Python convention, and ruff can enforce it.

`lifespan`: startup and shutdown.

- `Redis.from_url(settings.redis_url, decode_responses=True)` creates the async Redis client and its connection pool from the URL in the settings. Connections open lazily, on first use. `decode_responses=True` makes Redis replies arrive as Python text (`str`) instead of raw bytes, which is easier to read when debugging.
- `app.state.limiter = TokenBucket(...)` builds the one limiter object, injecting the Redis client and the two numbers, and stores it where the middleware will look for it.
- `yield` hands control to FastAPI: the app now serves requests until you press Ctrl+C.
- `await redis.aclose()` runs at shutdown and closes the connections politely, so Redis isn't left holding dead ones.

Why build the limiter here and not at the top of the file? Startup is the right moment: the event loop that `async` code needs is running, and shutdown has a matching place for cleanup. Creating it at import time would also make every import of `main.py`, for example by a test, open connections.

`app = FastAPI(title="RateGate", lifespan=lifespan)` creates the application. `title` appears on the `/docs` page. This object, named `app`, is what Uvicorn runs, which is why the start command says `app.main:app`.

`app.add_middleware(RateLimitMiddleware)` registers the gatekeeper. From now on, every request passes through `dispatch` before reaching a route.

The routes. `@app.get("/health")` registers `health()` for `GET /health`. Returning a Python dictionary is enough: FastAPI turns it into JSON with status 200. `/api/resource` stands in for your real API. It contains no rate-limit code at all, which is the point: protection is added from outside.

One request traced through every file. You run `curl.exe -H "X-API-Key: alice" http://127.0.0.1:8000/api/resource`:

1. **curl** opens a connection to port 8000 and sends the HTTP request.
2. **Uvicorn** (started by `uv run uvicorn app.main:app`) reads it and hands it to the FastAPI `app` from `main.py`.
3. **`middleware.py`**: `dispatch` runs. The path isn't exempt. `identify_client` finds the header and returns `key:alice`.
4. **`token_bucket.py`**: `check("key:alice")` builds `rategate:tb:key:alice` and sends `EVALSHA` with that key and `[10, 1.0, 1]` to Redis on port 6379.
5. **`token_bucket.lua`** runs inside Redis, atomically: gets the time, loads alice's bucket, refills, takes a token, saves, resets the expiry, and returns `{1, 9, 0, 1000}`.
6. **`token_bucket.py`** turns that into `Decision(allowed=True, limit=10, remaining=9, retry_after=0.0, reset_after=1.0)`.
7. **`middleware.py`** builds the headers, sees `allowed`, and calls `call_next`.
8. **`main.py`**: `resource()` runs and returns `{"message": "You got through!"}`.
9. **`middleware.py`** adds the headers to the 200 response on its way out.
10. **Uvicorn** sends it back, and curl prints it.

`config.py` took part too, earlier: at startup it supplied the Redis URL, the 10 and the 1.0.

Running the server

The server runs in one terminal and stays running; every check is done from a second terminal.

Before starting, every session:

1. Open Docker Desktop and wait for "Engine running" in its bottom-left corner.
2. Make sure Redis is up: `docker start redis`, then `docker exec redis redis-cli ping` should print `PONG`.
3. In VS Code, open a terminal in `D:\RateGate`. The prompt shows `(rategate) PS D:\RateGate>` when the project environment is active.

Start the server (terminal 1):

```
uv run uvicorn app.main:app --reload
```

Part	Meaning
`uv run`	Run the next command inside the project's private Python 3.12 environment
`uvicorn`	The ASGI server
`app.main:app`	Module `app/main.py`, object named `app` inside it
`--reload`	Watch the files and restart automatically when you save a change. For development only

Reading the startup log:

```
INFO:     Will watch for changes in these directories: ['D:\\RateGate']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [11244] using WatchFiles
INFO:     Started server process [11908]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

- "Will watch" and "Started reloader process": `--reload` is active. The reloader is a small supervisor process that restarts the real server when files change.
- "Uvicorn running on http://127.0.0.1:8000": the address to send requests to.
- "Started server process": the process running your app. The numbers in brackets are process IDs Windows assigned.
- "Waiting for application startup" then "Application startup complete": the `lifespan` function ran up to `yield` without errors. If Redis settings or imports were broken, an error would appear here instead.

Each request adds a log line, such as `127.0.0.1:52708 - "GET /api/resource HTTP/1.1" 200 OK`: the client's address and port, the method and path, and the status code.

Why the browser showed 404. Opening `http://127.0.0.1:8000/` asks for the path `/`, and no route exists there, so FastAPI answers `404 Not Found`. Browsers also ask for `/favicon.ico`, the little tab icon, which doesn't exist either. Both are expected, and they prove the server is responding. Useful browser addresses:

Address	What you get
`http://127.0.0.1:8000/api/resource`	The protected route. Refresh 11 times quickly to see the 429 message
`http://127.0.0.1:8000/health`	`{"status":"ok"}`, never limited

Address	What you get
`http://127.0.0.1:8000/docs`	FastAPI's interactive documentation. Click a route, then "Try it out" and "Execute" to see the response and headers

Second terminal. Click the **+** in VS Code's terminal panel, then `cd D:\RateGate`. Leave terminal 1 alone: closing it or pressing Ctrl+C stops the server.

Stopping the server. Press Ctrl+C in terminal 1. You'll see "Waiting for application shutdown", which is the code after `yield` closing the Redis connections.

Verifying it works: 7 checks in order

Seven checks, run from terminal 2 while the server runs in terminal 1, prove every behaviour Phase 1 promises. All seven passed in a full timed run on 2026-11-02, recorded below.

The 10-second clock. With capacity 10 and refill 1 per second, three timing rules decide what you'll see:

Rule	Consequence
A bucket is full again 10 s after its last request	Wait at least 10 s before a test to start fresh
The Redis key is deleted 10 s after its last request	Inspect Redis in the same command line as the requests
Each second of waiting returns 1 token	Wait 5 s and about 5 requests succeed

Reading the test command. Most checks use this pattern:

```
1..15 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} " http://127.0.0.1:8000/api/resource
}
```

Part	Meaning
`1..15`	The numbers 1 to 15
`|` (pipe)	Hand each number to the next command
`ForEach-Object { ... }`	Run the braces once per number, so 15 requests
`curl.exe`	Send one HTTP request
`-s`	Silent: hide the progress bar
`-o NUL`	Throw the response body away (`NUL` is Windows' "nowhere")
`-w "%{http_code} "`	Print only the status code, then a space
`-i` (other checks)	Include the response headers in the output
`-H "X-API-Key: alice"`	Add a request header
`;`	Run the next command right after, on the same line

Check 1: the server is alive.

- Run: `curl.exe http://127.0.0.1:8000/health`
- Expect: `{"status":"ok"}`
- Proves: Uvicorn is running and the app started. `/health` is exempt, so this works even when you're rate limited.

Check 2: the limit blocks after 10.

- When: after at least 10 idle seconds.
- Run: the 15-request command above.
- Expect: `200 200 200 200 200 200 200 200 200 200 429 429 429 429 429`
- Our result: exactly that.
- Proves: the bucket starts full with 10 tokens and blocks when empty.

Check 3: the headers.

- When: straight after Check 2, while blocked.
- Run: `curl.exe -i http://127.0.0.1:8000/api/resource`
- Our result:

```
HTTP/1.1 429 Too Many Requests
x-ratelimit-limit: 10
x-ratelimit-remaining: 0
x-ratelimit-reset: 10
retry-after: 1
content-type: application/json

{"detail":"Too many requests. Slow down."}
```

- Proves: blocked clients are told when to retry. Header names appear in lowercase because HTTP header names are case-insensitive and Uvicorn writes them in lowercase. Run it again after 15 s: you get `200 OK` and `x-ratelimit-remaining: 9`.

Check 4: refill.

- Run (the 5-second wait is built in):

```
1..12 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} " http://127.0.0.1:8000/api/resource
}; Start-Sleep -Seconds 5; ""; 1..12 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} "
http://127.0.0.1:8000/api/resource }
```

- Our result: `200 × 10, 429 × 2`, then `200 × 5, 429 × 7`.
- Proves: tokens return at 1 per second. Getting 4 or 6 is also fine, because the wait plus the requests' own time is never exactly 5.000 s.

Check 5: look inside Redis.

- When: in the same command line as a burst, before the 10-second expiry.
- Run:

```
1..12 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} " http://127.0.0.1:8000/api/resource
}; docker exec redis redis-cli KEYS "rategate:*"; docker exec redis redis-cli HGETALL
"rategate:tb:ip:127.0.0.1"; docker exec redis redis-cli PTTL "rategate:tb:ip:127.0.0.1"
```

- `docker exec redis redis-cli ...` runs the Redis command-line client inside the container. `KEYS` lists keys matching a pattern, `HGETALL` prints the hash, and `PTTL` prints the milliseconds before expiry.
- Our result: key `rategate:tb:ip:127.0.0.1`, `tokens 0.22000000000000203`, `ts 1793618507825`, `PTTL 9602`. Eleven seconds later, `KEYS` was empty.
- Proves: state really lives in Redis, and idle buckets clean themselves up. Running the Redis commands by hand, more than 10 s later, shows nothing. That isn't a bug: it's the expiry working.

Check 6: separate buckets per client.

- Run:

```
1..12 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} " http://127.0.0.1:8000/api/resource
}; ""; 1..12 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} " -H "X-API-Key: alice"
http://127.0.0.1:8000/api/resource }
```

- Our result: both lines `200 × 10, 429 × 2`.
- Proves: the second line started with 200s although your IP's bucket was empty, so alice has her own bucket, `rategate:tb:key:alice`.

Check 7: configuration without code changes.

- Terminal 1: Ctrl+C, then `$env:RATEGATE_CAPACITY = "3"; uv run uvicorn app.main:app --reload`
- Terminal 2, after 10 s: `1..6 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} " http://127.0.0.1:8000/api/resource }`
- Expect: `200 200 200 429 429 429`
- Undo, in terminal 1: Ctrl+C, then `Remove-Item Env:RATEGATE_CAPACITY; uv run uvicorn app.main:app --reload`
- Proves: `config.py` reads the environment. Phase 5's Docker setup depends on this.

Phase 1 is done when all seven match. Then commit: `git add .`, `git commit -m "Phase 1: token bucket rate limiter using atomic Redis Lua script"`, `git push`.

Full check run (2026-11-02, 17:28 to 17:30)

All 7 checks were run as one timed sequence, taking 76 seconds, and every result matched the expectation. The raw outputs and what each one means follow.

How the run was set up. A server ran on port 8000 with the default settings. For Check 7, a second server ran on port 8001 with `RATEGATE_CAPACITY=3`. Running both side by side shows the same code behaving differently from configuration alone, with no restart. Both were stopped afterwards.

The schedule, and why each wait is there:

Time	Step	Wait before it	Why that wait
17:28:32	Pre-check: list keys	none	Confirms a clean start: no buckets in Redis
17:28:32	Checks 1, 2, 3	none	Run back to back: Check 3 must see the bucket Check 2 emptied
17:28:44	Check 3b	11 s	Lets the bucket refill completely, to see the allowed headers
17:28:55	Check 4	11 s	Check 4 must start from a full bucket
17:29:00	Check 4, second burst	exactly 5 s (built into the command)	The wait *is* the test: 5 s should return 5 tokens
17:29:11	Check 5	11 s	A full bucket, so the stored numbers are predictable
17:29:12	Redis inspection	none, same command line	The key disappears 10 s after the last request
17:29:23	Keys after expiry	11 s	Proves the key really expires

Time	Step	Wait before it	Why that wait
17:29:23	Check 6	none	The IP bucket's key had just expired, so both buckets start full
17:29:48	Check 7 (port 8001)	11 s	Check 6 had emptied the IP bucket

The rule of thumb: **wait 11 seconds** (10 s for the bucket, plus a margin) whenever a check needs a full bucket. **Don't wait at all** when a check needs the state the previous one left behind, or when you inspect Redis.

Pre-check: keys before starting

```
(empty)
```

No `rategate:*` keys existed, so every client started with a full bucket.

Check 1: `/health`

```
{"status":"ok"}
```

The server was running and its startup (the `lifespan`) completed. `/health` never touches the limiter.

Check 2: burst of 15

```
200 200 200 200 200 200 200 200 200 200 429 429 429 429 429
```

The 15 requests took about 0.4 s. The first 10 each took a token. By the 11th, under half a token had dripped back, so requests 11 to 15 were blocked. This proves the capacity of 10.

Check 3: headers while blocked

```
HTTP/1.1 429 Too Many Requests
x-ratelimit-limit: 10
x-ratelimit-remaining: 0
x-ratelimit-reset: 10
retry-after: 1
content-type: application/json

{"detail":"Too many requests. Slow down."}
```

- `429`: blocked by the middleware; the route never ran.
- `x-ratelimit-limit: 10`: the capacity.
- `x-ratelimit-remaining: 0`: under 1 token left.
- `x-ratelimit-reset: 10`: about 10 s until the bucket is full (10 missing tokens at 1 per second, rounded up).
- `retry-after: 1`: one token is less than a second away.
- The JSON body is the message `middleware.py` returns.

Check 3b: headers after waiting 11 s

```
HTTP/1.1 200 OK
x-ratelimit-limit: 10
x-ratelimit-remaining: 9
x-ratelimit-reset: 1
```

```
{"message":"You got through!"}
```

The bucket refilled to 10, this request took 1, leaving 9. `reset: 1` because only one token is missing. There's no `retry-after`, since it's only sent when blocked. The body comes from the `resource()` route, which proves the request reached it.

Check 4: refill

```
200 200 200 200 200 200 200 200 200 200 429 429      (17:28:55)
200 200 200 200 200 429 429 429 429 429 429 429      (17:29:00, after exactly 5 s)
```

The first line empties the bucket. During the 5-second sleep, 5 tokens dripped back, and the second line spent exactly those 5. This proves the refill rate of 1 token per second.

Check 5: inside Redis

```
KEYS     → rategate:tb:ip:127.0.0.1
HGETALL  → tokens 0.23300000000000104, ts 1793620752101
PTTL     → 9562
(11 s later) KEYS → (empty)
```

- One bucket exists, named after your IP address.
- `tokens 0.233`: 0.233 of a token had dripped back since the 10th request. The long tail of digits is floating-point noise.
- `ts 1793620752101`: the Redis time of the last update, in milliseconds since 1970 (here, 17:29:12.101 local time).
- `PTTL 9562`: the key would be deleted 9.56 s later. It was set to 10,000 ms and about 0.44 s had passed.
- Empty after 11 s: the expiry worked. Idle clients cost no memory.

Check 6: separate buckets

```
200 200 200 200 200 200 200 200 200 200 429 429      (your IP)
200 200 200 200 200 200 200 200 200 200 429 429      (X-API-Key: alice)
KEYS → rategate:tb:key:alice, rategate:tb:ip:127.0.0.1
```

The second line started with 200s although the IP bucket had just been emptied, so alice was counted separately. `KEYS` shows the two buckets.

Check 7: capacity 3, from configuration

```
200 200 200 429 429 429
HTTP/1.1 429 Too Many Requests
x-ratelimit-limit: 3
x-ratelimit-remaining: 0
x-ratelimit-reset: 3
retry-after: 1
HGETALL → tokens 0.1589999999999997
PTTL    → 2685
```

- Only 3 allowed: the server on port 8001 read `RATEGATE_CAPACITY=3`.
- `x-ratelimit-limit: 3` and `reset: 3` confirm the setting reached every calculation.
- `PTTL 2685`: the expiry dropped from 10 s to about 3 s by itself, because the script computes it as capacity ÷ refill rate (3 ÷ 1). This wasn't in the original check list, but it confirms the expiry formula.

Bonus: the first automated test (not yet added)

One short pytest file can repeat Check 2 automatically, in about a second, every time you change code. There's no `tests/` folder in the project yet. Adding this is optional now, and Phase 3 builds on it.

Create `tests/test_token_bucket.py`:

```
from fastapi.testclient import TestClient
from redis import Redis

from app.config import settings
from app.main import app


def test_blocks_after_capacity():
    Redis.from_url(settings.redis_url).delete("rategate:tb:ip:testclient")

    with TestClient(app) as client:
        codes = [client.get("/api/resource").status_code for _ in range(settings.capacity + 5)]

    assert codes[: settings.capacity] == [200] * settings.capacity
    assert codes[settings.capacity :] == [429] * 5
```

Run it with `uv run pytest -v` (`-v` = verbose: one line per test). A pass prints `PASSED`. A failure prints both lists, showing exactly where they differ.

How pytest works. pytest looks in the `tests` folder (set in `pyproject.toml` in Phase 0) for files named `test_*.py`, runs every function whose name starts with `test_`, and reports which passed. A test passes if it finishes; it fails if any `assert` is false or an error is raised.

Line by line:

- `TestClient(app)` lets the test send requests straight into the app in memory, without Uvicorn or a network. It comes from FastAPI and uses httpx, which is why httpx was installed in Phase 0.
- `from redis import Redis` is the **normal, non-async** Redis client. A simple test function isn't `async`, so it can't `await`; the synchronous client is used for the one cleanup command.
- `.delete("rategate:tb:ip:testclient")` removes any leftover bucket. `TestClient` reports its own client address as `testclient`, so that's the key it uses. Without the delete, a test run 3 seconds after a previous one would start with only 3 tokens and fail randomly. A good test always starts from a known state.
- `with TestClient(app) as client:` the `with` block runs the app's `lifespan`: startup when entering, shutdown when leaving. Without `with`, `app.state.limiter` would never be created.
- `codes = [... for _ in range(settings.capacity + 5)]` is a **list comprehension**: a compact loop that builds a list. It sends 15 requests and keeps their status codes. `_` is the conventional name for a loop variable you don't use.
- `codes[: settings.capacity]` is **slicing**: the first 10 items. `codes[settings.capacity :]` is everything after them.
- `[200] * settings.capacity` builds a list of ten 200s. The two `assert` lines therefore say: the first 10 were allowed and the last 5 were blocked.

Why it reads `settings.capacity` instead of writing 10. If the default changes to 20, the test still checks the right thing.

What kind of test this is. It needs a running Redis, so it's an **integration test**: it checks real pieces working together, not one function alone. That's deliberate. The behaviour that matters most, the Lua script, only exists inside a real Redis.

Troubleshooting

Most problems come from three places: Docker or Redis not running, the wrong folder or terminal, or timing. The first four rows below actually happened during Phase 1.

What you see	Cause	Fix
Browser shows 404 at `http://127.0.0.1:8000/`	No route exists at `/`	Expected. Use `/api/resource`, `/health` or `/docs`
`KEYS` or `HGETALL` prints nothing	The key expired, 10 s after the last request	Run the Redis command on the same line as the requests (Check 5)
Clicking `6379:6379` in Docker Desktop shows a blank page	Redis speaks RESP, not HTTP	Expected. Use `docker exec -it redis redis-cli`, or the container's Exec tab
`tokens 0.22000000000000203`	Floating-point rounding noise	Harmless
`uv` is not recognized	The terminal was opened before uv was installed	Open a new terminal, or restart VS Code
`failed to connect to the docker API`	Docker Desktop isn't running	Open Docker Desktop and wait for "Engine running"
`ConnectionError` / `Error 10061` when requesting	Redis container stopped	`docker start redis`
`ModuleNotFoundError: No module named 'app'`	Wrong folder, or a missing `__init__.py`	`cd D:\RateGate`; check both `__init__.py` files exist
`FileNotFoundError ... token_bucket.lua`	Script misplaced or misnamed	It must be at `app/limiter/scripts/token_bucket.lua`
`ResponseError` mentioning a script	A typo in the Lua; the message names the line	Compare with the code above. Lua uses `end`, `then`, and `~=` for "not equal"
`curl` gives a strange PowerShell error	`curl` is a PowerShell alias	Type `curl.exe`
Never any 429	Requests are too slow, or capacity is high	Use the one-line loop; check `RATEGATE_CAPACITY` isn't set
Everything is 429	The bucket is still empty from a previous test	Wait 10 s, or `docker exec redis redis-cli DEL "rategate:tb:ip:127.0.0.1"`
`[Errno 10048] address already in use`	Another server is still on port 8000	Close the other terminal running Uvicorn, or press Ctrl+C there

A general method for any error: read the **last** lines of the error first. Python prints a **traceback**, the chain of calls that led to the failure, and the actual cause is at the bottom. Then ask which layer it belongs to: Docker, Redis, Python imports, or your code. The file and line number in the traceback tell you where to look.

Check your understanding

These are common interview questions about this design. Answer each aloud in your own words before reading the answer.

1. Why can't the token count live in a Python variable? Each server process has its own memory. With 3 servers, each would keep its own count, and a client could get 10 from each: 30 in total. Redis is one shared store, so there's one count per client.

2. Describe the race condition the Lua script prevents. Two requests read the same token count before either writes back. Both see 1 token, both allow, both write 0, so two requests pass on one token. Redis runs a Lua script atomically, so the second request only reads after the first has written.

3. Why use Redis's `TIME` instead of Python's `time.time()`? Server clocks drift apart. If one server saves the timestamp and another measures elapsed time with a different clock, refills come out wrong, even negative. Using Redis's clock means every measurement uses the same clock.

4. Why does the key expire, and why is deleting it safe? Without expiry, every client that ever called would use memory forever. After capacity ÷ refill_rate seconds (10 s) of silence, any bucket is full, and a missing key is treated as a full bucket. Deleting it changes no answer.

5. With capacity 10 and refill 1 per second, how many requests can a client make in the first 60 seconds? About 70: 10 from the initial full bucket, plus 60 that drip in over the minute.

6. Why doesn't a blocked request use up a token? It receives nothing, so charging it would be unfair, and a client that waits as told by `Retry-After` will get through. The script only subtracts when `tokens >= cost`.

7. What's the advantage of a token bucket over counting requests per calendar minute? A fixed window resets at the minute boundary, so a client can send a full limit at 12:00:59 and another at 12:01:00, which is double the limit in two seconds. Tokens return smoothly, so there's no boundary to exploit.

8. Why is rate limiting done in middleware instead of in each route? One place protects every route, the routes need no rate-limit code, and blocked requests never reach your real code, so a flood costs it nothing.

9. Why does `TokenBucket` receive a Redis client instead of creating one? Dependency injection: the whole app shares one connection pool, and a test can pass in a different Redis without editing the class.

10. What is the cost of this design? Every request makes one extra network trip to Redis, usually well under a millisecond. And Redis becomes something every request depends on. Phase 4 decides what happens when it's down.

Phase 2: Three algorithms behind one interface (done 2026-11-02)

RateGate now runs any of three algorithms (token bucket, sliding window log, sliding window counter), chosen by one setting, and `middleware.py` was never edited. All 16 automated tests pass, and the work is committed as `2630d51` and pushed to GitHub.

This makes one resume line fully true: *"Implemented token bucket, sliding window log, and sliding window counter algorithms behind a common strategy interface, using atomic Redis Lua scripts."*

Result	Evidence
All three algorithms enforce 10 per window	Each allowed 10 of 15 rapid requests
They recover differently after a burst	Check 5: 5 seconds later, token bucket 5, log 0, counter 0 to 2
Switching needs no code change	`RATEGATE_ALGORITHM=sliding_log` alone changed the behaviour
A typo can't slip through	`leaky_bucket` stopped the server at startup with the valid choices listed
The middleware didn't change	`git diff` of `middleware.py` was empty
Tests are stable	16 passed, 3 runs in a row, about 4.6 s each

What changed in the project:

File	Change
`app/limiter/base.py`	New: the shared interface, `Decision`, and the script loader
`app/limiter/token_bucket.py`	Refactored onto the interface; its Lua script is untouched
`app/limiter/sliding_log.py` + `scripts/sliding_log.lua`	New: algorithm 2
`app/limiter/sliding_counter.py` + `scripts/sliding_counter.lua`	New: algorithm 3
`app/limiter/factory.py`	New: builds the chosen algorithm
`app/config.py`	Added `algorithm`, `limit`, `window_seconds`
`app/main.py`	2 lines: use the factory
`tests/conftest.py`, `tests/test_limiters.py`	New: 16 tests
`pyproject.toml`	pytest-asyncio settings; ruff excludes `docs/`
`README.md`	Was empty; now explains the project, algorithms and settings
`app/middleware.py`	**Unchanged**, on purpose

The step-by-step guide with the same code is in the project at `docs/PHASE_2_GUIDE.md`. This journal adds the reasoning, every command explained, and the pitfalls met along the way.

Theory 1: The design ideas

Phase 2 is mostly about structure: making three algorithms interchangeable so the rest of the app never needs to know which one runs.

The problem. In Phase 1, `main.py` named `TokenBucket` directly. Adding a second algorithm that way would mean `if algorithm == ...` checks scattered through the app, and every new algorithm would touch several files. That's how codebases become fragile.

The strategy pattern. Define the job's shape once, as the methods every version must have. Each version is a separate class that fills in the shape. Code that uses it only knows the shape. Here the job is "decide whether this client may make a request", and the shape is one method: `check(client_id, cost) → Decision`.

An analogy is a phone charger socket. The socket's shape is fixed, and any charger that fits it works. The phone doesn't care who made the charger, and a new charger brand doesn't require a new phone.

Abstract base classes (ABCs). Python expresses "the shape" with an abstract base class:

- `class RateLimitStrategy(ABC)` declares the shape. You can't create a `RateLimitStrategy()` itself, since it has no real behaviour.
- `@abstractmethod` on `check` means every subclass **must** provide `check`. If one forgets, Python refuses to create it, with an error naming the missing method. The contract is enforced, not merely documented.
- `class TokenBucket(RateLimitStrategy)` declares that `TokenBucket` **is a** `RateLimitStrategy`. This relationship is called **inheritance**.

The contract has two halves. The input half is the `check` method. The output half is the `Decision` dataclass: whatever the algorithm, it answers with the same five fields. The middleware builds its headers from those fields alone, which is why it needed no change.

The open/closed principle. Software should be *open for extension* (you can add behaviour) but *closed for modification* (you don't edit working code to do it). Adding a fourth algorithm now means one new Lua script, one new class, and one new line in the factory. The middleware, the routes, and the other algorithms stay untouched.

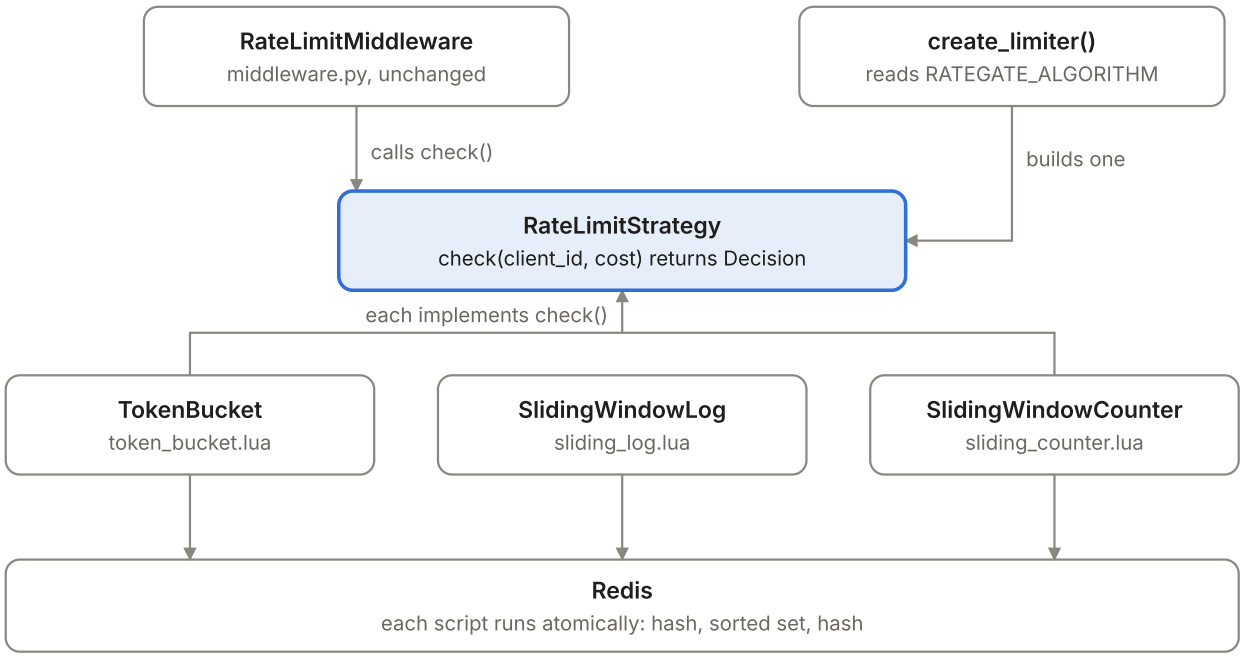
The factory. Something has to read the setting and build the matching class. A **factory function** is a function whose job is to create objects. `create_limiter` is the only place that knows all three classes exist. `main.py` asks it for "the limiter" and gets back something with the right shape.

Refactoring and regression checks. Refactoring means improving code's structure without changing what it does. Step 2 refactored `TokenBucket` onto the new interface before any new algorithm was added. A **regression** is something that used to work and broke after a change. A **regression check** re-runs an old test after a change to prove nothing broke. Here that was the Phase 1 burst: still 10 allowed, then 429s.

Doing the refactor first, alone, is deliberate. If the refactor and two new algorithms had all been written before the first test, a failure could have come from any of them. One change at a time keeps every failure easy to locate.

DRY (Don't Repeat Yourself). Things that all three algorithms need were moved to `base.py` once: finding a script file (`load_script`) and turning a script's four numbers into a `Decision` (`Decision.from_script`). Copying them three times would mean three places to fix every future bug.

The middleware only knows the interface; the factory picks the strategy



Phase 2 structure · one interface, three strategies

At startup the factory builds exactly one strategy and stores it in `app.state.limiter`. On each request, the middleware calls `check()` on it without knowing which of the three it is.

Theory 2: The sliding window log

The sliding window log records the exact time of every allowed request, and allows a new one only if fewer than *limit* fall inside the last *window* seconds. It's exact, at the cost of storing one entry per request.

"Sliding window" means the window moves with the clock. At 12:00:07 with a 10-second window, it covers 11:59:57 to 12:00:07. One second later, it covers 11:59:58 to 12:00:08. It isn't a fixed calendar slice.

On each request, three steps:

1. Delete log entries older than now − window: they've slid out.
2. Count what's left.
3. If count + cost ≤ limit, allow it and add now to the log. Otherwise, block it.

Worked example with limit 3 and window 10 s:

Time	Log after cleanup	Count	Decision	Log after
0 s	empty	0	allow	0
2 s	0	1	allow	0, 2
4 s	0, 2	2	allow	0, 2, 4
6 s	0, 2, 4	3	**block**: the 0 s entry leaves at 10 s, so retry in 4 s	0, 2, 4
10.5 s	2, 4 (0 slid out)	2	allow	2, 4, 10.5

Blocked requests are **not** logged. A client that keeps retrying doesn't push its own recovery further away.

Redis sorted sets. A **sorted set** stores unique **members**, each with a numeric **score**, always kept in score order. Using the request time as the score turns each step into one fast command:

Command	What it does	Used for
`ZADD key score member`	Add a member with a score	Log a request
`ZREMRANGEBYSCORE key -inf X`	Remove members with score ≤ X (`-inf` = from the smallest)	Drop entries that slid out
`ZCARD key`	Count members	How many requests in the window
`ZRANGE key i i WITHSCORES`	Member at position i, plus its score (0 = oldest, −1 = newest)	When will the oldest leave?
`INCR key`	Add 1 to a number and return it	Unique sequence numbers

Why each entry needs a sequence number. Members must be unique: adding an existing member only updates its score. Two requests in the same millisecond would both be named `1793623325995`, and the second would silently disappear, letting an extra request through. So each member is the time plus a counter, such as `1793623325995-1`, with the counter kept in a second key and increased with `INCR`.

Trade-offs. Exact: there's no estimation at all. But memory grows with the limit. A limit of 1,000 per minute stores up to 1,000 entries for every active client. There's also no gradual refill: after a burst, nothing comes back until the oldest request is a full window old, and then the whole allowance returns at once. You saw this in Check 5: 0 allowed after 5 seconds, then all 10 after 11.

Theory 3: The sliding window counter

The sliding window counter estimates the sliding log's answer from just two counts, so it uses almost no memory. In exchange, it's approximate.

Step 1: fixed windows. Cut time into equal slices: [0, 10 s), [10 s, 20 s), and so on. A slice's number is floor(now ÷ window). With a 10,000 ms window and now = 1793621490482 ms, the window number is 179362149, and we are 482 ms into it. Every server computes the same number from Redis's clock, so windows line up everywhere.

The flaw of counting per slice. A plain **fixed window** limiter just counts requests in the current slice and resets at each boundary. A client can send 10 requests at 9.9 s and 10 more at 10.1 s: 20 requests in 0.2 seconds against a limit of 10 per 10 seconds. The boundary is a loophole.

Step 2: weighting. The real "last 10 seconds" overlaps two slices: part of the previous one and all of the current one. The counter assumes the previous slice's requests were spread evenly, and counts only the overlapping share:

$$\text{estimate} = \text{prev} \times \frac{\text{window} - \text{elapsed}}{\text{window}} + \text{curr}$$

A request is allowed if estimate + cost ≤ limit.

Worked example, limit 10 and window 10 s. The previous slice had 8 requests, the current one has 3, and we're 2.5 s into the current slice:

1. weight = (10 − 2.5) ÷ 10 = 0.75, so 75% of the previous slice still overlaps the last 10 seconds.
2. estimate = 8 × 0.75 + 3 = 9.
3. 9 + 1 ≤ 10, so it's **allowed**, and the current count becomes 4.

As time moves through the current slice, the weight falls from 1 to 0, so the previous slice's requests are gradually forgotten.

Rolling forward. Each client's hash stores the slice number it was last updated in, plus the two counts. When a request arrives:

Saved slice compared with now	Meaning	Action
Same slice	Nothing moved on	Use counts as they are
Exactly one slice earlier	One boundary passed	prev = curr, curr = 0
Two or more earlier, or none	Idle for over a whole window	prev = 0, curr = 0

Why the results vary. The estimate depends on how far into the current slice you are, which depends on when your burst happened relative to the boundaries. In our two Check 5 runs, the same requests gave 0 then 5 allowed in one run, and 2 then 7 in the other. Only the burst's position in its slice differed. This is the approximation, not a bug.

Trade-offs. Three numbers per client, whatever the limit. It smooths away the fixed-window loophole. But it's approximate, and it can be off when traffic within a slice is very uneven. Cloudflare uses this algorithm at global scale and measured it as wrong on only a tiny fraction of requests (see Real-world context).

The three algorithms side by side

All three allow the same first burst. They differ in memory, accuracy, and above all in how a client recovers afterwards.

	Token bucket	Sliding window log	Sliding window counter
Stored per client	2 numbers	1 entry per request in the window	3 numbers
Redis type	Hash	Sorted set (+ a counter key)	Hash
Accuracy	Exact	Exact	Approximate
Settings	`capacity`, `refill_rate`	`limit`, `window_seconds`	`limit`, `window_seconds`
Recovery after a burst	Gradual: 1 token per second	All at once, a full window later	Gradual, from the next slice on
Key prefix	`rategate:tb:`	`rategate:swl:`	`rategate:swc:`
Expiry	capacity ÷ refill_rate (10 s)	1 window (10 s)	2 windows (20 s)
Known users	Stripe, AWS	Strict, low-volume limits	Cloudflare

Which to choose? The token bucket suits APIs that want to allow short bursts but cap the steady rate. The sliding log suits small, strict limits where exactness matters and memory doesn't, such as 5 login attempts per 15 minutes. The sliding counter suits very large traffic, where memory per client must stay tiny and a slight error is acceptable.

The defaults (10 tokens refilling at 1 per second, and 10 requests per 10 seconds) give all three the same long-run rate, 1 per second, so they can be compared fairly.

Setup: one library and five config lines

Phase 2 needed one new development library and two small config additions, each solving a specific problem.

1. Install pytest-asyncio.

```
uv add --dev pytest-asyncio
```

Part	Meaning
`uv add`	Add a library to the project: download it, record it in `pyproject.toml`, pin its exact version in `uv.lock`
`--dev`	Only for development, not needed to run the app in production
`pytest-asyncio`	A pytest plugin that can run `async def` test functions

Why: our limiters' `check` method is `async`, so it must be `await`ed. Plain pytest calls test functions normally and can't `await`. pytest-asyncio runs each async test inside an event loop.

2. Two pytest settings in `pyproject.toml`, under `[tool.pytest.ini_options]`:

```
asyncio_mode = "auto"
asyncio_default_fixture_loop_scope = "function"
```

- `asyncio_mode = "auto"`: treat every `async def test_...` as an async test automatically. Without it, each test would need a `@pytest.mark.asyncio` marker.
- `asyncio_default_fixture_loop_scope = "function"`: give every test its own fresh event loop. Leaving it unset works, but prints a long "PytestDeprecationWarning" on every run, which we saw the first time.

3. Tell ruff to skip `docs/`:

```
[tool.ruff]
extend-exclude = ["docs"]
```

Why: this version of ruff also formats Python code blocks **inside Markdown files**. Running `ruff format .` would have rewritten the guides' examples, and turned deliberately aligned "remove / add" snippets into misleading code. `extend-exclude` adds `docs` to ruff's default skip list rather than replacing the list. It also keeps the formatting check in Phase 6's CI from failing on documentation.

What uv changed by itself: `pyproject.toml` gained `"pytest-asyncio>=1.4.1.post1"` in the dev list, and `uv.lock` gained the exact versions. Both files are committed so any machine, including CI, installs the identical set.

Build order and the regression checkpoint

The files were written in dependency order again, with one extra rule: change the working code first and prove it still works, before adding anything new.

Order	File	Why at this point
1	`limiter/base.py` (new)	Every limiter file imports from it
2	`limiter/token_bucket.py` (rewrite)	Fit the existing, working algorithm into the new shape first
✓	**Regression checkpoint**	Prove the refactor changed nothing before building on it

Order	File	Why at this point
3	`config.py` (edit)	New settings must exist before anything reads them
4–5	`sliding_log.lua`, then `sliding_log.py`	Logic first, then its Python wrapper
6–7	`sliding_counter.lua`, then `sliding_counter.py`	The same pattern again
8	`limiter/factory.py` (new)	Imports all three classes, so it comes after them
9	`main.py` (2 lines)	The wiring, last
10	`tests/`	Needs everything above
11	`README.md`	Describes what now exists

Why the checkpoint sits after Step 2. At that moment `main.py` still imported `TokenBucket` directly, so the app could run with only the refactor in place. That made it possible to test the refactor alone. If something broke there, the cause had to be in two files, not in nine.

The checkpoint run (server on port 8000, fresh API key):

```
200 200 200 200 200 200 200 200 200 200 429 429 429 429 429
HTTP/1.1 429 Too Many Requests
x-ratelimit-limit: 10
x-ratelimit-remaining: 0
x-ratelimit-reset: 10
retry-after: 1
```

The output is identical to Phase 1, burst and headers both, so the refactor passed.

Code 1: `base.py` and the token bucket refactor

`base.py` holds everything the three algorithms share, so each algorithm file only contains what's unique to it.

`app/limiter/base.py`:

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from pathlib import Path

SCRIPTS_DIR = Path(__file__).parent / "scripts"


def load_script(name: str) -> str:
    return (SCRIPTS_DIR / name).read_text(encoding="utf-8")


@dataclass(frozen=True)
class Decision:
    allowed: bool
    limit: int
    remaining: int
    retry_after: float  # seconds until the next request would be allowed (0 if allowed)
    reset_after: float  # seconds until the client's full allowance is back

    @classmethod
    def from_script(cls, result: list[int], limit: int) -> "Decision":
```

```
        allowed, remaining, retry_ms, reset_ms = result
        return cls(
            allowed=bool(allowed),
            limit=limit,
            remaining=int(remaining),
            retry_after=retry_ms / 1000,
            reset_after=reset_ms / 1000,
        )


class RateLimitStrategy(ABC):
    @abstractmethod
    async def check(self, client_id: str, cost: int = 1) -> Decision:
        """Spend `cost` from the client's allowance if there is enough, and report the result."""
```

Piece by piece:

- `from abc import ABC, abstractmethod`: `abc` is Python's built-in module for abstract base classes.
- `SCRIPTS_DIR = Path(__file__).parent / "scripts"`: the `scripts` folder beside this file, computed once. `__file__` is this file's own path, so it works whichever folder you start Python from.
- `load_script(name)`: reads one Lua file by name. Phase 1 had this logic inside `token_bucket.py`; now all three algorithms call the same function.
- `Decision` moved here unchanged, except the `reset_after` comment now says "full allowance" instead of "bucket", because windows don't have buckets.
- `@classmethod def from_script(cls, ...)`: a **classmethod** is called on the class itself, `Decision.from_script(result, 10)`, rather than on an existing object. `cls` stands for the class, the way `self` stands for an object. Classmethods are the usual way to offer "another way to build this object". Here it builds a `Decision` from a script's raw `[allowed, remaining, retry_ms, reset_ms]` list, converting 1/0 to True/False and milliseconds to seconds.
- `-> "Decision"` is in quotes because, at that line, the `Decision` class is still being defined. Quoting delays the lookup.
- `result: list[int]` is a type hint meaning "a list of integers".
- `class RateLimitStrategy(ABC)` with an `@abstractmethod` `check`: the interface. Its body is just a **docstring**, the triple-quoted text that documents the promise every implementation makes. An abstract method needs no code.

`app/limiter/token_bucket.py`, after the refactor:

```
from redis.asyncio import Redis

from app.limiter.base import Decision, RateLimitStrategy, load_script

LUA_SCRIPT = load_script("token_bucket.lua")


class TokenBucket(RateLimitStrategy):
    def __init__(self, redis: Redis, capacity: int, refill_rate: float) -> None:
        self.capacity = capacity
        self.refill_rate = refill_rate
        self._script = redis.register_script(LUA_SCRIPT)

    async def check(self, client_id: str, cost: int = 1) -> Decision:
        key = f"rategate:tb:{client_id}"
        result = await self._script(
            keys=[key], args=[self.capacity, self.refill_rate, cost]
```

```
        )
        return Decision.from_script(result, self.capacity)
```

Phase 1	Phase 2	Why
`class TokenBucket:`	`class TokenBucket(RateLimitStrategy):`	It formally follows the contract
`Decision` defined in this file	Imported from `base`	Shared by all algorithms
Built the script path itself	`load_script("token_bucket.lua")`	Shared helper
Unpacked 4 values, built `Decision` by hand	`Decision.from_script(result, self.capacity)`	Shared converter

The file went from 35 lines to 19, and `token_bucket.lua` didn't change at all. The `self._script(...)` call is split over three lines only because ruff wraps lines longer than 88 characters.

Code 2: `config.py`, the new settings

Three settings were added: which algorithm to run, and the two numbers the window algorithms use.

```
from typing import Literal

from pydantic_settings import BaseSettings, SettingsConfigDict

Algorithm = Literal["token_bucket", "sliding_log", "sliding_counter"]


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="RATEGATE_", env_file=".env")

    redis_url: str = "redis://localhost:6379/0"
    algorithm: Algorithm = "token_bucket"

    # Token bucket
    capacity: int = 10  # max tokens in a bucket (burst size)
    refill_rate: float = 1.0  # tokens added per second

    # Sliding window log and sliding window counter
    limit: int = 10  # max requests per window
    window_seconds: float = 10.0  # window length


settings = Settings()
```

`Literal[...]` is a type meaning "exactly one of these values, nothing else". Pydantic enforces it when the settings are read. With `RATEGATE_ALGORITHM=leaky_bucket`, the app stops before it starts, listing the three valid names. A misspelled algorithm can't silently fall back to something else.

`Algorithm = Literal[...]` gives the type a name, called a **type alias**. The field line stays short, and the list of valid names sits in one obvious place.

`algorithm: Algorithm = "token_bucket"`: the default is the token bucket, so with no setting, the app behaves exactly as in Phase 1.

`limit` and **`window_seconds`** are new because the window algorithms think in "N requests per window", not in tokens and refill rates. They're kept separate from `capacity` and `refill_rate` so each setting means one thing.

Comments as section labels. `# Token bucket` and `# Sliding window ...` group the fields by who uses them, which matters once a file has several.

Each field is still set by an environment variable with the `RATEGATE_` prefix: `RATEGATE_ALGORITHM`, `RATEGATE_LIMIT`, `RATEGATE_WINDOW_SECONDS`.

Code 3: The sliding window log

The Lua script does the three steps from Theory 2 atomically; the Python class is the same 15-line wrapper shape as the token bucket.

`app/limiter/scripts/sliding_log.lua`:

```
-- Sliding window log, executed atomically inside Redis.
-- KEYS[1] = sorted set of this client's request times, KEYS[2] = counter that keeps entries
unique
-- ARGV[1] = limit (requests per window), ARGV[2] = window length (ms), ARGV[3] = cost of this
request

local key = KEYS[1]
local seq_key = KEYS[2]
local limit = tonumber(ARGV[1])
local window_ms = tonumber(ARGV[2])
local cost = tonumber(ARGV[3])

local t = redis.call('TIME')
local now = tonumber(t[1]) * 1000 + math.floor(tonumber(t[2]) / 1000)

-- Forget requests that have slid out of the window, then count the rest.
redis.call('ZREMRANGEBYSCORE', key, '-inf', now - window_ms)
local count = redis.call('ZCARD', key)

local allowed = 0
local retry_after_ms = 0
if count + cost <= limit then
  -- Log this request. Two requests can share a millisecond, so each entry gets a sequence number.
  for i = 1, cost do
    local seq = redis.call('INCR', seq_key)
    redis.call('ZADD', key, now, now .. '-' .. seq)
  end
  count = count + cost
  allowed = 1
  -- Both keys can go once this newest request has left the window.
  redis.call('PEXPIRE', key, window_ms)
  redis.call('PEXPIRE', seq_key, window_ms)
else
  -- Wait until enough of the oldest requests slide out of the window.
  local must_leave = count + cost - limit
  local entry = redis.call('ZRANGE', key, must_leave - 1, must_leave - 1, 'WITHSCORES')
  if entry[2] then
    retry_after_ms = tonumber(entry[2]) + window_ms - now
  else
    retry_after_ms = window_ms
  end
end

-- The full allowance is back when the newest logged request leaves the window.
local reset_ms = 0
local newest = redis.call('ZRANGE', key, -1, -1, 'WITHSCORES')
if newest[2] then
  reset_ms = tonumber(newest[2]) + window_ms - now
```

```
end

return {allowed, limit - count, retry_after_ms, reset_ms}
```

Block by block:

- **Inputs.** Two keys this time: the log and its sequence counter. Every key a script touches is passed in `KEYS`, never made up inside the script. Redis, and especially Redis Cluster, which spreads keys over several servers, relies on knowing a script's keys in advance.
- **Time** comes from Redis's `TIME`, exactly as in the token bucket, so every server uses one clock.
- **Cleanup and count.** `ZREMRANGEBYSCORE key -inf (now − window)` deletes every entry at least a window old. `ZCARD` counts the rest. Both run inside the same atomic script, so no other request can sneak in between them.
- **Allowed branch.** `for i = 1, cost do ... end` is a Lua loop, adding one entry per unit of cost (normally 1). `..` joins text in Lua, so the member is `1793623325995-1`. `count` is increased locally so the returned `remaining` is correct.
- **Expiry only when allowed.** The key only needs to live until its newest entry is a window old. The newest entry only changes when a request is allowed, so only that branch resets the timer.
- **Blocked branch: when to retry.** If count + cost − limit entries must leave first, the one that matters is that many places from the oldest, at index `must_leave − 1` (indexes count from 0). `ZRANGE ... WITHSCORES` returns `{member, score}`, so `entry[2]` is its time. It leaves at score + window, so retry = score + window − now. The `else` is a safety net for impossible requests (cost bigger than the limit).
- **Reset.** The whole allowance is back when the **newest** entry leaves.
- **Return.** The same four numbers as the token bucket: `{allowed, remaining, retry_ms, reset_ms}`. That shared format is what lets `Decision.from_script` serve every algorithm.

`app/limiter/sliding_log.py`:

```
from redis.asyncio import Redis

from app.limiter.base import Decision, RateLimitStrategy, load_script

LUA_SCRIPT = load_script("sliding_log.lua")


class SlidingWindowLog(RateLimitStrategy):
    def __init__(self, redis: Redis, limit: int, window_seconds: float) -> None:
        self.limit = limit
        self.window_ms = int(window_seconds * 1000)
        self._script = redis.register_script(LUA_SCRIPT)

    async def check(self, client_id: str, cost: int = 1) -> Decision:
        key = f"rategate:swl:{client_id}"
        result = await self._script(
            keys=[key, f"{key}:seq"], args=[self.limit, self.window_ms, cost]
        )
        return Decision.from_script(result, self.limit)
```

The same shape as `TokenBucket`, with four differences:

- It takes `limit` and `window_seconds` instead of `capacity` and `refill_rate`.
- `int(window_seconds * 1000)` converts seconds to whole milliseconds once, at startup, because the Lua works in milliseconds.
- The key prefix `swl` keeps its keys apart from the token bucket's `tb`, so switching algorithms never reads another algorithm's data.

- It passes two keys: `rategate:swl:<client>` and `rategate:swl:<client>:seq`.

Code 4: The sliding window counter

The counter's script finds the current slice, rolls the stored counts forward, computes the weighted estimate, and decides, all in one atomic step.

`app/limiter/scripts/sliding_counter.lua`:

```
-- Sliding window counter, executed atomically inside Redis.
-- KEYS[1] = hash holding this client's window number and two counts
-- ARGV[1] = limit (requests per window), ARGV[2] = window length (ms), ARGV[3] = cost of this
request

local key = KEYS[1]
local limit = tonumber(ARGV[1])
local window_ms = tonumber(ARGV[2])
local cost = tonumber(ARGV[3])

local t = redis.call('TIME')
local now = tonumber(t[1]) * 1000 + math.floor(tonumber(t[2]) / 1000)

-- Time is cut into fixed windows numbered 0, 1, 2, ...; find ours and how far into it we are.
local window = math.floor(now / window_ms)
local elapsed = now - window * window_ms

-- Load the saved counts and roll them forward if time has moved on.
local state = redis.call('HMGET', key, 'window', 'curr', 'prev')
local saved_window = tonumber(state[1])
local curr = tonumber(state[2]) or 0
local prev = tonumber(state[3]) or 0
if saved_window == nil or saved_window < window - 1 then
  -- New client, or idle for over a whole window: nothing counts any more.
  prev = 0
  curr = 0
elseif saved_window == window - 1 then
  -- One window boundary passed: the current count becomes the previous one.
  prev = curr
  curr = 0
end

-- Estimate the requests in the last window_ms: all of this window, plus
-- the share of the previous window that still overlaps it.
local weight = (window_ms - elapsed) / window_ms
local estimate = prev * weight + curr

local allowed = 0
local retry_after_ms = 0
if estimate + cost <= limit then
  curr = curr + cost
  estimate = estimate + cost
  allowed = 1
else
  local room = limit - cost - curr
  if room < 0 or prev == 0 then
    -- This window alone is full: wait for the next one.
    retry_after_ms = window_ms - elapsed
  else
    -- Wait until the previous window's share shrinks enough.
    retry_after_ms = math.ceil(window_ms * (1 - room / prev) - elapsed)
  end
```

```
end

-- After two idle windows both counts are irrelevant, so the key can go.
redis.call('HSET', key, 'window', window, 'curr', curr, 'prev', prev)
redis.call('PEXPIRE', key, 2 * window_ms)

-- Everything is forgotten once this window and the next have passed.
local reset_ms = window_ms - elapsed
if curr > 0 then
  reset_ms = reset_ms + window_ms
end

return {allowed, math.max(0, math.floor(limit - estimate)), retry_after_ms, reset_ms}
```

Block by block:

- **Slice number and position.** `math.floor(now / window_ms)` is the slice number. `elapsed` is how far into it we are. Example: now = 1793621490482 and window_ms = 10000 give slice 179362149, elapsed 482 ms.
- **Load with defaults.** `HMGET` reads three fields at once. In Lua, `x or 0` means "x, or 0 if x is nil", so a new client starts with zero counts.
- **Roll forward** (the `if / elseif`), following the table in Theory 3. When the saved slice is the current one, neither branch runs.
- **Why one hash, and not one key per slice?** The slice number is computed *inside* the script from `TIME`. Keys must be passed in from outside via `KEYS`, so a key name can't depend on it. Storing the slice number as a field sidesteps the problem.
- **Estimate and decision**: the formula from Theory 3.
- **Retry when blocked**, in two cases. `room = limit − cost − curr` is how much of the previous slice's share we can afford. If it's negative, the current slice alone is too full, so wait for the next slice: window − elapsed. Otherwise, solve prev × (window − e) ÷ window ≤ room for the position e at which enough has faded: e = window × (1 − room ÷ prev). Retry = e − elapsed, rounded up. Like the algorithm, it's an estimate.
- **Always save.** Unlike the log, this script saves even when blocking, because the roll-forward may have changed the counts.
- **Expiry of 2 windows.** After that long, both counts would be reset anyway.
- **Reset.** If the current slice has requests, they matter until the *next* slice ends too, hence the extra window. That's why Check 5 showed `x-ratelimit-reset: 18` for the counter.
- **Remaining** = floor(limit − estimate), never below 0. The estimate is a decimal, such as 8.3, so rounding down gives an honest whole number.

`app/limiter/sliding_counter.py`:

```
from redis.asyncio import Redis

from app.limiter.base import Decision, RateLimitStrategy, load_script

LUA_SCRIPT = load_script("sliding_counter.lua")


class SlidingWindowCounter(RateLimitStrategy):
    def __init__(self, redis: Redis, limit: int, window_seconds: float) -> None:
        self.limit = limit
        self.window_ms = int(window_seconds * 1000)
        self._script = redis.register_script(LUA_SCRIPT)
```

```
    async def check(self, client_id: str, cost: int = 1) -> Decision:
        key = f"rategate:swc:{client_id}"
        result = await self._script(keys=[key], args=[self.limit, self.window_ms, cost])
        return Decision.from_script(result, self.limit)
```

The same shape as the log, with the prefix `swc` and a single key. By now the pattern is clear: every strategy is about 15 lines of Python around its Lua script.

Code 5: The factory and `main.py`

The factory turns the setting into an object, and `main.py` changed by just two lines to use it.

`app/limiter/factory.py`:

```
from redis.asyncio import Redis

from app.config import Settings
from app.limiter.base import RateLimitStrategy
from app.limiter.sliding_counter import SlidingWindowCounter
from app.limiter.sliding_log import SlidingWindowLog
from app.limiter.token_bucket import TokenBucket


def create_limiter(redis: Redis, settings: Settings) -> RateLimitStrategy:
    if settings.algorithm == "token_bucket":
        return TokenBucket(redis, settings.capacity, settings.refill_rate)
    if settings.algorithm == "sliding_log":
        return SlidingWindowLog(redis, settings.limit, settings.window_seconds)
    if settings.algorithm == "sliding_counter":
        return SlidingWindowCounter(redis, settings.limit, settings.window_seconds)
    raise ValueError(f"Unknown algorithm: {settings.algorithm}")
```

- **The return type is `RateLimitStrategy`**, the shape, not a specific class. That's the promise to callers: you'll get *something* with a `check` method.
- **It imports the `Settings` class, not the global `settings` object**, and receives the settings as a parameter. Tests can then pass `Settings(algorithm="sliding_log")` directly instead of changing environment variables. Handing a function what it needs is dependency injection, as with Redis in Phase 1.
- **Each `if` returns immediately**, so no `elif` is needed.
- **The final `raise`** normally can't be reached, because `Literal` already rejected bad names. It's a safety net for one specific future mistake: adding a name to `Algorithm` in `config.py` but forgetting to add a branch here. Without it, the function would quietly return `None` and the app would crash later with a confusing error.

`app/main.py`, two lines changed:

```
from app.limiter.token_bucket import TokenBucket       # removed
from app.limiter.factory import create_limiter         # added
```

```
    app.state.limiter = TokenBucket(redis, settings.capacity, settings.refill_rate)   # removed
    app.state.limiter = create_limiter(redis, settings)                               # added
```

After this change, `main.py` doesn't name any algorithm. Nothing else in the file changed.

`app/middleware.py` : **not touched.** It calls `limiter.check(...)` and reads `decision.allowed`, `.limit`, `.remaining`, `.retry_after`, `.reset_after`. Every strategy provides exactly those, so the middleware works with all three as it was written in Phase 1.

Code 6: The 16 tests

Six test functions, most run once per algorithm, give 16 test cases that check every algorithm against the real Redis in under 5 seconds.

`tests/conftest.py` : pytest automatically loads a file with this exact name, and any test in the folder can use what it defines.

```
from uuid import uuid4

import pytest
from redis.asyncio import Redis

from app.config import settings


@pytest.fixture
async def redis():
    client = Redis.from_url(settings.redis_url, decode_responses=True)
    yield client
    await client.aclose()


@pytest.fixture
def client_id():
    # A fresh id per test, so tests never share a bucket or wait for old keys to expire.
    return f"test-{uuid4()}"
```

- A **fixture** is reusable setup. A test receives one simply by naming it as a parameter: `async def test_x(redis, client_id)`. pytest sees the names, runs the matching fixtures, and passes in the results.
- In `redis`, the code before `yield` is setup (open a client) and the code after is cleanup (close it). It runs for every test, even one that fails.
- `uuid4()` produces a random, practically unique ID such as `test-3f2a9c1e-7b1d-4e8a-9c21-7d4e5f6a8b90`. Each test therefore gets a brand-new client in every algorithm. No test inherits another's state, and nobody waits for keys to expire.

`tests/test_limiters.py` :

```
import asyncio

import pytest
from pydantic import ValidationError

from app.config import Settings
from app.limiter.factory import create_limiter
from app.limiter.sliding_counter import SlidingWindowCounter
from app.limiter.sliding_log import SlidingWindowLog
from app.limiter.token_bucket import TokenBucket

# Every algorithm allows 5 requests, plus how long to wait until it allows again.
ALGORITHMS = {
    "token_bucket": (
        lambda redis: TokenBucket(redis, capacity=5, refill_rate=1.0),
```

```
        1.1,
    ),
    "sliding_log": (
        lambda redis: SlidingWindowLog(redis, limit=5, window_seconds=1),
        1.1,
    ),
    "sliding_counter": (
        lambda redis: SlidingWindowCounter(redis, limit=5, window_seconds=1),
        2.1,
    ),
}


def make(name, redis):
    build, _ = ALGORITHMS[name]
    return build(redis)


@pytest.mark.parametrize("name", ALGORITHMS)
async def test_allows_up_to_limit_then_blocks(redis, client_id, name):
    limiter = make(name, redis)
    results = [(await limiter.check(client_id)).allowed for _ in range(7)]
    assert results == [True] * 5 + [False] * 2


@pytest.mark.parametrize("name", ALGORITHMS)
async def test_blocked_decision_says_when_to_retry(redis, client_id, name):
    limiter = make(name, redis)
    for _ in range(5):
        await limiter.check(client_id)
    decision = await limiter.check(client_id)
    assert not decision.allowed
    assert decision.remaining == 0
    assert decision.retry_after > 0


@pytest.mark.parametrize("name", ALGORITHMS)
async def test_clients_have_separate_limits(redis, client_id, name):
    limiter = make(name, redis)
    for _ in range(5):
        await limiter.check(client_id)
    assert (await limiter.check(f"{client_id}-other")).allowed


@pytest.mark.parametrize("name", ALGORITHMS)
async def test_allows_again_after_waiting(redis, client_id, name):
    build, wait_seconds = ALGORITHMS[name]
    limiter = build(redis)
    for _ in range(5):
        await limiter.check(client_id)
    assert not (await limiter.check(client_id)).allowed
    await asyncio.sleep(wait_seconds)
    assert (await limiter.check(client_id)).allowed


@pytest.mark.parametrize(
    ("name", "expected_class"),
    [
        ("token_bucket", TokenBucket),
        ("sliding_log", SlidingWindowLog),
        ("sliding_counter", SlidingWindowCounter),
    ],
)
```

```
async def test_config_picks_the_algorithm(redis, name, expected_class):
    assert isinstance(create_limiter(redis, Settings(algorithm=name)), expected_class)


def test_unknown_algorithm_is_rejected():
    with pytest.raises(ValidationError):
        Settings(algorithm="leaky_bucket")
```

The building blocks:

- `ALGORITHMS` is a dictionary from name to a pair: how to build that limiter, and how long it needs before allowing again. All three are set to allow **5**, so one set of assertions fits all.
- `lambda redis: ...` is a tiny unnamed function. Storing a recipe instead of a ready object means each test builds a fresh limiter with its own Redis client.
- **Short windows (1 s)** keep the suite fast. The production defaults of 10 seconds would make the waiting test take 21 seconds.
- `@pytest.mark.parametrize("name", ALGORITHMS)` runs the test once per dictionary key. pytest prints each run with its value in brackets, such as `[sliding_log]`.
- `[... for _ in range(7)]` is a list comprehension: 7 checks, keeping each `.allowed`. `_` is the usual name for an unused loop variable.
- `[True] * 5 + [False] * 2` builds the expected list: five Trues, then two Falses.

The six tests:

Test	Cases	Proves
`test_allows_up_to_limit_then_blocks`	3	Exactly 5 allowed, then blocked, for each algorithm
`test_blocked_decision_says_when_to_retry`	3	A blocked `Decision` has `remaining == 0` and a positive `retry_after`
`test_clients_have_separate_limits`	3	Exhausting one client doesn't affect another
`test_allows_again_after_waiting`	3	Each algorithm recovers after its own wait
`test_config_picks_the_algorithm`	3	The factory builds the right class for each name
`test_unknown_algorithm_is_rejected`	1	`Settings` refuses an invalid name

Why the wait times differ: the token bucket regains a token after 1 second, so wait 1.1. The log forgets an entry exactly one window later, so 1.1. The counter's old count only disappears completely after **two** slice boundaries, so 2.1. Each value is the algorithm's own recovery time, plus a 0.1 s margin so the tests aren't flaky.

`pytest.raises(ValidationError)` passes only if the code inside it raises that error. `isinstance(obj, Cls)` checks an object's class. These tests use the real Redis, so they're integration tests: Redis must be running.

Command reference: every command used in Phase 2

Every command run during Phase 2, grouped by purpose, with what each part means and why it was used. All run in PowerShell from `D:\RateGate` unless noted.

Tests and code quality

Command	What each part means	Why we used it
`uv run pytest -v`	`uv run`: use the project's environment. `pytest`: find and run tests. `-v` (verbose): one line per test	See each of the 16 cases by name, with PASSED or FAILED
`uv run pytest -q`	`-q` (quiet): only the summary line	Repeated runs where only "16 passed" matters
`foreach ($i in 1..3) { uv run pytest -q }`	A PowerShell loop running the suite 3 times	A test that passes once might be lucky; passing every time shows it's stable
`uv run ruff check .`	`ruff check`: the **linter**, which looks for likely bugs and bad style. `.`: this folder and below	Catch unused imports, undefined names and similar mistakes
`uv run ruff format --check .`	`ruff format`: the **formatter**, which fixes layout. `--check`: only report, don't change files	Confirm layout is consistent without touching anything; CI will run the same check
`uv run ruff format .`	Without `--check`, it rewrites files	Apply the layout fixes

Running servers

Command	What each part means	Why we used it
`uv run uvicorn app.main:app --reload`	Start the app; `--reload` restarts it when a file changes	Normal development
`uv run uvicorn app.main:app --port 8002`	`--port`: listen on a door other than 8000	Run several servers at once, one per algorithm
`$env:RATEGATE_ALGORITHM = "sliding_log"`	Set an environment variable in **this terminal**	Choose the algorithm without editing code
`Remove-Item Env:RATEGATE_ALGORITHM`	Delete that variable from this terminal	Go back to the default; otherwise it silently stays set
`Get-NetTCPConnection -LocalPort 8000 -State Listen`	List programs listening on port 8000	Find a leftover server occupying a port

Sending requests

Command	What each part means	Why we used it
`1..15 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code} " -H "X-API-Key: c3" http://127.0.0.1:8002/api/resource }`	15 requests; print only status codes; send an API key header	A burst, readable at a glance
`curl.exe -i ...`	`-i`: include response headers	See `x-ratelimit-*` and `retry-after`
`$k = "cmp" + (Get-Random)`	`Get-Random` returns a random number; `+` joins it to text	A brand-new API key, so no earlier state interferes and no waiting is needed
`Start-Sleep -Seconds 5`	Pause for exactly 5 seconds	The waits in Check 5 are part of the test
`function burst($p) { ... }`	Define a reusable PowerShell function with a parameter `$p` (the port)	Send the same burst to three servers without repeating the long command

Command	What each part means	Why we used it
`(...) -join ' '`	Combine a list into one line, separated by spaces	Print 15 status codes on one line

Looking inside Redis (`docker exec redis redis-cli <command>` runs a Redis command inside the `redis` container)

Redis command	Shows	Used for
`HGETALL rategate:tb:key:c3`	Every field of a hash	Token bucket (`tokens`, `ts`) and counter (`window`, `curr`, `prev`)
`ZCARD rategate:swl:key:c3`	How many members a sorted set holds	How many requests the log holds
`ZRANGE rategate:swl:key:c3 0 -1 WITHSCORES`	Members from first (0) to last (−1), each followed by its score	The log's actual entries and times
`GET rategate:swl:key:c3:seq`	A plain value	The log's sequence counter
`PTTL <key>`	Milliseconds until the key expires	Confirm each algorithm's expiry: about 10 s, 10 s, and 20 s

Git

Command	What each part means	Why we used it
`git status --short`	Changed files, one per line: `M` modified, `A` added, `??` untracked	See exactly what will be committed
`git diff --stat HEAD -- app/middleware.py`	Compare one file with the last commit (`HEAD`); `--stat`: summary only; `--` separates options from file names	Check 7: empty output proves the middleware wasn't edited
`git add -A`	Stage every change, including new and deleted files	Prepare the commit
`git commit -F - <<'EOF' ... EOF` (in Bash)	`-F -`: read the message from standard input; the heredoc supplies a multi-line message	A commit message with a title plus a bullet list
`git log --oneline -3`	The last 3 commits, one line each	Confirm the commit exists
`git push`	Upload new commits to GitHub	Publish the work
`git status -sb`	Short status plus the branch line, such as `## main...origin/main`	Confirm local and GitHub are in sync (nothing ahead or behind)

Verification: 7 checks, all passed (2026-11-02, 18:10 to 18:13)

All seven checks were run in the real project before the commit. Four servers ran side by side: the default on port 8000, and one per algorithm on ports 8001 (token bucket), 8002 (sliding log) and 8003 (sliding counter).

Timing rules for Phase 2. Each algorithm forgets on its own clock:

Algorithm	Fresh state after	Why
Token bucket	10 s idle	The bucket refills completely
Sliding window log	10 s idle	Every entry has left the window

Algorithm	Fresh state after	Why
Sliding window counter	up to 20 s idle	Counts must pass two slice boundaries

The shortcut used throughout: **a new API key per check**, such as `-H "X-API-Key: c3-12345"`. A new key is a new client with a full allowance, so no check had to wait for an old one to be forgotten. Redis is inspected on the **same command line** as the requests, before keys expire.

Check 1: automated tests. No waiting needed: each test uses its own random client.

```
run 1 : 16 passed in 4.60s
run 2 : 16 passed in 4.66s
run 3 : 16 passed in 4.67s
```

All 16 cases, three times in a row. The roughly 4.6 seconds is mostly the deliberate waits (1.1 + 1.1 + 2.1 s).

Check 2: the default is still Phase 1. Port 8000, no setting:

```
200 200 200 200 200 200 200 200 200 200 429 429 429 429 429
```

With no setting, the algorithm defaults to the token bucket, and it behaves exactly as before the refactor.

Check 3: switching by setting. A fresh key on each server:

```
token_bucket    (8001) 200 200 200 200 200 200 200 200 200 200 429 429 429 429 429
sliding_log     (8002) 200 200 200 200 200 200 200 200 200 200 429 429 429 429 429
sliding_counter (8003) 200 200 200 200 200 200 200 200 200 200 429 429 429 429 429
```

The same code, three behaviours chosen by one environment variable each. On a first burst they look identical; the differences appear over time.

Check 4: inside Redis, on the same line as Check 3:

```
token_bucket    HGETALL: tokens 0.29999999999999905 ts 1793623325932
sliding_log     ZCARD: 10 | seq counter: 10
sliding_log     first 3: 1793623325995-1 1793623325995 1793623326018-2 1793623326018
1793623326039-3 1793623326039
sliding_counter HGETALL: window 179362332 curr 10 prev 0
PTTL ms: tb=8330 swl=8448 swc=18790
```

- **Token bucket:** 2 numbers, as in Phase 1.
- **Sliding log:** 10 entries, one per allowed request; the 5 blocked ones were not logged. Each entry prints as `member score`. The entries are about 20 ms apart: the time each curl took. The sequence counter reached 10.
- **Sliding counter:** slice 179362332 holds 10 requests, and the previous slice none.
- **Expiry:** about 8 s left for the first two (set to 10 s), about 19 s for the counter (set to 20 s). These match each script's rule.

This is the memory trade-off made visible: 10 stored entries against 2 or 3 numbers.

Check 5: recovery over time. A fresh random key, then a burst of 15 at t = 0, again after 5 seconds, and again 6 seconds later:

```
             token_bucket   sliding_log   sliding_counter
t=0          10 allowed     10 allowed    10 allowed
```

```
t=5           5 allowed      0 allowed     2 allowed
t=11          7 allowed     10 allowed     7 allowed
```

- **Token bucket:** 5 tokens dripped back in 5 seconds, then about 7 in the next 7.
- **Sliding log:** nothing at t = 5, because all 10 requests were still inside the last 10 seconds. At t = 11 all 10 were older than the window, so the full allowance came back at once.
- **Sliding counter:** the t = 0 burst happened about 5 s into a slice. By t = 5 a new slice had begun, about 2 s in: weight 0.8, estimate 10 × 0.8 = 8, so 2 fit. By t = 11 it was about 9 s in: weight 0.1, estimate 1 + 2 = 3, so 7 fit.

An earlier run, with the burst at the start of a slice, gave the counter 0 and then 5. The token bucket and the log gave the same numbers in both runs. Only the approximate algorithm depends on where the boundary falls.

Headers on one more request straight after the first burst:

```
token_bucket     HTTP/1.1 200 OK, x-ratelimit-remaining: 0, x-ratelimit-reset: 10
sliding_log      HTTP/1.1 429, x-ratelimit-reset: 10, retry-after: 10
sliding_counter  HTTP/1.1 429, x-ratelimit-reset: 14, retry-after: 4
```

The token bucket answered **200**: about a second had passed, so a token had dripped back. The log said retry in 10 s, when its oldest entry leaves. The counter said retry in 4 s, when its slice ends, and reset in 14 s, at the end of the following slice.

Check 6: a misspelled setting. `RATEGATE_ALGORITHM=leaky_bucket`:

```
exit code: 1
pydantic_core._pydantic_core.ValidationError: 1 validation error for Settings
algorithm
  Input should be 'token_bucket', 'sliding_log' or 'sliding_counter' [type=literal_error,
input_value='leaky_bucket', input_type=str]
port 8009 listening? False
```

The server refused to start and listed the valid choices. Failing loudly at startup beats running with a wrong limiter.

Check 7: the middleware. `git diff --stat HEAD -- app/middleware.py` printed nothing. Three algorithms run behind it, and it wasn't edited once.

Finish: `ruff check` passed, 14 files already formatted, and the commit `2630d51` (16 files) was pushed. `git status -sb` showed `## main...origin/main`, meaning in sync with GitHub.

Common pitfalls

Five of these actually happened during Phase 2 (marked ★); the rest are the mistakes this design most often invites.

Pitfall	Symptom	Why it happens	Fix
★ Multi-line commit message in PowerShell	`error: pathspec '...' did not match any file(s)`, and nothing committed	`git commit -F -` reads the message from standard input. A PowerShell here-string written after it isn't sent to standard input, so Git reads the text as file names	Use a Bash heredoc (`git commit -F - <<'EOF' ... EOF`), a message file (`-F msg.txt`), or several `-m "..."` flags

Pitfall	Symptom	Why it happens	Fix
★ ruff reformats Markdown	`ruff format --check` reports `docs\PHASE_1_GUIDE.md` and `PHASE_2_GUIDE.md`	This ruff version also formats Python code blocks inside `.md` files	`[tool.ruff] extend-exclude = ["docs"]` in `pyproject.toml`
★ pytest-asyncio deprecation warning	A long `PytestDeprecationWarning` about `asyncio_default_fixture_loop_scope`	The setting was left unset	Add `asyncio_default_fixture_loop_scope = "function"`
★ "LF will be replaced by CRLF" warnings	One warning per file on `git add`	Windows and Linux end lines differently; Git converts automatically (`core.autocrlf`)	Harmless, ignore
★ A "blocked" check returns 200	The token bucket's header check showed `200 OK` after a burst	A second passed between the burst and the check, so a token dripped back	Expected; check headers immediately, or read it as proof of refilling
Algorithm setting stays on	A later run uses `sliding_log` when you expected the default	`$env:` variables last until the terminal closes	`Remove-Item Env:RATEGATE_ALGORITHM`, or open a new terminal
Counter gives different numbers each run	Check 5's counter row differs from the guide	Its estimate depends on where the burst sits in its 10-second slice	Expected: that's the approximation
Old state leaks into a check	Fewer `200`s than expected at the start	The same API key was used within the last 10–20 s	Use a fresh key per check (`Get-Random`), or wait 20 s
Checking Redis too late	`ZRANGE` or `HGETALL` prints nothing	The key expired (10 s for log and bucket, 20 s for the counter)	Put the Redis command on the same line as the burst
Importing `Decision` from the old place	`ImportError: cannot import name 'Decision' from 'app.limiter.token_bucket'`	`Decision` moved to `base.py`	`from app.limiter.base import Decision`
A strategy without `check`	`TypeError: Can't instantiate abstract class ... without an implementation for abstract method 'check'`	`@abstractmethod` enforces the contract	Add `check`, spelled exactly the same
Async tests not running	`async def functions are not natively supported`, tests skipped	pytest-asyncio missing, or `asyncio_mode = "auto"` not set	`uv add --dev pytest-asyncio` and the setting

Pitfall	Symptom	Why it happens	Fix
Port already in use	`[Errno 10048] ... address already in use`	A previous server is still running on that port	Ctrl+C in its terminal; `Get-NetTCPConnection -LocalPort 8000 -State Listen` shows which process
Same member twice in the log	The log lets more requests through than the limit under heavy load	Two requests in one millisecond share a member name, so the second overwrites the first	Already prevented by the `-seq` suffix: don't remove it
Computing key names inside a Lua script	Works locally, breaks on Redis Cluster	Redis needs every key declared in `KEYS`	Pass keys in; store window numbers as fields, as the counter does
Changing several things before testing	A failure with no obvious cause	Too many new parts at once	Refactor first, run the regression check, then add new code

Check your understanding (Phase 2)

Answer each aloud before reading the answer. These come up in system-design and code interviews.

1. What is the strategy pattern, and what proves it works here? Several interchangeable implementations of one job behind a shared interface, so callers only know the interface. The proof is that `middleware.py` runs all three algorithms and wasn't edited.

2. Why is `check` marked `@abstractmethod`? To enforce the contract. A strategy class missing `check` can't be created, so the mistake shows up immediately, not when a request arrives.

3. Why refactor the token bucket before adding the new algorithms? To test one change at a time. The regression check after the refactor proved it changed nothing, so any later failure had to come from the new code.

4. Why does each sliding log entry carry a sequence number? Sorted-set members must be unique. Two requests in the same millisecond would share a name, and the second would overwrite the first, letting an extra request through.

5. A client may make 1,000 requests a minute. What does each algorithm store for them? Token bucket: 2 numbers. Sliding log: up to 1,000 entries. Sliding counter: 3 numbers.

6. In Check 5, why did the token bucket allow 5 at t = 5 when the log allowed none? The bucket refills continuously at 1 token per second. The log only frees capacity when old requests become a full window old, and all of them were under 10 seconds old.

7. Previous slice 6, current slice 2, 70% into the current slice, limit 10. Is the next request allowed? Weight = 0.3, so estimate = 6 × 0.3 + 2 = 3.8. Since 3.8 + 1 ≤ 10, yes.

8. Why does the counter store its slice number in the hash instead of using a key per slice? The slice number is computed inside the script from Redis's clock, but keys must be declared in `KEYS` before the script runs. A field avoids needing a computed key name.

9. Why does the counter's key expire after 2 windows, not 1? The previous slice's count still affects the estimate throughout the next slice. Only after two boundaries is it irrelevant.

10. Why does `create_limiter` take settings as a parameter instead of importing the global? So tests can pass `Settings(algorithm="sliding_log")` directly, without changing environment variables. This is dependency injection.

11. Why is a misspelled algorithm a startup error rather than a fallback to the default? A silent fallback would run the wrong limiter in production without anyone noticing. Failing loudly at startup is caught immediately.

12. When would you choose each algorithm? Token bucket for bursty but rate-capped APIs. Sliding log for small, strict limits such as login attempts. Sliding counter for huge traffic where per-client memory must be tiny and slight error is acceptable.

Phase 3: Correctness under concurrency (done 2026-11-02)

200 requests fired at the same instant against a limit of 100 now allow exactly 100, for every algorithm, and a test fails if that ever stops being true. Writing those tests exposed two real bugs in the Phase 1 and 2 code, both now fixed. The work is committed as `d7d9c57` and pushed.

This turns a resume claim into evidence: *"using atomic Redis Lua scripts to prevent race conditions under concurrent requests."*

Result	Evidence
No algorithm lets extra requests through	200 simultaneous requests → exactly 100 allowed, for all three
It holds across several servers	4 independent Redis clients sharing the load → still exactly 100
The test can detect a race	A deliberately non-atomic limiter let **200 of 200** through
It works end to end	Over real HTTP: 100 × `200` and 100 × `429`, no errors
Stable, not lucky	Full suite 3/3 runs (31 passed); concurrency tests 10/10 runs

The two bugs the tests found:

Bug	What a user would have seen	Fix
The Redis connection pool raised an error when more than 100 commands were in flight	Under a burst of 200 requests, **77 to 100 answered `500 Internal Server Error`**	A `BlockingConnectionPool`, which waits briefly for a free connection
The sliding counter's `Retry-After` was too short	A client that waited exactly as told was **still refused**	The retry formula now accounts for the full window's count carrying into the next window

What changed in the project:

File	Change
`tests/test_concurrency.py`	New: 7 concurrency tests, including the broken "control" limiter
`tests/test_limiters.py`	8 boundary tests appended (60 lines, nothing removed)
`app/redis_client.py`	New: `create_redis()`, the one place Redis clients are built
`app/config.py`	Two pool settings
`app/main.py`, `tests/conftest.py`	Use `create_redis()`
`app/limiter/scripts/sliding_counter.lua`	`Retry-After` fix
`scripts/concurrent_burst.py`	New: fires a burst over real HTTP
`README.md`, `ROADMAP.md`	Testing section, pool settings, Phase 3 ticked
`app/middleware.py`	Still unchanged since Phase 1

The step-by-step guide is in the project at `docs/PHASE_3_GUIDE.md`.

Theory 1: Why earlier tests couldn't catch a race

A race condition only happens when operations overlap, and every Phase 1 and 2 test sent requests strictly one after another, so even a broken limiter would have passed them.

Sequential requests. Phase 2's tests send requests like this:

```
results = [(await limiter.check(client_id)).allowed for _ in range(7)]
```

Each `await` waits for its request to **finish** before the next one starts. A race needs one request to read the count *before* another has written it back. With one-at-a-time requests that never happens, so these tests can't tell an atomic limiter from a broken one.

Creating real overlap with `asyncio.gather`:

```
decisions = await asyncio.gather(*(limiter.check(client_id) for _ in range(200)))
```

1. `limiter.check(client_id)` written **without** `await` doesn't run yet. It creates a **coroutine**: a piece of work, paused before its first line.
2. `(... for _ in range(200))` makes 200 of them. The `*` unpacks them into 200 separate arguments.
3. `asyncio.gather(...)` starts all 200 at once and waits until every one is done. It returns their results in the original order.

Inside, each coroutine runs until its first `await`, which sends a command to Redis, then pauses so the next one can start. Within a few milliseconds all 200 commands are travelling to Redis together, like 200 users clicking at the same moment.

Concurrency, not parallelism. This is **concurrency**: many tasks in progress at once, taking turns on one thread. **Parallelism** means several CPU cores working at literally the same instant. Races don't need parallelism. All they need is for one request's read to fall between another request's read and write, and concurrency produces that constantly.

Summing booleans. Tests count allowed requests with `sum(d.allowed for d in decisions)`. In Python `True` behaves as 1 and `False` as 0, so the sum is the number of `True`s.

Theory 2: Testing the test (a negative control)

A passing test only means something if it could have failed. So the test suite also runs a deliberately broken limiter, and checks that the same test catches it.

The problem. Suppose the concurrency test passes. Maybe the limiter is correct. Or maybe the requests didn't really overlap, and the test would pass for broken code too. From a pass alone, you can't tell.

The control. Scientists handle this with a **negative control**: run the same experiment on something known to be broken, and confirm the experiment detects it. Here that's `NaiveCounter`. It reads the count, decides in Python, and writes it back, as three separate Redis calls. That's exactly the design the Lua scripts exist to avoid.

What happened. Same 200 simultaneous requests, same limit of 100, same Redis, five runs each:

Limiter	Allowed
Naive (read, decide, write separately)	**200 of 200**, every run
Token bucket (atomic Lua script)	**100 of 200**, every run

The naive limiter didn't just leak a few requests; it had no effect at all. Every overlapping request read "0" before anyone wrote "1". The only difference between the two rows is atomicity, which is the whole point of the project.

The test `test_naive_limiter_lets_too_many_through` asserts that the naive limiter allows **more than** 100. If it ever stops doing so, the concurrency tests are no longer creating real overlap, and their passes can't be trusted. This is also the strongest single answer to an interviewer asking "how do you know it's race-free?"

Theory 3: Connection pools and the burst crash

The app's Redis client could hold at most 100 connections and threw an error for the 101st, so a burst of 200 requests turned half of them into `500` errors. A pool that waits instead of failing fixed it.

What a connection pool is. Opening a network connection to Redis costs a handshake, so a Redis client keeps a **pool** of open connections. Each command borrows one and returns it when done. A pool is like a stack of trays in a canteen: people take one, use it, and put it back for the next person.

When every connection is busy, the two kinds of pool behave differently:

Pool type	When all connections are in use	Default size in redis-py
`ConnectionPool` (what Phases 1 and 2 used)	Raises `MaxConnectionsError` immediately	100
`BlockingConnectionPool`	Waits up to `timeout` seconds for one to come back	50

How that became a 500 error. With 200 requests at once, request 101 found all 100 connections busy and the pool raised `MaxConnectionsError`. Nothing in the middleware catches that error, so FastAPI answered `500 Internal Server Error`. The limit itself was still enforced (exactly 100 were allowed), but the refused requests crashed instead of receiving a polite `429`.

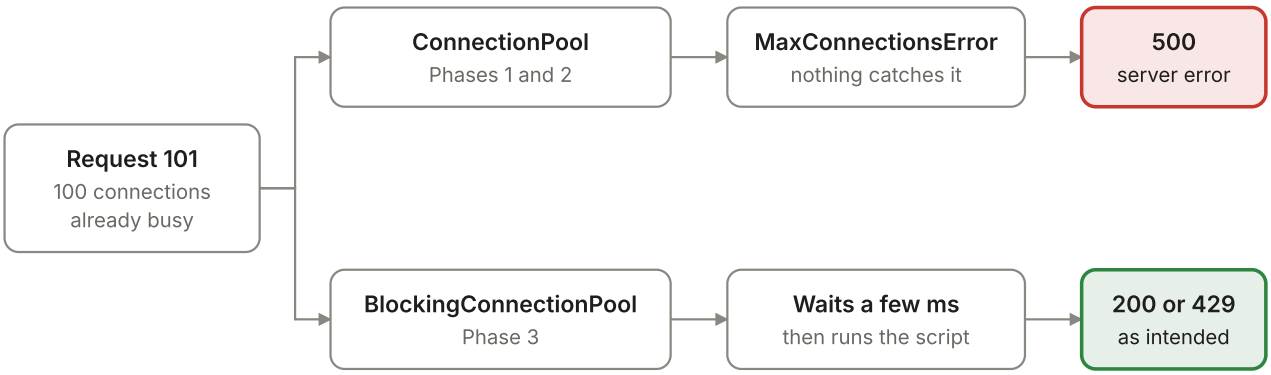
Why the test exposed it, and the app's own tests didn't. Every earlier test sent one request at a time, using one connection. Nothing had ever asked the pool for more than a handful.

Why a pool of 50 is enough. Redis executes one command at a time, each in microseconds. 50 connections can carry thousands of requests per second, and a request that has to wait for a connection waits only milliseconds. Bigger pools mostly cost memory on both sides. Redis accepts up to 10,000 connections by default, so in Phase 5, three servers × 50 = 150 is far below its limit.

Why a timeout. `redis_pool_timeout = 5.0` caps the wait. If Redis became very slow, waiting forever would make requests pile up silently inside the app. With a cap, the wait ends in an error after 5 seconds, which Phase 4 will turn into a deliberate fail-open or fail-closed answer.

One builder for app and tests. Phase 3 moved client creation into `create_redis()` in `app/redis_client.py`, used by both `main.py` and the test fixture. Before, the app and the tests each built their own client. They happened to match, but nothing guaranteed it, and a test that builds Redis differently from production can pass while production fails.

Request 101 crashed with the old pool and waits with the new one



In both cases exactly 100 were allowed; the difference is how the other 100 were refused.

Request 101 under a burst · old pool vs new pool

The only change is the pool type, made in `create_redis()`. The limiter and the middleware are the same in both rows.

Theory 4: Boundary testing and the `Retry-After` bug

Bugs gather at the edges where behaviour changes. For a rate limiter those edges are moments in time, and testing one of them caught the sliding counter's second bug.

The three time boundaries tested:

Boundary	The question	The test
The moment a blocked client may retry	Is `Retry-After` honest?	Exhaust the limit, wait exactly `retry_after`, and the next request **must** be allowed
The moment an idle key expires	Is memory cleaned up on schedule?	After one request, the key's time to live must be within the expected range; after waiting it out, the key must be gone
A window boundary	Can a client double its burst across it?	5 requests just before a boundary and 5 just after must total exactly 5 allowed

The third test checks Phase 2's main promise about the window algorithms: the fixed-window loophole is closed. A plain fixed-window limiter would allow all 10.

The bug. When the sliding counter's **current** slice alone was full, the old code said "retry when this slice ends". But at the start of the next slice, the old count becomes `prev` with a weight of almost 1, so the estimate is still full.

Worked example with limit 10 and window 10 s. A client makes 10 requests 3 seconds into a slice:

1. **Old answer:** retry in 7 s, when the slice ends.
2. **At that moment:** prev = 10, weight = (10 − 0) ÷ 10 = 1, estimate = 10. Still full, so the request is **refused**.
3. **Correct answer:** the 7 seconds, plus the time for 10 × weight to fall to 9 so one more fits. The weight must reach 0.9, which happens 1 second into the new slice. Retry in **8 s**.

The general rule, when the current slice alone is full:

$$\text{retry} = (\text{window} - \text{elapsed}) + \text{window} \times \left(1 - \frac{\text{limit} - \text{cost}}{\text{curr}}\right)$$

What the test saw. With a 1-second window and limit 5, the counter answered `retry_after=0.137`. The test waited 0.137 seconds and was still refused. A client obeying `Retry-After` should never be refused. Only the counter failed; the token bucket and the log were already exact.

Using Redis's clock in tests. The window test has to start 100 ms before a boundary, and windows are cut by Redis's clock, not Python's. So the helper `sleep_until` reads the time from Redis (`redis.time()`), the same `TIME` the scripts use, and sleeps until the right offset.

Theory 5: Write the failing test first

Each Phase 3 test was written and run **before** the fix it targets, so it was seen failing ("red") and then passing ("green").

Why watch a test fail first?

- **It proves the test can detect the bug.** A test you've only ever seen pass might be testing nothing at all.
- **It proves your fix is what made the difference.** Red before the change and green after it, with nothing else changed, leaves only one explanation.
- **It shows what is really broken,** which isn't always what you expected. Here the concurrency tests didn't fail by allowing too many requests; they crashed with `MaxConnectionsError`, revealing a bug nobody was looking for.

This habit is called **test-driven development (TDD)**: red, then green, then tidy up. You don't need it for everything, but for bugs it's the professional standard: first a failing test that reproduces the bug, then the fix. The test then stays in the suite as a **regression test**, so the same bug can never quietly return.

How it was built: red, green, red, green

Phase 3 was built in nine steps, and both bugs were reproduced in the real project before being fixed.

Step	Action	Result in the project
1	Wrote `tests/test_concurrency.py`, temporarily using plain `Redis.from_url`	—
2	Ran it	**Red:** 4 failed, 3 passed. `MaxConnectionsError: Too many connections`
3	Added `create_redis()` with a blocking pool; updated config, `main.py`, conftest	`git diff` showed only those changes
4	Ran it again	**Green:** 7 passed in 3.09 s
5	Appended the boundary tests to `tests/test_limiters.py`	**Red:** 23 passed, 1 failed: the counter's `retry_after=0.137` wasn't enough
6	Fixed the counter's Lua	**Green:** retry test 3/3
7	Full suite × 3, concurrency × 10	31 passed every time; 7 passed every time
8	Added `scripts/concurrent_burst.py`, ran it over HTTP	100 × `200`, 100 × `429` for every algorithm
9	README, roadmap, commit `d7d9c57`, push	In sync with GitHub

Step 2's red output, decoded:

```
test_simultaneous_requests_never_exceed_limit[token_bucket] FAILED
test_simultaneous_requests_never_exceed_limit[sliding_log] FAILED
test_simultaneous_requests_never_exceed_limit[sliding_counter] FAILED
test_limit_holds_across_separate_instances[token_bucket] PASSED
test_limit_holds_across_separate_instances[sliding_log] PASSED
test_limit_holds_across_separate_instances[sliding_counter] PASSED
test_naive_limiter_lets_too_many_through FAILED
E   redis.exceptions.MaxConnectionsError: Too many connections
```

The single-client tests crashed: 200 commands on one client exceeded its 100 connections. The four-client tests passed, because each client carried only 50. Even the naive control crashed before it could show anything.

Step 5's red output, decoded:

```
test_waiting_retry_after_is_enough[sliding_counter] FAILED
E   +  where False = Decision(allowed=False, limit=5, remaining=0, retry_after=0.137,
reset_after=0.937).allowed
```

pytest printed the whole `Decision` of the refused request. The counter had been told to wait 0.137 s, waited, and was refused again.

Code 1: The concurrency tests

Three tests in `tests/test_concurrency.py` say together: one server is safe, several servers are safe, and the test would have noticed if they weren't.

```
import asyncio

import pytest
from redis.asyncio import Redis

from app.config import settings
from app.limiter.sliding_counter import SlidingWindowCounter
from app.limiter.sliding_log import SlidingWindowLog
from app.limiter.token_bucket import TokenBucket
from app.redis_client import create_redis

LIMIT = 100
ATTEMPTS = 200

# Every algorithm allows exactly LIMIT, with nothing refilling during the test.
BUILDERS = {
    "token_bucket": lambda redis: TokenBucket(redis, capacity=LIMIT, refill_rate=0.001),
    "sliding_log": lambda redis: SlidingWindowLog(
        redis, limit=LIMIT, window_seconds=60
    ),
    "sliding_counter": lambda redis: SlidingWindowCounter(
        redis, limit=LIMIT, window_seconds=60
    ),
}


class NaiveCounter:
    """Deliberately broken limiter: it reads, decides and writes in separate steps."""

    def __init__(self, redis: Redis, limit: int) -> None:
```

```
        self.redis = redis
        self.limit = limit

    async def check(self, client_id: str) -> bool:
        key = f"rategate:naive:{client_id}"
        count = int(await self.redis.get(key) or 0)
        if count >= self.limit:
            return False
        await self.redis.set(key, count + 1, px=60_000)
        return True


@pytest.mark.parametrize("name", BUILDERS)
async def test_simultaneous_requests_never_exceed_limit(redis, client_id, name):
    limiter = BUILDERS[name](redis)
    decisions = await asyncio.gather(
        *(limiter.check(client_id) for _ in range(ATTEMPTS))
    )
    assert sum(d.allowed for d in decisions) == LIMIT


@pytest.mark.parametrize("name", BUILDERS)
async def test_limit_holds_across_separate_instances(client_id, name):
    # Four separate Redis clients and limiters, like four API servers sharing one Redis.
    clients = [create_redis(settings) for _ in range(4)]
    limiters = [BUILDERS[name](client) for client in clients]
    try:
        decisions = await asyncio.gather(
            *(limiters[i % 4].check(client_id) for i in range(ATTEMPTS))
        )
    finally:
        for client in clients:
            await client.aclose()
    assert sum(d.allowed for d in decisions) == LIMIT


async def test_naive_limiter_lets_too_many_through(redis, client_id):
    limiter = NaiveCounter(redis, LIMIT)
    results = await asyncio.gather(*(limiter.check(client_id) for _ in range(ATTEMPTS)))
    assert sum(results) > LIMIT
```

Piece by piece:

- **`LIMIT = 100`, `ATTEMPTS = 200`.** Constants are written in capitals by convention and named once. Twice the limit means exactly half must be refused, which is easy to check.
- **Nothing refills mid-test.** The token bucket gets `refill_rate=0.001`, about one token every 17 minutes, and the windows last 60 s. If tokens could drip back during the burst, 101 might be a correct answer and the test would need a vague range. Removing refill makes the expected answer **exactly** 100.
- **`NaiveCounter`.** The read (`get`), the decision (`if count >= ...`) and the write (`set`) are separate network round trips. `or 0` turns a missing key into 0. `px=60_000` sets an expiry in milliseconds; the `_` in `60_000` is just a readable thousands separator. Its `rategate:naive:` prefix keeps it apart from the real algorithms. It lives in the test file, never in `app/`, because it's a measuring tool, not a feature.
- **Single client test.** One client, 200 overlapping checks. This test exposed the pool crash.
- **Four-instance test.** It creates four independent clients, each with its own pool, imitating four servers. `limiters[i % 4]` deals requests out round robin: `%` gives the remainder, so `i % 4` cycles 0, 1, 2, 3, 0, 1..., as nginx will in Phase 5. It doesn't use the `redis` fixture because it needs four clients of its own.
- **`try` / `finally`.** The `finally` block runs even if something inside `try` fails, so the four clients are always closed and no connections leak.

- **The control test** asserts `> LIMIT`, not `== 200`. It only needs to prove the race is detectable; demanding exactly 200 would make the test fragile for no benefit.

Code 2: The connection pool fix

One new file builds every Redis client with a blocking pool, and three files changed a line or two to use it.

`app/config.py`: two new settings, under `redis_url`:

```
    redis_url: str = "redis://localhost:6379/0"
    redis_max_connections: int = 50  # connections shared by all requests
    redis_pool_timeout: float = 5.0  # seconds to wait for a free connection
```

As settings, both can be changed per environment, such as `RATEGATE_REDIS_MAX_CONNECTIONS=100`, without editing code.

`app/redis_client.py` (new):

```
from redis.asyncio import BlockingConnectionPool, Redis

from app.config import Settings


def create_redis(settings: Settings) -> Redis:
    # A blocking pool makes a burst of requests wait briefly for a free connection
    # instead of failing with "Too many connections".
    pool = BlockingConnectionPool.from_url(
        settings.redis_url,
        max_connections=settings.redis_max_connections,
        timeout=settings.redis_pool_timeout,
        decode_responses=True,
    )
    return Redis.from_pool(pool)
```

- **`BlockingConnectionPool.from_url(...)`** builds a pool that waits instead of raising. It takes the same URL as before, plus the pool size and the longest wait. `decode_responses=True` moved here from `main.py`.
- **`Redis.from_pool(pool)`** creates a client that **owns** the pool: closing the client (`await redis.aclose()`) also closes the pool and every connection in it. The alternative, `Redis(connection_pool=pool)`, would leave the pool open after the client closes.
- **It takes `Settings` as a parameter** rather than importing the global, so a test could pass different settings. That's dependency injection again.

`app/main.py`: two lines. The `from redis.asyncio import Redis` import became `from app.redis_client import create_redis`, and in `lifespan`:

```
    redis = Redis.from_url(settings.redis_url, decode_responses=True)   # before
    redis = create_redis(settings)                                      # after
```

`main.py` no longer imports anything from the redis library directly.

`tests/conftest.py`: the same two lines, so the `redis` test fixture is built exactly like the app's client:

```
@pytest.fixture
async def redis():
    client = create_redis(settings)
```

```
    yield client
    await client.aclose()
```

Now the app and every test go through one function. If Redis setup changes again, say in Phase 4 with network timeouts, it changes in one place and the tests automatically cover it.

Code 3: The boundary tests

The boundary tests were appended to `tests/test_limiters.py`, so they can reuse its `ALGORITHMS` dictionary and `make` helper (5 requests per 1-second window).

```
# Boundaries: the edges of time where off-by-one and estimation bugs hide.

# How each algorithm names its key, and how long an idle key should live.
EXPIRY = {
    "token_bucket": (
        lambda redis: TokenBucket(redis, capacity=5, refill_rate=5.0),
        "rategate:tb:",
        1.0,
    ),
    "sliding_log": (
        lambda redis: SlidingWindowLog(redis, limit=5, window_seconds=1),
        "rategate:swl:",
        1.0,
    ),
    "sliding_counter": (
        lambda redis: SlidingWindowCounter(redis, limit=5, window_seconds=1),
        "rategate:swc:",
        2.0,
    ),
}


async def sleep_until(redis, offset_ms, window_ms=1000):
    """Sleep until Redis's clock is `offset_ms` into a window."""
    seconds, micros = await redis.time()
    elapsed = (seconds * 1000 + micros // 1000) % window_ms
    await asyncio.sleep(((offset_ms - elapsed) % window_ms) / 1000)


@pytest.mark.parametrize("name", ALGORITHMS)
async def test_waiting_retry_after_is_enough(redis, client_id, name):
    limiter = make(name, redis)
    for _ in range(5):
        await limiter.check(client_id)
    blocked = await limiter.check(client_id)
    assert not blocked.allowed
    await asyncio.sleep(blocked.retry_after + 0.05)
    assert (await limiter.check(client_id)).allowed


@pytest.mark.parametrize("name", EXPIRY)
async def test_key_expires_when_idle(redis, client_id, name):
    build, prefix, ttl_seconds = EXPIRY[name]
    await build(redis).check(client_id)
    key = f"{prefix}{client_id}"
    assert 0 < await redis.pttl(key) <= ttl_seconds * 1000
    await asyncio.sleep(ttl_seconds + 0.1)
    assert await redis.exists(key) == 0
```

```
@pytest.mark.parametrize("name", ["sliding_log", "sliding_counter"])
async def test_no_double_burst_across_window_boundary(redis, client_id, name):
    limiter = make(name, redis)
    await sleep_until(redis, 900)  # 100 ms before a window boundary
    before = [(await limiter.check(client_id)).allowed for _ in range(5)]
    await sleep_until(redis, 50)  # just after it
    after = [(await limiter.check(client_id)).allowed for _ in range(5)]
    assert sum(before) + sum(after) == 5
```

Piece by piece:

- `EXPIRY` gives each algorithm settings where its key should live a known time: 1 s for the token bucket (capacity 5 ÷ refill 5 per second), 1 s for the log (one window), 2 s for the counter (two windows). It also gives each key prefix, so the test can find the key in Redis.
- `sleep_until`. `redis.time()` returns Redis's clock as seconds and microseconds; `//` is whole-number division, turning microseconds into milliseconds. `% window_ms` gives how far into the current 1-second window we are. The second `% window_ms` handles wrap-around: at 950 ms, waiting until 50 ms means 100 ms into the next window, and `(50 − 950) % 1000` is exactly 100.
- `test_waiting_retry_after_is_enough`. Five requests use up the limit; the sixth is refused and says how long to wait. The test sleeps that long plus 50 ms (a margin for scheduling noise), and the next request must be allowed. It runs for all three algorithms.
- `test_key_expires_when_idle`. `pttl` returns the milliseconds a key has left. Right after one request it must be positive and no more than the expected lifetime. After waiting that long plus 0.1 s, `exists` must return 0: the key is gone, so idle clients use no memory.
- `test_no_double_burst_across_window_boundary`. Five requests 100 ms before a boundary and five just after must total exactly 5 allowed. A plain fixed-window limiter would allow all 10. Only the log and the counter are tested, because the token bucket has no windows.
- **Margins** (`+ 0.05`, `+ 0.1`) exist because timing tests on a real machine are never exact to the millisecond. Without them, a test can fail one run in fifty for no real reason: a **flaky** test.

Code 4: The sliding counter fix

Only the block that computes `retry_after_ms` for a blocked request changed in `sliding_counter.lua`; nothing else in the script moved.

Before:

```
  local room = limit - cost - curr
  if room < 0 or prev == 0 then
    -- This window alone is full: wait for the next one.
    retry_after_ms = window_ms - elapsed
  else
    -- Wait until the previous window's share shrinks enough.
    retry_after_ms = math.ceil(window_ms * (1 - room / prev) - elapsed)
  end
```

After:

```
  local room = limit - cost - curr
  if room >= 0 then
    -- Wait until the previous window's share shrinks enough.
    retry_after_ms = math.ceil(window_ms * (1 - room / prev) - elapsed)
  else
    -- This window alone is full. In the next one its count becomes the previous
    -- count, so also wait for that share to shrink enough.
```

```
    local next_wait = 0
    if curr > 0 then
      next_wait = math.max(0, window_ms * (1 - (limit - cost) / curr))
    end
    retry_after_ms = math.ceil(window_ms - elapsed + next_wait)
  end
```

What changed and why:

- **The condition became simply `room >= 0`,** with the cases swapped. The `prev == 0` check could go because if `prev` were 0, the estimate would just be `curr`, and a blocked request means curr + cost > limit, so `room` would already be negative. Therefore `room >= 0` guarantees `prev > 0`, and dividing by `prev` is safe.
- **"This window alone is full" now adds `next_wait`:** the time into the next window for the carried-over count, weighted, to shrink enough that one more request fits. This is the formula from Theory 4.
- **`if curr > 0`** prevents a division by zero. `curr` can only be 0 here if a single request costs more than the whole limit, which never happens today, but a crash inside a Redis script would be an ugly way to find out.
- **`math.max(0, ...)`** stops the extra wait ever being negative.
- **`math.ceil`** rounds up to a whole millisecond, so the client is never told to come back a fraction too early.

Code 5: The end-to-end burst script

`scripts/concurrent_burst.py` sends a burst through the whole running system (HTTP, Uvicorn, middleware, pool, Redis) and counts the status codes. Phase 5 will reuse it against three servers behind nginx.

```
"""Fire many requests at a running RateGate server at the same moment and count the answers."""

import argparse
import asyncio
from collections import Counter
from uuid import uuid4

import httpx


async def burst(url: str, total: int, api_key: str) -> Counter:
    limits = httpx.Limits(max_connections=total)
    async with httpx.AsyncClient(limits=limits, timeout=30) as client:
        responses = await asyncio.gather(
            *(client.get(url, headers={"X-API-Key": api_key}) for _ in range(total))
        )
    return Counter(response.status_code for response in responses)


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--url", default="http://127.0.0.1:8000/api/resource")
    parser.add_argument("--total", type=int, default=200)
    parser.add_argument("--api-key", default=f"burst-{uuid4().hex[:8]}")
    args = parser.parse_args()

    counts = asyncio.run(burst(args.url, args.total, args.api_key))
    print(f"{args.total} requests at once, API key {args.api_key}:")
    for status, count in sorted(counts.items()):
```

```
        print(f"  {status}: {count}")


if __name__ == "__main__":
    main()
```

Piece by piece:

- **The docstring** (the first quoted line) describes the script. `argparse` reuses it as the `--help` text via `description=__doc__`.
- **`httpx.AsyncClient`** is an async HTTP client, the HTTP counterpart of the async Redis client. `async with` closes it automatically at the end of the block.
- **`httpx.Limits(max_connections=total)`** lets httpx open one connection per request. Its default cap is 100, which would quietly turn a 200-request burst into two smaller waves.
- **`timeout=30`** stops the script hanging forever if the server stalls.
- **`Counter`** (from Python's `collections` module) counts how often each value appears: `Counter([200, 429, 200])` gives `{200: 2, 429: 1}`.
- **`argparse`** reads command-line options. `--url`, `--total` and `--api-key` each have a default. `type=int` turns the text "200" into the number 200. `--api-key` becomes `args.api_key`: dashes turn into underscores.
- **A random default API key** (`uuid4().hex[:8]`, 8 random characters) makes every run a brand-new client, so nobody waits for old state to expire.
- **`asyncio.run(...)`** starts an event loop and runs the async function in it: the doorway from ordinary code into async code. In tests, pytest-asyncio does this for you.
- **`if __name__ == "__main__":`** runs `main()` only when the file is executed directly (`python scripts/concurrent_burst.py`), not when another file imports it.

Command reference: Phase 3

Every command run during Phase 3, what each part means, and why it was used. Commands already explained in Phase 2 are only listed where they were used differently.

Running tests

Command	What each part means	Why we used it
`uv run pytest tests/test_concurrency.py -v`	Run only one test file, one line per test	Focus on the new tests while watching red turn green
`uv run pytest tests/test_limiters.py -k retry_after_is_enough -v`	`-k`: only tests whose name contains this text	Re-run just the counter's retry test after the Lua fix
`foreach ($i in 1..3) { uv run pytest -q | Select-Object -Last 1 }`	Run the suite 3 times, keep only the summary line of each	Show the whole suite is stable
`foreach ($i in 1..10) { uv run pytest -q tests/test_concurrency.py | Select-Object -Last 1 }`	The concurrency tests 10 times	The roadmap's "passes repeatedly": timing varies, so one pass could be luck

End to end over HTTP

Command	What each part means	Why we used it
`$env:RATEGATE_CAPACITY = "100"; $env:RATEGATE_REFILL_RATE = "0.01"`	Token bucket of 100, refilling one token every 100 s	A burst result of exactly 100, not 101 from a token dripping back mid-burst
`$env:RATEGATE_LIMIT = "100"; $env:RATEGATE_WINDOW_SECONDS = "60"`	Window algorithms: 100 per 60 s	The same for the log and the counter
`uv run uvicorn app.main:app --port 8002 --log-level warning`	`--log-level warning`: print only warnings and errors	200 requests would otherwise print 200 log lines
`uv run python scripts/concurrent_burst.py`	Run the burst script with its defaults (200 requests, port 8000, random key)	The end-to-end check
`... concurrent_burst.py --url http://127.0.0.1:8002/api/resource`	Point it at another server	One server per algorithm
`Remove-Item Env:RATEGATE_LIMIT, Env:RATEGATE_WINDOW_SECONDS`	Delete several variables at once (comma-separated)	Clean up so later runs use the defaults
`Get-ChildItem Env:RATEGATE_*`	List every `RATEGATE_` variable still set in this terminal	Catch leftovers before running tests

Git, used to verify the changes

Command	What each part means	Why we used it
`git --no-pager diff -- app tests/conftest.py`	Show changes since the last commit, limited to these paths; `--no-pager` prints straight out instead of opening a scrolling viewer	Confirm the pool fix changed exactly what the guide says
`git diff --numstat -- tests/test_limiters.py`	Lines added and removed, as numbers	Prove the boundary tests were only **appended** (60 added, 0 removed)
`git add -A`, `git commit -F - <<'EOF' ... EOF`, `git push`	Stage everything, commit with a multi-line message from a Bash heredoc, upload	Record and publish Phase 3 as `d7d9c57`

Inside the test code (Redis commands sent by redis-py):

Call	Redis command	Returns
`await redis.time()`	`TIME`	Redis's clock: (seconds, microseconds)
`await redis.pttl(key)`	`PTTL`	Milliseconds until the key expires (−2 if it doesn't exist)
`await redis.exists(key)`	`EXISTS`	1 if the key exists, 0 if not
`await redis.get(key)` / `set(key, v, px=...)`	`GET` / `SET ... PX`	Used only by the naive control limiter

Verification: 5 checks, all passed

Every check passed in the real project before the commit. None of them needs waiting between runs: the tests use a random client per test, and the burst script a random API key per run.

Check 1: the full suite, 3 times

```
31 passed in 16.92s
31 passed in 16.69s
31 passed in 16.80s
```

31 tests: 16 from Phase 2, 8 boundary tests and 7 concurrency tests. Around 17 seconds, mostly deliberate sleeps in the timing tests.

Check 2: the concurrency tests, 10 times

```
7 passed in 3.36s     7 passed in 3.43s
7 passed in 3.27s     7 passed in 3.73s
7 passed in 3.20s     7 passed in 3.22s
7 passed in 3.40s     7 passed in 3.05s
7 passed in 3.22s     7 passed in 3.18s
```

Races depend on timing, which differs every run. Ten clean runs make a lucky pass very unlikely. This is the roadmap's "passes repeatedly".

Check 3: the negative control, in numbers. The naive control let 200 of 200 through on every run; the atomic token bucket let exactly 100 (table in Theory 2).

Check 4: end to end over HTTP, 200 requests at once, limit 100, one server per algorithm. The Phase 2 code was measured too, before the fix:

Code	Algorithm	Run	200 OK	429	500 error
Phase 2 (before fix)	Token bucket	1	100	0	**100**
Phase 2 (before fix)	Token bucket	2	100	23	**77**
Phase 2 (before fix)	Token bucket	3	100	23	**77**
Phase 3	Token bucket	1 and 2	100	100	0
Phase 3	Sliding log	1 and 2	100	100	0
Phase 3	Sliding counter	1 and 2	100	100	0

Before the fix, the limit was still enforced (exactly 100 allowed), but most refused requests crashed instead of being refused politely. After it, every refused request got a proper `429`.

Check 5: lint and the middleware

```
All checks passed!
17 files already formatted
(git diff of app/middleware.py: empty)
```

The linter and formatter are happy, and the middleware is still untouched since Phase 1: Phase 3 changed the infrastructure under it, not its logic.

Common pitfalls

Concurrency and timing tests fail in their own particular ways. The two marked ★ actually happened in this project.

Pitfall	Symptom	Why	Fix
★ Default connection pool under a burst	`MaxConnectionsError: Too many connections`; HTTP 500s	The default pool raises once 100 connections are busy	`BlockingConnectionPool`, built in `create_redis()`
★ Retry estimate that ignores the next window	A client obeying `Retry-After` is refused again	The full window's count carries into the next one at weight ≈ 1	Add the next window's wait (Code 4)
Testing concurrency with a loop of `await`s	Tests pass even for broken code	One-at-a-time requests never overlap	`asyncio.gather`
No negative control	A green test you can't trust	Nothing shows the test can fail	Keep `NaiveCounter` and its test
Tests build Redis differently from the app	Tests pass, production fails	Two setups drift apart	Both call `create_redis(settings)`
Refill during a burst test	101 allowed instead of 100, now and then	A token dripped back mid-burst	`refill_rate=0.001` in tests; `RATEGATE_REFILL_RATE=0.01` over HTTP
Using Python's clock for window timing	A boundary test that sometimes fails	Windows follow Redis's clock, which differs slightly from Python's	`sleep_until` reads `redis.time()`
No timing margin	Rare, random failures (a flaky test)	Scheduling and the network add milliseconds	Add `+ 0.05` or `+ 0.1` seconds
Extra clients not closed	"Unclosed connection" warnings	A failed assertion skips the cleanup line	Close them in `finally:`
httpx's default connection cap	A "200 at once" burst that's really two waves	httpx allows 100 connections by default	`httpx.Limits(max_connections=total)`
Leftover `RATEGATE_*` variables	Later runs unexpectedly use a limit of 100	`$env:` lasts until the terminal closes	`Remove-Item Env:...`, or check with `Get-ChildItem Env:RATEGATE_*`
Declaring victory after one run	A race slips through later	Timing differs every run	Repeat the concurrency tests (Check 2)
Fixing before reproducing	A "fix" that changed nothing, or the wrong thing	No proof of cause and effect	Red first, then green

Check your understanding (Phase 3)

Answer each aloud before reading the answer.

1. Why couldn't Phase 2's tests detect a race, even for a broken limiter? Each request finished before the next began, so no read ever fell between another request's read and write.

2. What does `asyncio.gather` do differently from a loop of `await`s? It starts all the coroutines at once and waits for them together, so their Redis commands overlap.

3. What is a negative control, and what did ours show? The same test run on something known to be broken, to prove the test can fail. The naive limiter let 200 of 200 through, so the test creates real overlap.

4. Before the fix, why did the single-client test crash but the four-client test pass? One client carried all 200 commands, over its 100-connection pool limit. Four clients carried 50 each, under it.

5. How did a pool problem become HTTP 500 errors? The pool raised `MaxConnectionsError`, nothing caught it, so FastAPI answered with an internal server error instead of 429.

6. Why do the app and the tests both use `create_redis`? So tests exercise exactly the Redis setup production uses. If they differ, tests can pass while production fails.

7. Limit 10, window 10 s, 10 requests 3 s into a window. What should `Retry-After` be? 8 seconds: 7 to the end of the window, plus 1 into the next, when 10 × weight falls to 9. The old answer of 7 led to a refusal.

8. Why does the concurrency test use `refill_rate=0.001`? So no token can drip back during the burst, making the correct answer exactly 100.

9. Why does `sleep_until` read Redis's clock? The Lua scripts cut windows using Redis's `TIME`. Python's clock may differ slightly, which would misplace the test around the boundary.

10. Why write the failing test before the fix? To prove the test detects the bug, and that the fix, and nothing else, made it pass.

11. Was the limit itself broken by the pool bug? No. Exactly 100 were still allowed. The bug was in how refused requests failed: crashing with 500 instead of a 429.

Phase 4: Failure handling, fail-open or fail-closed (done 2026-11-02)

When Redis is down or frozen, RateGate now gives a deliberate answer in 0.5 to 2 seconds, instead of a `500` error after up to 5 seconds. You choose the answer with one setting. Once Redis returns, limiting resumes by itself, with no restart. The work is committed as `8bd2123` and pushed.

This proves the resume line *"configurable fail-open/fail-closed handling for Redis outages"*.

Before and after, measured with the real `docker stop` and `docker pause` commands on 2026-11-02:

Redis problem	Phase 3 code	Phase 4, fail-open	Phase 4, fail-closed
Up	`200`	`200` + rate-limit headers	`200` + rate-limit headers
Stopped (`docker stop`)	`500` after up to 4.1 s	`200` in 1 to 2 s, no rate-limit headers	`503` in 1 to 2 s, `Retry-After: 5`
Hung (`docker pause`)	`500` after a 5.0 s freeze	`200` in 0.51 to 0.54 s	`503` in 0.51 to 0.54 s
Back up (`docker start` / `unpause`)	`200`	Limited normally on the very next request	Limited normally on the very next request
Log output	Full traceback per request (59 lines)	One warning line per request	One error line per request

Results:

Result	Evidence
Both modes behave as configured	Fail-open → `200`; fail-closed → `503` with `Retry-After: 5`, in tests and live
Outages are noticed quickly	Hung Redis: about 0.5 s instead of a 5 s freeze
Recovery is automatic	After `docker start`, the next request took 0.01 to 0.03 s and was limited normally
An outage doesn't use up tokens	Test sequence before, during and after an outage gives `200, 503, 503, 200, 429`
Nothing older broke	Full suite 36 passed, 3 runs out of 3; concurrency tests 10 runs out of 10

What changed in the project:

File	Change
`tests/test_fail_modes.py`	New: 5 tests, written first and seen failing
`app/config.py`	A `FailMode` type and 3 settings: connect timeout, command timeout, fail mode
`app/redis_client.py`	The timeouts and one immediate retry, passed to every connection
`app/middleware.py`	Catches Redis errors and applies the fail mode. This is its first change since Phase 1
`app/main.py`	One line: hands the fail mode to the middleware
`README.md`, `ROADMAP.md`	Failure handling section, 3 settings, Phase 4 ticked

In total, 31 lines were added to the app code and 1 replaced. The step-by-step guide is in the project at `docs/PHASE_4_GUIDE.md`.

Theory 1: Three ways Redis can fail

Redis can be down, hung or freshly restarted. Each looks different to the app and needs its own defence, so Phase 4 has three parts, not one.

Why this matters at all. Every request asks Redis "may this client go ahead?" before it reaches the route. So Redis is a **dependency** of the whole API: if it breaks and the app has no plan, every endpoint breaks with it. The rate limiter exists to protect the API, yet it could become the thing that takes the API down. Phase 4 removes that risk.

Failure	Real-world cause	What the app experiences	How we simulate it	What it needs
Down	Redis crashed or is being restarted; a firewall refuses the connection	Opening a connection fails	`docker stop redis`	Notice fast and apply a policy
Hung	Redis overloaded; a network fault; a frozen machine	Connected, but the reply never comes	`docker pause redis`	A limit on how long to wait for a reply
Restarted	Redis came back after being down	Connections in the pool are dead ("stale") until replaced	`docker restart redis`	Replace dead connections without the user noticing

The hung case is the dangerous one. A refused connection is at least an answer. A frozen Redis gives no answer at all, so without a limit the request just waits. The Phase 3 code waited 5 seconds per request, the library's default, then crashed. Imagine 1,000 users each waiting 5 seconds: the server fills up with stuck requests.

`docker pause` vs `docker stop`. `stop` shuts the container down, so the port closes and connections are refused. `pause` freezes every process inside the container without closing anything. The port stays open, connections stay open, but nothing answers. It is the closest simple imitation of an overloaded or unreachable server.

What a "stale" connection is. The pool from Phase 3 keeps connections open and reuses them. When Redis restarts, its end of each connection is gone, but the app's end still looks open. The app only finds out when it sends a command and gets "Connection closed by server". Nothing is wrong with Redis by then; the app is just holding an old line.

Theory 2: Fail-open vs fail-closed

When the limiter can't check a request, there are only two honest answers: let it through unchecked (**fail-open**) or refuse it (**fail-closed**). RateGate makes this a setting, `RATEGATE_FAIL_MODE`, with `open` as the default.

The names come from electrical engineering and physical security. A door that **fails open** unlocks when the power goes, so people can get out. A door that **fails closed** stays locked, so nobody can get in. Neither is better in general; it depends on what you are protecting.

Mode	What happens during an outage	Protects	Risks	Typical use
Fail-open	Request goes through, unchecked; a warning is logged	**Availability**: the API keeps working	No rate limiting until Redis returns	Public APIs, websites, read-only endpoints
Fail-closed	`503` with `Retry-After: 5`; an error is logged	**Protection**: no unlimited traffic, ever	The API is down for as long as Redis is	Login, password reset, payments, expensive endpoints

How to choose. Ask: what is worse for this API, a few minutes without limits or a few minutes without service?

- A news site: a few minutes without limits costs almost nothing, while going down loses readers. Fail-open.
- A login endpoint: limits are what stop password-guessing attacks. An attacker who can knock Redis over would get unlimited guesses under fail-open. Fail-closed.

The default is `open` because, for most APIs, a broken limiter should not take down the service it protects. Real systems often mix the two per endpoint; Real-world context below covers that.

Why `503` and not `429`? The status code tells the client *whose* problem it is.

- `429 Too Many Requests` means "**you** are over your limit". That would be false: the client did nothing wrong.
- `503 Service Unavailable` means "**the server** can't handle this right now". That is the truth.

This matters in practice. A well-written client that gets `429` slows itself down, thinking it was greedy. Monitoring tools count `5xx` codes as server faults and alert the on-call engineer, which is exactly what should happen when Redis is down. `Retry-After: 5` asks clients to wait 5 seconds rather than retry instantly and pile more load on a struggling system.

Why fail-open sends no `X-RateLimit-*` headers. Those headers report the client's real limit and remaining allowance. During an outage the app doesn't know them. Sending a guess (say `Remaining: 10`) would mislead a client into planning around numbers that aren't true. Sending nothing is honest: "no information right now".

Theory 3: Two timeouts, chosen by measurement

RateGate waits at most **1.0 s to open** a Redis connection and at most **0.25 s for a reply** to a command. The two values differ because the two waits differ by about 500 times. Both were picked from measurements on this machine, after a first guess failed (see What went wrong, Mistake 1).

What a timeout is. The longest you will wait for something before giving up. Giving up is not a failure of the code; it is the point. Without a timeout a frozen Redis would hold every request forever. With one, the wait ends in a `TimeoutError` after a known time, and the middleware turns that error into the fail-mode answer.

A Redis connection has two separate waits:

- **Connect timeout** (`socket_connect_timeout`): opening a new connection. This involves the network handshake, Docker's port forwarding, and Redis accepting the connection.
- **Socket timeout** (`socket_timeout`): waiting for the reply to one command on a connection that is already open. A "socket" is the operating system's name for one end of a network connection.

What they measured (a timing script, run while Redis was healthy):

What was timed	Median	Slowest
1 new connection	17 ms	18 ms
50 new connections at once	about 110 ms	about 155 ms
200 new connections at once	about 430 ms	**about 600 ms**
One command on an open connection	0.4 ms	1.25 ms

Reading the table. Commands are fast and steady: even the slowest took 1.25 ms. Connections are slow and get much slower when many open at once, because Redis accepts new connections one at a time and each also passes through Docker's network layer. So:

- **Command timeout 0.25 s** is about 200 times the slowest command. A healthy Redis never comes close, yet a frozen one is noticed within a quarter of a second.
- **Connect timeout 1.0 s** leaves room above the 600 ms worst case. It looks generous, but it rarely matters: the pool reuses connections, so new ones are opened only at startup and after a restart.

Worst case per request = 2 × the timeout, because of the one retry (Theory 4). That is why a hung Redis is detected in about 0.5 s (2 × 0.25 s). (Since Phase 5, timeouts are no longer retried, so it is one timeout: about 0.25 s hung, 1.0 s stopped.)

The Windows detail. When nothing listens on a port, Linux refuses a connection almost instantly. Windows instead retries the connection quietly for about **2 seconds** before reporting "refused". So on your machine, while Redis is stopped, it is the 1 s connect timeout that ends each attempt, giving 1 to 2 s per request. That is why the stopped case is slower than the hung case on Windows, the opposite of what you might expect. On Linux, which Phase 5's containers run, a stopped Redis should be noticed much faster; Phase 5 will show whether it is. (Corrected in Phase 5: it isn't. Inside Docker a stopped container's address disappears, so nothing refuses and the connect timeout ends the wait, as on Windows. See Phase 5, What went differently, problem 8.)

For reference: a plain redis-py client with every setting left at its default took **27 seconds** to give up on one command against a closed port. The Phase 3 pool already avoided most of that; Phase 4 sets the values on purpose.

Both values are settings (`RATEGATE_REDIS_CONNECT_TIMEOUT`, `RATEGATE_REDIS_SOCKET_TIMEOUT`), because the right numbers depend on where Redis runs. A Redis in another data centre, 40 ms away, would need more than 0.25 s.

Theory 4: Exactly one retry, immediately

A failed Redis command is tried once more, straight away, and then the app gives up. Zero retries would break on every Redis restart; more retries, with pauses, would make every outage painfully slow.

What a retry is. Sending the same command again after it failed, in case the failure was a brief blip. The usual pattern is **exponential backoff**: retry several times, waiting longer before each attempt (for example 0.1 s, then 0.2 s, then 0.4 s). That is good for a busy service that needs a moment to recover.

Why many retries are wrong here. During a real outage, every retry is just more waiting before the inevitable failure, and the user is stuck the whole time. With redis-py's defaults we measured **27 seconds** for one request to fail against a closed port. Nobody waits 27 seconds for a web page.

Why zero retries is also wrong. After `docker restart redis`, the pool still holds the old, dead connections (Theory 1). The first command on each fails with "Connection closed by server", although Redis is healthy again. With no retry, the middleware would treat that as an outage and answer `503` (fail-closed) or skip

limiting (fail-open), for no real reason.

One immediate retry fixes both. When a command fails on a stale connection, redis-py drops that connection, opens a fresh one, and sends the command again. Measured right after `docker restart redis`: the first request took **0.01 s and was allowed**. The user never notices. During a real outage, the retry fails too, so the cost is at most one extra timeout.

Retry policy	After a Redis restart	During a real outage
None	False outage on the first request per connection	Fastest failure
One, immediately (RateGate)	Invisible: 0.01 s, allowed	2 × the timeout (about 0.5 s hung, 1 to 2 s stopped)
Library default, with backoff	Fine	Up to 27 s per request

In code this is `Retry(NoBackoff(), retries=1)`: redis-py's own `Retry` class, with a "backoff" of zero seconds between attempts.

> **Corrected in Phase 5.** This retry had a flaw. redis-py's `Retry` reacts to timeouts as well as connection errors, and after a timeout the script may already have run, so retrying it counted one request twice (20 late replies left 40 entries in Redis). Since Phase 5 only connection errors are retried: `Retry(NoBackoff(), retries=1, supported_errors=(RedisConnectionError,))`. The stale-connection case above still works, and outages are noticed faster. The full story is in Phase 5, What went differently, part 2.

Theory 5: Where the failure is caught, and what gets logged

The middleware catches every Redis error, and only Redis errors, then applies the fail mode and writes one log line. The algorithms don't change at all.

Exceptions, briefly. When Python code hits a problem it can't handle, it **raises an exception**: an object describing what went wrong. The exception travels up through every function that called the failing one, until some code **catches** it with `try` / `except`. If nothing catches it, the web framework does, and answers `500 Internal Server Error`. That is exactly what happened in Phase 3.

The Redis exception family. redis-py's exceptions form a tree. Catching a parent also catches all its children:

```
RedisError
├── ConnectionError          can't connect, or the connection was closed
│   └── MaxConnectionsError  the pool problem from Phase 3
└── TimeoutError             connecting or the reply took too long
```

So `except RedisError` covers every way Redis can fail, in one line.

Why not catch `Exception`? `Exception` is the parent of almost every error in Python, including bugs in our own code: a typo, a `TypeError`, a missing key. Catching it would silently hide those bugs behind fail-open: the API would "work", with no limiting, and nobody would know why. A bug should still crash loudly as a `500`, so it gets noticed and fixed. Catch what you expect; let the rest surface.

Why the middleware is the right place. The decision is about the **HTTP response**: pass the request on, or answer `503`. Turning limiter results into HTTP is the middleware's job already: it makes the `429`. The algorithms (token bucket, sliding log, sliding counter) shouldn't know about HTTP, and the strategy pattern from Phase 2 keeps them that way. So all three get failure handling from one change in one file.

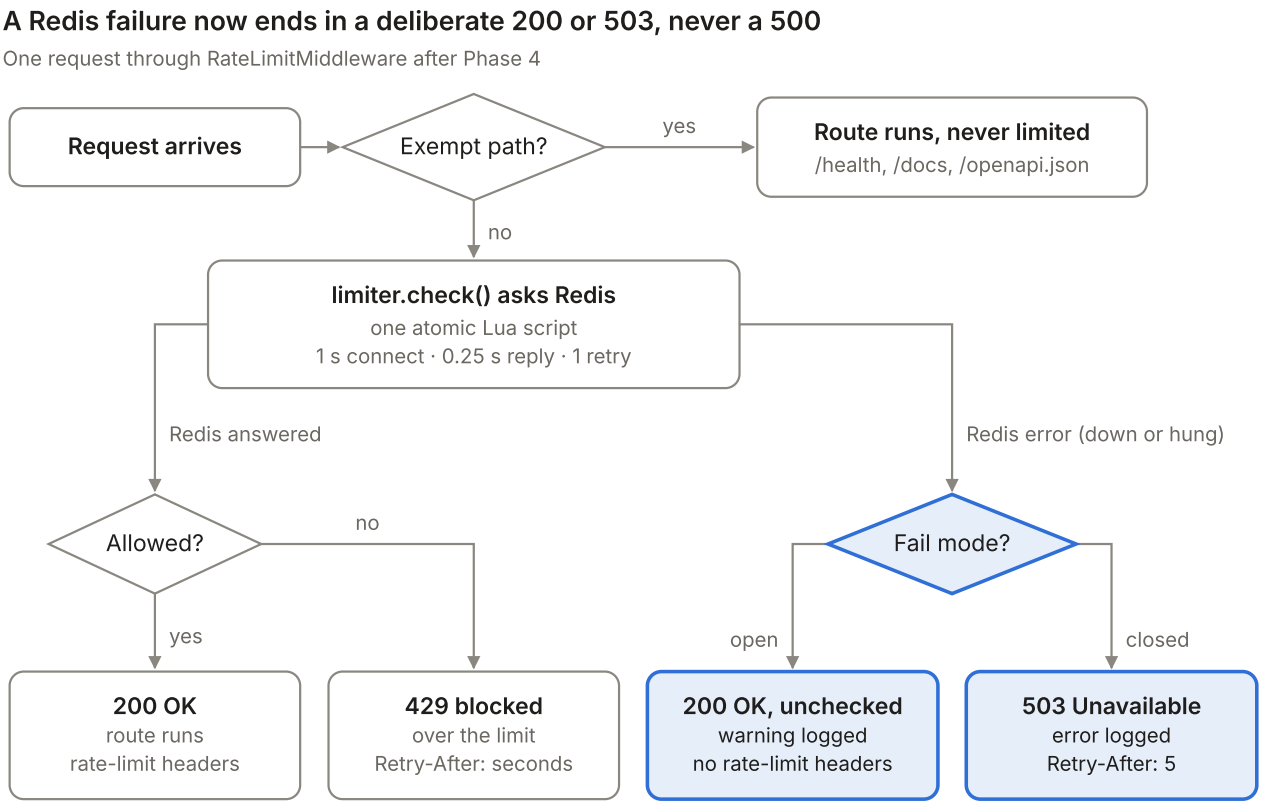
This is the first change to `middleware.py` since Phase 1. Phase 2 deliberately left it untouched because *algorithms* were changing. Phase 4 changes *HTTP behaviour*, which belongs exactly there.

Logging. When the limiter is bypassed or refusing users, the person running the service must find out. Python's built-in `logging` module writes timestamped lines to the server's output:

- `logging.getLogger("rategate")` gets a logger named after our app, so its lines can be filtered from other libraries' lines.
- **Levels** say how serious a line is. Fail-open logs `warning`: degraded, but users are served. Fail-closed logs `error`: users are being refused.
- `logger.warning("... %s", error)` fills `%s` with the error text only if the line is actually written. That is cheaper than building an f-string that might be thrown away.

The Phase 3 code printed a full traceback for every failed request: 59 lines for a handful of requests, mostly library internals. Phase 4 prints one readable line each, naming the cause, such as `Timeout connecting to server` or `Connection closed by server.`

A known limit, and what comes next. During a long outage, *every* request still pays the timeout (1 to 2 s while stopped, on Windows). Large systems add a **circuit breaker**: after several failures in a row, stop calling Redis for a few seconds and apply the fail mode instantly, then try Redis again. It works like a household fuse that trips and is later reset. It isn't built here; it's a natural Phase 8 extension and a good answer to "what would you add next?"



One request through the middleware · Phase 4

When Redis answers, nothing changed since Phase 1: `200` with headers, or `429`. When the call raises a Redis error, even after its one retry, the fail mode decides between `200` unchecked and `503`.

What went wrong while building it

Three mistakes happened while Phase 4 was built in a scratch copy. All three were caught before anything reached the project, and none was found by thinking harder: each was found by running or reading something real.

	What was assumed	What showed otherwise	Lesson
1	A 100 ms timeout is plenty	Running the **full** test suite, then measuring	Measure before choosing a number
2	The new test checks the new feature	Running the new tests on the **old** code	A test proves something only if it can fail
3	`with_retry()` exists	Reading the library's **source**	Check an API exists before relying on it

Mistake 1: one timeout, set too low

Context. Phase 4 needs timeouts so a broken Redis is noticed quickly (Theory 3). The roadmap suggested 50 to 100 ms.

What the code should do. Give up on Redis only when Redis is really in trouble. A healthy Redis, even a busy one, must never trip a timeout.

What was expected. The first attempt set **0.1 s for both** the connect and the command timeout. A command takes about 1 ms, so 100 ms looked like a huge safety margin, and it would make outages show up very fast.

What actually happened. A hung Redis was detected quickly, as intended. But running the **whole** suite, not only the new tests, broke Phase 3's tests, while Redis was perfectly healthy:

```
redis.exceptions.TimeoutError: Timeout connecting to server
FAILED tests/test_concurrency.py::test_simultaneous_requests_never_exceed_limit[token_bucket]
FAILED tests/test_concurrency.py::test_simultaneous_requests_never_exceed_limit[sliding_log]
FAILED tests/test_concurrency.py::test_simultaneous_requests_never_exceed_limit[sliding_counter]
...
======================== 5 failed, 26 passed ========================
```

How it was found. The error said *connecting*, not *reading*. So commands were fine; opening connections was slow. A small timing script then measured both waits separately (the table in Theory 3): one connection 17 ms, but 200 at once up to **600 ms**, while commands never exceeded 1.25 ms.

Root cause. The concurrency tests fire 200 requests at once into an empty pool, so many connections open together. Redis accepts them one at a time, and each also goes through Docker's network layer, so the later ones wait in line. A 100 ms limit cut those off and reported them as an outage. The mistake was treating "connect" and "command" as the same kind of wait, when they differ by about 500 times.

Why it mattered beyond the tests. In production this happens right after every deploy or restart: the pool is empty, a traffic spike arrives, connections are slow. The app would declare Redis "down" while it was healthy. Fail-open would let everyone through unlimited; fail-closed would refuse everyone with `503`. A false alarm, caused by the very feature built to handle real alarms.

The fix. Two separate values from the measurements: 0.25 s per command and 1.0 s to connect. One knock-on effect: a new test had hard-coded "an outage must be noticed in under 1 s", which failed once the connect timeout became 1 s. It now computes its limit from the settings, `2 × connect timeout + 0.5`, so a changed setting can't silently make the test wrong.

Mistake 2: a test that passed for the wrong reason

Context. `RATEGATE_FAIL_MODE` must accept only `open` or `closed`. A typo such as `closd` should stop the app at startup rather than run in an unknown mode. This uses `FailMode = Literal["open", "closed"]`, the same technique Phase 2 used for `Algorithm`.

What the test should do. Fail if the app accepts a bad value; pass if it refuses one. The first version was:

```
def test_fail_mode_accepts_only_open_or_closed():
    with pytest.raises(ValidationError):
        Settings(fail_mode="maybe")
```

In words: building the settings with `fail_mode='maybe'` must raise a `ValidationError`.

What was expected. Every new test is run on the old code first, the red step from Phase 3. The Phase 3 code had no fail mode at all, so all 5 new tests should fail.

What actually happened. 4 failed and **this one passed**.

How it was found. Only because the red step was actually run and its result read. A test that passes before its feature exists is a warning sign: either the feature already exists, or the test isn't checking what you think.

Root cause. The same calls, run against both versions of `config.py`:

```
=== OLD config (Phase 3, no fail mode)
fail_mode field exists? False
Settings(fail_mode='maybe')  -> REJECTED: extra_forbidden: Extra inputs are not permitted
Settings(fail_mode='closed') -> REJECTED: extra_forbidden: Extra inputs are not permitted

=== NEW config (Phase 4)
fail_mode field exists? True
Settings(fail_mode='maybe')  -> REJECTED: literal_error: Input should be 'open' or 'closed'
Settings(fail_mode='closed') -> accepted, fail_mode=closed
```

pydantic-settings refuses **any setting name it doesn't recognise**. The old `Settings` had no field called `fail_mode`, so it rejected the whole thing with "I don't know this setting" (`extra_forbidden`). It never looked at whether `"maybe"` was a valid value. The test only asked "was there *some* error?", so the wrong error counted as a pass. It would have stayed green even if the feature were never built, or built wrongly.

The fix. Add the other half: a valid value must be **accepted**.

```
def test_fail_mode_accepts_only_open_or_closed():
    assert Settings(fail_mode="closed").fail_mode == "closed"
    with pytest.raises(ValidationError):
        Settings(fail_mode="maybe")
```

On the old code the first line now fails, because "closed" is rejected too. On the new code both lines pass. The test can now tell the two versions apart, which is the whole point of a test. Tests of the form "expect an error" are especially easy to get wrong, because any error satisfies them.

Mistake 3: a method that doesn't exist

Context. After a Redis restart, pooled connections are stale (Theory 1), and one immediate retry replaces them (Theory 4).

What was written first, from memory, because some libraries work this way:

```
return Redis.from_pool(pool).with_retry(Retry(NoBackoff(), retries=1))
```

How it was found. Before relying on it, redis-py's installed source code was searched for `retry`. There was no `with_retry` method. Instead the search showed `conn.retry.call_with_retry(...)`: each **connection** carries its own retry rule, set with a `retry=` argument when the connection is created. Running the original line confirms it:

```
has with_retry? False
AttributeError: 'Redis' object has no attribute 'with_retry'
Connection accepts retry= ? True
```

What would have happened. `create_redis()` runs when the app starts and in every test's `redis` fixture. The app would have crashed on startup and every test would have errored. At least it would have been loud rather than silent.

The fix. Pass the rule to the pool, which hands it to every connection it creates: `retry=Retry(NoBackoff(), retries=1)` inside `BlockingConnectionPool.from_url(...)`. Then confirm it does its job: right after `docker restart redis`, the first request took 0.01 s and was allowed.

The common thread. Each assumption was reasonable, and each was wrong. This is why every guide makes you watch tests fail first, run the whole suite several times, and do the live `docker stop` and `docker pause` checks by hand. These also make good interview stories: "tell me about a bug you found" (Mistake 1

shows you measure) and "how do you know your tests work?" (Mistake 2).

How it was built

Phase 4 followed the same pattern as Phase 3: see the problem, write the tests and watch them fail, make the smallest change, then run everything. The order below matters, and each step says why it comes where it does.

Step	What	Why at this point
1	See the problem live: stop and pause Redis under the Phase 3 code	Know exactly what you are fixing, and have "before" numbers to compare against
2	Write `tests/test_fail_modes.py` and run it: **red**	Proves the tests can detect today's behaviour
3	`config.py`: fail mode and two timeouts	Every later step reads these settings
4	`redis_client.py`: pass the timeouts and one retry to every connection	Detect failure fast; recover from restarts
5	`middleware.py`: catch Redis errors, apply the fail mode	Decides the HTTP answer
6	`main.py`: one line, hand the fail mode to the middleware	The middleware reads it from app state
7	Run the new tests (**green**), then the whole suite 3 times and the concurrency tests 10 times	Confirms the fix, and that the new timeouts didn't break Phase 3
8	README and ROADMAP	Record what was built

Step 1's result, Phase 3 code, measured with `curl`: `200` while Redis was up; `500` after up to 4.1 s while stopped; `500` after a 5.0 s freeze while paused. The server's terminal filled with tracebacks.

Step 2's result, the new tests on the Phase 3 code:

```
test_fail_open_allows_requests_when_redis_is_down FAILED
test_fail_closed_rejects_requests_when_redis_is_down FAILED
test_outage_is_detected_within_the_timeouts FAILED
test_limiting_resumes_after_an_outage FAILED
test_fail_mode_accepts_only_open_or_closed FAILED
```

Each outage test took about 2 s and crashed with the Redis error: the same `500` as Step 1, seen from inside a test. The last test fails only since Mistake 2 was fixed; its first version passed here, which is how that mistake was caught.

Step 7's result: 5 passed; full suite `36 passed` in about 29 s, three runs in a row; concurrency tests `7 passed`, ten runs in a row. Most of those 29 s come from the outage tests, which wait on purpose: each waits out the 1 s connect timeout twice against the closed port.

Code 1: The tests (`tests/test_fail_modes.py`)

Five tests send real HTTP requests through the real app and middleware, with Redis made unreachable on purpose. They were written first and seen failing (How it was built).

```
import time

import httpx
import pytest
from pydantic import ValidationError
```

```
from app.config import Settings, settings
from app.limiter.token_bucket import TokenBucket
from app.main import app
from app.redis_client import create_redis

# Nothing listens on this port, so every Redis command fails like during an outage.
DOWN_URL = "redis://127.0.0.1:6399/0"


@pytest.fixture
async def redis_down():
    client = create_redis(Settings(redis_url=DOWN_URL))
    yield client
    await client.aclose()


async def call_api(limiter, fail_mode, api_key, times=1):
    """Send requests through the real app and middleware, using the given limiter."""
    app.state.limiter = limiter
    app.state.fail_mode = fail_mode
    transport = httpx.ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport, base_url="http://test") as client:
        headers = {"X-API-Key": api_key}
        return [
            await client.get("/api/resource", headers=headers) for _ in range(times)
        ]


async def test_fail_open_allows_requests_when_redis_is_down(redis_down, client_id):
    [response] = await call_api(TokenBucket(redis_down, 10, 1.0), "open", client_id)
    assert response.status_code == 200
    assert "x-ratelimit-limit" not in response.headers


async def test_fail_closed_rejects_requests_when_redis_is_down(redis_down, client_id):
    [response] = await call_api(TokenBucket(redis_down, 10, 1.0), "closed", client_id)
    assert response.status_code == 503
    assert response.headers["retry-after"] == "5"


async def test_outage_is_detected_within_the_timeouts(redis_down, client_id):
    # Worst case: the attempt and its one retry each wait the full connect timeout.
    budget = 2 * settings.redis_connect_timeout + 0.5
    start = time.perf_counter()
    await call_api(TokenBucket(redis_down, 10, 1.0), "open", client_id)
    assert time.perf_counter() - start < budget


async def test_limiting_resumes_after_an_outage(redis, redis_down, client_id):
    healthy = TokenBucket(redis, capacity=2, refill_rate=0.001)
    broken = TokenBucket(redis_down, capacity=2, refill_rate=0.001)
    before = await call_api(healthy, "closed", client_id)
    during = await call_api(broken, "closed", client_id, times=2)
    after = await call_api(healthy, "closed", client_id, times=2)
    codes = [response.status_code for response in before + during + after]
    assert codes == [200, 503, 503, 200, 429]


def test_fail_mode_accepts_only_open_or_closed():
    assert Settings(fail_mode="closed").fail_mode == "closed"
```

```
    with pytest.raises(ValidationError):
        Settings(fail_mode="maybe")
```

Piece by piece:

- **Imports.** `time` for the stopwatch; `httpx` to send HTTP requests; `ValidationError` is what pydantic raises for a bad setting. Note both `Settings` (the class, to build custom settings) and `settings` (the app's actual settings object, to read the real timeout).
- **`DOWN_URL`.** Port 6399 has nothing listening. A client pointed at it fails exactly as if Redis were down, without stopping the real Redis. Stopping Docker inside tests would be slow, would break tests running alongside, and wouldn't work in CI (Phase 6).
- **`redis_down` fixture.** Builds that broken client with the app's own `create_redis`, so it has the same timeouts and retry as production. `Settings(redis_url=DOWN_URL)` makes a settings object with just that one value changed; everything else keeps its default. After the test, `aclose()` closes the client.
- **`call_api` helper.** The test's way into the app. It puts the chosen limiter and fail mode into `app.state`, exactly where `main.py` normally puts them. `httpx.ASGITransport(app=app)` connects httpx straight to the FastAPI app in memory: no server, no network, but the request still goes through the real middleware. `base_url="http://test"` is required by httpx but goes nowhere. It returns a list of responses, one per request.
- **`[response] = await call_api(...)`.** Unpacks a one-item list into one variable. It also quietly checks there is exactly one item; two would raise an error.
- **Test 1, fail-open.** Redis down, mode open: the answer must be `200`, and there must be no `x-ratelimit-limit` header, because the app has no real numbers (Theory 2). httpx lowercases header names, hence the lowercase.
- **Test 2, fail-closed.** Redis down, mode closed: `503`, with `retry-after` exactly `"5"` (a string, because headers are text).
- **Test 3, speed.** `time.perf_counter()` is a precise stopwatch; the difference between two readings is the seconds elapsed. The limit, `budget`, is calculated from the real setting: the attempt plus its one retry, each up to the 1 s connect timeout, plus 0.5 s spare. If someone changes the timeout, the test follows (Mistake 1).
- **Test 4, recovery.** The most important one. Two limiters share one client id and a 2-token bucket: one on the healthy Redis, one on the dead port. Refill is nearly zero (`0.001`), so no tokens come back during the test. The sequence: one request **before** the outage (`200`, uses token 1); two **during** it (`503`, `503`); two **after** (`200` uses token 2, then `429` because the bucket is empty). This proves the outage used no tokens, and limiting picked up exactly where it left off.
- **Test 5, validation.** "closed" must be accepted and "maybe" refused. The first line is what makes this test fail on the old code (Mistake 2).
- **`client_id` and `redis` fixtures** come from `tests/conftest.py` (Phase 2 and 3): a fresh random id per test, and a healthy Redis client.

Code 2: The settings and the Redis client

Four lines in `config.py` define the new settings; three arguments in `redis_client.py` hand them to every Redis connection.

`app/config.py`: a type and three settings. Next to `Algorithm`:

```
Algorithm = Literal["token_bucket", "sliding_log", "sliding_counter"]
FailMode = Literal["open", "closed"]
```

and after `redis_pool_timeout`:

```
    redis_pool_timeout: float = 5.0  # seconds to wait for a free connection
    redis_connect_timeout: float = 1.0  # seconds to open a connection
    redis_socket_timeout: float = 0.25  # seconds to wait for a command's reply
    fail_mode: FailMode = "open"  # what to do when Redis can't be reached
    algorithm: Algorithm = "token_bucket"
```

- **`FailMode = Literal["open", "closed"]`** is a **type alias**: a name for "exactly one of these two strings". pydantic checks it when settings are loaded, so `RATEGATE_FAIL_MODE=closd` stops the app at startup with a clear error, instead of quietly running as something else. Failing loudly at startup is far better than failing quietly during an outage, which is exactly when you'd discover the typo.
- **The two timeouts** are `float`s (numbers with decimals) in seconds, with the measured values as defaults (Theory 3). Each becomes an environment variable automatically, thanks to the `RATEGATE_` prefix: `RATEGATE_REDIS_CONNECT_TIMEOUT`, `RATEGATE_REDIS_SOCKET_TIMEOUT`.
- **`fail_mode` defaults to `"open"`** (Theory 2).
- **Order in the file** groups the Redis settings together, before the algorithm settings. Python doesn't care about the order; readers do.

`app/redis_client.py`: the whole file now.

```
from redis.asyncio import BlockingConnectionPool, Redis
from redis.asyncio.retry import Retry
from redis.backoff import NoBackoff

from app.config import Settings


def create_redis(settings: Settings) -> Redis:
    # A blocking pool makes a burst of requests wait briefly for a free connection
    # instead of failing with "Too many connections".
    pool = BlockingConnectionPool.from_url(
        settings.redis_url,
        max_connections=settings.redis_max_connections,
        timeout=settings.redis_pool_timeout,
        socket_connect_timeout=settings.redis_connect_timeout,
        socket_timeout=settings.redis_socket_timeout,
        # One immediate retry replaces a connection that went stale (e.g. Redis
        # restarted); more retries would only delay noticing a real outage.
        retry=Retry(NoBackoff(), retries=1),
        decode_responses=True,
    )
    return Redis.from_pool(pool)
```

- **Two new imports.** `Retry` is redis-py's retry policy for its async client; `NoBackoff` is the policy "wait 0 seconds between attempts". Neither is our code.
- **Three new arguments.** `from_url` passes on any argument the pool itself doesn't use to **every connection** the pool creates. So:
  - `socket_connect_timeout` becomes each connection's connect timeout (1.0 s).
  - `socket_timeout` becomes each connection's reply timeout (0.25 s).
  - `retry=Retry(NoBackoff(), retries=1)` gives each connection one immediate retry. This is the correct form of what Mistake 3 first got wrong.

- **Three different timeouts now exist; don't mix them up.** `timeout` (5 s, from Phase 3) is how long a request waits for a *free connection from the pool* when all 50 are busy. `socket_connect_timeout` is for *opening* a connection. `socket_timeout` is for *a reply*. They protect against different things: a busy pool, an unreachable Redis, a frozen Redis.
- **Why it all lives here.** Phase 3 made `create_redis` the one place Redis clients are built, used by the app and by the tests. That paid off: this change reached both at once, and the `redis_down` test fixture automatically got the production timeouts.

Code 3: The middleware and `main.py`

One line of the middleware became a `try` / `except` block that applies the fail mode; `main.py` gained one line. The rest of the middleware is unchanged since Phase 1.

`app/middleware.py`, the whole file now:

```
import logging
import math

from redis.exceptions import RedisError
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse

EXEMPT_PATHS = {"/health", "/docs", "/openapi.json"}
OUTAGE_RETRY_AFTER = "5"  # seconds a client should wait while Redis is unreachable

logger = logging.getLogger("rategate")


def identify_client(request: Request) -> str:
    api_key = request.headers.get("X-API-Key")
    if api_key:
        return f"key:{api_key}"
    host = request.client.host if request.client else "unknown"
    return f"ip:{host}"


class RateLimitMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        if request.url.path in EXEMPT_PATHS:
            return await call_next(request)

        limiter = request.app.state.limiter
        try:
            decision = await limiter.check(identify_client(request))
        except RedisError as error:
            if request.app.state.fail_mode == "closed":
                logger.error(
                    "Redis unavailable, rejecting request (fail-closed): %s", error
                )
                return JSONResponse(
                    status_code=503,
                    content={"detail": "Rate limiter unavailable. Try again shortly."},
                    headers={"Retry-After": OUTAGE_RETRY_AFTER},
                )
            logger.warning("Redis unavailable, allowing request (fail-open): %s", error)
            return await call_next(request)

        headers = {
            "X-RateLimit-Limit": str(decision.limit),
```

```
            "X-RateLimit-Remaining": str(decision.remaining),
            "X-RateLimit-Reset": str(math.ceil(decision.reset_after)),
        }

        if not decision.allowed:
            headers["Retry-After"] = str(max(1, math.ceil(decision.retry_after)))
            return JSONResponse(
                status_code=429,
                content={"detail": "Too many requests. Slow down."},
                headers=headers,
            )

        response = await call_next(request)
        response.headers.update(headers)
        return response
```

What's new, top to bottom:

- `import logging` brings in Python's built-in logging module (Theory 5).
- `from redis.exceptions import RedisError` imports the parent of every Redis error, so the `except` can name it.
- `OUTAGE_RETRY_AFTER = "5"` is a **constant**: a value set once, named in capitals by convention so readers know not to change it at runtime. It is a string because header values are text. Naming it means the number isn't buried inside the code, and the comment beside it says what it means.
- `logger = logging.getLogger("rategate")` creates our named logger once, when the module loads, rather than on every request.
- `limiter = request.app.state.limiter` was previously part of the one line that called `check`. It moved out because the `try` should cover only the Redis call, nothing else.
- `try:` means "run this, but watch for errors". Only one line sits inside it: the call that talks to Redis. Keeping the `try` that small means a bug anywhere else can't be mistaken for an outage.
- `except RedisError as error:` runs only if that call raised a Redis error. `as error` keeps the error object so it can be logged.
- `if request.app.state.fail_mode == "closed":` reads the mode from app state, where `main.py` put it. Reading it from state rather than importing `settings` directly lets each test choose its own mode per request.
- **The fail-closed branch** logs at `error` level, then returns a `JSONResponse` straight away: status `503`, a short JSON body explaining why, and the `Retry-After` header. `return` ends `dispatch` here, so the route never runs.
- **The fail-open branch** logs at `warning` level, then `return await call_next(request)`: the request goes to the route as normal, and its response is returned untouched, with no `X-RateLimit-*` headers.
- **Everything after the `except`** is unchanged from Phase 1: build the headers, answer `429` if blocked, otherwise run the route and attach the headers. It only runs when Redis answered.

`app/main.py`, one new line in `lifespan`, after the limiter is created:

```
    app.state.limiter = create_limiter(redis, settings)
    app.state.fail_mode = settings.fail_mode
    yield
```

`app.state` is FastAPI's shared storage for objects the whole app needs. The lifespan fills it once at startup; the middleware reads it on every request. The fail mode joins the limiter there for the same reason the limiter is there: the middleware gets everything it needs from one place, and tests can swap either one.

Command reference

Every command used in Phase 4, what it does and why it was used. They run in PowerShell from `D:\RateGate` unless noted.

Simulating outages (Docker)

Command	What it does	Why in Phase 4
`docker stop redis`	Shuts the Redis container down; its port stops accepting connections	Simulates Redis being **down**
`docker start redis`	Starts the stopped container again, with the same settings	Checks recovery without restarting RateGate
`docker pause redis`	Freezes every process in the container; connections stay open but nothing answers	Simulates a **hung** Redis
`docker unpause redis`	Unfreezes it	Checks recovery from a hang
`docker restart redis`	Stop then start in one command	Creates **stale** pooled connections, to test the retry
`docker exec redis redis-cli ping`	Runs `redis-cli` inside the container and sends `PING`	`PONG` proves Redis is healthy again. Always finish with this

Running two servers with different settings

Command	What it does	Why
`$env:RATEGATE_FAIL_MODE = "open"`	Sets an environment variable in **this terminal only**, until it closes	pydantic-settings reads it as `fail_mode`
`uv run uvicorn app.main:app --port 8030`	Starts RateGate on port 8030 instead of 8000	Two servers (open on 8030, closed on 8031) can run side by side, one per terminal, and be compared
`Get-ChildItem Env:RATEGATE_*`	Lists environment variables starting with `RATEGATE_`	Cleanup check: a leftover `closed` would surprise a later server
`Remove-Item Env:RATEGATE_FAIL_MODE`	Deletes that variable from this terminal	Undoes the setting without closing the terminal

Measuring responses with curl

```
curl.exe -s -i -m 15 -w "\n%{http_code} in %{time_total}s\n" http://127.0.0.1:8030/api/resource
```

Part	Meaning
`curl.exe`	The real curl; plain `curl` in Windows PowerShell can be an alias for a different command
`-s`	Silent: hide the progress bar
`-i`	Include the response headers in the output, to see `retry-after` and `x-ratelimit-*`
`-o NUL` (used in Step 1 instead of `-i`)	Throw the body away; only the summary line is wanted. `NUL` is Windows' "nowhere" file
`-m 15`	Give up after 15 seconds, so a frozen server can't hang the terminal forever

Part	Meaning
`-w "..."`	After the response, print this text. `%{http_code}` becomes the status code and `%{time_total}` the seconds taken. That is how every timing in this phase was measured

Tests

Command	What it does	Why
`uv run pytest tests/test_fail_modes.py -v`	Runs only the new file; `-v` lists each test by name with PASSED or FAILED	The red and green steps
`uv run pytest tests/test_fail_modes.py -v --durations=5`	Same, then lists the 5 slowest tests with their times	Confirms each outage test takes about 2 s: the designed worst case, not a hang
`foreach ($i in 1..3) { uv run pytest -q | Select-Object -Last 1 }`	Runs the whole suite 3 times; `-q` is quiet output, and `Select-Object -Last 1` keeps only the summary line	Proves the new timeouts didn't break anything, repeatedly
`foreach ($i in 1..10) { uv run pytest -q tests/test_concurrency.py | Select-Object -Last 1 }`	The concurrency tests 10 times	These are the ones Mistake 1 broke; 10 runs out of 10 shows it is really fixed

`foreach ($i in 1..3) { ... }` is a PowerShell loop: `1..3` is the list 1, 2, 3, and the block in braces runs once for each. `|` (a pipe) sends one command's output into the next.

Finishing

Command	Why
`uv run ruff check .` and `uv run ruff format --check .`	Lint and formatting; both must be clean before committing
`git add .`	Stage every change
`git commit -m "Phase 4: fail-open/fail-closed handling with short Redis timeouts and one retry"`	Save a snapshot with a message saying what and why
`git push`	Send it to GitHub

Verification

All four checks passed on 2026-11-02 against the project code. Check 2 was done by hand with real `docker stop` and `docker pause`, because the automated tests only imitate an outage with a closed port.

Check 1: the test suite, repeated.

```
36 passed in 29.27s
36 passed in 28.55s
36 passed in 29.65s
```

and `7 passed` for the concurrency tests, 10 runs out of 10. The 36 are Phase 3's 31 plus the 5 new ones. Repeating matters for the same reason as in Phase 3: timing-related failures can appear only sometimes, and Mistake 1 was exactly that kind of problem.

Check 2: live outages, both modes side by side. Two servers ran at once: fail-open on port 8030 and fail-closed on port 8031, each started in its own terminal after setting `$env:RATEGATE_FAIL_MODE`. A third terminal sent one `curl` to each server per scenario, with Docker commands in between: stop, wait 5 s, request; start, wait 2 s, request; pause, request; unpause, request.

Scenario	Fail-open (8030)	Fail-closed (8031)
A. Redis up	`200` in 0.01 s, rate-limit headers present	`200` in 0.01 s, rate-limit headers present
B. `docker stop redis`	`200` in 1.00 to 2.02 s, **no** rate-limit headers	`503` in 1.00 to 2.02 s, `retry-after: 5`
C. `docker start redis`, no RateGate restart	`200` in 0.02 s, headers back	`200` in 0.02 s, headers back
D. `docker pause redis`	`200` in 0.50 to 0.53 s	`503` in 0.50 to 0.53 s
E. `docker unpause redis`	`200` in 0.03 s, headers back	`200` in 0.02 s, headers back

What each row means:

- **A** proves nothing changed when Redis is healthy.
- **B** proves each mode applies its policy. It takes 1 to 2 s on Windows because of the 2-second refusal delay, cut short by the 1 s connect timeout (Theory 3). The Phase 3 code answered `500` here after up to 4.1 s.
- **C** proves automatic recovery: the one retry replaced the dead pooled connections, so the very first request after the restart was limited normally.
- **D** proves a frozen Redis is noticed in 2 × 0.25 s, about half a second. The Phase 3 code froze for 5.04 s, then answered `500`.
- **E** proves recovery from a hang too.

One honest note: in one early run, fail-open answered in 0.05 s right after `docker stop`, while Redis was still shutting down. The numbers above are the steady state, measured 5 s after stopping. Report what is typical, not the luckiest first number.

Check 3: the logs. Each server's terminal printed one line per affected request. Fail-open:

```
Redis unavailable, allowing request (fail-open): Connection closed by server.
```

Fail-closed:

```
Redis unavailable, rejecting request (fail-closed): Timeout connecting to server
```

In total: 8 outage lines per server and 0 tracebacks. The Phase 3 code printed `ERROR: Exception in ASGI application` and a full traceback each time, 59 lines for the same handful of requests. The causes named in the lines match the scenarios: a closed connection when Redis stopped, a connect timeout while it stayed down, a read timeout while paused.

Check 4: cleanup and lint.

```
PONG
All checks passed!
18 files already formatted
```

Redis answers `PONG`, so it was left running and unpaused. `Get-ChildItem Env:RATEGATE_*` listed nothing, so no fail mode is left set. ruff found no problems. Then the work was committed as `8bd2123` and pushed.

Common pitfalls

Failure handling is code that only runs when something is already wrong, so its mistakes hide until the worst moment. These are the ones met in this phase, or most likely to catch you.

Pitfall	Symptom	Why it happens	Fix
No timeouts	Requests freeze 5 s or more while Redis hangs	The app waits for a reply that never comes	Set `socket_timeout` and `socket_connect_timeout`
One timeout for everything, set too low	Outage mode switches on during traffic spikes, while Redis is healthy	Opening connections under load takes about 500 times longer than a command	Separate values from measurements: 1.0 s connect, 0.25 s command
Library-default retries with backoff	One request takes up to 27 s to fail	Several retries, with growing pauses	`Retry(NoBackoff(), retries=1)`
Zero retries	Errors right after a Redis restart, though Redis is fine	Pooled connections are stale	One immediate retry
Answering `429` when Redis is down	Clients think *they* did something wrong and back off; monitoring sees no server fault	`429` means "you're over your limit"	`503` with `Retry-After`
Made-up rate-limit headers in fail-open	Clients plan around numbers that aren't true	The app doesn't know the real numbers	Send no `X-RateLimit-*` headers
Catching `Exception` instead of `RedisError`	Real bugs hide behind fail-open; the API "works" with no limiting	The catch is too broad	Catch only `RedisError`, and keep the `try` to the one Redis call
A test that passes on code without the feature	False confidence	It checks something the old code also does	Run new tests on the old code first; they must fail
Calling a method from memory	`AttributeError` at startup	The method doesn't exist in the library	Check the docs or source first
Forgetting to restart Redis after a live check	Every later test fails, or a server allows everything	Redis is still stopped or paused	`docker start redis` / `docker unpause redis`, then `ping`
`RATEGATE_FAIL_MODE` left set in a terminal	A later server unexpectedly answers `503`	`$env:` values last until the terminal closes	`Remove-Item Env:RATEGATE_FAIL_MODE`
A hard-coded time limit in a test	The test breaks when a setting changes	The limit doesn't follow the setting	Compute it from `settings`
Trusting the first measurement	Misleadingly fast or slow numbers	Transitional moments, such as Redis mid-shutdown	Wait for the steady state, and repeat
Expecting `docker stop` to be noticed instantly on Windows	Each request takes 1 to 2 s while Redis is stopped	Windows takes about 2 s to report a refused connection	Expected here; a circuit breaker (Phase 8) would remove it

Check your understanding

Answer each in your own words first, then compare. These are also likely interview questions about this project.

1. Name the three ways Redis can fail, and what each needs from the app. Down: notice fast and apply the policy. Hung: a limit on how long to wait for a reply. Restarted: replace stale connections without the user noticing.

2. When would you choose fail-closed over fail-open? Give one endpoint for each. Fail-closed when unlimited traffic is worse than downtime, such as login or payments, where limits stop attacks. Fail-open when downtime is worse, such as a public news page.

3. Why `503` and not `429` when Redis is down? `429` says the client is over its limit, which is false. `503` says the server can't serve right now, which is true, and it makes monitoring raise an alert.

4. Why are the two timeouts different, and how were they chosen? Opening a connection took up to 600 ms under a burst of 200; a command never took more than 1.25 ms. So: 1.0 s to connect, 0.25 s per command, both from measurements on this machine.

5. What went wrong when both timeouts were 0.1 s? The burst tests opened many connections at once; some took longer than 0.1 s, so a healthy Redis was reported as unreachable. In production, a traffic spike after a deploy would falsely switch on outage mode.

6. Why exactly one retry, with no pause? One retry replaces a stale connection after a restart. More retries with pauses only delay noticing a real outage: library defaults took 27 s.

7. Why does a stopped Redis take 1 to 2 s per request on Windows, but a hung one only about 0.5 s? Windows takes about 2 s to report a refused connection, so the 1 s connect timeout (twice, with the retry) ends each attempt. A hung Redis accepts the connection but never replies, so the 0.25 s reply timeout applies, twice.

8. Why catch `RedisError` and not `Exception`? `Exception` also catches bugs in our own code, which would then hide behind fail-open. Only Redis problems should trigger the fail mode; real bugs should surface as `500`.

9. Why does fail-open send no `X-RateLimit-*` headers? The app can't know the real limit or remaining count without Redis. Invented numbers would mislead clients.

10. What does `test_limiting_resumes_after_an_outage` prove? The codes `200, 503, 503, 200, 429` show that requests during the outage used no tokens, and limiting continued exactly where it left off once Redis was back.

11. What is a circuit breaker, and what would it fix here? After several failures in a row, it stops calling Redis for a while and applies the fail mode instantly, then tests Redis again. It would remove the 1 to 2 s each request pays during a long outage.

12. Why did the first version of the validation test pass on the old code? The old settings rejected `fail_mode` as an unknown setting name, so the expected error appeared for the wrong reason. Checking that `"closed"` is accepted made the test able to fail.

Phase 5: Horizontal scaling, 3 copies behind nginx (done 2026-11-02)

RateGate now runs as three identical API copies behind an nginx load balancer, all sharing one Redis, started with one command. 200 requests fired at once against a limit of 100 allow **exactly 100**, although each copy allowed only about 33 of them. The work is committed as `5569b27` and pushed.

This proves the resume's headline claim: a **distributed**, **horizontally scaled** rate limiter.

Phase 5 also went differently from the plan. Building it exposed a real bug in Phase 4 (a retry could count one request twice), a flaky Phase 3 test, two nginx surprises, and a Windows test environment that was slower than the real one. All are fixed; the three "What went differently" sections below tell each story in full.

Results, measured on 2026-11-02:

Result	Evidence
Copies take turns	14 requests in a row answered by copy 1, 2, 3, 1, 2, 3 ...
One shared limit	Those 14 drew from one bucket: 10 × `200`, then `429`, from whichever copy answered
Exact under load	200 at once, limit 100: exactly 100 allowed, in every run, for all three algorithms
The test can tell right from wrong	Each copy with its own separate store: 300 of 400 allowed
A dying copy doesn't matter	Stopping a copy while requests flowed: 300 of 300 answered `200`, slowest 1.01 s
Clients can't fake their IP	Requests with fake `X-Forwarded-For` headers stayed in their real bucket
Scales with one command	5 copies: still exactly 10 allowed (the default capacity)
Phase 4 still works inside containers	Redis stopped: fail-open `200`, fail-closed `503`, each in 1.0 s
Nothing older broke	40 tests, 4 runs out of 4; concurrency tests 30 runs out of 30

Before and after Phase 5:

	Before	After
How it runs	One `uvicorn` process on your PC	Redis + 3 API containers + nginx, via `docker compose up`
Who answered?	Unknown	`X-Instance-Id` on every response
Client IP	Whoever connected	The real client, taken safely from nginx's `X-Forwarded-For`
A late Redis reply	Retried, so the request could be **counted twice**	Not retried; counted once
Hung Redis noticed in	about 0.5 s	about 0.25 s
Stopped Redis noticed in	1 to 2 s	1.0 s

What changed in the project:

File	Change
`Dockerfile`, `.dockerignore`	New: package the API into an image
`docker-compose.yml`	New: Redis, 3 API copies, nginx
`nginx/nginx.conf`	New: the load balancer
`app/config.py`, `app/main.py`	An `instance_id` setting and the `X-Instance-Id` middleware (11 lines)
`app/redis_client.py`	Retry only connection errors (the Phase 4 bug fix)
`tests/test_instance_id.py`	New: 3 tests
`tests/test_fail_modes.py`	A late-reply regression test; a tighter outage-time budget
`tests/test_concurrency.py`	Warms its connection pools before the burst
`scripts/sequential_requests.py`	New: shows the copies taking turns
`scripts/concurrent_burst.py`	Also reports the answers per copy
`.gitignore`	Keeps a per-machine `.env` out of git
`README.md`, `ROADMAP.md`	Full-stack section, new setting, Phase 5 ticked

The step-by-step guide is in the project at `docs/PHASE_5_GUIDE.md`.

Theory 1: Scaling out, and why the copies must be stateless

Real APIs grow by adding identical copies of the server, not by buying one bigger machine. That only works if no copy keeps important data in its own memory, and RateGate was built that way from Phase 1.

Two ways to handle more traffic:

	Vertical ("scale up")	Horizontal ("scale out")
How	A bigger machine: more CPU, more memory	More copies running the same code
Limit	The biggest machine you can buy	Practically none: add copies
One copy crashes	Everything is down	The others keep serving
Deploying new code	Downtime while it restarts	Replace copies one at a time

Stateless. For scaling out to work, **any copy must be able to answer any request**. A load balancer decides which copy gets each request, and the client has no say. So a copy must not remember anything important in its own memory, such as how many requests a client has made. A server that keeps no such data of its own is called **stateless**. Its state lives somewhere shared instead.

For RateGate the important state is the counters: tokens, timestamps, logs of requests. From Phase 1 they have lived in **Redis**, not inside the Python process. Every copy reads and updates the same keys, using atomic Lua scripts so two copies can never interleave their reads and writes (proved in Phase 3). Phase 5 is the first time more than one copy actually runs.

What a copy does keep is harmless: its connection pool, its settings, its name. Losing any of these when a copy dies costs nothing; a replacement builds its own in milliseconds.

Theory 2: Images, containers and the Dockerfile

A **Dockerfile** is a recipe that packages the API, with its exact Python and libraries, into an **image**. Each running copy of that image is a **container**. Three containers from one image give three identical API copies.

Image vs container. You have used containers since Phase 0 (`docker run ... redis:7`). The two words are easy to mix up:

- An **image** is a packaged, read-only snapshot: the files of a small Linux system, Python, our libraries and our code. Think of a frozen ready meal.
- A **container** is a running instance of an image, with its own processes, memory and network address. You can heat as many portions from the same meal as you like.

Why package the app at all? On your PC, RateGate runs because you installed uv, Python 3.12 and the libraries. Another machine would need all of that again, in the same versions. An image carries everything inside it, so it runs the same way on any machine with Docker: your laptop, a CI server, or a cloud server.

The instructions a Dockerfile uses:

Instruction	Meaning
`FROM python:3.12-slim`	Start from an official image that already has Python 3.12 on a small Debian Linux
`COPY src dest`	Copy files from the project (or from another image) into the image
`WORKDIR /app`	Work in `/app` for every following instruction, creating it if needed
`ENV NAME=value`	Set an environment variable inside the image
`RUN command`	Run a command **while building**, such as installing libraries
`EXPOSE 8000`	Document the port the app listens on (it does not open it to the outside)
`CMD [...]`	The command each container runs **when it starts**

Layers and the build cache. Each instruction produces a **layer**, a saved snapshot of the changes it made. When you rebuild, Docker reuses every layer whose inputs are unchanged, and redoes everything from the first layer that changed. So the order of instructions matters:

1. Copy only `pyproject.toml`, `uv.lock` and `.python-version`.
2. Install the libraries (`uv sync`). This is the slow step.
3. Copy `app/`, the code.

Code changes all the time; dependencies rarely do. With this order, changing a line of code rebuilds only step 3. Measured: after a code change the rebuild took **1.6 s**, with every dependency layer reported as `CACHED`. Copying the code first would reinstall every library on every change.

`.dockerignore` lists files the build must never see: `.venv` (it holds Windows programs that can't run in a Linux container, and it's large), `.git`, caches, `tests`, `docs`, `scripts`, and `.env` (which may hold secrets). Smaller builds, and nothing private leaks into the image.

The finished image is 356 MB: mostly the Python base image and our libraries.

Theory 3: Docker Compose

Docker Compose starts a whole system of containers from one file, `docker-compose.yml`, with one command: `docker compose up`. Each kind of container is a **service**: `redis`, `api` and `nginx`.

Starting five containers by hand, each with the right settings, network and start order, would take a page of commands and be easy to get wrong. Compose writes that down once.

What Compose provides:

- **One private network.** Every service joins it automatically. Containers on it can talk to each other; the outside world can't reach them unless a port is published.
- **Names instead of addresses.** Containers get IP addresses that change every time they are recreated. So Docker runs a small **DNS** server (the internet's phone book: it turns names into addresses) on the network. Each service is reachable by its **service name**: the API connects to `redis://redis:6379`, and nginx forwards to `api:8000`.
- **Replicas.** `deploy: replicas: 3` starts three containers from one service: `rategate-api-1`, `-2` and `-3`. Asked for `api`, Docker's DNS answers with all three addresses.
- **Health checks.** A small command Docker runs every 2 seconds to ask "are you ready?": `redis-cli ping` for Redis, a request to `/health` for each API copy. `docker compose ps` shows `(healthy)`.
- **Start order.** `depends_on` with `condition: service_healthy` waits until a service is healthy, not merely started: Redis first, then the API copies, then nginx.
- **Published ports.** Only nginx has one: `8080:80` means "your PC's port 8080 leads to nginx's port 80". Redis and the API copies have none, so the only way in is through nginx. This matters for security (Theory 6).

Two Redis servers now exist, on purpose.

	Your Phase 0 `redis` container	The stack's `rategate-redis-1`
Reached at	`localhost:6379`, from Windows	`redis:6379`, only inside the stack's network
Used by	`uv run pytest`, the dev server	The 3 API copies
Published	Yes	No

They never clash and never share data. Commands must target the right one: `docker exec redis ...` for yours, `docker compose exec redis ...` for the stack's.

Settings flow in from your terminal. In `docker-compose.yml`, a setting listed with no value, such as `RATEGATE_CAPACITY:`, means "copy it from the terminal that runs `docker compose`, if it is set there". So `$env:RATEGATE_CAPACITY = "100"` before `docker compose up` changes the limit in all three copies at once. When it isn't set, the variable isn't passed at all and the app's defaults apply; we checked inside a container. After changing such a setting, `docker compose up -d` again **recreates** the copies (it prints `Recreated`), because a container's environment is fixed when it is created.

Theory 4: nginx, the load balancer

nginx ("engine-x") is a fast, widely used web server. Here it stands in front of the three copies, hands each request to them in turn, and routes around a copy that has died.

Two roles in one:

- **Reverse proxy.** Clients talk only to nginx. nginx forwards each request to an API copy and passes the answer back. Clients never see the copies or know how many there are.
- **Load balancer.** It spreads requests across the copies. The default method is **round-robin**: copy 1, copy 2, copy 3, copy 1, ...

The upstream. nginx calls a group of backend servers an **upstream**:

```
upstream rategate_api {
    zone rategate_api 64k;
    server api:8000;
}
```

`server api:8000` is a single line, but when nginx starts it asks Docker's DNS for `api`, gets three addresses, and so has three servers. nginx looks the name up **only at startup**. After changing the number of copies, run `docker compose restart nginx` so it looks again.

Worker processes and `zone`. nginx handles connections with several **worker processes** running side by side, one per CPU core: 16 on your machine. Without the `zone` line, each worker keeps its own round-robin count, all starting at the first server. That made one copy receive almost everything (What went differently, part 1). `zone` puts the count in memory shared by all workers.

When a copy dies. nginx uses **passive health checks**: it doesn't poll the copies, it notices failures while serving real requests.

1. Connecting to a copy fails, or takes longer than `proxy_connect_timeout`.
2. nginx sends that request to the next copy instead, so the client still gets an answer.
3. It marks the failed copy "temporarily disabled" and skips it for 10 seconds (the `fail_timeout` default), then tries it again.

The default connect limit is 60 seconds, which is far too long. We set `proxy_connect_timeout 1s`, because copies on the same machine connect in milliseconds. Result: stopping a copy while requests flowed cost at most one request about 1 s; everything else was answered normally.

Health checks in two places, for two jobs. Docker's health check (Theory 3) decides **when to start** nginx. nginx's passive checks decide **where to send** each request. Docker does not tell nginx when a copy dies; nginx finds out by itself.

Theory 5: Why the limit stays global, proved both ways

With one shared Redis, three copies together allowed exactly the limit: 100. With a separate store per copy, the same experiment allowed three times as much: 300. Showing both is what makes the 100 convincing.

The risk. Each copy is its own Python process with its own memory. If a copy counted requests in its own memory, each copy would allow a client the full limit. Behind a round-robin load balancer that happens automatically: the client doesn't even have to try, because nginx spreads its requests across the copies anyway. With 3 copies, the limit would silently become 3 times what you configured.

Why RateGate doesn't have that problem. Every copy counts in Redis, in the same keys, with atomic Lua scripts. A request answered by copy 2 sees the token that copy 1 used a millisecond earlier.

The proof. Requests fired at once through nginx, limit 100, nothing refilling during the test:

Setup	Requests	Allowed	Allowed per copy
All copies share one Redis (RateGate)	200	**100**	34, 33, 33
Each copy has its own separate store (deliberately wrong)	400	**300**	100, 100, 100

The second row is a **negative control**, like Phase 3's `NaiveCounter`: run the same experiment on something known to be wrong, and confirm it detects the problem. It was built in the scratch copy only, by pointing each copy at a different Redis database number (Redis has 16 numbered databases that share nothing). 400 requests were sent so each copy had room to reach its own 100.

The RateGate row was repeated for every algorithm in the project: exactly 100 in 3 out of 3 runs with the token bucket, and exactly 100 with the sliding window log and with the sliding window counter. No single copy ever allowed more than 39.

Theory 6: The client's real IP behind a proxy

Behind nginx, every request reaches the copies **from nginx**, so the app would see the same IP address for every user. nginx passes the real address in the `X-Forwarded-For` header, and uvicorn reads it. Getting the trust right is the difference between correct limits and none at all.

Why it matters. Clients without an `X-API-Key` are identified by IP address (`identify_client`, Phase 1). If every request seems to come from nginx's address, all those users share one bucket. One busy user would then block everyone.

The standard solution, in two halves:

1. **nginx** writes the address it actually saw into a header: `X-Forwarded-For`.
2. **uvicorn** reads that header and puts the address in `request.client.host`, where our middleware already looks. That's the `--proxy-headers --forwarded-allow-ips "*"` at the end of the Dockerfile. No RateGate code changed.

The danger. `X-Forwarded-For` is an ordinary header, and **any client can send one**, with any address in it. Whether that matters depends on what nginx does with a header that a client already sent. We built all three possible setups and looked at the keys Redis stored. The requests came from your PC, whose address on Docker's network is `172.19.0.1`; nginx's is `172.19.0.6`.

Setup	Keys stored in Redis	What it means
A. uvicorn ignores the header	`ip:172.19.0.6` (nginx)	**Every** client shares one bucket
B. nginx **appends** its address to the client's header (`$proxy_add_x_forwarded_for`)	`ip:1.1.1.1`, `ip:2.2.2.2`, `ip:3.3.3.3`	A client sending fake addresses gets a **fresh bucket every time**: no limit at all
C. nginx **overwrites** the header (`$remote_addr`), as in RateGate	`ip:172.19.0.1` only, even with fake headers	Correct

Setup B is the one many tutorials show, and it is useful when you need the full chain of proxies. For a rate limiter it is a hole: the client controls the address it is limited by.

Why trusting `"*"` is safe here. `--forwarded-allow-ips "*"` tells uvicorn to believe the header whoever sends it. That is safe **only** because the API copies' port is never published: nothing but nginx can reach them, and nginx always overwrites the header. If the copies were reachable directly, you would list nginx's address instead of `"*"`.

What went differently, part 1: building the stack

The plan for Phase 5 was four tasks: a Dockerfile, a Compose file, an `X-Instance-Id` header, and a script showing the limit holds. Eight things went differently from that plan. This part covers the three that came from nginx; parts 2 and 3 cover the rest.

All eight at a glance:

#	What went differently	How it showed up	Fix
1	nginx sent every request to the same copy	All 14 responses had the same `X-Instance-Id`	`zone` in the upstream

#	What went differently	How it showed up	Fix
2	A dead copy made one request wait 38.8 s	Timing a request right after stopping a copy	`proxy_connect_timeout 1s`
3	nginx's automatic DNS option caused failures	A `502` while a copy was stopped	Not used; restart nginx after scaling
4	A Phase 3 test failed half the time	Repeated test runs	Open the connections before the burst
5	**A real Phase 4 bug:** one request counted twice	`assert 88 == 100`	Retry only connection errors
6	The regression test for #5 was itself wrong	`assert [1, 9, 0, 60000] == 1`	Copy redis-py's real timeout behaviour
7	In the project, tests failed 7 times in 10	Repeated test runs	Per-machine timeouts in `.env`
8	A Phase 4 prediction was wrong	Measuring inside the containers	Corrected in Phase 4's section

1. nginx sent every request to the same copy

Context. nginx should hand requests out in turn: copy 1, 2, 3, 1, 2, 3. The first `nginx.conf` listed only `server api:8000;` in the upstream.

Expected. 14 requests in a row would show three different `X-Instance-Id` values, rotating.

What happened. All 14 came from one copy:

```
 1  200  instance=79524934a17f  remaining=9
 2  200  instance=79524934a17f  remaining=9
 ...
14  429  instance=79524934a17f  remaining=0
```

How the cause was found, one question at a time:

1. *Does nginx even know about three copies?* `nslookup api`, run inside the nginx container, returned three addresses (`172.19.0.3`, `.4`, `.5`). So DNS was fine and nginx had all three.
2. *Then why only one?* nginx handles connections with several worker processes. Counting them gave **16**, the same as the number of CPU cores (`nproc`).
3. *Root cause.* Without extra configuration, each worker keeps its **own** round-robin count, and every count starts at the first server. Each `curl` request opened a new connection, which usually landed on a different worker, and each worker sent its first request to copy 1.

The fix. `zone rategate_api 64k;` gives the upstream 64 KB of memory shared by all workers, so they share one count. The same 14 requests then rotated perfectly: `79524934…`, `d89abe42…`, `900ecddf…`, `79524934…`, and so on.

Lesson. "Configured" isn't "working". Without the `X-Instance-Id` header this would have gone unnoticed: the limit would still have held, but one copy would have done all the work.

2. A dead copy made one request wait 38.8 seconds

Context. If one copy dies, the others should carry on, and nginx should route around the dead one.

Expected. After `docker stop rategate-api-2`, every request still succeeds, quickly.

What happened. Every request got `200`, but the first took this long:

```
200 in 38.778807s
200 in 0.006218s
```

(The very first attempt at this test stopped nothing: `docker compose stop api --index 2` isn't a valid command, and the fallback after it never ran. It showed up because all three IDs kept answering, and the test was redone with `docker stop rategate-api-2`.)

How the cause was found. nginx's error log:

```
connect() failed (113: Host is unreachable) while connecting to upstream ... upstream:
"http://172.19.0.3:8000/health"
upstream server temporarily disabled while connecting to upstream
```

nginx kept trying to connect to the stopped copy until the operating system gave up with "Host is unreachable". A stopped container doesn't **refuse** connections: its address disappears, so nothing answers at all. nginx's default connect limit is 60 s. Only after the failure did it try another copy and mark the dead one "temporarily disabled".

A second surprise. After `docker start rategate-api-2`, that copy got no traffic for a while. A changed IP address was the first suspect, but `docker network inspect` showed the same `172.19.0.3`. Asked again a few seconds later, it was answering: nginx skips a failed copy for 10 seconds (`fail_timeout`) before retrying it. Normal behaviour, not a bug.

The fix. `proxy_connect_timeout 1s;`. Copies on the same machine connect in milliseconds, so 1 s is generous, and after it nginx moves on.

How it was checked. A request every 0.1 s while stopping a copy, repeated 5 times: **250 of 250 answered `200`, slowest 1.01 s.** In the project, once more: 50 of 50, slowest 1.01 s.

3. nginx's automatic DNS option: tried and rejected

Context. nginx looks up `api` only when it starts, so copies added later are invisible until it restarts. nginx 1.27 can re-check DNS by itself (`resolver 127.0.0.11` plus `server api:8000 resolve;`), which sounded strictly better.

What happened. New copies were found automatically, as promised. But in one stop test a request failed with `502 Bad Gateway`:

```
upstream timed out (110: Operation timed out) while connecting to upstream
no live upstreams while connecting to upstream
```

After one copy failed, nginx declared "no live upstreams" instead of trying the two healthy ones.

How it was checked. One `502` could be bad luck, so both setups ran the same test (stop a copy while sending 50 requests):

Setup	Result
Plain (look up at startup)	50 of 50 answered `200`, in each of 5 trials
With `resolve`	A `502` in 2 of 3 trials

Decision. Failed requests are a real problem; restarting nginx after scaling is only a small chore. RateGate uses the plain setup, and the docs say to run `docker compose restart nginx` after changing the number of copies. Scaling to 5 copies and back was then checked that way: all 5 answered, and exactly 10 requests were allowed either way.

What went differently, part 2: a real bug in Phase 4

The most important find of Phase 5 was a bug written in Phase 4. Its one retry could run the limiter's script twice for one request. A single strange test result exposed it, and fixing it also made outages faster to notice.

5. One request could be counted twice

Context. Phase 4 added **one immediate retry** to every Redis command: `Retry(NoBackoff(), retries=1)`. It was meant for one situation. After Redis restarts, the connections waiting in the pool are dead, and the first command on each fails with "Connection closed by server", although Redis is fine again. One retry on a fresh connection hides that from users.

How it showed up. While problem 4 (part 3) was being investigated, the concurrency tests were running again and again. Most failures were timeouts. One was different:

```
assert 88 == 100
```

That is not a timeout: it is a **wrong count**. 200 requests, limit 100, and only 88 allowed. Every earlier failure had been an error; this one was the limiter giving a wrong answer. A rate limiter must never allow more than its limit, and this one allowed fewer, so something was counting requests too eagerly.

The hypothesis. There are two kinds of failure, and they mean different things:

Failure	What it means	Did the script run in Redis?
Connection error	The connection was already dead	**No**: the command never arrived
Timeout (`Timeout reading`)	The reply was late	**Maybe**: Redis may have run it and replied too slowly

If a late reply is retried, the script runs a second time, and one request uses up two slots of the client's limit. Twelve such requests would turn the expected 100 into 88.

How it was checked, step by step:

1. *Does redis-py retry after timeouts?* Its source code says yes: the `Retry` class reacts to `(ConnectionError, TimeoutError)` by default.
2. *Does the double counting really happen?* That needed an experiment, and it took three attempts to get one that measured the right thing:
   - **Attempt 1:** a tiny 0.5 ms reply timeout, so every reply would be "late". Result: 0 entries stored for every setting. The timeout also cut off the connection's opening handshake, so the script never reached Redis. Nothing was measured.
   - **Attempt 2:** patch redis-py's reply-reading code to delay replies. Result: muddled numbers, because the patch also delayed the handshake replies.
   - **Attempt 3:** delay **only the script's reply** (`EVALSHA`, the command redis-py uses to run a stored script), after it has been sent, then fail the way a real read timeout does. That is exactly the situation in question.

   20 requests to the sliding window log, each with a late reply:

Retry setting	Entries stored in Redis
Phase 4: retry connection errors **and** timeouts	**40**: every request counted twice
Retry connection errors only	**20**

Retry setting	Entries stored in Redis
No retry at all	20

Hypothesis confirmed.

The fix. One argument in `create_redis`: `retry=Retry(NoBackoff(), retries=1, supported_errors=(RedisConnectionError,))`. Connection errors are still retried, which is safe because the command never reached Redis. Timeouts are raised straight away and handled by the fail mode.

Checking nothing else broke. The retry's original job still works: with a full pool, Redis was restarted, and the next request was allowed in 0.009 s.

A bonus. Timeouts are no longer retried, so an outage costs one timeout instead of two:

Redis problem	Phase 4	After the fix
Hung (`docker pause`)	about 0.5 s	about 0.25 s
Stopped (`docker stop`)	1 to 2 s	1.0 s
First request after a restart	0.01 s, allowed	0.01 to 0.02 s, allowed

How serious was it? The error only ever made the limiter **stricter**: clients were blocked a little early, never let through extra. Phase 4's tests couldn't see it, because it needs replies that are late but not lost, which only happens under heavy load. Still, a rate limiter that miscounts is wrong.

Lesson. A flaky test is information, not bad luck. Re-running until it went green would have buried a real bug.

6. The regression test for the bug was itself wrong

Context. A fixed bug needs a test that fails if it ever comes back. `test_a_late_reply_is_counted_once` makes the script's reply late on purpose, then checks Redis stored exactly **one** entry.

What happened. The first version failed on the **fixed** code, with a value that made no sense:

```
assert [1, 9, 0, 60000] == 1
```

`zcard` counts the entries in a sorted set and returns one number. `[1, 9, 0, 60000]` is what our limiter **script** returns: allowed, remaining, and its timing values.

Root cause. The fake timeout raised the error but left the late reply sitting unread on the connection. The connection went back to the pool, and the next command, `zcard`, read the script's leftover reply as if it were its own answer.

How the real behaviour was found. redis-py's `read_response` code shows what a real read timeout does:

```
if disconnect_on_error:
    await self.disconnect(nowait=True)
raise TimeoutError(f"Timeout reading from {host_error}")
```

It **closes the connection first**, precisely so a leftover reply can't confuse the next command. The fake skipped that step.

The fix. The test now closes the connection before raising, exactly as redis-py does. The experiment from problem 5 was re-run with this corrected fake: still 40 vs 20, so the conclusion stands. Then the test was checked in both directions:

- **Old code:** `assert 2 == 1`: one request, two entries. The test catches the bug.
- **Fixed code:** passes.

Lesson. A fake must copy the real behaviour it stands in for, or the test ends up testing the fake. The way to find the real behaviour was the same as in Phase 4's Mistake 3: read the library's source.

What went differently, part 3: the test environment

The tests run on Windows and reach Redis through Docker Desktop's port forwarding. That path turned out slower, and more variable through the day, than the network the real copies use. It caused two rounds of failing tests and needed a per-machine setting, not a code change.

4. A Phase 3 test started failing half the time

Context. `test_limit_holds_across_separate_instances` makes 4 separate Redis clients (like 4 servers) fire 200 requests at once, then checks that exactly 100 were allowed. In Phase 4 it passed 10 runs out of 10.

What happened. Early in Phase 5 (Docker Desktop had been off and was just started), it failed **5 runs out of 10**:

```
redis.exceptions.TimeoutError: Timeout connecting to server
```

How the cause was found, ruling things out one at a time:

1. *Is the new code to blame?* Phase 5's changes don't touch this test, and your unchanged Phase 4 project code also failed it (1 run in 10).
2. *Is the running stack using up the machine?* With all five containers stopped, it still failed 6 runs in 10. Not the stack.
3. *Has the machine changed since Phase 4?* Phase 4's connection measurement, repeated:

Opening 200 Redis connections at the same moment	Slowest
Phase 4, the morning of 2026-11-02	about 600 ms
Phase 5, that afternoon	1.5 s, then **2.5 s**, getting worse with each try

Connecting now took longer than the 1 s connect timeout. But was Redis slow, or the way to it?

4. *Redis or the path?* The same measurement from **inside** Docker's network, skipping Windows: slowest **0.84 s**, getting faster with each try. So the slow part was Docker Desktop's **port forwarding**, the program (`wslrelay`) that passes each connection from Windows into Docker. It slowed down after Docker Desktop's restart.

Root cause. The test opens up to 200 brand-new connections in one instant, from one Windows process, through that forwarding. A running server never does this: its pool opens gradually as traffic arrives, and is then reused.

The fix. The test exists to prove **atomicity** (no extra requests let through), not how fast 200 connections open. So it now "warms up": each client opens its connections first with 50 `PING`s, then the burst runs on open connections. Production timeouts stayed as they were: inside Docker's network, replies during a 200-request burst took at most 41 ms.

After this fix, 19 runs of 20 passed. The remaining failure led to problem 5 (part 2).

7. In the project, the tests failed 7 times in 10

Context. All of the above happened in a scratch copy, where the final runs passed 10 of 10. An hour later the same code was built into your project, following the guide.

What happened. The full suite failed 1 or 2 tests per run, and the concurrency tests failed **7 runs in 10**, now with a different message:

```
redis.exceptions.TimeoutError: Timeout reading from localhost:6379
```

The code was identical to the code that had just passed.

How the cause was found:

1. *The stack?* Stopped: 7 failures in 10. Running: 3 in 10. Not the stack.
2. *An overloaded PC?* Per-process CPU use over 5 seconds was low (Chrome and VS Code, about 1% each). Two clues: `wslrelay` had used 267 CPU-seconds that day, and Windows was holding 9,285 recently closed connections to port 6379 (it keeps closed connections around for a couple of minutes).
3. *How late are replies, exactly?* The test's workload, run 30 times (6,000 replies), with timeouts so long that nothing was cut off:

Where it ran	Replies over 250 ms	Slowest reply
From Windows, through Docker Desktop (round 1)	0	162 ms
From Windows, through Docker Desktop (round 2)	**50**	**269 ms**
Inside Docker's network	0	47 ms

Root cause. The Windows → Docker path sometimes delays replies past the 0.25 s reply timeout, and how often varies through the day. That's why scratch passed at one moment and the project failed the next. The real copies never take this path.

The decision. The production timeouts are right for where the app runs, so they stay. Phase 4 made them **settings** precisely because the right value depends on where Redis runs, and "tests on a Windows laptop through Docker Desktop" is a different place. The app already reads a `.env` file from the project folder (`env_file=".env"`, since Phase 1), so this machine gets its own values there:

```
RATEGATE_REDIS_SOCKET_TIMEOUT=1.0
RATEGATE_REDIS_CONNECT_TIMEOUT=1.5
```

Choosing the numbers:

- **Replies: 1.0 s**, almost 4 times the slowest reply measured (269 ms). With only this change, 18 runs of 20 passed; the failures were now `Timeout connecting`.
- **Connecting: 1.5 s.** 4,000 connections, opened 50 at a time, took at most 458 ms. The value must also stay **under 2 s**: Windows takes about 2 s to report a refused connection, and a refusal is a connection error, which is still retried once. Above 2 s, an outage would wait for two refusals and the outage-speed test would fail.

Checking it:

- 30 of 30 concurrency runs and 4 of 4 full suites (40 tests) passed.
- **Containers are unaffected:** inside one, the settings read `0.25 1.0`. `.dockerignore` keeps `.env` out of the image, and Compose doesn't pass those two variables.

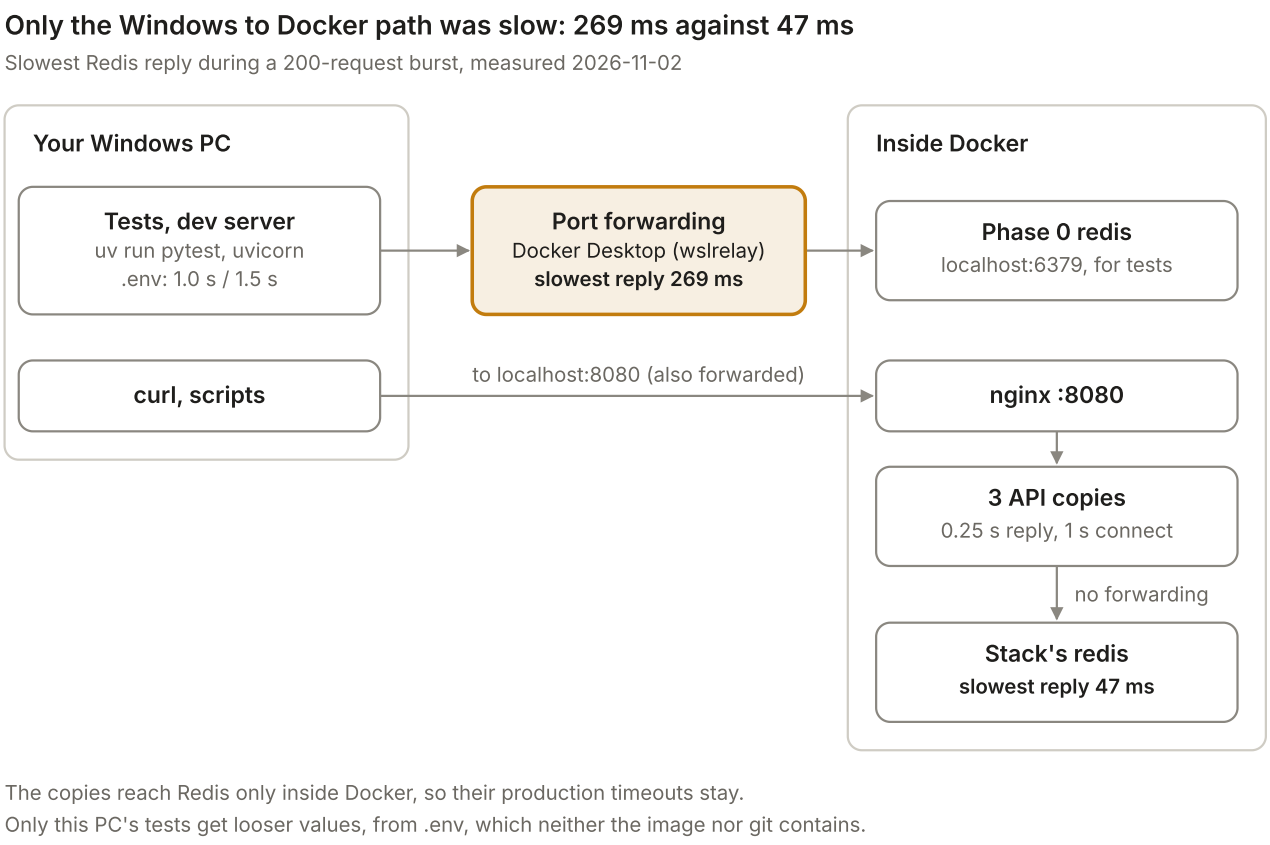
- **Git is unaffected:** `.env` was added to `.gitignore`, so GitHub never sees it, and Phase 6's tests on GitHub's Linux machines will use the real 0.25 s and 1 s.
- VS Code noticed the new `.env` and created `.vscode/settings.json` (`"python.terminal.useEnvFile": true`), so its terminals load it too. That's an editor file, so it wasn't committed.

Lesson. Timeouts belong to an environment, not to the code. The production values were measured where production runs; the test machine needed its own measurement.

8. A Phase 4 prediction was wrong

Phase 4's journal said that on Linux, which the containers run, a stopped Redis "should be noticed much faster", because Linux refuses connections to a closed port instantly. Measured inside the stack: **not faster.** Before the retry fix it took 1 to 2 s per request, exactly as on Windows, and the log said `Timeout connecting to server`, not "refused".

The reason is the same as in problem 2. Stopping a container doesn't close a port on a machine that is still there: the container's whole address disappears. Nothing answers at all, not even with a refusal, so the connect timeout is what ends the wait. Since the retry fix, that is 1.0 s in every environment. Phase 4's section now carries a correction.



Two paths to Redis · slowest reply under a burst

The tests reach Redis through Docker Desktop's port forwarding, which delayed some replies past 0.25 s. The API copies talk to their Redis directly on Docker's network, where the slowest reply was 47 ms.

How it was built

As in every phase: build and measure in a scratch copy, write the guide from what actually worked, then build it in the project step by step, watching each new test fail before its fix.

Step	What	Why at this point
Setup	Start Docker Desktop and the Phase 0 Redis; create this machine's `.env`; add `.env` to `.gitignore`	Everything after needs Docker; the tests need the per-machine timeouts
1	Retry fix: two tests first (**red**), then the fix (**green**)	A correctness bug; everything after relies on exact counts
2	Warm the pools in the 4-client test	A trustworthy test suite before adding to it
3	`X-Instance-Id`: 3 tests (**red**), then setting and middleware (**green**)	The one app change Phase 5 needs, testable without Docker
4	`Dockerfile` + `.dockerignore`; build; run **one** container	Check the packaged app alone before running three
5	`nginx/nginx.conf`	The load balancer's rules
6	`docker-compose.yml`	Describes the whole system
7	`docker compose up --build -d --wait`	Start it and see every piece healthy
8	The two scripts	Make turn-taking and the shared limit visible
9	Full suite 3 times, concurrency tests 10 times, lint	Prove nothing older broke
10	README and ROADMAP	Record it

Step 1, red. The two new tests on the Phase 4 code:

```
test_outage_is_detected_within_the_timeouts FAILED
test_a_late_reply_is_counted_once FAILED
E       assert (185228.5415013 - 185226.5083437) < 1.5
E       assert 2 == 1
```

The outage took 2.03 s: the connect timed out, was retried, and timed out again. And one late request left two entries in Redis. With the fix: `6 passed`.

Step 3, red. On the Phase 4 code: `3 failed`, as `KeyError: 'x-instance-id'` twice (the header didn't exist) and `AttributeError: 'Settings' object has no attribute 'instance_id'`. After the change: `3 passed`.

Step 4. The image built in 11 s. One container, given the address of your Phase 0 Redis, answered with `x-instance-id: de59446787e4`, its own container ID.

Step 7. All five services were healthy after 10 s, and only nginx was published (`0.0.0.0:8080->80/tcp`).

Step 9. First attempt: 1 to 2 failures per full suite and 7 failed concurrency runs in 10. That was problem 7 (What went differently, part 3); the `.env` was created then. After it: `40 passed` in about 27 s in every run, and 30 of 30 concurrency runs.

The scratch copy also contains experiments that were never meant for the project: the separate-store negative control, the three setups for `X-Forwarded-For`, the `resolve` comparison, and the double-counting measurement. Their results are recorded above; their code stayed in the scratch folder and never entered the project.

Code 1: The retry fix, its regression test and the warm-up

Three small changes to Phase 3 and 4 code: one argument in `create_redis`, one new test plus a tighter time budget, and a warm-up in one concurrency test.

`app/redis_client.py`, the whole file now:

```
from redis.asyncio import BlockingConnectionPool, Redis
from redis.asyncio.retry import Retry
from redis.backoff import NoBackoff
from redis.exceptions import ConnectionError as RedisConnectionError

from app.config import Settings


def create_redis(settings: Settings) -> Redis:
    # A blocking pool makes a burst of requests wait briefly for a free connection
    # instead of failing with "Too many connections".
    pool = BlockingConnectionPool.from_url(
        settings.redis_url,
        max_connections=settings.redis_max_connections,
        timeout=settings.redis_pool_timeout,
        socket_connect_timeout=settings.redis_connect_timeout,
        socket_timeout=settings.redis_socket_timeout,
        # One immediate retry replaces a connection that went stale (e.g. Redis
        # restarted); more retries would only delay noticing a real outage.
        # Only connection errors are retried: after a timeout the script may already
        # have run, and running it again would count the request twice.
        retry=Retry(NoBackoff(), retries=1, supported_errors=(RedisConnectionError,)),
        decode_responses=True,
    )
    return Redis.from_pool(pool)
```

- **`supported_errors=(RedisConnectionError,)`** lists the errors this retry policy reacts to. Anything else, including `TimeoutError`, is raised straight away and reaches the middleware's fail mode. The trailing comma makes it a tuple of one item; without it, the brackets would just be grouping.
- **`as RedisConnectionError`**: Python has its own built-in `ConnectionError`. Importing redis-py's under another name avoids hiding Python's and makes clear which one is meant.

`tests/test_fail_modes.py`: the regression test, added at the end, with new imports `asyncio`, `Redis`, `RedisError`, `TimeoutError as RedisTimeoutError` and `SlidingWindowLog`:

```
async def test_a_late_reply_is_counted_once(redis, client_id, monkeypatch):
    limiter = SlidingWindowLog(redis, limit=10, window_seconds=60)
    await limiter.check(f"{client_id}-warm")  # makes sure Redis has the script loaded
    original = Redis.parse_response

    async def late_reply(self, connection, command_name, **options):
        # The script has been sent and Redis runs it; only its answer is "late".
        # Do what redis-py does on a real read timeout: drop the connection, then raise.
        if command_name == "EVALSHA":
            await asyncio.sleep(0.05)
            await connection.disconnect(nowait=True)
            raise RedisTimeoutError("Timeout reading from Redis (simulated)")
        return await original(self, connection, command_name, **options)

    monkeypatch.setattr(Redis, "parse_response", late_reply)
    with pytest.raises(RedisError):
        await limiter.check(client_id)
    monkeypatch.undo()
    # One request, one entry. A retry after the timeout would have run it twice.
    assert await redis.zcard(f"rategate:swl:{client_id}") == 1
```

- **`monkeypatch`** is a fixture built into pytest. `monkeypatch.setattr(Redis, "parse_response", late_reply)` temporarily replaces redis-py's method for reading a command's reply; `monkeypatch.undo()` puts the original back, and pytest also restores it automatically when the test ends.
- **`late_reply`** passes every command through unchanged except the script (`EVALSHA`). For that one, the command has already been sent, so Redis runs it within a millisecond. Then the fake waits 50 ms, closes the connection and raises `TimeoutError`: exactly what redis-py does on a real late reply (What went differently, problem 6).
- **`original = Redis.parse_response`** saves the real method first, so `late_reply` can call it for every other command.
- **The warm-up check with another client id** makes sure Redis has the script stored. Otherwise the first `EVALSHA` would be refused as an unknown script and nothing would be counted: the test would pass or fail for the wrong reason (Phase 4's Mistake 2).
- **`pytest.raises(RedisError)`**: the request itself must still fail, because the app never learned the result.
- **`zcard`** counts the entries in the sliding log's sorted set. One request must mean one entry.

The tighter budget in `test_outage_is_detected_within_the_timeouts`:

```
    # A connect timeout is not retried, so the worst case is one connect timeout.
    budget = settings.redis_connect_timeout + 0.5
```

It was `2 * settings.redis_connect_timeout + 0.5`. With timeouts no longer retried, an outage costs one connect timeout, and the test now says so. On the old code it fails (2.03 s), which is what makes it a check of the fix.

`tests/test_concurrency.py`: the warm-up, a helper above the first test:

```
async def warm_up(client: Redis) -> None:
    # Open the pool's connections before the burst, as a running server's pool already
    # is. The test is about atomicity, not about opening 200 connections in one instant.
    await asyncio.gather(
        *(client.ping() for _ in range(settings.redis_max_connections))
    )
```

called in `test_limit_holds_across_separate_instances`, inside the `try`, before the burst:

```
    try:
        for client in clients:
            await warm_up(client)
        decisions = await asyncio.gather(
            *(limiters[i % 4].check(client_id) for i in range(ATTEMPTS))
        )
```

- **50 `PING`s at once** make each pool open its 50 connections, because each ping needs its own. Clients are warmed one after another, so at most 50 connections open at once instead of 200.
- **Inside the `try`**, so the clients are still closed in `finally` if a warm-up fails.

Code 2: `X-Instance-Id`

Every response names the copy that answered. It took one setting, one tiny middleware and three tests: 11 lines of app code in total.

`tests/test_instance_id.py` (new), written first:

```
import socket

import httpx

from app.config import Settings, settings
from app.limiter.token_bucket import TokenBucket
from app.main import app


async def get(path, limiter, api_key, times=1):
    """Send requests through the real app, the way nginx would hand them over."""
    app.state.limiter = limiter
    app.state.fail_mode = "open"
    transport = httpx.ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport, base_url="http://test") as client:
        headers = {"X-API-Key": api_key}
        return [await client.get(path, headers=headers) for _ in range(times)]


async def test_allowed_and_blocked_responses_name_the_instance(redis, client_id):
    limiter = TokenBucket(redis, capacity=1, refill_rate=0.001)
    responses = await get("/api/resource", limiter, client_id, times=2)
    assert [response.status_code for response in responses] == [200, 429]
    for response in responses:
        assert response.headers["x-instance-id"] == settings.instance_id


async def test_health_check_names_the_instance(redis, client_id):
    [response] = await get("/health", TokenBucket(redis, 10, 1.0), client_id)
    assert response.headers["x-instance-id"] == settings.instance_id


def test_instance_id_defaults_to_the_hostname():
    assert Settings().instance_id == socket.gethostname()
    assert Settings(instance_id="api-7").instance_id == "api-7"
```

- `get` is the same in-memory HTTP harness as Phase 4's `call_api`: it puts a limiter into `app.state` and sends real requests through the real middleware, with no server.
- **Test 1**: a bucket of 1 token gives `200` then `429`, and **both** must carry the header. The `429` is the important one: the rate-limit middleware makes it without ever reaching the route, so it proves the new middleware wraps the old one.
- **Test 2**: `/health` is exempt from limiting, but still labelled, which helps when checking the load balancer.
- **Test 3**: the default is the machine's name, and the setting can be overridden. `socket.gethostname()` is Python's way to ask the operating system for that name.

`app/config.py`: two imports at the top and one setting after `fail_mode`:

```
import socket
from typing import Literal

from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict
```

```
    fail_mode: FailMode = "open"  # what to do when Redis can't be reached
    instance_id: str = Field(default_factory=socket.gethostname)  # names this copy
```

- **`Field(default_factory=socket.gethostname)`**: a default that is **computed** when the settings are created, by calling the function, instead of a fixed value written in the code. Each container computes its own.
- **Inside Docker, a container's hostname is its short container ID**, such as `517428e7f289`. So each copy names itself with no configuration, and the name matches what `docker ps` shows.
- Like every setting, `RATEGATE_INSTANCE_ID` can override it.

`app/main.py`: after `app.add_middleware(RateLimitMiddleware)`:

```
@app.middleware("http")
async def add_instance_id(request, call_next):
    # Added last, so it wraps everything: 200s, 429s and 503s all say who answered.
    response = await call_next(request)
    response.headers["X-Instance-Id"] = settings.instance_id
    return response
```

- **`@app.middleware("http")`** is FastAPI's short way to write a middleware as one function, without a class like `RateLimitMiddleware`.
- **`call_next(request)`** hands the request inward (to the rate-limit middleware, then the route) and returns whatever response comes back, whether `200` from the route or `429` / `503` from the rate limiter. One header is added on the way out.
- **Order matters.** Middleware works like layers of an onion: the one added **last** is the **outermost**, so it runs first on the way in and last on the way out. Added after `RateLimitMiddleware`, it sees every response that middleware produces. Added before it, `429` and `503` answers would have gone out unlabelled, and test 1 would fail.

Code 3: The Dockerfile and `.dockerignore`

The Dockerfile is 31 lines, most of them comments. It installs dependencies before copying the code so rebuilds are fast, and starts uvicorn so it can see clients' real IP addresses through nginx.

`Dockerfile` (in the project root, no file extension):

```
# The Python version matches .python-version.
FROM python:3.12-slim

# uv, copied from its official image (pinned to the version used to write uv.lock).
COPY --from=ghcr.io/astral-sh/uv:0.13.6 /uv /bin/uv

WORKDIR /app

# Use the image's Python instead of downloading one; copy packages into .venv
# rather than linking them, and precompile them so the first request is fast.
ENV UV_PYTHON_DOWNLOADS=never \
    UV_LINK_MODE=copy \
    UV_COMPILE_BYTECODE=1

# Dependencies first: this layer is rebuilt only when the lock file changes,
# not on every code change.
COPY pyproject.toml uv.lock .python-version ./
RUN uv sync --frozen --no-dev

COPY app ./app

# Run the virtual environment's programs directly, with no uv at runtime.
ENV PATH="/app/.venv/bin:$PATH"
```

```
EXPOSE 8000

# --proxy-headers: take the client's IP from nginx's X-Forwarded-For header.
# --forwarded-allow-ips "*": trust that header from any address, which is safe
# only because the API port is never published; nginx is the only way in.
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", \
     "--proxy-headers", "--forwarded-allow-ips", "*"]
```

Line by line:

- **`FROM python:3.12-slim`**: the official Python 3.12 image, the same version `.python-version` pins on your PC. `slim` leaves out compilers and extras we don't need.
- **`COPY --from=ghcr.io/astral-sh/uv:0.13.6 /uv /bin/uv`**: copies only the `uv` program out of uv's official image. Pinning the exact version you use locally means the image is built the same way every time.
- **`WORKDIR /app`**: everything after happens in `/app`.
- **`ENV ... \`**: three uv settings. The `\` at the end of a line continues the instruction on the next line.
  - `UV_PYTHON_DOWNLOADS=never`: use the image's Python 3.12; never download another.
  - `UV_LINK_MODE=copy`: copy library files into `.venv`. uv normally links them from a cache, which doesn't work across Docker's layers and prints a warning.
  - `UV_COMPILE_BYTECODE=1`: compile the libraries' `.py` files to `.pyc` during the build, so the first request after a start isn't slowed by compiling.
- **`COPY pyproject.toml uv.lock .python-version ./`** then **`RUN uv sync --frozen --no-dev`**: install exactly the versions in `uv.lock`. `--frozen` means "never change the lock file; fail if it is out of date". `--no-dev` skips pytest and ruff, which production doesn't need. Only these three files are copied first, so this slow step stays cached until dependencies change (Theory 2).
- **`COPY app ./app`**: now the code. Tests, scripts and docs stay out of the image.
- **`ENV PATH="/app/.venv/bin:$PATH"`**: put the virtual environment's programs first on the search path, so `uvicorn` means the one in `.venv`. No `uv run` is needed at start-up.
- **`EXPOSE 8000`**: documents the port. It doesn't publish anything; Compose decides that.
- **`--host 0.0.0.0`**: listen on every network interface of the container. uvicorn's default, `127.0.0.1`, only accepts connections from **inside** the container itself, so nginx would get `502 Bad Gateway`.
- **`--proxy-headers --forwarded-allow-ips "*"`**: read the client's real IP from `X-Forwarded-For`, trusting it from any sender (Theory 6 explains why that is safe here, and only here).
- **The `CMD [...]` list form** runs uvicorn directly, without a shell in between. When Docker stops the container, uvicorn itself receives the stop signal and shuts down cleanly.

`.dockerignore`:

```
# Only what the Dockerfile copies is needed; everything else would slow the
# build and could leak local files into the image.
.git
.venv
.env
**/__pycache__
.pytest_cache
.ruff_cache
docs
tests
scripts
```

- **`.venv`** matters most: it contains **Windows** programs that can't run in a Linux container, and it's large.

- `.env` is this machine's settings and could later hold secrets. Keeping it out also guarantees the containers never see the Windows-only timeouts (What went differently, problem 7).
- `**/__pycache__` means a `__pycache__` folder at any depth.

Code 4: `nginx.conf` and `docker-compose.yml`

Two configuration files describe the running system: nginx's rules for sharing requests, and Compose's list of containers and how they connect.

`nginx/nginx.conf`:

```
upstream rategate_api {
    # Shared memory for the round-robin count. Without it, each nginx worker process
    # keeps its own count, all starting at the first server, so one replica gets
    # most requests.
    zone rategate_api 64k;
    # "api" is the Compose service name. When nginx starts, Docker's DNS answers it
    # with the address of every replica, so this one line becomes one server per
    # replica, used in turn. After changing the number of replicas, restart nginx.
    server api:8000;
}

server {
    listen 80;

    location / {
        proxy_pass http://rategate_api;
        # Replicas are on the same machine and answer in milliseconds. If one is dead,
        # give up on it after 1 s and try the next, instead of waiting the 60 s default.
        proxy_connect_timeout 1s;
        proxy_set_header Host $host;
        # Overwrite, never append: a client can't smuggle in a fake IP of its own.
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

nginx's own syntax: one setting per line, each ending in `;`, blocks in `{ }`, comments after `#`.

- **`upstream rategate_api { ... }`**: a named group of backend servers (Theory 4).
- **`zone rategate_api 64k;`**: 64 KB of memory shared by all worker processes, holding the group's round-robin count. Without it, one copy got all the traffic (What went differently, problem 1).
- **`server api:8000;`**: the copies, by Compose service name and port. Becomes three servers when nginx starts.
- **`server { listen 80; ... }`**: the site nginx serves, on port 80 inside its container. Compose publishes it as your PC's port 8080.
- **`location / { ... }`**: rules for every path that starts with `/`, which is all of them.
- **`proxy_pass http://rategate_api;`**: forward the request to the group.
- **`proxy_connect_timeout 1s;`**: give up on a copy that doesn't accept the connection within 1 s, and try the next (problem 2).
- **`proxy_set_header Host $host;`**: pass on the `Host` header the client sent. Without it, nginx would send the group's name, `rategate_api`. Names starting with `$` are nginx's built-in variables.
- **`proxy_set_header X-Forwarded-For $remote_addr;`**: `$remote_addr` is the address of whoever connected to nginx. Setting the header to it **replaces** anything the client sent, so a fake address is thrown away (Theory 6).

`docker-compose.yml`:

```
name: rategate

services:
  redis:
    image: redis:7
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 2s
      retries: 10

  api:
    build: .
    environment:
      RATEGATE_REDIS_URL: redis://redis:6379/0
      # Passed through from your terminal when set there; otherwise the app's defaults.
      RATEGATE_ALGORITHM:
      RATEGATE_CAPACITY:
      RATEGATE_REFILL_RATE:
      RATEGATE_LIMIT:
      RATEGATE_WINDOW_SECONDS:
      RATEGATE_FAIL_MODE:
    deploy:
      replicas: 3
    depends_on:
      redis:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 2s
      retries: 15

  nginx:
    image: nginx:1.27-alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      api:
        condition: service_healthy
```

YAML is indentation-based: whatever is indented under a name belongs to it. Use spaces, never tabs.

- **`name: rategate`**: the project name. Containers are named after it: `rategate-api-1`, `rategate-nginx-1`.
- **`redis`**: the official Redis 7 image. Its **health check** runs `redis-cli ping` every 2 s and marks it unhealthy after 10 failures. No `ports:`, so it is private to the stack.
- **`api` → `build: .`**: build the image from our `Dockerfile` instead of downloading one.
- **`environment:`**: `RATEGATE_REDIS_URL` points at the service called `redis`. The names with no value are passed through from your terminal when set there (Theory 3). The two timeout settings are deliberately **not** listed, so this PC's `.env` values can never reach the containers.
- **`deploy: replicas: 3`**: three containers from this one service.
- **`depends_on: redis: condition: service_healthy`**: start the copies only once Redis answers `PONG`.
- **The API health check** runs a one-line Python program inside the container that fetches `/health`. If the request fails, Python exits with an error and Docker counts a failure. The `python:3.12-slim` image has no `curl`, but it has Python.

- `nginx` → `image: nginx:1.27-alpine`: official nginx 1.27 on the tiny Alpine Linux. A fixed version gives the same nginx every time.
- `ports: - "8080:80"`: your PC's port 8080 leads to nginx's port 80. The **only** way into the stack.
- `volumes:`: show our config file inside the nginx container at the path where nginx looks for its site configuration, replacing its default site. `:ro` means read-only.
- `depends_on: api: condition: service_healthy`: start nginx only after the copies are healthy, so its one DNS lookup of `api` finds all three.

Code 5: The two scripts

One script sends requests one at a time to show the copies taking turns; the other now reports a burst's answers per copy.

`scripts/sequential_requests.py` **(new):**

```
"""Send requests one at a time and show which server copy answered each, and how."""

import argparse
from uuid import uuid4

import httpx


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--url", default="http://127.0.0.1:8080/api/resource")
    parser.add_argument("--total", type=int, default=14)
    parser.add_argument("--api-key", default=f"seq-{uuid4().hex[:8]}")
    args = parser.parse_args()

    print(f"API key {args.api_key}")
    with httpx.Client(timeout=10) as client:
        for n in range(1, args.total + 1):
            response = client.get(args.url, headers={"X-API-Key": args.api_key})
            instance = response.headers.get("X-Instance-Id", "(no header)")
            remaining = response.headers.get("X-RateLimit-Remaining", "-")
            print(f"{n:3}  {response.status_code}  {instance}  remaining={remaining}")


if __name__ == "__main__":
    main()
```

- `httpx.Client`, without `async`: send one request, wait for its answer, then send the next.
- **A fresh random API key per run** (`uuid4`), so every run starts with a full bucket.
- `response.headers.get(name, default)` returns the default instead of crashing when a header is missing, for example `X-RateLimit-Remaining` on a fail-open response.
- `{n:3}` pads the number to 3 characters so the columns line up.
- **14 requests by default**: enough to empty the default bucket of 10 and show several `429`s, from different copies.

`scripts/concurrent_burst.py` **(changed):** `burst` now returns the responses themselves instead of only their status codes, and `main` adds a summary per copy:

```
from collections import Counter, defaultdict
...
async def burst(url: str, total: int, api_key: str) -> list[httpx.Response]:
```

```
    limits = httpx.Limits(max_connections=total)
    async with httpx.AsyncClient(limits=limits, timeout=30) as client:
        return await asyncio.gather(
            *(client.get(url, headers={"X-API-Key": api_key}) for _ in range(total))
        )
...
    # Which server copy answered, and how: only meaningful behind a load balancer.
    by_instance = defaultdict(Counter)
    for response in responses:
        instance = response.headers.get("X-Instance-Id", "(no header)")
        by_instance[instance][response.status_code] += 1
    print("By instance:")
    for instance, statuses in sorted(by_instance.items()):
        summary = ", ".join(f"{s}: {n}" for s, n in sorted(statuses.items()))
        print(f"  {instance}  {summary}")
```

- **`defaultdict(Counter)`**: a dictionary that creates an empty `Counter` the first time an unseen key is used. So `by_instance[instance][status] += 1` works without first checking whether that copy has an entry yet.
- **`", ".join(...)`** glues strings together with `, ` between them, giving lines like `200: 34, 429: 33`.
- **`list[httpx.Response]`** in the signature says what `burst` returns now: one response object per request.

Command reference

Every command used in Phase 5, what it does and why. All run in PowerShell from `D:\RateGate`.

Building and running one container

Command	What it does	Why
`docker build -t rategate-api .`	Builds an image from the `Dockerfile` in `.` (this folder) and names it (**t**ag) `rategate-api`	Package the app (Step 4)
`docker run --rm -d --name rategate-single -p 8001:8000 -e RATEGATE_REDIS_URL=redis://host.docker.internal:6379/0 rategate-api`	Starts one container: `--rm` deletes it when it stops, `-d` runs it in the background, `--name` names it, `-p 8001:8000` publishes its port 8000 as your PC's 8001, `-e` sets a variable inside it	Check the packaged app alone, before running three
`docker stop rategate-single`	Stops that container (and `--rm` then deletes it)	Clean up
`docker ps`	Lists running containers with their IDs	Compare IDs with `X-Instance-Id`

`host.docker.internal` is a special name Docker Desktop provides inside containers: "the PC this container runs on". It let the single container reach your Phase 0 Redis on `localhost:6379`.

The Compose stack

Command	What it does	Why
`docker compose up --build -d --wait`	Builds the API image (`--build`), creates and starts every service, in the background (`-d`), and returns only when all are healthy (`--wait`)	Start the stack; `--build` is needed after every code change
`docker compose ps`	Lists the stack's containers, their health and ports	Check every piece is `(healthy)` and only nginx is published

Command	What it does	Why
`docker compose up -d --wait`	Same without rebuilding; recreates copies whose settings changed	Apply new `$env:RATEGATE_*` values
`docker compose restart nginx`	Restarts only nginx	Make nginx look up the copies again after they change (Theory 4)
`docker compose up -d --scale api=5 --wait`	Runs 5 API copies instead of 3	Show scaling with one command
`docker compose stop redis` / `start redis`	Stops / starts only the stack's Redis	Check fail modes inside the containers
`docker compose exec redis redis-cli ...`	Runs `redis-cli` inside the **stack's** Redis	Inspect keys; not your Phase 0 Redis
`docker compose logs api`	Shows the copies' output, each line prefixed with its copy (`api-2  |`)	See who logged an outage
`docker compose down`	Stops and **removes** the stack's containers and network (the image stays)	Clean up; the next `up` is fast

Inspecting what's going on

Command	What it does	Why it was used
`docker exec rategate-nginx-1 nslookup api`	Asks Docker's DNS, from inside nginx's container, for the addresses of `api`	Problem 1: did nginx know about all three copies?
`docker exec rategate-nginx-1 nproc`	Prints the number of CPU cores the container sees	Problem 1: how many nginx workers there are
`docker compose logs nginx`	nginx's access and error log	Problems 2 and 3: why a request was slow or failed
`docker network inspect rategate_default`	Lists the containers on the stack's network, with their addresses	Problem 2: had the restarted copy changed address?
`docker stop rategate-api-2` / `docker start rategate-api-2`	Stops / starts one copy by its container name	Check that a dying copy doesn't matter
`docker exec rategate-api-1 python -c "..."`	Runs Python inside a copy	Problem 7: confirm the containers still use 0.25 s / 1 s

The scripts

Command	Why
`uv run python scripts/sequential_requests.py`	14 requests one at a time: see the copies take turns and share one count
`uv run python scripts/concurrent_burst.py --url http://127.0.0.1:8080/api/resource --total 200`	200 at once through nginx: totals and answers per copy

Settings for one run

Command	Why
`$env:RATEGATE_CAPACITY = "100"; $env:RATEGATE_REFILL_RATE = "0.001"`	Limit 100 with nothing refilling, for exact counts (passed into the copies by Compose)
`Remove-Item Env:RATEGATE_*`	Remove every such variable from this terminal; otherwise the next `up` would quietly use them

Measuring time with curl

```
curl.exe -s -o NUL -w "%{http_code} in %{time_total}s`n" http://127.0.0.1:8080/health
```

`-w` prints the status and the seconds taken after each request; `` `n `` inside a double-quoted PowerShell string is a newline, so each result gets its own line. This is how every timing in this phase was measured.

Verification

Every check passed in the project on 2026-11-02, with the stack running. The outputs below are from that run; container IDs will differ on every start.

Check 1: the stack starts healthy. `docker compose up --build -d --wait`, then `docker compose ps`:

```
NAME               IMAGE               STATUS                   PORTS
rategate-api-1     rategate-api        Up 4 seconds (healthy)   8000/tcp
rategate-api-2     rategate-api        Up 4 seconds (healthy)   8000/tcp
rategate-api-3     rategate-api        Up 4 seconds (healthy)   8000/tcp
rategate-nginx-1   nginx:1.27-alpine   Up 1 second              0.0.0.0:8080->80/tcp, [::]:8080-
>80/tcp
rategate-redis-1   redis:7             Up 7 seconds (healthy)   6379/tcp
```

All healthy in 10 s. Only nginx shows an arrow (`0.0.0.0:8080->80/tcp`): published. The others list a port without an arrow: private to the stack. (nginx has no health check, so it shows no `(healthy)`.)

Check 2: the copies take turns and share one count. `uv run python scripts/sequential_requests.py`:

```
  1  200  d1d21e5ac434  remaining=9
  2  200  11f76c9c258c  remaining=8
  3  200  1f499cd72640  remaining=7
  4  200  d1d21e5ac434  remaining=6
  ...
 10  200  d1d21e5ac434  remaining=0
 11  429  11f76c9c258c  remaining=0
 12  429  1f499cd72640  remaining=0
```

Three IDs in a repeating order: round-robin works. `remaining` falls 9, 8, 7 … across **different** copies: one shared bucket. Request 11 is refused by a copy that had served only 3 requests itself; it refuses because of what the other copies did.

Check 3: bursts stay exactly at the limit. Default settings (token bucket, capacity 10), 200 at once: `200: 10, 429: 190`, split 3, 4 and 3 across the copies. Then limit 100, nothing refilling, 200 at once:

Algorithm	Run	Allowed	Per copy
Token bucket	1	**100**	33, 33, 34
Token bucket	2	**100**	32, 32, 36
Token bucket	3	**100**	34, 33, 33

Algorithm	Run	Allowed	Per copy
Sliding window log	1	**100**	33, 33, 34
Sliding window counter	1	**100**	33, 34, 33

After `Remove-Item Env:RATEGATE_*` and `docker compose up -d --wait`, each copy had only `RATEGATE_REDIS_URL` set: back to defaults.

Check 4: the real client IP, no spoofing. After `flushall`, one normal request and one with `X-Forwarded-For: 1.2.3.4`. Keys in the stack's Redis:

```
rategate:tb:ip:172.19.0.1
```

One key, your PC's address on Docker's network. Not nginx's address, and not the fake one: both requests used the same bucket.

Check 5: a copy dies. `rategate-api-2` stopped while `/health` was requested every 0.1 s: **50 of 50 answered `200`, slowest 1.01 s.** Then only two IDs answered, until the copy was started again.

Check 6: Redis dies, inside the stack.

Mode	Redis stopped	Redis started again
Fail-open (default)	`200` in 1.007, 1.022, 1.019 s	`200` in 0.010, 0.025, 0.006 s
Fail-closed	`503` in 1.010, 1.007, 1.008 s	`200` in 0.010, 0.008, 0.009 s

`docker compose logs api` showed lines like `api-1  | Redis unavailable, rejecting request (fail-closed): Timeout connecting to server`. The prefix says which copy wrote it. Phase 4's behaviour holds in every copy, now with a 1.0 s detection.

Check 7: scale to 5 and back. `--scale api=5`, `restart nginx`, 50 at once: `200: 10, 429: 40`, five IDs (two got only `429`s: their requests arrived after the bucket was empty). Back at 3 copies: three IDs, exactly 10 allowed.

Check 8: tests and lint. With this machine's `.env` in place:

```
40 passed in 26.93s
40 passed in 26.81s
40 passed in 26.61s
```

plus `40 passed in 26.01s` once more before committing, `7 passed` in 30 runs out of 30, `All checks passed!` and `20 files already formatted`. The 40 are Phase 4's 36, 3 instance-id tests and the late-reply test.

Check 9: clean up. `docker compose down` removed the stack; your Phase 0 Redis answered `PONG`. Committed as `5569b27` and pushed. `.vscode/` was left out on purpose.

Common pitfalls

Most of these were met in this phase; the rest are the usual ways a Docker and nginx setup goes wrong for beginners.

Pitfall	Symptom	Why	Fix
Docker Desktop not running	`failed to connect to the docker API ... dockerDesktopLinuxEngine`	The Docker engine is off	Start Docker Desktop and wait for "Engine running"
No `zone` in the upstream	Every request answered by the same copy	Each nginx worker keeps its own round-robin count	`zone rategate_api 64k;`
Default `proxy_connect_timeout`	One request hangs for about 40 s after a copy dies	nginx waits up to 60 s for a vanished address	`proxy_connect_timeout 1s;`
Not restarting nginx after scaling	New copies get no traffic	nginx looked up `api` only at startup	`docker compose restart nginx`
nginx's `resolve` option	Occasional `502` while a copy is down	It sometimes gave up instead of trying another copy	Plain `server api:8000;` and restart after scaling
nginx appends to `X-Forwarded-For`	Clients escape their limit with fake addresses	The header is client-controlled	`proxy_set_header X-Forwarded-For $remote_addr;`
uvicorn ignores the header	Every client shares nginx's bucket	The app only sees nginx's address	`--proxy-headers --forwarded-allow-ips`
Publishing the copies' port	Clients can skip nginx and fake their address	`"*"` trusts anyone who can connect	Publish only nginx
uvicorn on `127.0.0.1` in a container	nginx gets `502 Bad Gateway`	That address only accepts connections from inside the container	`--host 0.0.0.0`
`localhost` as the Redis address in a container	The copies can't reach Redis	Inside a container, `localhost` is the container itself	The service name: `redis://redis:6379/0`
No `.dockerignore`	Slow builds; Windows `.venv` and `.env` copied into the image	Docker sends the whole folder to the build	`.dockerignore`
Copying `app/` before installing libraries	Every code change reinstalls everything	The cache is lost from the first changed layer onwards	Lock files, `uv sync`, then `app/`
Forgetting `--build` after a code change	The change doesn't appear	Compose reuses the old image	`docker compose up --build -d --wait`
`$env:RATEGATE_*` left set	A later `up` quietly uses odd limits	Compose passes them into the copies	`Remove-Item Env:RATEGATE_*`
Retrying after a timeout	Requests counted twice	The script may have run before its reply was late	Retry connection errors only
A fake that skips the real behaviour	A test fails (or passes) for a reason that can't happen in reality	The fake left an unread reply on the connection	Copy the library's real behaviour; read its source
Production timeouts on a Windows test machine	Tests fail now and then with `Timeout reading` / `Timeout connecting`	Docker Desktop's port forwarding adds delays under bursts	This machine's own values in `.env`

Pitfall	Symptom	Why	Fix
A local connect timeout of 2 s or more	The outage-speed test fails	Windows takes about 2 s to refuse, and refusals are retried	Keep it under 2 s (1.5 s)
Committing `.env`	Machine-specific values, later secrets, end up in git and CI	It's just a file in the folder	`.env` in `.gitignore`
`flushall` on the wrong Redis	Data you wanted is gone	Two Redis servers exist	`docker compose exec redis ...` targets the stack's
Port 8080 already in use	`port is already allocated`	Another program has it	Stop it, or change `"8080:80"`
Re-running a flaky test until it passes	A real bug stays hidden	"Sometimes fails" was treated as luck	Collect the errors; a wrong number is a clue (problem 5)

Check your understanding

Answer each in your own words first, then compare. Questions 11 to 15 are good interview stories.

1. What is the difference between vertical and horizontal scaling, and what does horizontal scaling require of the app? Vertical means a bigger machine; horizontal means more identical copies. Any copy must be able to answer any request, so the copies must be stateless: important data lives in shared storage (Redis), not in each copy's memory.

2. What is the difference between an image and a container? An image is a packaged, read-only snapshot of a system and app. A container is a running instance of it. One image ran as three containers here.

3. Why does the Dockerfile copy `pyproject.toml` and `uv.lock` before `app/`? Layers are cached. Installing libraries is slow and rarely needed; code changes all the time. With this order a code change rebuilds only the last layer: 1.6 s.

4. How does `server api:8000;` become three servers? When nginx starts, it asks Docker's DNS for `api`, which answers with the address of every replica.

5. Why did every request go to one copy before `zone` was added? nginx has 16 worker processes, and each kept its own round-robin count starting at the first copy. `zone` makes them share one count.

6. If each copy counted requests in its own memory, how many of 400 requests would a limit of 100 allow with 3 copies, and why does RateGate allow 100? 300: each copy allows its own 100. RateGate counts in one shared Redis with atomic scripts, so all copies see one count.

7. Why does the API need `--proxy-headers`, and why must nginx overwrite `X-Forwarded-For` rather than append to it? Without it, every request seems to come from nginx, so all users share one bucket. Appending keeps whatever address the client sent, so a client could fake a fresh address on every request and never be limited.

8. When is `--forwarded-allow-ips "*"` safe? Only when nothing but the proxy can reach the app, as here, where the copies' port is never published. Otherwise list the proxy's address.

9. Why is the `X-Instance-Id` middleware added after the rate-limit middleware? The middleware added last is the outermost, so it also labels the `429` and `503` answers that the rate-limit middleware returns without reaching a route.

10. What happens when a copy dies, and what made it fast? nginx fails to connect, sends that request to another copy, and skips the dead one for 10 s. `proxy_connect_timeout 1s` limits the failed attempt to 1 s instead of up to 60 s; without it one request took 38.8 s.

11. Why was a retry after a timeout a bug? A timeout means the reply was late, and the script may already have run. Retrying ran it again, counting the request twice: 20 late requests left 40 entries. Only connection errors, where the command never reached Redis, are safe to retry.

12. How was that bug noticed? By a test that failed with a wrong number, `88 == 100`, instead of an error. Treating it as information rather than bad luck led to the cause.

13. Why did the first regression test print `[1, 9, 0, 60000] == 1`? Its fake timeout left the script's reply unread on the connection, and the next command read it. A real redis-py timeout closes the connection first; the fixed test does the same.

14. Why do the tests on this PC use different timeouts from the containers, and why is that not cheating? Replies from Windows through Docker Desktop's port forwarding reached 269 ms, past the 0.25 s production timeout, while inside Docker the slowest was 47 ms. Timeouts belong to an environment, which is why they are settings. The containers and CI still use and test the real values.

15. Why must this PC's connect timeout stay under 2 s? Windows takes about 2 s to refuse a connection, and a refusal is a connection error, which is retried once. Above 2 s, an outage would wait for two refusals instead of one timeout.

16. Why did the containers notice a stopped Redis no faster than Windows did? A stopped container's address disappears, so nothing answers, not even with a refusal. The connect timeout ends the wait in both places: 1.0 s.

Phase 6: Continuous integration with GitHub Actions (done 2026-11-03)

Every push to GitHub is now checked automatically on GitHub's own machines: lint, formatting, all 40 tests against a real Redis, and the full Phase 5 stack with a burst of requests through nginx. The result shows as a green check or red cross on each commit, and as a badge at the top of the README, which now reads **"CI – passing"**. The work is committed as `08ec5bd` and pushed.

This proves the resume line *"GitHub Actions CI"*.

Results, measured on GitHub between 18:53 and 19:05 UTC on 2026-11-02 (shortly after midnight on 2026-11-03 in India):

Result	Evidence
The workflow passes	4 green runs in a row on a trial branch, then green on `main`
It's quick	Each job takes 29 to 91 s; the two jobs run at the same time
It catches a broken test	A deliberately wrong assertion turned the run red at "Run tests", exit code 1
It checks the real timeouts	CI has no `.env`, so the 40 tests pass with the production 0.25 s / 1 s
It checks Phase 5's claim on every push	200 requests through nginx, limit 100: exactly 100 allowed, across 3 copies
The badge works	GitHub's badge for `main` reads "CI – passing"

The very first run failed in 4 seconds, before running any of our steps, because a version of a tool didn't exist under the name used. The full story, including how that was tracked down without being logged in to GitHub, is below.

What changed in the project:

File	Change
`.github/workflows/ci.yml`	New: the workflow, with two jobs
`scripts/concurrent_burst.py`	`--expect-allowed` and `--expect-instances`: exit with an error when the numbers differ (13 lines)
`README.md`	CI badge at the top; a paragraph on what CI checks
`ROADMAP.md`	Phase 6 ticked; progress-log row
`docs/PHASE_6_GUIDE.md`	New: the step-by-step guide

No application code changed. Phase 6 adds checks around the code, not features inside it.

Theory 1: What CI is, and why

Continuous integration (CI) means that every time code is pushed, a fresh machine downloads it, installs everything from scratch, and runs the checks. If anything fails, everyone sees it straight away.

Until Phase 6, RateGate's checks only ran when you remembered to run them, on your PC, with whatever happened to be installed there. That has three weaknesses:

Without CI	With CI
"I forgot to run the tests before pushing"	Every push is checked, every time
"Works on my machine", because of something only your machine has	A clean machine each time: anything missing from git makes CI fail
Nobody can tell whether `main` works	A green check or red cross on every commit, and a badge in the README

The middle row turned out to matter directly. In Phase 5 your PC got a `.env` file with looser timeouts, because Docker Desktop on Windows is slow. CI doesn't have that file, so it tests the production timeouts. If a future change only worked with the looser values, CI would catch it.

Why it matters for this project. In real teams, CI is what stops one person's mistake from breaking everyone else's work. For a resume project, it's also visible proof: a reviewer sees the badge and knows the tests exist, run, and pass, without cloning anything.

Theory 2: GitHub Actions vocabulary

GitHub Actions is GitHub's built-in CI service, free for public repositories. It reads **workflow** files from the folder `.github/workflows/` in your repository, and runs them when the events they name happen.

Word	Meaning	In RateGate
Workflow	One YAML file describing what to run and when	`.github/workflows/ci.yml`, named "CI"
Trigger (`on:`)	The events that start the workflow	Every `push`, to any branch, and every `pull_request`
Run	One execution of the workflow, for one commit	One per push; 7 so far
Job	A group of steps that runs on its own fresh machine. Jobs run **at the same time** unless told otherwise	`test` ("Lint and test") and `stack` ("Docker stack")
Runner	The machine a job runs on: created for the job, thrown away after it	`ubuntu-latest`, a Linux virtual machine GitHub provides
Step	One command (`run:`) or one ready-made action (`uses:`), run in order	Install uv, lint, run tests ...
Action	A reusable step someone published, referenced as `owner/name@version`	`actions/checkout@v7`, `astral-sh/setup-uv@v10.3.3`
Service container	A container GitHub starts next to a job, for the job to use	`redis:7`

Pass or fail. A job passes when every step finishes with **exit code 0** (Theory 4). The first step that fails stops the job: the steps after it are skipped unless they say otherwise. One failed job turns the whole run red.

Pull requests. A pull request asks to merge one branch into another. With `pull_request` as a trigger, every proposed change is checked before it is merged. RateGate has no pull requests yet, but the trigger costs nothing and is standard.

Theory 3: A fresh machine has nothing

Each job starts on a brand-new Linux machine that has common tools (git, Docker, curl) but none of RateGate's. So the workflow repeats, automatically, what you did by hand in Phase 0, and adds Redis as a service container.

What each job has to do, in order:

1. **Get the code.** `actions/checkout` clones exactly the commit being checked.
2. **Install uv.** `astral-sh/setup-uv`, pinned to `0.13.6`, the version on your PC.
3. **Install Python and the libraries.** `uv sync --locked`. uv reads `.python-version` (3.12), downloads that Python, and installs exactly the versions in `uv.lock`.
4. **Start Redis** (lint-and-test job): a service container.
5. **Run the checks**: the same commands you run locally.

`--locked` makes `uv sync` **fail** if `uv.lock` doesn't match `pyproject.toml`, instead of quietly updating it. That catches a classic mistake: adding a library with `uv add`, then committing `pyproject.toml` but forgetting `uv.lock`. Locally everything works; in CI the lock file is out of date and the run turns red.

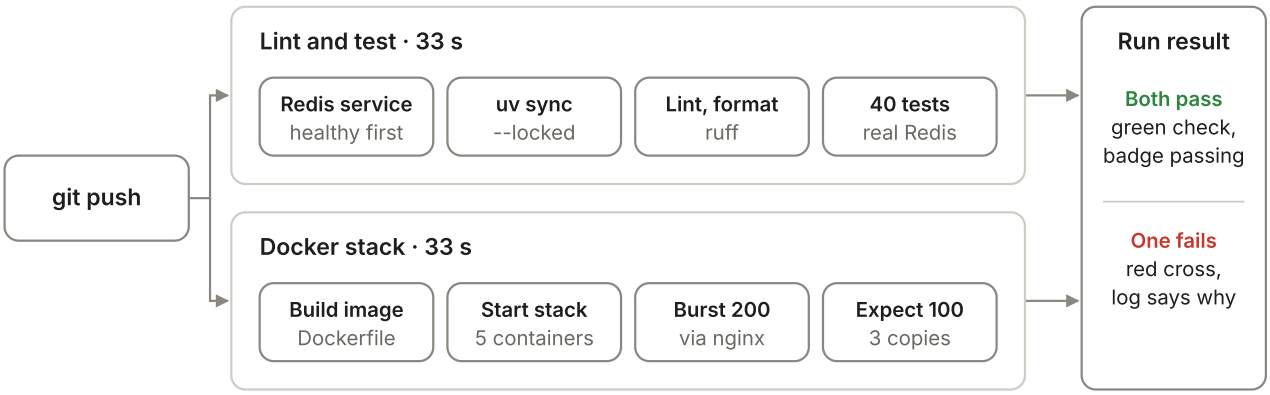
Nothing from your PC is there: no `.venv`, no Phase 0 Redis, no `.env`. Only what is in git.

Service containers. The tests expect Redis at `localhost:6379`. In the workflow:

```
    services:
      redis:
        image: redis:7
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 2s
          --health-timeout 3s
          --health-retries 15
```

- GitHub starts a `redis:7` container **before** the job's steps and removes it afterwards. The step list shows this as "Initialize containers" (5 s) and "Stop containers".
- `ports: - 6379:6379` publishes it on the runner's port 6379, so the tests' default address works unchanged. No CI-only settings are needed.
- `options:` are extra flags for `docker run`. The four `--health-*` flags give the container a health check, and GitHub **waits until it is healthy** before running the first step. Without it, the first tests could start before Redis accepts connections and fail at random.
- `>-` is YAML for "join the following lines into one line": the options are one long string, split for readability.

The Docker-stack job needs no service container: `docker compose` starts its own Redis, exactly as on your PC.



A failed step stops its own job (later steps skipped); the other job carries on, but the run is red.

What happens on every push · two jobs in parallel

One push starts one run. Its two jobs start together on separate machines, and the run is green only when both finish without a failed step.

Theory 4: Exit codes, and making a check able to fail

CI doesn't read a program's output; it only looks at its **exit code**. So any check that should be able to turn CI red must end with a non-zero exit code when something is wrong. The Phase 5 burst script couldn't, so Phase 6 taught it to.

What an exit code is. When any program finishes, it hands the operating system one number: **0 means success**, anything else means failure. You rarely see it, but every tool relies on it:

- `pytest` exits 0 when all tests pass and 1 when any fails.
- `ruff check` exits 1 when it finds a problem.
- A Python script exits 0 when it reaches the end, and 1 if it crashes with an uncaught error.

In PowerShell the last program's exit code is in `$LASTEXITCODE`; in bash it is `$?`.

The problem. `scripts/concurrent_burst.py` printed its results (`200: 100`, `429: 100`) and then always exited 0, whatever the numbers were. A human reading the output would spot `200: 300`, but CI would report it as a success. A check that only prints can never fail a CI job.

The fix. Two optional options, `--expect-allowed 100 --expect-instances 3`. When given, the script compares the real result with them and, if anything differs, ends with:

```
raise SystemExit("FAILED: expected 99 allowed, got 100; expected 4 instances, got 3")
```

`SystemExit` with a text message prints the message and exits with code **1**. Measured locally both ways: the correct expectations gave exit code 0, and deliberately wrong ones (99 and 4) printed that `FAILED` line and gave exit code 1.

Without the options the script behaves exactly as before, so nothing that used it changes. This is the same lesson as Phase 4's Mistake 2, seen from the other side: there, a test passed when it should have failed; here, a script could never fail at all.

Theory 5: Pinning versions, and version tags

Every tool in the workflow is fixed to a version, so a red run means *your* change broke something, not that a tool changed overnight. How you name a version depends on what its publisher provides, and that was the cause of the first failed run.

What is pinned:

Thing	Pinned by	Version
The checkout action	Tag in the workflow	`actions/checkout@v7`
The uv installer action	Tag in the workflow	`astral-sh/setup-uv@v10.3.3`
uv itself	`version:` input	`0.13.6`, the same as your PC
Python	`.python-version`	3.12
Every library	`uv.lock`, enforced by `--locked`	Exact versions
Container images	`docker-compose.yml`, the service, the Dockerfile	`redis:7`, `nginx:1.27-alpine`, `python:3.12-slim`

The one exception is `runs-on: ubuntu-latest`: GitHub moves it forward over time. (Its runs currently carry a notice that it moves to Ubuntu 26 from 19 October 2026.) Nothing in RateGate's workflow depends on the Ubuntu version.

Version tags. An action is referenced by a **git tag**: a name attached to one commit in the action's own repository. Publishers usually provide two kinds:

- **Full versions**, such as `v7.0.1`: always the same code.
- **Major tags**, such as `v7`: moved by the publisher to the newest `v7.x.y`. You get bug fixes without breaking changes.

Major tags are the common way to reference actions, but only if the publisher maintains them. `actions/checkout` does (`v7`). `astral-sh/setup-uv` stopped after `v7`: from `v8` onwards it publishes full versions only. So `astral-sh/setup-uv@v10` doesn't exist, while `astral-sh/setup-uv@v10.3.3` does. That is exactly what the first run tripped over (The full story, below).

The full story

Phase 6 took 7 runs on GitHub. One failed by mistake, one failed on purpose, five passed. Here is each one in order: what was expected, what happened, how it was investigated, and what it led to.

All 7 runs at a glance (times in UTC, 2026-11-02):

#	Time	Branch	What was being tried	Result
1	18:53	`ci-trial`	The first version of the workflow	**Red** in 4 s: an action version that doesn't exist
2	18:55	`ci-trial`	Same, with the version fixed	Green: 53 s and 50 s
3	18:57	`ci-trial`	Repeat, nothing changed	Green: 29 s and 32 s
4	18:57	`ci-trial`	Repeat	Green: 91 s and 45 s
5	18:57	`ci-trial`	Repeat	Green: 31 s and 35 s
6	18:59	`ci-trial`	A deliberately broken test	**Red**, as intended, at "Run tests"

#	Time	Branch	What was being tried	Result
7	19:04	`main`	The finished Phase 6	Green: 33 s and 33 s

The two times in each green row are the "Lint and test" job and the "Docker stack" job.

Before the first push: two decisions

A trial branch, not `main`. A CI workflow can only really be tested by GitHub, so the first versions had to be pushed somewhere. Pushing experiments to `main` would put broken commits in your project's history and could turn the badge red. Instead, the work was done in a separate **clone** (a second copy of the repository, in the scratch folder) on a temporary branch, `ci-trial`. Your project folder wasn't touched until the workflow was proven.

Watching without being logged in. The usual tool for following runs, GitHub's `gh` command, isn't installed on your PC. But your repository is public, so GitHub's **REST API** (web addresses that return data instead of pages) answers without a login. A small Python script asked `https://api.github.com/repos/diyalibiswas1998/RateGate/actions/runs?head_sha=<commit>` every 20 seconds until the run finished, then listed every job and step with its result. Full logs need a login, but each failed job's **annotations** (short error notes) are public, and they were enough every time.

Checked before writing: the latest versions of both actions (from GitHub's API: checkout `v7.0.1`, setup-uv `v10.3.3`), and the inputs setup-uv v10 accepts, read from its `action.yml` through the API. It accepts `version` and caches by default. (The direct file address, `raw.githubusercontent.com`, didn't connect from this machine, so the API route was used.)

Run 1: red in 4 seconds

Expected. Both jobs green, or at worst a failing test.

What happened. Both jobs failed at the very first step, **"Set up job"**, 2 to 4 seconds after starting. None of our steps ran.

How it was investigated. A failure at "Set up job" means GitHub couldn't even prepare the job: our commands never started. The annotations said exactly why:

```
Unable to resolve action `astral-sh/setup-uv@v10`, unable to find version `v10`
```

The workflow said `@v10` by analogy with `actions/checkout@v7`. Listing both repositories' tags through the API settled it:

Repository	Short major tags	Newest full versions
`actions/checkout`	`v7`, `v6`, `v5` …	`v7.0.1`, `v7.0.0`
`astral-sh/setup-uv`	only up to `v7`	`v10.3.3`, `v10.3.2`, `v10.3.1` …

setup-uv stopped publishing major tags after `v7`, so `v10` had never existed.

Fix. `astral-sh/setup-uv@v10.3.3`, in both jobs.

Lesson. The same as Phase 4's Mistake 3: check that something exists instead of assuming it follows another tool's pattern. The checking took one API call; the assumption cost one red run.

Run 2: green

Both jobs passed: "Lint and test" in 53 s, "Docker stack" in 50 s. The step timings showed where the time goes:

Job	Step	Time
Lint and test	Initialize containers (start Redis, wait until healthy)	5 s
Lint and test	Install dependencies (`uv sync --locked`)	2 s
Lint and test	Lint, Check formatting	under 1 s each
Lint and test	**Run tests**	**17 s**
Docker stack	**Start the stack** (build the image, start 5 containers, wait until healthy)	**19 s**
Docker stack	Burst through nginx	2 s

Two things this confirmed:

- **The production timeouts work on a normal machine.** CI has no `.env`, so all 40 tests ran with the 0.25 s reply and 1 s connect timeouts that failed so often from Windows in Phase 5. That supports Phase 5's conclusion: the problem was Windows' path into Docker, not the timeouts.
- **The tests are faster on Linux:** 17 s against about 27 s on your PC. On Linux a closed port refuses a connection instantly, so the outage tests (which connect to the dead port 6399) don't wait for Windows' 2-second refusal.

Runs 3 to 5: is it stable?

One green run could be luck, and Phase 5 showed how timing problems can come and go. Three empty commits (`git commit --allow-empty`) triggered three more identical runs. All three were green.

One detail: in run 4 the test job took **91 s** instead of about 30. The step list showed nothing failing; shared CI machines simply vary in speed. That's normal, and why `timeout-minutes: 10` leaves plenty of room.

Run 6: red, on purpose

The roadmap's "done when" asks for proof that a broken test turns CI red. A check that has never failed might be unable to (Theory 4).

What was changed, on the trial branch only, in `tests/test_instance_id.py`:

```
    assert Settings(instance_id="api-7").instance_id == "api-8"
```

Expected. "Lint and test" fails at "Run tests"; "Docker stack" passes, because it doesn't run the test suite.

What happened, exactly that:

- "Lint and test" failed at step 8, **Run tests**, with `Process completed with exit code 1`. The steps after it were skipped, except the cleanup ones.
- "Docker stack" passed: the jobs are independent.
- The run as a whole turned **red**.

The same test run locally showed the report the CI log contains:

```
>       assert Settings(instance_id="api-7").instance_id == "api-8"
E       AssertionError: assert 'api-7' == 'api-8'
FAILED tests/test_instance_id.py::test_instance_id_defaults_to_the_hostname
```

Cleaning up

The trial branch had done its job, so it was deleted from GitHub with `git push origin --delete ci-trial`. Afterwards, `git ls-remote --heads origin` listed only `main`. The trial runs stay in the Actions history, because deleting runs needs a login. That's harmless, and run 6 is a useful record that the check really can fail.

Nothing from the trial reached your project except the proven workflow and script. In particular, the broken test never left the trial branch: copying files into the project, only `ci.yml`, `concurrent_burst.py`, `README.md` and the guide changed.

Run 7: `main`

Before pushing, every check CI runs was run locally first: `uv sync --locked` ("Checked 32 packages", nothing to change), ruff (`All checks passed!`, `20 files already formatted`), `40 passed`, and the stack with the burst (`200: 100`, three copies, exit code 0). Then the Phase 6 commit `08ec5bd` was pushed to `main`.

Both jobs passed in 33 s each. Finally the badge itself was fetched from GitHub: its SVG title reads `CI - passing`.

What the story shows

Moment	What helped
Run 1	Reading the exact error before guessing; checking the publisher's tags
Runs 3–5	Repeating a green result before trusting it
Run 6	Proving the check can fail, without breaking `main`
Run 7	Running everything locally first, so `main` was green on its first try

How it was built

The order follows one idea: prove each piece where it is cheapest to fail. Locally first (seconds), then on a throwaway branch on GitHub (minutes), and only then on `main`.

Step	What	Why at this point
1	Pass/fail options for the burst script; tested both ways locally	CI can only fail on an exit code (Theory 4)
2	Write `.github/workflows/ci.yml`; check its YAML parses with the intended jobs and steps	Catch typos before GitHub does
3	Push to a trial branch from a scratch clone; watch the run	Only GitHub can really test a workflow; `main` stays clean
4	Fix (the setup-uv tag), repeat until green, then 3 repeat runs	A single green run could be luck
5	Break a test on the trial branch; see red; delete the branch	Prove the check can fail
6	Write the guide from what actually happened	Every number in it measured
7	Copy the proven files into the project; README badge; ROADMAP	Same files that went green
8	Run every CI check locally; commit; push to `main`; watch	`main` green on its first try

The guide (`docs/PHASE_6_GUIDE.md`) gives you the same steps for doing it yourself, including the red check on a temporary branch of your own (`ci-red-check`).

Code 1: The burst script's pass/fail options

Thirteen lines in `scripts/concurrent_burst.py` let it fail on wrong numbers. Without the new options it behaves exactly as before.

Two new options, after `--api-key`:

```
    parser.add_argument("--api-key", default=f"burst-{uuid4().hex[:8]}")
    # For automated checks (CI): exit with an error if the result is different.
    parser.add_argument("--expect-allowed", type=int, help="exact number of 200s")
    parser.add_argument("--expect-instances", type=int, help="number of copies")
    args = parser.parse_args()
```

- **No `default=`**, so when an option isn't given its value is `None`, meaning "don't check".
- **`type=int`** makes `argparse` convert `"100"` from the command line into the number 100, and refuse something like `"abc"` with a clear message.
- **`help=`** text appears in `uv run python scripts/concurrent_burst.py --help`.

The check, at the end of `main()`, after the per-instance summary is printed:

```
    problems = []
    if args.expect_allowed is not None and counts[200] != args.expect_allowed:
        problems.append(f"expected {args.expect_allowed} allowed, got {counts[200]}")
    if args.expect_instances is not None and len(by_instance) != args.expect_instances:
        problems.append(
            f"expected {args.expect_instances} instances, got {len(by_instance)}"
        )
    if problems:
        raise SystemExit("FAILED: " + "; ".join(problems))
```

- **`is not None`**: only check what was asked for. Writing `if args.expect_allowed and ...` would be a bug, because `--expect-allowed 0` would then be skipped (0 counts as false in Python).
- **`counts[200]`**: a `Counter` returns 0 for a key it has never seen, so this works even if nothing at all was allowed.
- **`len(by_instance)`**: how many different `X-Instance-Id` values answered.
- **Collect, then report.** All problems go into a list first, so one run reports everything that's wrong instead of stopping at the first.
- **After the printout.** The results are printed before the check, so a failing CI log still shows the full numbers.
- **`raise SystemExit("FAILED: ...")`** prints the message and exits with code 1 (Theory 4).

Code 2: `ci.yml`, line by line

The whole workflow is 83 lines, about a quarter of them comments and blank lines. It lives at `.github/workflows/ci.yml`; GitHub finds it there automatically.

```
# Runs on every push and pull request. Both jobs must pass for a green check.
name: CI

on:
  push:
  pull_request:
```

```
jobs:
  test:
    name: Lint and test
    runs-on: ubuntu-latest
    timeout-minutes: 10

    # A real Redis for the tests, reachable at localhost:6379 like on your machine.
    services:
      redis:
        image: redis:7
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 2s
          --health-timeout 3s
          --health-retries 15

    steps:
      - uses: actions/checkout@v7

      # The same uv version as on the development machine.
      - uses: astral-sh/setup-uv@v10.3.3
        with:
          version: "0.13.6"

      # --locked: fail if uv.lock is out of date with pyproject.toml.
      - name: Install dependencies
        run: uv sync --locked

      - name: Lint
        run: uv run ruff check .

      - name: Check formatting
        run: uv run ruff format --check .

      - name: Run tests
        run: uv run pytest -v

  stack:
    name: Docker stack (3 copies behind nginx)
    runs-on: ubuntu-latest
    timeout-minutes: 10

    steps:
      - uses: actions/checkout@v7

      - uses: astral-sh/setup-uv@v10.3.3
        with:
          version: "0.13.6"

      # Only the script's dependency (httpx); the API itself runs in the containers.
      - name: Install dependencies
        run: uv sync --locked

      # Limit 100 with nothing refilling, so the expected result is exact.
      - name: Start the stack
        env:
          RATEGATE_CAPACITY: "100"
          RATEGATE_REFILL_RATE: "0.001"
        run: docker compose up --build -d --wait
```

```
      - name: Burst through nginx stays exactly at the limit
        run: >-
          uv run python scripts/concurrent_burst.py
          --url http://127.0.0.1:8080/api/resource --total 200
          --expect-allowed 100 --expect-instances 3

      - name: Show container logs
        if: failure()
        run: docker compose logs

      - name: Stop the stack
        if: always()
        run: docker compose down
```

The top:

- `name: CI`: the workflow's name, shown in the Actions tab and in the badge.
- `on:` **with** `push:` **and** `pull_request:`: run on every push to any branch, and on every pull request. A key with nothing after it means "with default settings". Any branch matters: that's how the trial branch got its runs.

The `test` job:

- `name: Lint and test`: the display name shown next to each commit.
- `runs-on: ubuntu-latest`: a fresh Linux machine for this job.
- `timeout-minutes: 10`: stop the job if it takes longer. The default is 6 hours; a hung test shouldn't run that long. The slowest job so far took 91 s.
- `services: redis:`: Theory 3.
- `steps:`: run in order. Each item starts with `-`, and is either `uses:` (an action) or `run:` (a shell command).
- `uses: actions/checkout@v7`: clone the commit being checked.
- `uses: astral-sh/setup-uv@v10.3.3` **with** `version: "0.13.6"`: install uv. `with:` passes inputs to an action. The version is in quotes so YAML reads it as text; unquoted, a value like `0.10` could be turned into the number 0.1.
- `name: Install dependencies` and `run: uv sync --locked`: `name:` is only the label shown on GitHub; `run:` is the command.
- **Lint, Check formatting, Run tests**: the same commands you run locally. `-v` lists every test by name, so a log shows which one failed.

The `stack` job:

- **No** `services:`: Compose starts its own Redis.
- `uv sync --locked` installs `httpx` for the burst script; the API itself runs inside the containers.
- `env:` **on "Start the stack"**: environment variables for that step only. Compose passes them into the API copies (the pass-through list in `docker-compose.yml`, Phase 5): limit 100 with no refill, so the expected answer is exactly 100.
- `docker compose up --build -d --wait`: the same command as on your PC. `--wait` returns only when every container is healthy, so the burst never hits a half-started stack.
- `run: >-` **over three lines**: folded into one command, just for readability.
- `if: failure()`: run this step **only** if an earlier step failed, printing every container's log so a red run explains itself.
- `if: always()`: run this step whatever happened, so the stack is always stopped.

Why `ruff` doesn't check this file. ruff checks Python. YAML mistakes show up when GitHub reads the file, as a failure at "Set up job" or an "Invalid workflow file" error. That's why the YAML was parsed locally before the first push.

Command reference

Every command used in Phase 6, what it does and why. PowerShell from `D:\RateGate` unless noted.

Git: branches for experiments

Command	What it does	Why
`git switch -c ci-red-check`	Creates a new branch and switches to it	Experiment without touching `main`
`git commit -am "..."`	Commits every change to files git already tracks (`-a`), with a message (`-m`)	Quick commit of the broken test
`git commit --allow-empty -m "..."`	Makes a commit that changes nothing	Trigger another identical CI run (runs 3 to 5)
`git push -u origin ci-red-check`	Sends the branch to GitHub; `-u` remembers where it goes	Start a CI run on that branch
`git switch main`	Goes back to `main`	Leave the experiment
`git branch -D ci-red-check`	Deletes the branch locally (`-D` even if unmerged)	Clean up
`git push origin --delete ci-red-check`	Deletes the branch on GitHub	Clean up
`git ls-remote --heads origin`	Lists the branches GitHub has	Confirm only `main` is left
`git clone <path> <folder>`	Makes a full second copy of a repository	The scratch copy used for the trial

Checking before pushing (the guide's Step 3)

Command	Why
`uv sync --locked`	Fails if `uv.lock` doesn't match `pyproject.toml`, exactly as in CI
`uv run ruff check .` / `uv run ruff format --check .`	Lint and formatting, as in CI
`uv run pytest -q`	The tests (with this PC's `.env` timeouts)
`uv run python scripts/concurrent_burst.py ... --expect-allowed 100 --expect-instances 3`	The stack check, as in CI
`echo "exit code: $LASTEXITCODE"`	Shows the last program's exit code: 0 passed, anything else failed

Looking at GitHub without logging in (public repositories only)

Address	What it returns
`https://api.github.com/repos/diyalibiswas1998/RateGate/actions/runs?head_sha=<commit>`	The runs for one commit, with status and result
`…/actions/runs/<run id>/jobs`	Each job, its steps, and their results and times

Address	What it returns
`…/check-runs/<job id>/annotations`	The short error notes of a job, such as `unable to find version v10`
`https://api.github.com/repos/astral-sh/setup-uv/tags`	An action's tags: which versions exist
`https://github.com/diyalibiswas1998/RateGate/actions/workflows/ci.yml/badge.svg`	The badge image itself; its title says `CI - passing` or `CI - failing`

In everyday use you don't need any of these: the **Actions** tab on your repository shows the same things, including full logs, once you're logged in.

Verification

The roadmap's "done when" is met: a pushed commit on `main` shows a green check, and a deliberately broken test turned a run red. All checks below are from 2026-11-02 (UTC).

Check	How	Result
1. Green on `main`	Run 7, commit `08ec5bd`	Both jobs passed, 33 s each
2. Badge	The badge SVG from GitHub	`CI - passing`
3. Stable	Runs 2 to 5, identical code	4 green out of 4
4. Can fail	Run 6, broken assertion on a trial branch	Red at "Run tests", exit code 1; the stack job unaffected
5. Real timeouts	"Run tests" in CI, with no `.env`	Passed, so all 40 tests pass with 0.25 s / 1 s
6. Phase 5's claim	"Burst through nginx stays exactly at the limit"	Passed: exactly 100 of 200, 3 copies
7. Lock file in sync	"Install dependencies" with `--locked`	Passed
8. Local run before pushing	Step 3 of the guide	`Checked 32 packages`, `All checks passed!`, `20 files already formatted`, `40 passed in 26.35s`, burst `200: 100` across 3 copies, exit code 0
9. `main` untouched by experiments	`git ls-remote --heads origin` after cleanup	Only `main`; the broken test never reached it

Job times across the green runs:

Run	Lint and test	Docker stack
2	53 s	50 s
3	29 s	32 s
4	91 s	45 s
5	31 s	35 s
7 (`main`)	33 s	33 s

Because the two jobs run at the same time, a whole run takes as long as its slower job: usually 30 to 50 s.

Common pitfalls

The first row happened in this phase; the rest are the usual ways a first CI workflow goes wrong.

Pitfall	Symptom	Why	Fix
A version tag that doesn't exist	Job fails at "Set up job" in seconds: `unable to find version`	Not every action publishes short major tags	Check its tags; use one that exists (`v10.3.3`)
Workflow file in the wrong place	Nothing happens on push	GitHub only reads `.github/workflows/*.yml`	That exact path, leading dot included
Tabs in YAML	`Invalid workflow file`	YAML allows only spaces for indentation	Spaces only
Unquoted version numbers in YAML	A wrong tool version is installed	`0.10` reads as the number 0.1	Quote versions: `"0.13.6"`
No health check on the service container	Early tests fail now and then with connection errors	Steps start before Redis is ready	The `--health-*` options
Not committing `uv.lock`	`uv sync --locked` fails in CI	CI only has what's in git	Commit lock-file changes together with `pyproject.toml`
Relying on something only your PC has	Green locally, red in CI	`.env`, a global tool, an untracked file	That's CI doing its job: add what's missing to git or to the workflow
A check script that only prints	CI stays green even with wrong numbers	CI only looks at exit codes	`--expect-...` options with `SystemExit`
`if args.x:` instead of `if args.x is not None:`	`--expect-allowed 0` is silently skipped	0 counts as false	Compare with `None`
Testing CI on `main`	Broken commits in history; red badge for everyone	The badge shows `main`	A temporary branch, deleted afterwards
No `timeout-minutes`	A hang runs for up to 6 hours	That's the default	`timeout-minutes: 10`
Trusting one green run	A flaky problem slips through	Timing problems come and go (Phase 5)	Repeat runs before trusting them
Re-running a red job until it's green	A real problem stays hidden	Failure treated as noise	Read the failing step's log or annotations first

Check your understanding

Answer each in your own words first, then compare.

1. Which problems does CI solve that running the tests locally doesn't? It checks every push, without anyone remembering to; it uses a clean machine, so anything missing from git is caught; and it shows everyone whether `main` works.

2. What's the difference between a workflow, a run, a job and a step? A workflow is the YAML file. A run is one execution of it for one commit. A job is a group of steps on its own fresh machine; jobs run at the same time. A step is one command or action, run in order.

3. Why must the workflow install uv, Python and the libraries itself? Each job starts on a brand-new machine with none of the project's tools. Only what's in git is there.

4. What does `--locked` add to `uv sync`, and which mistake does it catch? It fails instead of updating `uv.lock` when the lock doesn't match `pyproject.toml`. That catches a library added to `pyproject.toml` without committing the new lock file.

5. What is a service container, and why does ours need a health check? A container GitHub starts beside a job for it to use: here Redis on port 6379. The health check makes GitHub wait until Redis answers before the tests start.

6. How can a Python script make a CI step fail? By ending with a non-zero exit code, for example `raise SystemExit("FAILED: ...")`, which exits with 1.

7. Why did run 1 fail at "Set up job", and what does a failure at that step usually mean? `astral-sh/setup-uv@v10` doesn't exist: that publisher only has full versions after v7. A failure at "Set up job" means GitHub couldn't prepare the job at all (a missing action, a bad version, invalid YAML), so none of your commands ran.

8. Why does CI test the production timeouts, while your PC uses looser ones? CI has no `.env`. That's intended: the looser values only work around Docker Desktop's slow path on Windows, and CI proves the real values work on a normal machine.

9. Why are the tests faster in CI (17 s) than on your PC (about 27 s)? On Linux a closed port refuses a connection instantly; on Windows it takes about 2 s. The outage tests connect to a closed port.

10. Why was the broken test pushed to a temporary branch? So `main`, its history and its badge never showed a red run. The branch was deleted afterwards.

11. Why run three more identical CI runs after the first green one? One green run could be luck; timing problems can appear only sometimes. Four green runs in a row are much stronger evidence.

12. What do `if: failure()` and `if: always()` do? `failure()` runs a step only if an earlier one failed (here, printing the container logs). `always()` runs it no matter what (here, stopping the stack).

Phase 7: Load testing with Locust (done 2026-11-03)

RateGate was put under sustained load, and the real numbers are now in the README. At 230 requests per second it answers in 4 ms (p50) and 6–7 ms (p95). It never refuses a client within its limit, and it gives clients far over the limit almost exactly their configured 1 request per second. Under maximum load, adding API copies adds throughput: 740 served requests per second with 1 copy, 1,920 with 3, 2,730 with 6. The work is committed as `176397f` and pushed, and CI is green.

This proves the resume line *"load-tested with Locust"*, with numbers to quote instead of adjectives.

Results (the project run, 2026-11-03; a scratch run before it agreed within about 6%):

Result	Evidence
Correct under load	Polite clients (under their limit): 0 of 1,180 requests refused, for every algorithm
Exact under load	Greedy clients (20 × their limit): allowed 1.02/s (bucket), 1.00/s (log), 0.88/s (counter); configured 1/s
Fast	230 requests/s: p50 4 ms, p95 6–7 ms
The limiter's cost is invisible here	`/health`, which skips Redis, also has a p50 of 4 ms
Scaling out adds capacity	1 copy 740/s (with 4,900 `502` errors); 3 copies 1,920/s; 6 copies 2,730/s, no errors
The bottleneck is the Python copies	About one CPU core each at full load; Redis at 28%

Three traps distorted the first measurements, and each would have put wrong numbers in the README. They're the most useful part of this phase, told in full in The full story:

1. **The test caused a traffic jam.** All clients started at the same moment, so the polite ones looked 8 times slower than they were.
2. **The tool changed what was tested.** `docker compose run` silently recreated the API copies with default settings, so two of the three "algorithm" runs actually tested the token bucket.
3. **The load generator ran out first.** One Locust process hit 100% CPU at about 1,050 requests per second, long before RateGate did.

What changed in the project:

File	Change
`loadtest/locustfile.py`	New: 40 polite clients, 10 greedy clients, 5 health checks
`loadtest/capacity.py`	New: clients that never pause, to find the ceiling
`loadtest/summarize.py`	New: Locust's CSV files turned into the README table
`docker-compose.yml`	A `locust` service, only started on request (a Compose profile)
`.gitignore`	`loadtest/results/`: raw results stay out of git
`README.md`	Load testing section: both tables, the machine, the conditions, how to rerun
`ROADMAP.md`, `docs/PHASE_7_GUIDE.md`	Phase 7 ticked; the step-by-step guide

No application code changed, and nothing was installed: Locust runs from its official Docker image.

Theory 1: What a load test measures

A **load test** sends many requests, from many simulated users, for a sustained time, and records how the system copes. Unit tests ask "is it correct?"; a load test asks "is it still correct, and how fast, **under pressure**?"

Three kinds of number matter:

Number	Meaning	RateGate's question
Throughput: requests per second (RPS)	How much work gets done	How much traffic can 3 copies serve? Does adding copies help?
Latency: response time	How long one request takes, from sending it to receiving the full answer	Does the Redis check slow requests down?
Outcomes	How requests ended: allowed, limited, or errors	Is anyone refused who shouldn't be? Do heavy users get exactly their limit?

For most services the outcomes column is just "error rate". For a rate limiter it's the heart of the test: the *right* requests must be refused, at the *right* rate, while everything is busy.

Two kinds of load test were run:

- **A realistic mix** at a planned rate (230 requests per second). It checks behaviour and normal response times.
- **A capacity test**, where clients never pause. It finds the ceiling: the most requests per second the stack can serve, and which part gives up first.

Why measure at all? The roadmap's intent for this phase is "measure, don't guess". Before Phase 7, any statement about RateGate's speed would have been a guess. Now the README says what was measured, how, and on what machine, so anyone can check it or rerun it.

Theory 2: Percentiles, and why averages lie

Response times are never all the same, so one number can't describe them. **Percentiles** describe the spread: p50 is the typical request, p95 the slow ones a user notices now and then, p99 the worst 1 in 100.

How they work. Sort 100 response times from fastest to slowest:

- **p50** (the median) is the 50th: half the requests were faster.
- **p95** is the 95th: 95% were faster, 5% slower.
- **p99** is the 99th: only 1 in 100 was slower.

Why not the average? A worked example. 95 requests take 5 ms; 5 take 500 ms.

Measure	Value	What it suggests
Average	about 30 ms	"Everything takes about 30 ms": true of **no** request
p50	5 ms	The typical request is fast
p95, p99	about 500 ms	1 request in 20 is very slow

The average blends two very different groups into a number nobody experienced. Percentiles keep them apart, which is why services are judged by them: "p95 under 100 ms" is a common kind of promise.

In RateGate's results, p50 and p95 are reported for every run, and the raw CSV files also have p99. At 230 requests per second, p50 4 ms and p95 6–7 ms means almost every request is fast and the slow ones are only slightly slower.

One warning, from the capacity runs: when a system is full, latency mostly measures **queueing**, the time a request waits behind others. Capacity-run latencies (12–240 ms) say "the system is busy", not "the system is slow". RateGate's response time is the 4 ms from the realistic mix.

Theory 3: Locust, and why a 429 counts as a success

Locust is a free load-testing tool in which simulated users are ordinary Python classes. You describe what one user does; Locust runs as many as you ask for and records every request.

Word	Meaning	In RateGate
User (class)	One kind of simulated client; Locust runs many instances of it	`PoliteClient`, `GreedyClient`, `HealthCheck`, `FloodClient`
Task	What a user does, over and over (`@task`)	`GET /api/resource` with the user's own API key
`wait_time`	The pause between a user's tasks	`constant_throughput(0.5)`: one task every 2 s
`fixed_count`	Exactly this many users of a class	40 polite, 10 greedy, 5 health
`-u` / `-r`	Total users / how many to start per second (the **spawn rate**)	`-u 55 -r 5`
`-t`	How long to run	`-t 70s`
`--headless`	No web page: run, print the results, stop	Repeatable command-line runs
`--reset-stats`	Discard what was recorded while users were still starting	Only full load is measured
`--csv PREFIX`	Write the results to `PREFIX_stats.csv` and related files	Fed to `summarize.py`
`--processes N`	Split the users across N Locust processes	More load than one CPU core can create

Rows by name. Locust groups its statistics by request **name**, with one row each. RateGate's locustfile names each outcome: `polite allowed`, `polite limited`, `greedy allowed`, `greedy limited`, `health (no limiter)`. One run then shows what a plain total would mix together, such as whether *any* polite request was refused.

Why a 429 is a success here. By default Locust treats any answer outside 2xx as a **failure**. For a rate limiter a `429` is the feature working, so counting it as a failure would report "86% errors" for a perfectly behaving system. The locustfile takes over the decision (`catch_response=True`):

- `200` is recorded as a success, named "... allowed".
- `429` is recorded as a success too, named "... limited".
- Anything else (`500`, `502`, a lost connection) is a **failure**, with the status in its message.

So Locust's failure count means "something actually went wrong", and in every realistic-mix run it was 0.

Theory 4: The scenario, and the numbers expected before running

The test mixes three kinds of client, each with a job, and every expected number was worked out before the first run. Without expectations, any output looks plausible.

The limits under test are the defaults: a token bucket of 10 refilling 1 per second, and sliding windows of 10 per 10 seconds. Both mean a long-run rate of **1 request per second per client**.

Client	How many	Pace	What it checks	Expected
Polite	40	1 request every 2 s: half its limit	Are well-behaved clients ever refused by mistake?	**0** limited
Greedy	10	20 requests/s: 20 times its limit	Does an abuser get exactly its limit, not more and not less?	About **1 allowed per second** each; about 95% of its requests limited
Health	5	2 requests/s to `/health`	What does the Redis check cost? `/health` takes the same path without it	Same latency as the others if the limiter is cheap
Flood (capacity test only, a separate file)	20–150	No pause at all	How many requests per second can the stack serve?	Unknown: that's the question

Each client has its own API key, so its own bucket, like separate real customers. Without that, all 55 clients would share one limit.

Expected total load: 40 × 0.5 + 10 × 20 + 5 × 2 = **230 requests per second**. If the measured rate is lower, something can't keep up: either RateGate or the load generator.

Expected share of 429s: the greedy clients send 200 of the 230 requests per second and about 190 of those are refused, so roughly 83–87% of all answers should be `429`. A high share is not a problem; it's the traffic mix.

A subtlety for the token bucket. Each greedy client also starts with a full bucket of 10 tokens. Over a 60-second measurement that adds up to 10 extra requests, so slightly **more** than 1.00 per second is expected for the bucket (measured: 1.02). The sliding window log remembers every request, so it should give exactly 1.00 (measured: 1.00).

Theory 5: Where the load generator runs

Locust runs as a container on the stack's own Docker network, talking to `nginx:80` like any other service. Run from Windows instead, every request crosses Docker Desktop's port forwarding, and the numbers would partly measure Windows rather than RateGate.

Measured both ways, with the same test and the same stack (token bucket):

Locust runs ...	p50	p95	Polite p99	Polite limited	Per greedy client
On Windows, to `localhost:8080`	7 ms	20 ms	46 ms	0	1.02/s
Inside Docker, to `nginx:80`	**4 ms**	**7 ms**	**13 ms**	0	1.02/s

The behaviour is identical: correctness doesn't depend on where the test runs. Only the timing does, and the Windows path adds both delay and variation, as in Phase 5. The README reports the inside-Docker numbers, and says so.

How Locust runs there. A `locust` service in `docker-compose.yml`, using Locust's official image (`locustio/locust:2.47.0`), so nothing is installed on your PC or added to the project's dependencies. It has a Compose **profile**, `loadtest`, so a plain `docker compose up` (and the Phase 6 CI job) never starts it; `docker compose run ... locust` starts it on request. Your `loadtest/` folder is shared with the container, so the locustfiles are read from it and the CSV results land in `loadtest/results/` on your PC.

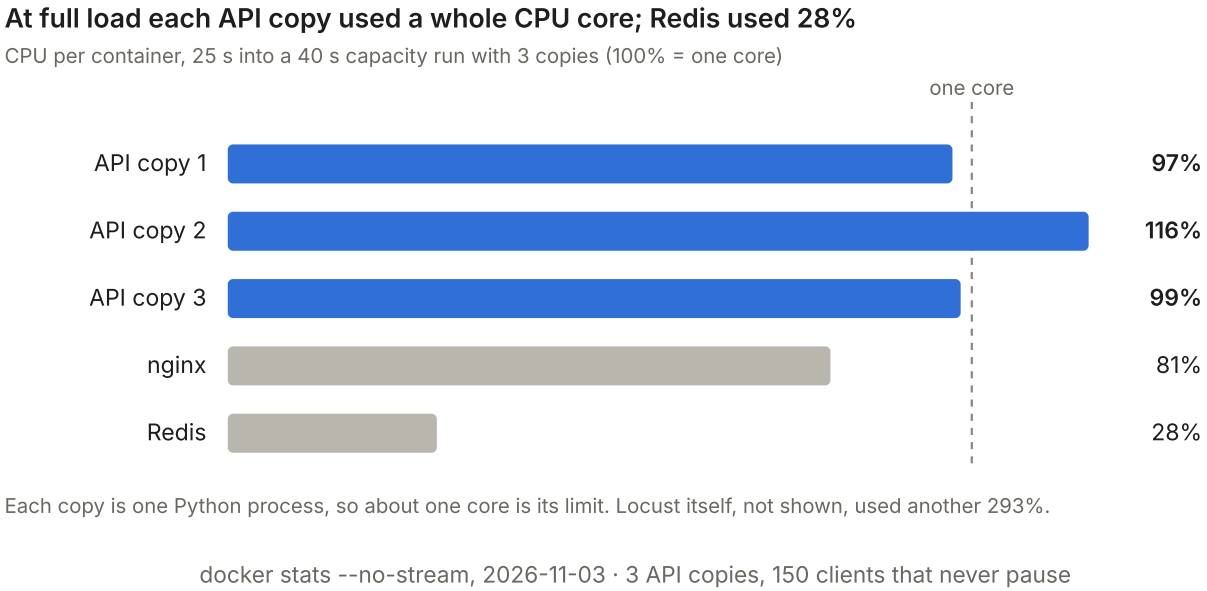
For comparison only, the Windows run used `uvx --from locust==2.47.0 locust ...`. `uvx` runs a tool in a temporary environment without adding it to the project, which suits a one-off check like this.

Theory 6: Finding the bottleneck, and what scaling out bought

Under maximum load, each API copy ran out of CPU while Redis stayed mostly idle. The limit is the Python copies, not the shared Redis, so adding copies raised throughput: exactly what horizontal scaling (Phase 5) promised.

What a bottleneck is. In any chain of parts (Locust → nginx → API copies → Redis), the slowest part limits the whole, like the narrowest point of a bottle. Making any other part faster changes nothing. So the first question in a capacity test is *which* part is full.

How to find it. `docker stats --no-stream` shows each container's CPU use at that moment; 100% means one full CPU core. Here it is, sampled 25 seconds into the project's 3-copy capacity run:



docker stats --no-stream, 2026-11-03 · 3 API copies, 150 clients that never pause

Reading it. Each API copy is **one Python process**, and one Python process can't use much more than one core. All three were there. Redis, which every request also visits, used about a quarter of a core: each check is a tiny Lua script that Redis runs in microseconds. So the copies were full and Redis wasn't, which means more copies should help. A copy can briefly show more than 100% because a process has a few helper threads besides the main one.

What scaling out bought (40 s, 150 clients that never pause, token bucket):

API copies	Served per second	Errors	p95	Compared with 1 copy
1	740	4,900 `502`s	240 ms	–
3	1,920	0	240 ms	2.6 times
6	2,730	0	170 ms	3.7 times

- **1 copy:** overloaded, it couldn't accept new connections within nginx's 1-second connect timeout (from Phase 5). nginx then marked it "temporarily disabled" and, with no other copy to try, answered `502` instantly for a while. A single copy is a capacity limit **and** a single point of failure.
- **3 copies:** 2.6 times the throughput, with no errors at all.
- **6 copies:** more again, but less than double. At that point Locust's 6 processes were using about 5 cores and nginx a full one, so the test itself was near its limit. 2,730 is therefore a **lower bound**, not the stack's ceiling.

One laptop runs everything. Locust, nginx, the copies and Redis all share the same 16 CPU threads. On real servers each part would have its own machine, so these numbers describe this laptop, and the README says which laptop.

What would raise the ceiling further (not done in this project): more copies on more machines, or more uvicorn worker processes per copy (`--workers`). Redis has plenty of room either way.

The full story

Phase 7 was built in a scratch copy first, as usual. Three times the numbers looked fine, or at least plausible, and were wrong. Each time the clue was something that didn't fit, and each time a direct check settled it. Here is every step in order.

The key moments at a glance:

#	What was run	What it showed	What it led to
1	First trial, Locust on Windows, all users started at once	Polite clients 47 ms, greedy 8 ms	Something is distorting the timing
2	Same test inside Docker	Polite still 33 ms	Not Windows
3	One client alone, with and without pauses	2–5 ms everywhere	Not the server, not idle connections
4	Start 5 users per second instead of 55	Polite 32 ms → 6 ms	**Trap 1:** the test made its own traffic jam
5	Three algorithms, 70 s each	Identical rows	Looked plausible, but was wrong (see 8)
6	Capacity, one Locust process	~1,050/s with 20 or 100 users; CPU warning	**Trap 3:** the generator is full
7	Capacity, 4 Locust processes	~2,050/s; API copies at 100% CPU	The copies are the bottleneck: test scaling
8	Scaling with 1, 3, 6 copies	Wrong container counts in the CPU readings	**Trap 2:** `docker compose run` had recreated the copies
9	Everything again with `--no-deps`, with proof	Algorithms now differ; scaling clean	The real results
10	Rerun in the project	Within about 6%	The README's numbers

1. The first trial: a number that didn't fit

Locust was first added to the project's dev tools (`uv add --dev locust`, version 2.47.0) and run from Windows against `localhost:8080`: 55 users, all started within one second, 20 seconds.

Two things checked out at once. The renaming of rows worked: Locust's output showed separate `greedy allowed`, `greedy limited` and `polite allowed` rows, so the `request_meta["name"]` technique is sound. And the greedy clients got 290 allowed requests, close to the expected 10 clients × (10 burst + 20 s × 1/s) = 300.

One thing didn't fit:

Row	Median
greedy limited	9 ms
greedy allowed	8 ms
health (no limiter)	16 ms
polite allowed	**47 ms**

The polite clients send 40 times fewer requests than the greedy ones, yet took 6 times longer. A server doesn't slow down for its quietest clients, so the measurement itself was suspect.

2–4. Finding the cause, one hypothesis at a time

Hypothesis A: Windows' port forwarding, which caused trouble in Phase 5. Test: the identical run from inside Docker, using Locust's official image on the stack's network. Result: polite 33 ms, greedy 5–6 ms. Smaller, but the gap remained, so **not Windows**.

Hypothesis B: connections that sit idle are slow. A polite client's connection is idle for 2 seconds between requests, a greedy one's never. Test: a small script, run inside Docker, timing one client's requests on a kept-open connection after a 1-second pause and back to back, both through nginx and directly to one copy. (The first version used `httpx`, which isn't in the API image, so it was rewritten with Python's built-in `http.client`.)

Route	After a 1 s pause	Back to back
nginx → copy, `/health`	3.2 ms	1.7 ms
Copy directly, `/health`	1.8 ms	1.0 ms
nginx → copy, `/api/resource`	2.9 ms	2.4 ms
Copy directly, `/api/resource`	3.0 ms	1.9 ms

One client alone saw 2–5 ms either way, so **not the server and not idle connections**.

Hypothesis C: the clients were in lockstep. All 40 polite clients started within the same second, and `constant_throughput(0.5)` makes each fire exactly every 2 seconds. So they kept firing **at the same instant**, and 40 requests arriving together queue behind each other. The greedy clients, firing 20 times a second, spread out naturally. Test: the same run with users started 5 per second (`-r 5`), which spreads their timing over 11 seconds:

Spawn rate	polite p50	polite p95	greedy p50
55 per second (all at once)	32 ms	53 ms	5 ms
5 per second	**6 ms**	10 ms	4 ms

Confirmed. Trap 1: the test created its own traffic jam, a "thundering herd". Real users don't arrive in lockstep, so a test that makes them do so measures itself. Every run since uses `-r 5`, plus `--reset-stats` so the 11-second ramp-up isn't counted.

(This step also produced the `summarize.py` script. A quick text-parsing command garbled Locust's columns, so the results were read from Locust's CSV files instead, properly.)

Two decisions

- **Locust runs inside Docker,** as a Compose service with a profile. Its numbers then describe RateGate, not Windows, and nothing needs installing. The dev dependency added in step 1 was removed again; `pyproject.toml` and `uv.lock` ended up exactly as before. The CI job also stays lean: Locust brings about 20 packages that every CI run would otherwise install.
- **The Windows comparison was kept for the guide,** run later with `uvx`, which runs a tool without adding it to the project.

(A side note from this machine: Git Bash, the shell used for the scratch work, turned the container path `/mnt/loadtest/...` into `C:/Program Files/Git/mnt/...`, and Locust couldn't find its file. Setting `MSYS_NO_PATHCONV=1` stops that conversion. PowerShell, which the guide uses, doesn't do this.)

5. Three algorithms, three identical rows

The mixed test then ran 70 seconds for each algorithm, switching with `RATEGATE_ALGORITHM` and `docker compose up -d` between runs. All three rows came out the same: 230/s, 4 ms / 7 ms, 0 polite limited, 1.02/s per greedy client.

At 230 requests per second nothing is under stress, so identical latency was plausible. Identical **allowed rates** for three different algorithms were less so. The check that exposed this came three steps later.

6–7. Capacity: whose limit is it?

The capacity file's clients never pause. With 20 users: about 1,050 requests per second. With 100 users: also about 1,050, while latency rose. And Locust logged:

```
CPU usage above 90%! This may constrain your throughput and may even give inconsistent response
time measurements!
```

When five times the users gives the same throughput, something is full. The warning said what: **Locust itself**. One Locust process runs on one CPU core. **Trap 3: the load generator ran out before the system did.** With `--processes 4` the same test reached about 2,050/s, and `docker stats` showed each API copy at about 100% CPU while Redis sat near 23%. The copies were now the bottleneck, so the next question was obvious: does adding copies help?

8. The scaling run that didn't add up

Capacity runs with 1, 3 and 6 copies (`docker compose up -d --scale api=N`) gave numbers that made no sense:

- With **1 copy**: 5,415 failures, and the CPU readings taken during the run listed `api-2` and `api-3`, which shouldn't have existed.
- With **6 copies**: only `api-1` to `api-3` in the CPU readings.

The number of copies running wasn't the number requested. The suspect was the one command run between the scaling and the measurement: `docker compose run locust`. It starts the service's dependencies, and nginx depends on the API service. **The check:** start the copies with the sliding log, note their container IDs, run Locust for 3 seconds, look again.

```
after up:  rategate-api-1=5c8622c81086 rategate-api-2=521902911fd1 rategate-api-3=04b0238b67ed
RATEGATE_ALGORITHM=sliding_log
 Container rategate-api-1 Recreate
```

```
 ...
after run: rategate-api-1=17f4c11fe707 rategate-api-2=8b81d21165ea rategate-api-3=992b11f5a709
RATEGATE_ALGORITHM not set -> default token_bucket
```

New container IDs, and the algorithm setting gone. **Trap 2: `docker compose run` recreated the API copies from the Compose file**, with 3 copies and only the settings present in that terminal. Two consequences:

- the scaling runs had the wrong number of copies;
- the "sliding log" and "sliding counter" runs of step 5 had really tested the **token bucket**, which explains the three identical rows.

The fix: `docker compose run --no-deps` starts Locust only. Checked the same way: identical container IDs before and after, and `RATEGATE_ALGORITHM=sliding_log` still set.

Lesson. "Plausible" isn't "checked". Every run now ends by proving what it ran: the setting inside a copy, the number of copies, and the keys in Redis, whose prefix (`tb:`, `swl:` or `swc:`) shows which algorithm did the counting.

9. Everything again, with proof

All algorithm and scaling runs were repeated with `--no-deps`. Each algorithm run left only its own keys: 50 `tb` keys, 96 `swl` keys (the log also keeps a sequence counter per client), 50 `swc` keys. Now the algorithms differed, exactly where Phase 2's theory says they should: 1.02/s (bucket), 1.00/s (log), 0.89/s (counter).

The single copy still failed: 7,717 failures, all `unexpected status 502`. A second run gave 5,414. nginx's log explained it:

```
upstream timed out (110: Operation timed out) while connecting to upstream
upstream server temporarily disabled while connecting to upstream
```

The one overloaded copy couldn't accept connections within nginx's 1-second connect timeout. nginx marked it "temporarily disabled", and with no other copy to try it answered `502` for a while. Counting only **served** requests: about 740 per second.

Also done in this round:

- **The Windows comparison**, which showed the same behaviour with higher, noisier timing (Theory 5).
- **Locust's web page** checked on port 8089: HTTP 200, title "Locust".
- **The guide's PowerShell loop** run as written, shortened. Each algorithm was set correctly, and its first Redis key proved it.

10. The project run

After "do it", the same files went into the project and every measurement was repeated there, so the README's numbers come from the committed code. The trial run matched the scratch one. The algorithm runs left the right keys again (50 `tb`, 96 `swl`, 50 `swc`). The capacity runs had the right copy counts (1, 3, 6). The numbers agreed with the scratch runs within about 6%; the largest difference was 6 copies, 2,733/s against 2,906/s. Then: 40 tests passed, a plain `docker compose up` still started no Locust, the CI stack check allowed exactly 100, everything was committed as `176397f`, and CI went green.

What the story shows

Trap	The clue	The check that settled it
1. Lockstep clients	Quiet clients slower than busy ones	Change one thing (spawn rate) and measure again
2. Recreated copies	Container counts that didn't match; identical results for different algorithms	Container IDs and settings before and after one command
3. Generator saturated	More users, same throughput; a CPU warning	More generator processes and the throughput doubled

How it was built

The order is: decide what to expect, build the tools, prove each run measured what it claimed, then record the result.

Step	What	Why at this point
1	Write the users: `locustfile.py`, `capacity.py`	The scenario and its expected numbers come before any measurement
2	A `locust` service in Compose (profile `loadtest`); `loadtest/results/` in `.gitignore`	Run Locust inside the stack's network, with nothing to install
3	`summarize.py`	The README table built the same way from every run
4	A 40 s trial run	Check everything works before spending minutes on real runs
5	Mixed traffic, once per algorithm, each proved by its Redis keys	Correctness and latency under load
6	Capacity with 1, 3 and 6 copies, `docker stats` during each	Throughput, and which part runs out first
7	README section with the conditions	Numbers people can trust and reproduce
8	Tests, lint, CI stack check, commit, push, watch CI	Nothing older broke

The guide (`docs/PHASE_7_GUIDE.md`) gives the same steps with the exact PowerShell commands, including loops that run all three algorithms and all three copy counts in one go.

Code 1: `loadtest/locustfile.py`

Three kinds of simulated client in 60 lines. A shared template sends the request and files each answer under its outcome.

```
"""Load test: polite clients stay under their limit, greedy ones ask for 20 times it.

Run it inside the stack's network (see docker-compose.yml), for example:
    docker compose run --no-deps --rm locust -f /mnt/loadtest/locustfile.py --headless \
        -u 55 -r 5 -t 70s --reset-stats --host http://nginx:80 \
        --csv /mnt/loadtest/results/token_bucket
"""

from uuid import uuid4

from locust import HttpUser, constant_throughput, task
```

```
class RateGateClient(HttpUser):
    abstract = True  # a template for the client types below, never run itself
    kind = ""

    def on_start(self):
        # Each simulated client has its own API key, so its own limit.
        self.api_key = f"{self.kind}-{uuid4().hex[:8]}"

    @task
    def call_api(self):
        with self.client.get(
            "/api/resource", headers={"X-API-Key": self.api_key}, catch_response=True
        ) as response:
            if response.status_code == 200:
                outcome = "allowed"
            elif response.status_code == 429:
                outcome = "limited"
            else:
                response.failure(f"unexpected status {response.status_code}")
                return
            # A 429 is the limiter working, not an error: count it as a success,
            # but in its own row, so the statistics show allowed and limited apart.
            response.request_meta["name"] = f"{self.kind} {outcome}"
            response.success()


class PoliteClient(RateGateClient):
    """1 request every 2 seconds: half of the 1-per-second refill rate."""

    kind = "polite"
    fixed_count = 40
    wait_time = constant_throughput(0.5)


class GreedyClient(RateGateClient):
    """20 requests a second: 20 times what its limit allows."""

    kind = "greedy"
    fixed_count = 10
    wait_time = constant_throughput(20)


class HealthCheck(HttpUser):
    """/health skips the limiter: the same path without the Redis check, for comparison."""

    fixed_count = 5
    wait_time = constant_throughput(2)

    @task
    def health(self):
        self.client.get("/health", name="health (no limiter)")
```

Piece by piece:

- **`HttpUser`** is Locust's base class for a user that sends HTTP requests. `self.client` is its HTTP client, which keeps its connection open between requests, like a browser does.
- **`abstract = True`** marks `RateGateClient` as a template: it holds what polite and greedy clients share, and Locust never runs it directly.
- **`kind`** is a class attribute each subclass overrides. It goes into the API key and the row names.

- `on_start` runs once when a simulated user starts. A random API key (`uuid4().hex[:8]`, eight random hex characters) gives each its own bucket.
- `@task` marks the method Locust calls again and again, with `wait_time` between calls.
- `catch_response=True` with `with ... as response:` hands the success/failure decision to us instead of Locust's default (anything outside 2xx = failure).
- **The `if` / `elif` / `else`:** 200 is "allowed", 429 is "limited", and anything else is recorded as a real failure with its status code, then `return` skips the rest.
- `response.request_meta["name"] = ...`: Locust records each request under `request_meta["name"]` when the `with` block ends, so changing it inside the block files the request under `greedy limited`, for example. `request_meta` is a public attribute in Locust 2.47's source, but this use isn't in Locust's documentation. It was checked in the source and confirmed by a real run, and the guide notes to recheck it after upgrading Locust.
- `response.success()` records the outcome as a success.
- `fixed_count`: exactly this many users of the class. 40 + 10 + 5 = 55, so the run uses `-u 55`.
- `constant_throughput(0.5)`: Locust picks each pause so the task runs exactly 0.5 times per second, whatever the response time. A fixed pause (`constant(2)`) would drift slower whenever responses slowed. This exactness is also what kept the polite clients in lockstep in Trap 1.
- `HealthCheck` sends plain requests to `/health` with a fixed row name. `/health` is exempt from limiting, so its timing is the same path without Redis: the baseline for the limiter's cost.

Code 2: `capacity.py`, the Compose service and `.gitignore`

The capacity clients live in their own file so they can never be included by accident, and Locust is wired into Compose so that it only runs when asked.

`loadtest/capacity.py`:

```
"""Capacity test: clients that never pause, to find how many requests a second the stack handles.

Kept apart from locustfile.py so the normal test never includes it by accident. Run it
with several Locust processes, so the load generator isn't the bottleneck:
    docker compose run --no-deps --rm locust -f /mnt/loadtest/capacity.py --headless \
        -u 150 -r 150 -t 40s --processes 6 --reset-stats --host http://nginx:80
"""

from locust import constant
from locustfile import RateGateClient


class FloodClient(RateGateClient):
    """Sends the next request as soon as the last answer arrives."""

    kind = "flood"
    wait_time = constant(0)
```

- `from locustfile import RateGateClient` reuses the template, so flood clients also get their own keys and their `flood allowed` / `flood limited` rows. Locust runs every non-abstract user class in the file it's given. `PoliteClient` and the others aren't imported here, so only `FloodClient` runs.
- `constant(0)`: no pause at all between requests.
- **The two imports sit together** because ruff treats `locustfile`, like `locust`, as a third-party module and wants them in one group. ruff said so the first time.

`docker-compose.yml`, a new service at the end:

```
  # Load generator (Phase 7), only started on request:
  #   docker compose run --no-deps --rm locust -f /mnt/loadtest/locustfile.py ...
  # --no-deps matters: without it, "run" recreates the API copies from this file,
  # undoing --scale and any RATEGATE_* settings they were started with.
  # It runs on the stack's network, so its numbers don't include Windows' port forwarding.
  locust:
    image: locustio/locust:2.47.0
    profiles: ["loadtest"]
    volumes:
      - ./loadtest:/mnt/loadtest
    depends_on:
      - nginx
```

- **`image: locustio/locust:2.47.0`**: Locust's official image, pinned to the same version that was tested. Its program is `locust`, so anything written after `docker compose run ... locust` is passed to Locust as options.
- **`profiles: ["loadtest"]`**: a service with a profile is left out of a plain `docker compose up`. Checked: `docker compose config --services` lists `redis api nginx`, and only with `--profile loadtest` does `locust` appear. So the normal stack and the Phase 6 CI job are unchanged.
- **`volumes: - ./loadtest:/mnt/loadtest`**: your `loadtest` folder appears inside the container at `/mnt/loadtest`. Locust reads the locustfiles there, and the CSV files it writes there appear in your `loadtest/results/`.
- **`depends_on: - nginx`** documents what it talks to. It's always run with `--no-deps`, which ignores this, and the comment above the service says why (Trap 2).

`.gitignore`, two new lines:

```
# Load-test output (CSV files written by Locust); the summary goes in the README
loadtest/results/
```

Each run writes four CSV files, and their contents change every time. The README keeps the summary; the raw files stay on your PC. Checked with `git check-ignore`, and the Phase 7 commit indeed contains no results.

Code 3: `loadtest/summarize.py`

A small script that turns Locust's CSV files into the README's Markdown table. The README's algorithm table came out of it, so those numbers were never copied by hand. The capacity table was worked out from the same CSV files with a short one-off calculation: served requests = (requests − failures) ÷ duration.

```
"""Turn Locust's CSV results into a Markdown table, one row per run.

Usage: uv run python loadtest/summarize.py "Token bucket=loadtest/results/token_bucket" ...
Each argument is a label and the --csv prefix Locust was given.
"""

import argparse
import csv


def load(prefix: str) -> dict[str, dict[str, str]]:
    with open(f"{prefix}_stats.csv", newline="") as f:
        return {row["Name"]: row for row in csv.DictReader(f)}
```

```
def count(rows: dict, name: str) -> int:
    return int(rows[name]["Request Count"]) if name in rows else 0


def rate(rows: dict, name: str) -> float:
    return float(rows[name]["Requests/s"]) if name in rows else 0.0


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("runs", nargs="+", help="LABEL=CSV_PREFIX")
    parser.add_argument("--greedy-clients", type=int, default=10)
    args = parser.parse_args()

    print(
        "| Algorithm | Requests/s | p50 | p95 | Limited (429) "
        "| Polite clients limited | Allowed per greedy client |"
    )
    print("|---|---|---|---|---|---|---|")
    for run in args.runs:
        label, prefix = run.split("=", 1)
        rows = load(prefix)
        total = rows["Aggregated"]
        limited = count(rows, "polite limited") + count(rows, "greedy limited")
        answered = (
            limited + count(rows, "polite allowed") + count(rows, "greedy allowed")
        )
        polite = count(rows, "polite allowed") + count(rows, "polite limited")
        greedy_rate = rate(rows, "greedy allowed") / args.greedy_clients
        print(
            f"| {label} | {float(total['Requests/s']):.0f} "
            f"| {total['50%']} ms | {total['95%']} ms "
            f"| {100 * limited / answered:.1f}% "
            f"| {count(rows, 'polite limited')} of {polite} "
            f"| {greedy_rate:.2f}/s |"
        )


if __name__ == "__main__":
    main()
```

- **What Locust's CSV looks like.** `PREFIX_stats.csv` has a header row (`Type,Name,Request Count,Failure Count,Median Response Time,…,Requests/s,…,50%,66%,75%,80%,90%,95%,98%,99%,…`) and one row per request name, plus `Aggregated`.
- **`csv.DictReader`** turns each row into a dictionary keyed by the header names. **`{row["Name"]: row for ...}`** then lets the code ask for `rows["greedy allowed"]` directly.
- **`count` and `rate` return 0 for a missing row.** A perfect run has **no** `polite limited` row at all, because Locust only creates rows for names it has seen.
- **`"Aggregated"`** is Locust's total row across all names. Its p50 and p95 include the health checks: 10 of 230 requests per second, too few to move them.
- **Limited (429)** = limited ÷ all limiter answers (health checks excluded, since they never meet the limiter).
- **Allowed per greedy client** = the `greedy allowed` rate ÷ 10 clients, directly comparable to the configured 1 per second. `--greedy-clients` exists in case the scenario changes.
- **`nargs="+"`**: one or more `LABEL=PREFIX` arguments, so one command prints a row per algorithm. **`split("=", 1)`** splits only at the first `=`.

- **Only the standard library** (`argparse`, `csv`), so it runs with `uv run python` and needs nothing installed.

Command reference

Every command used in Phase 7, what it does and why. PowerShell from `D:\RateGate`, with the stack up (`docker compose up --build -d --wait`).

Running Locust

```
docker compose run --no-deps --rm locust -f /mnt/loadtest/locustfile.py --headless -u 55 -r 5 -t
70s --reset-stats --host http://nginx:80 --csv /mnt/loadtest/results/token_bucket --only-summary
```

Part	Meaning
`docker compose run`	Start a one-off container from a Compose service, with everything after the service name passed to its program
`--no-deps`	Don't start or recreate the service's dependencies. Essential here (Trap 2)
`--rm`	Delete the container when Locust finishes
`locust`	The service name; its program is Locust, so what follows are Locust's options
`-f /mnt/loadtest/locustfile.py`	The locustfile, at its path **inside** the container
`--headless`	Run without the web page; print the results and stop
`-u 55 -r 5`	55 users, started 5 per second (Trap 1)
`-t 70s`	Run 70 seconds: 11 s of ramp-up, then about 60 s measured
`--reset-stats`	Discard what was recorded during the ramp-up
`--host http://nginx:80`	Where to send requests: nginx's service name and its own port, inside the network
`--csv /mnt/loadtest/results/token_bucket`	Write `token_bucket_stats.csv` and three other files; they appear in your `loadtest\results\`
`--only-summary`	Print only the final table, not a progress line every few seconds
`--processes 6` (capacity runs)	Split the users across 6 Locust processes (Trap 3)

Reading the results

Command	What it does
`uv run python loadtest/summarize.py "Token bucket=loadtest/results/token_bucket" ...`	Prints the Markdown table, one row per `label=prefix`
`Import-Csv loadtest\results\scale_api3_stats.csv | Select-Object Name, "Request Count", "Failure Count", "Requests/s", "50%", "95%" | Format-Table`	PowerShell: read a CSV, pick columns, line them up
`docker stats --no-stream`	Each container's CPU and memory at this moment; run it in a second terminal during a capacity run

Proving what ran (Trap 2)

Command	Proves
`docker exec rategate-api-1 printenv RATEGATE_ALGORITHM`	The algorithm the copy was started with
`docker compose exec redis redis-cli --scan --pattern "rategate:*"`	The keys in the stack's Redis: `tb:`, `swl:` or `swc:` show which algorithm counted
`docker compose exec redis redis-cli flushall`	Empties the **stack's** Redis before a run, so only that run's keys remain
`docker ps --filter name=rategate-api`	How many copies are running, and their container IDs

Changing what's tested

Command	Why
`$env:RATEGATE_ALGORITHM = "sliding_log"` then `docker compose up -d --wait`	Recreate the copies with another algorithm
`docker compose up -d --scale api=6 --wait` then `docker compose restart nginx`	Change the number of copies; nginx looks up the addresses again
`Remove-Item Env:RATEGATE_ALGORITHM`	Back to the default afterwards

Optional: watch it live

```
docker compose run --no-deps --rm -p 8089:8089 locust -f /mnt/loadtest/locustfile.py --host
http://nginx:80
```

`-p 8089:8089` publishes Locust's web page; open `http://localhost:8089`, enter 55 users and a spawn rate of 5, and press Start. Checked: the page answers with the title "Locust".

One-off tools used during the investigation

Command	Why
`uvx --from locust==2.47.0 locust ...`	Run Locust from Windows once, for the comparison, without adding it to the project
`docker run --rm --network rategate_default ... locustio/locust:2.47.0 ...`	Run Locust on the stack's network before the Compose service existed

Verification

The roadmap's "done when" is met: the README contains real numbers from this machine. Below are the project run's results (2026-11-03), each run proved to be what it claims.

Trial run (40 s, 30 s measured): `exit code 0`; 230 requests/s, p50 4 ms, p95 6 ms, 0 of 580 polite requests limited, 1.04/s per greedy client.

Mixed traffic, 70 s per algorithm (60 s measured):

Algorithm	Request s/s	p50	p95	Limited (429)	Polite clients limited	Allowed per greedy client	Failures	Proof
Token bucket	230	4 ms	7 ms	86.4%	0 of 1180	1.02/s	0	50 `tb` keys
Sliding window log	230	4 ms	6 ms	86.4%	0 of 1180	1.00/s	0	96 `swl` keys

Algorithm	Request s/s	p50	p95	Limited (429)	Polite clients limited	Allowed per greedy client	Failures	Proof
Sliding window counter	230	4 ms	6 ms	87.0%	0 of 1180	0.88/s	0	50 `swc` keys

Health checks (no limiter) in all three runs: p50 4 ms, p95 7 ms.

Capacity, 40 s each, 150 clients, 6 Locust processes:

API copies	Copies actually running	Served per second	Failures	p50	p95
1	1	738	4,902 (`502`)	180 ms	240 ms
3	3	1,915	0	12 ms	240 ms
6	6	2,733	0	35 ms	170 ms

CPU during the 3-copy run is the chart in Theory 6. With 6 copies, Locust used 513% and nginx 111%: the test machine itself was near its limit.

Nothing else changed or broke:

Check	Result
Tests	`40 passed in 27.59s`
Lint and formatting	`All checks passed!` , `23 files already formatted`
Lock file	`uv sync --locked` : `Checked 32 packages` (no Locust added)
Plain `docker compose up`	3 API copies, nginx, Redis; no Locust
The CI stack check, locally	Exactly 100 of 200 allowed
Results kept out of git	`git check-ignore` confirms; the commit has no CSV files
GitHub CI on `176397f`	Both jobs green; badge "CI – passing"

Repeatability: the scratch runs gave the same picture: algorithms 1.02 / 1.00 / 0.89 per second, and capacity 740 / 1,940 / 2,906. The largest difference was 6%, for 6 copies.

Common pitfalls

The first three rows are the traps met in this phase; the rest are common ways load tests mislead.

Pitfall	Symptom	Why	Fix
Starting all users at once	Some clients look 5–8 times slower than others	Clients in lockstep queue behind each other (thundering herd)	`-r 5` ; `--reset-stats`
`docker compose run` without `--no-deps`	Different algorithms give identical results; `--scale` seems ignored	`run` recreates dependencies from the Compose file, dropping settings	`--no-deps` ; prove each run with `printenv` , Redis keys, container count
One Locust process	Throughput stops growing as users are added; `CPU usage above 90%` warning	The generator is the bottleneck	`--processes N` ; watch `docker stats`

Pitfall	Symptom	Why	Fix
Counting 429s as failures	"86% errors" for a correct limiter	Locust's default: anything outside 2xx fails	`catch_response`, `response.success()`, own row names
Measuring through Windows' port forwarding	Higher, noisier latency	Docker Desktop's forwarding (Phase 5)	Run Locust on the stack's network
Trusting the average	A "typical" time nobody saw	A few slow requests distort it	p50, p95, p99
Reading capacity-run latency as speed	"p95 240 ms"	A full system queues requests	Quote mixed-traffic latency for speed
No expected numbers	Any result looks plausible	Nothing to compare against	Work them out first (230/s, 0 polite limited, about 1/s per greedy client)
`-u` not matching the `fixed_count`s	Fewer clients of some kinds than planned	Locust fills fixed counts with the users it has	`-u 55` = 40 + 10 + 5
Forgetting `restart nginx` after scaling	New copies idle, or `502`s	nginx looked up the copies at startup (Phase 5)	`docker compose restart nginx`
`flushall` on the wrong Redis	Your other data gone	Two Redis servers exist	`docker compose exec redis ...` targets the stack's
Git Bash path conversion	`Could not find 'C:/Program Files/Git/mnt/...'`	Git Bash rewrites `/mnt/...` paths	PowerShell, or `MSYS_NO_PATHCONV=1`
Numbers without conditions	Nobody can compare or reproduce them	Results depend on machine, setup and scenario	State machine, date, limits, scenario, where the generator ran
Committing raw results	Large, ever-changing CSV files in git	Each run writes four files	`loadtest/results/` in `.gitignore`

Check your understanding

Answer each in your own words first, then compare. Questions 6, 7, 8 and 11 make good interview stories.

1. What do throughput, latency and percentiles each tell you? Throughput is how much work gets done (requests per second); latency is how long one request takes; percentiles describe the spread of latencies (p50 typical, p95 the slow ones, p99 the worst 1 in 100).

2. Why are p50 and p95 better than the average? The average blends fast and slow requests into a number that may match no real request; percentiles show both the typical case and the slow tail.

3. Why does each simulated client get its own API key? So each has its own bucket, like separate customers. With one shared key, all 55 would share one limit and nothing could be learned about per-client limiting.

4. Why are 429s recorded as successes, and how are they kept apart from allowed requests? A 429 is the limiter working. The locustfile marks it a success but files it under its own name (`… limited`), so allowed and limited show in separate rows; only unexpected statuses count as failures.

5. What was expected before running, and why does it matter? 230 requests/s, 0 polite clients limited, about 1 allowed per second per greedy client, and roughly 83–87% 429s. Without expectations, any output looks plausible, and wrong results go unnoticed.

6. Why did polite clients first look 8 times slower, and what fixed it? All 40 started together and fired every 2 s in lockstep, so their requests queued. Starting users 5 per second spread them out: 32 ms became 6 ms.

7. What did `docker compose run` do without `--no-deps`, and how was it noticed? It recreated the API copies from the Compose file, dropping their algorithm setting and resetting the number of copies. Noticed from container counts in the CPU readings that didn't match; confirmed by container IDs and settings before and after one run.

8. How do you tell whether the system or the load generator is the bottleneck? If more users don't raise throughput, something is full. Check the generator's CPU (Locust warns above 90%) and the system's (`docker stats`); add generator processes and see whether throughput rises.

9. Why does the sliding window counter allow 0.88/s while the log allows 1.00/s? The log remembers every request and is exact. The counter estimates the previous window's share with a weighted count, which over-counts under steady heavy traffic, so it errs on the strict side.

10. Why does the token bucket allow slightly more than 1/s? Each client starts with a full bucket of 10 tokens, used on top of the 1-per-second refill during the measurement.

11. What is the bottleneck in RateGate's stack, and what does it mean for scaling? The API copies: each is one Python process at about one CPU core, while Redis stayed at 28%. Adding copies adds capacity: 740 → 1,920 → 2,730 served per second.

12. Why did a single copy produce `502`s under full load, but three copies didn't? The overloaded copy couldn't accept connections within nginx's 1 s connect timeout; nginx marked it temporarily disabled and, with no other copy, answered `502`. With three copies there was always another to try.

13. Why must load-test numbers be published with their conditions? They depend on the machine, the setup, the limits, the scenario and where the generator ran. Without those, nobody can compare or reproduce them.

Phase 8: Polish: tiers, metrics, architecture and design decisions (done 2026-11-03)

RateGate now gives paid API keys 10 times the free limits, counts every decision it makes for a monitoring system, and its README opens with an architecture diagram and closes with the reasoning behind every design choice. The work is committed as `474465f` and pushed, and CI is green. With it, every phase of the roadmap is complete.

Phases 1–7 each proved a claim from the resume line. Phase 8 was optional polish: three features real rate limiters have, which make the project look and behave like a production service and give more to talk about in an interview.

Results (2026-11-03):

Result	Evidence
Tiers work	Free key: `limit: 10`, `tier: free`. Paid key: `limit: 100`, `tier: paid`. A burst of 200: free 10 allowed, paid 101
Metrics are exact	Counters summed over the 3 copies matched what the clients received, to the request: free 11 allowed / 190 limited, paid 102 / 99
Metrics cost nothing measurable	Phase 7's load test with metrics on: 230 requests/s, p50 4 ms, p95 6–7 ms, the same as without
`/metrics` isn't public	Through nginx: `404`
Tests first, red for the right reasons	7 new tests failed on the old code; the metrics tests failed for metrics reasons once tiers existed
Nothing broke	47 tests (40 + 7) pass 3 runs in a row; concurrency tests 5 of 5; CI's stack check: exactly 100 of 200

One thing couldn't be checked here: that the README's diagram draws on GitHub. Both ways to check it (rendering it on the mermaid.ink website, or with Mermaid's command-line tool) were blocked by Claude Code's permission system. The full story explains why; the diagram uses only standard syntax, and opening the README on GitHub settles it.

What changed in the project:

File	Change
`app/config.py`	A `Tier` type; `paid_api_keys` and four `paid_*` limit settings
`app/limiter/factory.py`	Builds a limiter for a given tier
`app/middleware.py`	Looks up the client's tier; adds `X-RateLimit-Tier`; counts and times every decision; `/metrics` exempt
`app/main.py`	One limiter per tier; the `/metrics` route
`app/metrics.py`	New: the two metrics
`tests/test_tiers.py`, `tests/test_metrics.py`	New: 4 + 3 tests
`tests/test_fail_modes.py`, `tests/test_instance_id.py`	Their harnesses set up limiters per tier

File	Change
`nginx/nginx.conf`	`/metrics` answers `404` from outside
`docker-compose.yml`	Passes `RATEGATE_PAID_API_KEYS` into the copies
`scripts/collect_metrics.py`	New: reads every copy's counters and adds them up
`pyproject.toml`, `uv.lock`	`prometheus-client` 0.26.0
`README.md`	Architecture diagram, how a request is checked, Tiers, Metrics, Design decisions, 5 settings, 47 tests
`ROADMAP.md`, `docs/PHASE_8_GUIDE.md`	Phase 8 ticked; the step-by-step guide

A separate later commit, `e1808f2`, added `.vscode/settings.json` (VS Code loads `.env` in its terminals), and was pushed too.

Theory 1: Tiers, different limits for different customers

Real APIs sell plans: a free plan with a small limit, paid plans with bigger ones. The limiter must apply each client's **own** limit. RateGate has two tiers, chosen by API key, each with its own limiter.

How it works:

Piece	What it does
Two tiers, `free` and `paid`	The free limits are the existing settings; the paid ones are new `paid_*` settings, 10 times bigger
`RATEGATE_PAID_API_KEYS`	A JSON list of the keys that are paid, e.g. `'["key-1", "key-2"]'`. Everyone else is free, including unknown keys and clients without a key
One limiter per tier	At startup, two limiters with the same algorithm and different numbers, kept in `app.state.limiters = {"free": …, "paid": …}`
`client_tier(request)`	Returns `"paid"` if the request's key is in the paid list, otherwise `"free"`
`X-RateLimit-Tier`	A new response header saying which tier applied

The default numbers:

Tier	Token bucket	Sliding windows
`free`	10 burst, 1 per second	10 per 10 s
`paid`	100 burst, 10 per second	100 per 10 s

Why a dictionary of limiters, rather than one limiter that changes its numbers per request? Each limiter object is built once with fixed numbers, exactly as before; the middleware only chooses which one to ask. The algorithms don't change at all, so Phase 3's concurrency proofs still hold for both tiers.

In a real product, tiers would come from a customer database: a key is issued when someone signs up and pays. A setting keeps this project self-contained, and swapping it for a database lookup later would change only `client_tier`.

What happens when a customer upgrades? Their Redis key stays the same (`rategate:tb:key:<their key>`); from the next request on, the paid limiter checks it with the bigger numbers. The Lua scripts already cap tokens at the current capacity, so nothing breaks.

Theory 2: Metrics and Prometheus

Metrics are numbers a service keeps updating so operators can watch it without reading logs. RateGate publishes two in Prometheus' text format: a counter of decisions by outcome and tier, and a histogram of how long each Redis check takes.

Why metrics. Until Phase 8, the only way to see what RateGate did was to read its logs or run a test. In production, operators watch graphs drawn from metrics collected every few seconds, with alerts such as "half of all requests are being limited" or "fail-open is happening".

Prometheus is the standard tool. Each service publishes its numbers as plain text at `/metrics`, and Prometheus fetches (**scrapes**) that page regularly. The official Python library, `prometheus-client`, keeps the numbers and writes the page.

Three kinds of metric (RateGate uses two):

Kind	Meaning	In RateGate
Counter	Only goes up: "how many so far"	`rategate_decisions_total`: decisions by outcome and tier
Histogram	Counts values into ranges (**buckets**): "how many took under 1 ms, under 2.5 ms ..."	`rategate_check_seconds`: how long each Redis check took
Gauge	Goes up and down: "how many right now"	Not needed

Labels split one metric into several series, e.g. `rategate_decisions_total{outcome="limited",tier="free"}`. RateGate's labels are `outcome` (`allowed`, `limited`, `fail_open`, `fail_closed`) and `tier` (`free`, `paid`): at most 8 series.

Never label by API key or IP address. Every distinct label value creates a new series. Millions of clients would mean millions of series, and the monitoring system would slow down or run out of memory. Labels must have a small, fixed set of values.

What the page looks like, a real sample after 3 requests against a bucket of 2 (some bucket lines left out):

```
# HELP rategate_decisions_total Rate-limit decisions, by outcome (allowed, limited, fail_open,
fail_closed)
# TYPE rategate_decisions_total counter
rategate_decisions_total{outcome="allowed",tier="free"} 2.0
rategate_decisions_total{outcome="limited",tier="free"} 1.0
# TYPE rategate_check_seconds histogram
rategate_check_seconds_bucket{le="0.001"} 0.0
rategate_check_seconds_bucket{le="0.005"} 3.0
rategate_check_seconds_bucket{le="+Inf"} 3.0
rategate_check_seconds_count 3.0
rategate_check_seconds_sum 0.004158700030529872
```

- `# HELP` and `# TYPE` describe each metric.
- A counter created as `rategate_decisions` is published as `rategate_decisions_total`: the library adds `_total`.
- Histogram buckets are **cumulative**. `le="0.005"} 3.0` means 3 checks took at most 5 ms (`le` = "less than or equal"). Here none was under 1 ms, all 3 were under 5 ms, and together they took 4.2 ms (`_sum`).
- The library also prints `..._created` lines: when each series started, in seconds since 1970. Prometheus uses them to notice restarts; they can be ignored.

Theory 3: Metrics with several copies

Each API copy counts only the requests it answered, so the real total is the sum over all copies. Monitoring therefore reads every copy directly, while the public address hides `/metrics` altogether.

Counters live in each process. Each copy is a separate Python process with its own counters in its own memory. Behind round-robin, each sees about a third of the traffic. Two consequences:

- **A monitoring system reads every copy and adds them up.** Prometheus finds the copies through "service discovery". For this project, a script does the same: `scripts/collect_metrics.py`.
- **`/metrics` through nginx would be misleading.** Each request would reach a different copy and show a different third of the numbers. It would also expose internal numbers to anyone. So nginx answers `404` for `/metrics`, and monitoring reads the copies on the internal network.

Where the counting happens. Every decision passes through the middleware, so that's the one place to count:

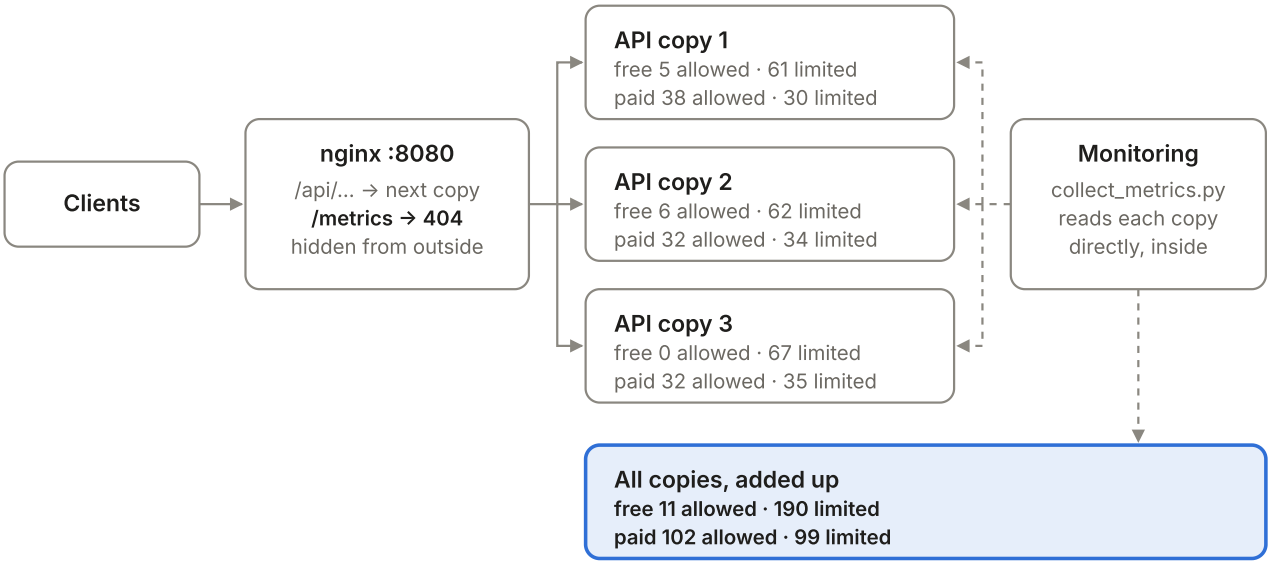
Situation	Response	Counted as
Redis says allowed	`200`	`allowed`
Redis says over the limit	`429`	`limited`
Redis unreachable, fail-open	`200`, unchecked	`fail_open`
Redis unreachable, fail-closed	`503`	`fail_closed`

The **timer** wraps only the Redis call, so it measures the limiter's own cost. It records failed checks too: a check that timed out shows up as about 0.25 seconds, which is exactly what an operator would want to notice.

`/metrics` is exempt from rate limiting, like `/health`. A monitoring system that received `429`s would go blind at exactly the moment traffic is highest.

Counters restart from 0 when a copy restarts, because they live in memory. That's normal: Prometheus recognises the drop as a restart, not as negative traffic.

The proof that this works: a live check on the 3-copy stack. A single request and a burst of 200 for a free key, then the same for a paid key. The counters, read from each copy and added up, matched what the clients received exactly. The diagram below shows the numbers.



The clients received: free 1 + 10 allowed, 190 limited; paid 1 + 101 allowed, 99 limited. The sums match exactly.

collect_metrics.py after the live check, project run, 2026-11-03 · 3 API copies

No single copy's numbers resemble the client's view; only the sum does. That's why monitoring must read every copy, and why `/metrics` through nginx would have been misleading as well as public.

Theory 4: The architecture diagram and the design decisions

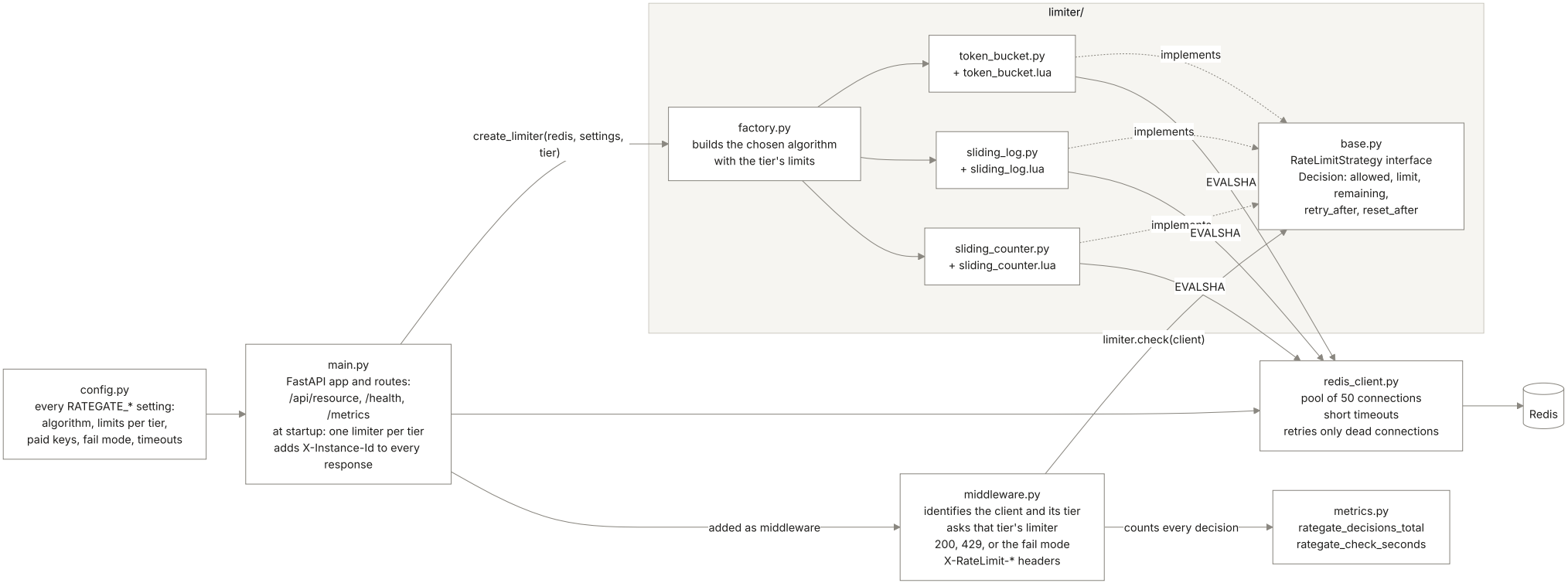
A reviewer opening the repository should understand the system in two minutes. So the README now opens with a picture of the system and closes with a table of every important choice and its reason.

Diagrams as text: Mermaid. GitHub draws diagrams written in **Mermaid**, a small text language, inside a ```` ```mermaid ```` code block. Because the diagram is text, it lives in git, changes alongside the code, and needs no drawing tool. The basics used in RateGate's diagram:

Mermaid	Meaning
`flowchart LR`	A flowchart, left to right
`name["label"]`	A box
`name[("label")]`	A cylinder, the usual symbol for a database
`a --> b`	An arrow
`a -->|"text"| b`	A labelled arrow
`a -.-> b`	A dotted arrow
` ` inside a label	A line break

The README pairs the diagram with **"How one request is checked"**, five numbered steps from nginx to the metrics counter. The picture shows *what* the parts are; the steps show *what happens*.

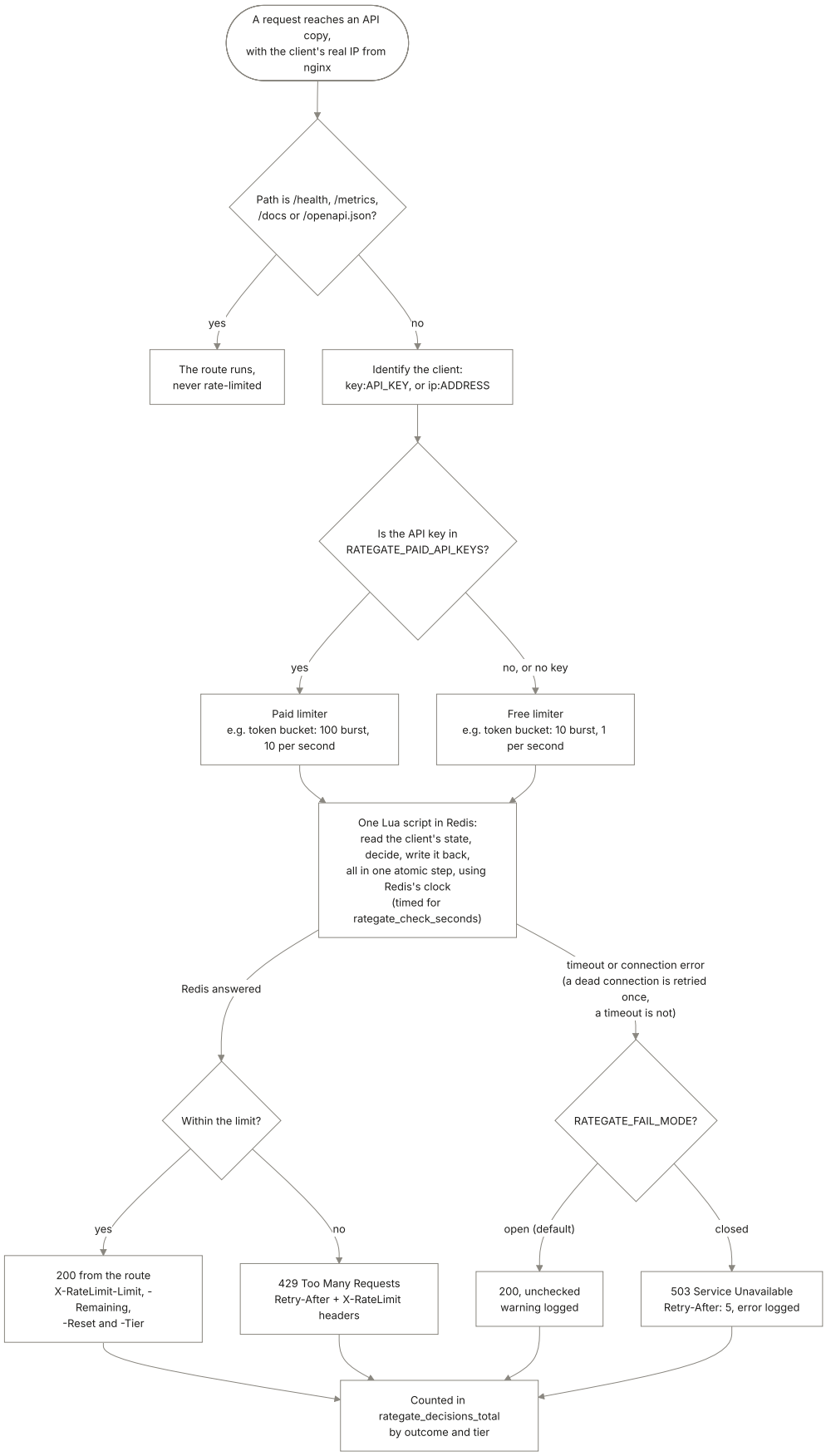
Inside one API copy



How the code is organised inside each copy (README, “Inside one API copy”)

Only the algorithm classes know about Redis keys and scripts; the middleware only knows the `RateLimitStrategy` interface, so switching algorithms is one setting (`RATEGATE_ALGORITHM`).

How one request is checked



Every path a request can take through an API copy (README, “How one request is checked”)

Every response, including `429` and `503`, also carries `X-Instance-Id`, naming the copy that answered.

The design-decisions table. Over eight phases, many choices were made for good reasons, and those reasons were scattered across commit messages, guides and this journal. The README now gathers 14 of them in one table: the decision, why it was made, and the evidence. For example:

Decision	Why	Evidence
Each check is one atomic Lua script	Read, decide and write can't interleave between concurrent requests	200 at once, limit 100: exactly 100; a non-atomic version let all 200 through
Retry only connection errors, never timeouts	After a timeout the script may already have run	20 late replies: 40 counts with retry-on-timeout, 20 without

The evidence column is what makes it convincing: each row points to something that was measured, not just asserted. In an interview, any row is a ready story.

The full story

Phase 8 went more smoothly than Phases 5–7: no hidden bugs surfaced. The interesting parts are the order the pieces had to be built in, the checks that proved the metrics exact and free, and one check that couldn't be done here. As always, it was built in a scratch copy first and then in the project.

The key moments at a glance:

#	Step	Result
1	Check the library's API before using it	Counter names gain `_total`; `REGISTRY.get_sample_value` reads values
2	7 new tests on the Phase 7 code	All red, but the metrics tests for the wrong reason
3	Tiers	Tier tests green; metrics tests now red for the right reason
4	Metrics	All 7 green; full suite 47 passed, 3 runs out of 3
5	Live check on the stack	`/metrics` hidden; tiers correct; summed counters exact
6	Load test with metrics on	Latency unchanged
7	Validate the README diagram	Blocked, twice
8	Build in the project, commit, push	Identical results; CI green

1. Checking the library first

Before writing any metrics code, `prometheus-client` (version 0.26.0) was installed and tried in a few lines: create a counter, add 1, print the page, read the value back. That showed three things the tests would rely on. A counter created as `x_decisions` is printed as `x_decisions_total`. Each counter also gets an `x_decisions_created` line. And `REGISTRY.get_sample_value("x_decisions_total", {...})` returns the current value. This is Phase 4's lesson again: check an API before relying on it.

2–3. Tests first, and the order that made them honest

The 7 new tests (4 for tiers, 3 for metrics) were written first and run on the Phase 7 code. All 7 failed:

```
AttributeError: 'Settings' object has no attribute 'paid_api_keys'      (1)
TypeError: create_limiter() takes 2 positional arguments but 3 were given  (1)
AttributeError: 'State' object has no attribute 'limiter'                (5)
```

The last line is the interesting one. The metrics tests set up the app the new way, with `app.state.limiters` (one per tier). The old middleware still read a single `app.state.limiter`, so the metrics tests crashed before reaching anything about metrics. A red test should be red **because of the missing feature** (Phase 4's Mistake 2), and these weren't yet.

So tiers were built **first**. After that the tier tests passed (4 of 4), and the metrics tests failed for the right reasons:

```
assert (0.0 - 0.0) == 1                                  (counters never increased)
assert [404, 429, 429, 429, 429] == [200, 200, 200, 200, 200]
```

The second line says two things: `/metrics` didn't exist (`404`), and it was being rate-limited (`429`). Both are exactly what the metrics step had to fix.

4. Metrics, and a few small decisions

- **The metrics live in their own module,** `app/metrics.py`. A metric can be registered only once (a second registration raises `Duplicated timeseries`), and a module runs its top-level code only once, however many files import it.
- **A plain route, not a mounted app.** The library also offers a ready-made app to mount at `/metrics`, but a mounted path can redirect `/metrics` to `/metrics/`. A four-line route that returns `generate_latest()` is clearer and behaves exactly like `/health`.
- **The timer wraps only the Redis call,** so it measures the limiter's cost, and it records failed checks too.
- **One count on every way out of the check:** allowed, limited, fail-open, fail-closed. The fail-mode test checks the last two by pointing a client at the dead port 6399, as in Phase 4.

All 7 new tests passed. The full suite gave `47 passed` three times, and the concurrency tests passed 5 of 5.

5. The live check: are the counters exact?

On the 3-copy stack, with `demo-paid` listed as a paid key:

```
/metrics through nginx: 404
demo-free:  x-ratelimit-limit: 10   x-ratelimit-remaining: 9   x-ratelimit-tier: free
demo-paid:  x-ratelimit-limit: 100  x-ratelimit-remaining: 99  x-ratelimit-tier: paid
burst, free key:  200: 10   429: 190
burst, paid key:  200: 101  429: 99
All copies  free allowed: 11, free limited: 190, paid allowed: 102, paid limited: 99
```

The client side and the metrics side, compared:

	Clients received	Counters, summed
Free allowed	1 + 10 = 11	11
Free limited	190	190
Paid allowed	1 + 101 = 102	102
Paid limited	99	99

Exact, to the request. The paid burst allowed 101, not 99: 99 tokens remained after the single request, and the paid tier refills 10 per second, so about 2 more arrived during the burst.

6. Does counting slow it down?

Phase 7's mixed load test was run again with metrics on: 230 requests per second, p50 4 ms, p95 6 ms in scratch and 7 ms in the project, against 4 ms and 6–7 ms in Phase 7. No measurable cost. Adding 1 to a number in memory takes far less time than the Redis round trip each request already makes.

7. The diagram that couldn't be checked

GitHub draws the Mermaid diagram in the visitor's browser, so a mistake in its syntax would only show on GitHub, as an error box. Two ways to check it beforehand were tried, and both were stopped by Claude Code's permission system:

Attempt	What it would have done	Why it was blocked
Render it on mermaid.ink	Send the diagram's text to a public website and get a picture back	Sending data to a third-party service needs your permission
Render it with Mermaid's command-line tool	Download that tool from npm and run it locally	Downloading and running outside code needs permission too

Your reply "do it with first option" (mermaid.ink) was tried once more and blocked again: the automatic check only accepts a permission rule in your Claude Code settings, not a chat reply. Neither block was worked around. The diagram uses only long-standing Mermaid syntax that GitHub supports, so it should draw, but this is the one claim in the phase without a check behind it. Opening the README on GitHub settles it.

8. The project build

After "do it", every step was repeated in the project in the guide's order, with the same results at each stage: 7 red with the same messages; tiers 4 green; metrics red for the right reasons; then 7 green. Every changed file was compared byte for byte with the tested scratch version, and all were identical. The live check gave the same exact totals, the load test the same latency, and the suite `47 passed` three times. The commit `474465f` went to GitHub and both CI jobs passed.

Afterwards, at your request, `.vscode/settings.json` (created by VS Code in Phase 5) was committed separately as `e1808f2` and pushed. It's one setting that makes VS Code's terminals load `.env`.

9. Removing Claude as a contributor

GitHub listed two contributors: you and "claude". The reason: from Phase 2 on, every commit message ended with a `Co-Authored-By: Claude <noreply@anthropic.com>` line, and GitHub counts co-authors as contributors. You asked for Claude to be removed.

What was done:

1. A local backup branch, `backup-before-rewrite`, keeps the old history in case it's ever needed.
2. `git filter-branch --msg-filter` rewrote the 8 affected commits (Phases 2–8 and the `.vscode` commit), deleting that line from each message. Phases 0 and 1 never had it and kept their IDs.
3. Checks before pushing: the code is byte-for-byte identical (`git diff` between old and new: nothing), no "claude" left in any message, and every commit kept its original date and message.
4. `git push --force-with-lease` replaced GitHub's `main`. "With lease" only overwrites if GitHub still has exactly the expected commit, so nothing anyone else pushed could be lost.
5. CI passed on the new history, and GitHub's contributors API lists only you (10 commits). The repository page's sidebar can take a while to catch up.

Rewriting gives each changed commit a new ID, so every commit ID in this journal was updated:

Phase	Old ID	New ID
2	`d59e1f9`	`2630d51`
3	`ad6ce06`	`d7d9c57`
4	`5f08b91`	`8bd2123`
5	`2186bcd`	`5569b27`
6	`dec1b27`	`08ec5bd`
7	`30d50cd`	`176397f`
8	`7893d33`	`474465f`
`.vscode`	`7a22cb8`	`e1808f2`

Future commits won't carry the line.

How it was built

Tests first, tiers before metrics so every red step was red for the right reason, then the stack, then the documentation.

Step	What	Why at this point
Setup	`uv add prometheus-client`	A runtime dependency: the API itself needs it
1	The 7 new tests → red	Prove they can detect the missing features
2	Tiers: settings, factory, middleware, `main.py`, the two existing test harnesses → tier tests green	The metrics tests depend on the new layout
3	Metrics tests again → red, now for metrics reasons	Phase 4's lesson
4	Metrics: `metrics.py`, counting, timing, `/metrics` → all 7 green	
5	Stack: nginx hides `/metrics`, Compose passes paid keys, `collect_metrics.py`; live check	Prove it on the real 3 copies, including exactness
6	README: architecture, request steps, tiers, metrics, design decisions, settings	Explain the whole project
7	Lock file, lint, 47 tests × 3, concurrency × 5, CI stack check, ROADMAP; commit, push, watch CI	Nothing older broke

The guide, `docs/PHASE_8_GUIDE.md`, gives the same steps with every file in full and the exact PowerShell commands.

Code 1: The tier and metrics tests

Seven tests, written before the code they test. The tier tests check behaviour through the real app; the metrics tests check that counters change by exactly the right amount.

`tests/test_tiers.py`:

```
import httpx

from app.config import Settings
from app.limiter.factory import create_limiter
```

```
from app.limiter.sliding_log import SlidingWindowLog
from app.limiter.token_bucket import TokenBucket
from app.main import app


async def send(limiters, paid_api_keys, api_key, times):
    """Send requests through the real app with the given limiters and paid keys."""
    app.state.limiters = limiters
    app.state.paid_api_keys = paid_api_keys
    app.state.fail_mode = "open"
    transport = httpx.ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport, base_url="http://test") as client:
        headers = {"X-API-Key": api_key} if api_key else {}
        return [
            await client.get("/api/resource", headers=headers) for _ in range(times)
        ]


def tiered(redis):
    # A small free limit and a bigger paid one, with nothing refilling during the test.
    return {
        "free": TokenBucket(redis, capacity=1, refill_rate=0.001),
        "paid": TokenBucket(redis, capacity=3, refill_rate=0.001),
    }


async def test_paid_keys_get_the_paid_limit(redis, client_id):
    free_key, paid_key = f"{client_id}-free", f"{client_id}-paid"
    free = await send(tiered(redis), {paid_key}, free_key, times=2)
    paid = await send(tiered(redis), {paid_key}, paid_key, times=4)
    assert [r.status_code for r in free] == [200, 429]
    assert [r.status_code for r in paid] == [200, 200, 200, 429]
    assert {r.headers["x-ratelimit-tier"] for r in free} == {"free"}
    assert {r.headers["x-ratelimit-tier"] for r in paid} == {"paid"}


async def test_clients_without_a_key_are_free(redis):
    await redis.delete("rategate:tb:ip:127.0.0.1")  # httpx's test client address
    responses = await send(tiered(redis), {"some-paid-key"}, None, times=2)
    assert [r.status_code for r in responses] == [200, 429]
    assert responses[0].headers["x-ratelimit-tier"] == "free"


def test_paid_api_keys_come_from_the_environment(monkeypatch):
    monkeypatch.setenv("RATEGATE_PAID_API_KEYS", '["key-a", "key-b"]')
    assert Settings().paid_api_keys == {"key-a", "key-b"}
    assert Settings().paid_capacity == 100


async def test_factory_builds_each_tier_with_its_own_limits(redis):
    settings = Settings(algorithm="sliding_log")
    free = create_limiter(redis, settings, "free")
    paid = create_limiter(redis, settings, "paid")
    assert isinstance(paid, SlidingWindowLog)
    assert (free.limit, paid.limit) == (settings.limit, settings.paid_limit)
```

- **`send`** is the usual in-memory harness (Phase 4), now setting `app.state.limiters`, a dictionary by tier, and `app.state.paid_api_keys`. `{...} if api_key else {}` lets a test send **no** key.
- **`tiered`** builds a free bucket of 1 and a paid bucket of 3, with nothing refilling (`0.001`), so the expected codes are exact.

- **Test 1:** the same app, two keys. The free key gets `[200, 429]`, the listed paid key `[200, 200, 200, 429]`. `{... for r in free}` is a **set**, so `== {"free"}` means every response said free.
- **Test 2:** a keyless client is free even when paid keys exist. Keyless clients are identified by IP; httpx's in-memory client reports `127.0.0.1`, and the test first deletes that bucket so earlier runs can't affect it.
- **Test 3:** `monkeypatch.setenv` sets an environment variable for this test only. pydantic-settings reads a `set[str]` setting as JSON, so `'["key-a", "key-b"]'` becomes `{"key-a", "key-b"}`.
- **Test 4:** the factory gives each tier its own numbers. With the sliding log, `limit` is 10 and `paid_limit` 100.

`tests/test_metrics.py`:

```
import httpx
from prometheus_client import REGISTRY

from app.config import Settings
from app.limiter.token_bucket import TokenBucket
from app.main import app
from app.redis_client import create_redis


def decisions(outcome, tier="free"):
    """The current value of one decision counter (0 if it hasn't been used yet)."""
    labels = {"outcome": outcome, "tier": tier}
    return REGISTRY.get_sample_value("rategate_decisions_total", labels) or 0.0


def checks():
    return REGISTRY.get_sample_value("rategate_check_seconds_count") or 0.0


async def send(limiter, api_key, times, path="/api/resource", fail_mode="open"):
    app.state.limiters = {"free": limiter, "paid": limiter}
    app.state.paid_api_keys = set()
    app.state.fail_mode = fail_mode
    transport = httpx.ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport, base_url="http://test") as client:
        headers = {"X-API-Key": api_key}
        return [await client.get(path, headers=headers) for _ in range(times)]


async def test_allowed_and_limited_requests_are_counted(redis, client_id):
    allowed, limited, timed = decisions("allowed"), decisions("limited"), checks()
    await send(TokenBucket(redis, capacity=1, refill_rate=0.001), client_id, times=3)
    assert decisions("allowed") - allowed == 1
    assert decisions("limited") - limited == 2
    assert checks() - timed == 3  # every check of Redis was timed


async def test_fail_open_and_fail_closed_are_counted(client_id):
    down = create_redis(Settings(redis_url="redis://127.0.0.1:6399/0"))
    try:
        limiter = TokenBucket(down, 10, 1.0)
        opened, closed = decisions("fail_open"), decisions("fail_closed")
        await send(limiter, client_id, times=1, fail_mode="open")
        await send(limiter, client_id, times=1, fail_mode="closed")
    finally:
        await down.aclose()
    assert decisions("fail_open") - opened == 1
    assert decisions("fail_closed") - closed == 1
```

```
async def test_metrics_endpoint_is_never_limited(redis, client_id):
    limiter = TokenBucket(redis, capacity=1, refill_rate=0.001)
    responses = await send(limiter, client_id, times=5, path="/metrics")
    assert [r.status_code for r in responses] == [200] * 5
    assert "rategate_decisions_total" in responses[0].text
    assert "rategate_check_seconds" in responses[0].text
```

- **`REGISTRY.get_sample_value(name, labels)`** reads one current value from the library's registry, where every metric lives. It returns `None` for a series not used yet, and `or 0.0` turns that into 0.
- **Measure the change, not the total.** The counters are shared by the whole test run, and other tests send requests too. So each test reads the values **before**, acts, and checks the **difference**. Checking totals would make the result depend on which tests ran first.
- **Test 1:** a bucket of 1, three requests, so +1 allowed, +2 limited, and 3 more timed checks.
- **Test 2:** a client pointed at the dead port 6399 (Phase 4), once with fail-open and once with fail-closed. One of each must be counted. `try` / `finally` closes the client even if a check fails.
- **Test 3:** `/metrics` five times with a bucket of 1, all `200` (never limited), and the page contains both metrics.

Code 2: Tiers

Tiers touched four app files and two test harnesses: 9 new settings lines, a factory that picks numbers by tier, a 3-line `client_tier` function, and two limiters instead of one at startup.

`app/config.py`: a type next to the others:

```
FailMode = Literal["open", "closed"]
Tier = Literal["free", "paid"]
```

and after the sliding-window settings:

```
    # Paid tier: these API keys get the paid_* limits instead (10 times the free ones).
    # From the environment as JSON, e.g. RATEGATE_PAID_API_KEYS='["key-1", "key-2"]'
    paid_api_keys: set[str] = set()
    paid_capacity: int = 100
    paid_refill_rate: float = 10.0
    paid_limit: int = 100
    paid_window_seconds: float = 10.0
```

- **`Tier = Literal["free", "paid"]`**: like `Algorithm` (Phase 2) and `FailMode` (Phase 4), a type that allows only these two strings.
- **`set[str]`**: a set of strings. Asking "is this key paid?" takes the same time however many keys there are. Empty by default, so there are no paid keys until you list some.
- The paid numbers mirror the free ones, times 10. Each is a setting, so `RATEGATE_PAID_CAPACITY=500` would change it without touching code.

`app/limiter/factory.py`, the whole file:

```
from redis.asyncio import Redis

from app.config import Settings, Tier
from app.limiter.base import RateLimitStrategy
from app.limiter.sliding_counter import SlidingWindowCounter
from app.limiter.sliding_log import SlidingWindowLog
from app.limiter.token_bucket import TokenBucket
```

```
def create_limiter(
    redis: Redis, settings: Settings, tier: Tier = "free"
) -> RateLimitStrategy:
    paid = tier == "paid"
    capacity = settings.paid_capacity if paid else settings.capacity
    refill_rate = settings.paid_refill_rate if paid else settings.refill_rate
    limit = settings.paid_limit if paid else settings.limit
    window_seconds = settings.paid_window_seconds if paid else settings.window_seconds

    if settings.algorithm == "token_bucket":
        return TokenBucket(redis, capacity, refill_rate)
    if settings.algorithm == "sliding_log":
        return SlidingWindowLog(redis, limit, window_seconds)
    if settings.algorithm == "sliding_counter":
        return SlidingWindowCounter(redis, limit, window_seconds)
    raise ValueError(f"Unknown algorithm: {settings.algorithm}")
```

- **`tier: Tier = "free"`**: a new parameter with a default, so existing calls such as `create_limiter(redis, settings)` in Phase 2's tests still work unchanged.
- **`paid = tier == "paid"`**: a true/false value, used four times below.
- **`x if paid else y`**: Python's one-line choice. Pick the paid number for the paid tier, otherwise the free one.
- The algorithm choice is unchanged; it just receives the chosen numbers.

`app/middleware.py`: a function after `identify_client`:

```
def client_tier(request: Request) -> str:
    # Only API keys listed as paid get the paid tier; everyone else, keyless included, is free.
    api_key = request.headers.get("X-API-Key")
    return "paid" if api_key in request.app.state.paid_api_keys else "free"
```

A missing header gives `None`, and `None in some_set` is simply `False`, so keyless clients are free with no special case.

In `dispatch`, the single limiter becomes a lookup, and the headers gain one entry:

```
        tier = client_tier(request)
        limiter = request.app.state.limiters[tier]
```

```
            "X-RateLimit-Tier": tier,
```

`app/main.py`: in `lifespan`, one limiter becomes two:

```
    app.state.limiters = {
        "free": create_limiter(redis, settings, "free"),
        "paid": create_limiter(redis, settings, "paid"),
    }
    app.state.paid_api_keys = settings.paid_api_keys
```

Both limiters share the same Redis client, and so the same connection pool.

The two existing test harnesses, `call_api` in `tests/test_fail_modes.py` and `get` in `tests/test_instance_id.py`: the line `app.state.limiter = limiter` became

```
    app.state.limiters = {"free": limiter, "paid": limiter}
    app.state.paid_api_keys = set()
```

They now set up the app the way `main.py` does; what they test is unchanged, and all their tests still pass.

Code 3: Metrics

A 15-line module defines the two metrics, the middleware counts and times every decision, and a four-line route serves the page.

`app/metrics.py` (new):

```
from prometheus_client import Counter, Histogram

# Each API copy keeps its own counts; a monitoring system reads every copy's /metrics
# and adds them up.
DECISIONS = Counter(
    "rategate_decisions",  # published as rategate_decisions_total
    "Rate-limit decisions, by outcome (allowed, limited, fail_open, fail_closed)",
    ["outcome", "tier"],
)

CHECK_SECONDS = Histogram(
    "rategate_check_seconds",
    "Time spent asking Redis whether a request may go ahead, in seconds",
    buckets=(0.001, 0.0025, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0),
)
```

- **`Counter(name, description, labels)`** creates the counter and registers it with the library. The description becomes the `# HELP` line.
- **`["outcome", "tier"]`** are the label names; values are supplied each time with `.labels(...)`.
- **`buckets`** are the upper edges of the time ranges, in seconds, from 1 ms to 1 s. Normal checks take 1–5 ms, and timeouts land near 0.25 s or 1 s, so the buckets cover both.
- **Upper-case names** (`DECISIONS`, `CHECK_SECONDS`) follow Python's convention for module-level values created once and shared.
- **Created once:** a module's top-level code runs only the first time it's imported, which matters because registering the same metric twice is an error.

`app/middleware.py`: the import, `/metrics` added to the exempt paths, and counting:

```
from app.metrics import CHECK_SECONDS, DECISIONS

EXEMPT_PATHS = {"/health", "/docs", "/openapi.json", "/metrics"}
```

```
        try:
            with CHECK_SECONDS.time():
                decision = await limiter.check(identify_client(request))
        except RedisError as error:
            if request.app.state.fail_mode == "closed":
                DECISIONS.labels("fail_closed", tier).inc()
                ...
            DECISIONS.labels("fail_open", tier).inc()
            logger.warning("Redis unavailable, allowing request (fail-open): %s", error)
            return await call_next(request)
```

```
        DECISIONS.labels("allowed" if decision.allowed else "limited", tier).inc()
```

- `with CHECK_SECONDS.time():` starts a stopwatch and, when the block ends (normally **or** with an error), records the elapsed time in the histogram. `await` inside the block is fine: the stopwatch measures real elapsed time.
- `DECISIONS.labels("fail_closed", tier)` picks the series for these label values, in the order the labels were declared. `.inc()` adds 1.
- **One `inc()` on every way out:** allowed, limited, fail-open, fail-closed. The `...` stands for the unchanged logging and `503` response from Phase 4.

`app/main.py`**:** two imports and the route:

```
from fastapi import FastAPI, Response
from prometheus_client import CONTENT_TYPE_LATEST, generate_latest
```

```
@app.get("/metrics")
async def metrics():
    # This copy's counters, in the text format Prometheus reads.
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
```

- `generate_latest()` produces the whole registry as Prometheus text, as bytes.
- `CONTENT_TYPE_LATEST` is the content type Prometheus expects: `text/plain; version=1.0.0; charset=utf-8`.
- `Response(...)` returns the text as it is. A normal route's return value would be turned into JSON, which Prometheus can't read.

Code 4: The stack

nginx hides `/metrics`, Compose passes in the paid keys, and a script collects every copy's counters the way a monitoring system would.

`nginx/nginx.conf`**:** inside `server { ... }`, before `location / {`:

```
    # Each copy's counters are for monitoring inside the network, not for the public.
    location = /metrics {
        return 404;
    }
```

- `location = /metrics`: `=` means **exactly** this path. An exact match takes priority over `location /`, which matches everything.
- `return 404;`: nginx answers by itself; the request never reaches a copy.

`docker-compose.yml`**:** one more pass-through setting under the API's `environment:` (Phase 5):

```
      RATEGATE_PAID_API_KEYS:
```

Set it in the terminal before `docker compose up`, e.g. `$env:RATEGATE_PAID_API_KEYS = '["demo-paid"]'`. The single quotes keep PowerShell from touching the double quotes inside the JSON.

`scripts/collect_metrics.py` **(new):**

```
"""Read /metrics from every API copy and add up the rate-limit decisions.
```

```
Each copy counts only the requests it answered, and /metrics isn't reachable through
nginx, so this asks each container directly, the way a monitoring system would.
"""

import subprocess
from collections import Counter

from prometheus_client.parser import text_string_to_metric_families

# Runs inside a container: fetch that copy's own /metrics.
FETCH = (
    "import urllib.request; "
    "print(urllib.request.urlopen('http://localhost:8000/metrics').read().decode())"
)


def api_containers() -> list[str]:
    result = subprocess.run(
        ["docker", "compose", "ps", "api", "--format", "{{.Name}}"],
        capture_output=True,
        text=True,
        check=True,
    )
    return sorted(result.stdout.split())


def decisions(container: str) -> Counter:
    result = subprocess.run(
        ["docker", "exec", container, "python", "-c", FETCH],
        capture_output=True,
        text=True,
        check=True,
    )
    counts = Counter()
    for family in text_string_to_metric_families(result.stdout):
        if family.name == "rategate_decisions":
            for sample in family.samples:
                if sample.name == "rategate_decisions_total":
                    key = (sample.labels["tier"], sample.labels["outcome"])
                    counts[key] += int(sample.value)
    return counts


def main() -> None:
    total = Counter()
    for container in api_containers():
        counts = decisions(container)
        total += counts
        summary = ", ".join(f"{t} {o}: {n}" for (t, o), n in sorted(counts.items()))
        print(f"{container}  {summary or 'nothing yet'}")
    summary = ", ".join(f"{t} {o}: {n}" for (t, o), n in sorted(total.items()))
    print(f"All copies  {summary or 'nothing yet'}")


if __name__ == "__main__":
    main()
```

- **`subprocess.run([...], capture_output=True, text=True, check=True)`** runs a command from Python and captures its output as text. `check=True` raises an error if the command fails, rather than carrying on with empty output.

- `docker compose ps api --format "{{.Name}}"` lists the API service's container names, one per line, however many copies are running.
- `docker exec <container> python -c FETCH` runs a one-line Python program **inside** each copy, fetching its own `http://localhost:8000/metrics`. No port has to be published.
- `text_string_to_metric_families` is the library's parser for the Prometheus text format. Each family groups its `samples`, the individual lines. The family is named `rategate_decisions`; its counting lines are `rategate_decisions_total` (the `_created` lines are skipped).
- `Counter` here is Python's own, from `collections`, not Prometheus'. It counts by key, `(tier, outcome)`, and `total += counts` adds two of them key by key.
- `summary or 'nothing yet'`: an empty string counts as false, so a copy with no traffic yet prints a readable line.

Code 5: The README additions

The README grew four sections: Architecture at the top, Tiers and Metrics in the middle, Design decisions near the end. It also gained five new settings and the new test count.

The order of the README now, each section answering the next question a reader would ask:

Section	Answers
Introduction + CI badge	What is it, and does it work?
Architecture (new)	What are the parts, and what happens to one request?
Run it / Run the full stack	How do I start it?
Algorithms, Failure handling	How does it limit, and what if Redis fails?
Tiers, **Metrics** (new)	Can customers get different limits? How do I watch it?
Testing, Load testing	How do I know it's correct, and how fast is it?
Design decisions (new)	Why is it built this way?
Configuration	What can I change?

The diagram's source:

```
flowchart LR
    clients["Clients"] -->|"HTTP, port 8080"| nginx["nginx<br/>round-robin load balancer"]
    nginx --> api1["API copy 1<br/>FastAPI + rate-limit middleware"]
    nginx --> api2["API copy 2"]
    nginx --> api3["API copy 3"]
    api1 -->|"one atomic Lua script per request"| redis[("Redis<br/>shared counters")]
    api2 --> redis
    api3 --> redis
    monitor["Monitoring<br/>reads each copy's /metrics"] -.-> api1
    monitor -.-> api2
    monitor -.-> api3
```

- Only the first copy's box and arrow carry the long labels. Copies 2 and 3 do the same, and repeating the words would crowd the picture.
- The monitoring arrows are **dotted** (`-.->`) because they're a different kind of traffic: reading counters, not serving clients.
- In the README it sits inside a code block marked `mermaid`, which is what tells GitHub to draw it instead of showing the text.

"How one request is checked" follows the diagram as five numbered steps. A request passes through nginx (`X-Forwarded-For` set), the middleware (client and tier identified), and Redis (one Lua script), then gets its answer (allowed, `429`, or the fail mode), and the decision is counted.

The design-decisions table has 14 rows, covering every phase:

- shared Redis counters;
- atomic Lua scripts;
- Redis's own clock;
- one interface for all algorithms;
- the token bucket as the default;
- fail-open as the default;
- short timeouts;
- retrying only connection errors;
- a blocking connection pool;
- nginx's shared zone and connect timeout;
- overwriting `X-Forwarded-For`;
- tiers from a list of keys;
- metrics per copy;
- tests against a real Redis with deliberately broken controls.

Each row gives the decision, why it was made, and the measured evidence where there is some.

Smaller changes: `X-RateLimit-Tier` added to the introduction's list of headers; the Testing section's count from 40 to 47, with bullets for tiers and metrics; five `RATEGATE_PAID_*` rows in the Configuration table.

Command reference

Every command used in Phase 8, what it does and why. PowerShell from `D:\RateGate`.

Setup and tests

Command	What it does	Why
`uv add prometheus-client`	Adds the library to `pyproject.toml`'s dependencies and updates `uv.lock`	The API itself needs it, so it's a normal dependency, not a dev tool
`uv run pytest tests/test_tiers.py tests/test_metrics.py -q`	Runs only the new tests, with short output	The red and green steps
`foreach ($i in 1..3) { uv run pytest -q | Select-Object -Last 1 }`	The whole suite, 3 times	Nothing older broke
`uv sync --locked`	Checks `uv.lock` matches `pyproject.toml`	What CI does first (Phase 6)

Trying tiers on the stack

Command	Why
`$env:RATEGATE_PAID_API_KEYS = '["demo-paid"]'`	Lists one paid key. Single quotes keep PowerShell from touching the double quotes inside

Command	Why
`docker compose up --build -d --wait`	Rebuilds the image (new library) and starts the copies with the paid key
`curl.exe -s -D - -o NUL -H "X-API-Key: demo-paid" http://127.0.0.1:8080/api/resource`	`-D -` prints the response headers: `x-ratelimit-limit: 100`, `x-ratelimit-tier: paid`
`Remove-Item Env:RATEGATE_PAID_API_KEYS`	Removes the setting from this terminal afterwards

Metrics

Command	Why
`curl.exe -s -o NUL -w "%{http_code}n" http://127.0.0.1:8080/metrics`	Through nginx: must print `404`
`uv run python scripts/collect_metrics.py`	Each copy's decisions and the total
`docker compose exec --index 2 api python -c "import urllib.request; print(urllib.request.urlopen('http://localhost:8000/metrics').read().decode())"`	The raw page of copy 2 (`--index` picks the copy); this is what the script does for each copy

Checking that metrics don't slow anything down (Phase 7's test)

```
docker compose run --no-deps --rm locust -f /mnt/loadtest/locustfile.py --headless -u 55 -r 5 -t 70s --reset-stats --host http://nginx:80 --csv /mnt/loadtest/results/with_metrics --only-summary
uv run python loadtest/summarize.py "Token bucket, metrics on=loadtest/results/with_metrics"
```

Git

Command	Why
`git add app tests nginx/nginx.conf docker-compose.yml scripts/collect_metrics.py pyproject.toml uv.lock README.md ROADMAP.md docs/PHASE_8_GUIDE.md`	Stages exactly the Phase 8 files
`git commit -m "Phase 8: ..."` then `git push`	Records the work and sends it to GitHub, which runs CI
`git add .vscode/settings.json` then `git commit`	The separate commit `e1808f2`, pushed afterwards

Verification

The roadmap's three Phase 8 items are done. Every check below was run in the project on 2026-11-03, except the diagram's rendering (see The full story).

Tests, red then green:

Stage	Result
New tests on the Phase 7 code	`7 failed`
After tiers	Tier tests `4 passed`; metrics tests `3 failed` (counters at 0; `/metrics` returning `404` and limited)
After metrics	`7 passed`

Stage	Result
Changed files vs the tested scratch version	Byte for byte identical, all 10 compared

Live on the 3-copy stack (paid key `demo-paid`):

Check	Result
`/metrics` through nginx	`404`
Free key, one request	`limit: 10`, `remaining: 9`, `tier: free`
Paid key, one request	`limit: 100`, `remaining: 99`, `tier: paid`
Burst of 200, free key	`200: 10`, `429: 190`
Burst of 200, paid key	`200: 101`, `429: 99`
Summed counters (diagram in Theory 3)	free 11 / 190, paid 102 / 99: exactly the client totals

Cost of metrics: Phase 7's mixed test with metrics on: 230 requests/s, p50 4 ms, p95 7 ms, 0 of 1,164 polite requests limited, 1.01/s per greedy client. Phase 7 without metrics: 4 ms / 6–7 ms.

Nothing older broke:

Check	Result
Lock file	`Checked 33 packages` (32 + `prometheus-client`)
Lint, formatting	`All checks passed!`, `27 files already formatted`
Full suite	`47 passed` in 27.75 s, 29.63 s, 29.66 s
Concurrency tests	`7 passed`, 5 runs out of 5
CI's stack check, locally	`200: 100`, `429: 100`, exit code 0
GitHub CI on `474465f`	Both jobs green; badge "CI – passing"

Still to confirm by eye: open the README on GitHub and check the Architecture section shows a drawn diagram, not an error box.

Common pitfalls

The usual ways tiers and metrics go wrong, including the ones this phase avoided on purpose.

Pitfall	Symptom	Why	Fix
Labelling metrics by API key or IP	Monitoring slows down or runs out of memory	Every label value is a new series	Only labels with a few fixed values (`outcome`, `tier`)
Reading `/metrics` through the load balancer	Numbers jump between requests	Each request reaches a different copy	Read each copy and add them up
Exposing `/metrics` publicly	Internal numbers visible to anyone	It's served like any route	nginx: `location = /metrics { return 404; }`
Rate-limiting `/metrics`	Monitoring goes blind under heavy traffic	It's a request like any other	Add it to `EXEMPT_PATHS`
Creating a metric twice	`Duplicated timeseries in CollectorRegistry`	Each name can be registered once	Define metrics once, in their own module

Pitfall	Symptom	Why	Fix
Checking counter totals in tests	Results depend on which tests ran first	Counters are shared by the whole test run	Compare before and after
Forgetting to count one way out	Some decisions missing from metrics	One `inc()` per path	Count allowed, limited, fail-open and fail-closed
Returning metrics as JSON	Prometheus can't read the page	A normal route's return value becomes JSON	`Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)`
`RATEGATE_PAID_API_KEYS=key-1`	Startup error from pydantic	A `set[str]` setting is read as JSON	`'["key-1"]'`
Double quotes around the JSON in PowerShell	The inner quotes disappear	PowerShell processes `"..."` strings	Single quotes outside: `'["key-1"]'`
Expecting counters to survive a restart	Back at 0 after a restart	They live in the process's memory	Normal; Prometheus handles resets
A red test for the wrong reason	Metrics tests fail on `app.state` errors	Tiers not built yet	Build tiers first
Changing `create_limiter`'s signature without a default	Older tests break	They call it with two arguments	`tier: Tier = "free"`
An unchecked diagram	An error box on GitHub instead of a picture	Mermaid is only drawn in the browser	Check it renders; here, open the README on GitHub

Check your understanding

Answer each in your own words first, then compare.

1. How does RateGate decide a client's tier, and where would that come from in a real product? If the request's API key is in `RATEGATE_PAID_API_KEYS` it's paid, otherwise free, including clients without a key. A real product would look the key up in a customer database.

2. Why is there one limiter per tier instead of one limiter with changing numbers? Each limiter is built once with fixed numbers, as before; the middleware only chooses which to ask. The algorithms stay unchanged, so earlier proofs still hold.

3. Why does `create_limiter` take `tier` with a default value? So existing calls with two arguments, such as Phase 2's tests, keep working.

4. What's the difference between a counter and a histogram? Give a RateGate example of each. A counter only goes up: `rategate_decisions_total`. A histogram counts values into ranges: `rategate_check_seconds`, how long Redis checks took.

5. Why must a metric never be labelled by API key? Every distinct label value creates a new series; millions of keys would overwhelm the monitoring system.

6. Why does each copy have its own counters, and how do you get the total? Each copy is a separate process that counts only what it answered. A monitoring system (here `collect_metrics.py`) reads every copy and adds them up.

7. Why does nginx hide `/metrics`, and why is `/metrics` exempt from rate limiting? Through nginx it would show a random copy's third and expose internal numbers. And a rate-limited `/metrics` would blind monitoring exactly when traffic is high.

8. What does `rategate_check_seconds_bucket{le="0.005"} 3.0` mean? Three checks took 5 ms or less. Buckets are cumulative.

9. Why do the metrics tests compare before and after? The counters are shared by the whole test run, so totals depend on what ran before; the difference doesn't.

10. Why were tiers built before metrics? The metrics tests use the new tiered setup; before tiers existed, they failed on `app.state` errors instead of on missing metrics.

11. How was it shown that the metrics are exact, and that they cost nothing measurable? The counters summed across the 3 copies matched the clients' results exactly (free 11 / 190, paid 102 / 99). Phase 7's load test with metrics on gave the same latency: p50 4 ms, p95 6–7 ms.

12. Why did the paid burst allow 101, not 99? 99 tokens were left after the single request, and the paid tier refills 10 per second, so about 2 more arrived during the burst.

13. Why couldn't the README's diagram be checked here, and how is it checked now? Both ways to render it (a public website, or downloading Mermaid's tool) need permissions that weren't granted. Opening the README on GitHub shows whether it draws.

What's next

All nine phases of the roadmap (0 to 8) are done: every claim in the project description is built, tested, measured and documented. Ideas beyond the roadmap, none of them built:

- **A circuit breaker** (from Phase 4): after several Redis failures in a row, skip Redis for a few seconds and apply the fail mode at once, instead of every request waiting out the 1 s timeout.
- **Prometheus and Grafana in the Compose file**, scraping the copies' `/metrics` and drawing live graphs.
- **Tiers from a database** instead of a list of keys in a setting.
- **Per-route limits**, e.g. a stricter limit on a login endpoint.
- **More uvicorn workers per copy** (`--workers`), since Phase 7 showed each single-process copy is the bottleneck.

Also still open: confirm the README's diagram draws on GitHub.

Real-world context

Real companies use RateGate's core idea, a shared counter in Redis that every server checks. At large scale they add layers on top and trade a little accuracy for speed. Stripe has described limiting its API with a token bucket in Redis, which is what Phase 1 builds.

For most companies, one Redis (with a standby copy) shared by all API servers is the whole solution. Everything below is what very large systems add.

Where the limit is checked

Large companies usually check limits in front of the app, not inside it. With 200 services written by different teams, a limiter in each would mean bugs, gaps and inconsistent rules. Blocked requests would also waste each service's CPU.

- **API gateway:** a proxy every request passes through first, such as Envoy, Kong, NGINX or AWS API Gateway. The rules are written once in its configuration.
- **Edge network (CDN):** servers in data centers worldwide, such as Cloudflare. A flood from one IP is blocked near its source and never reaches your servers.

RateGate's middleware is the same logic. Real systems move it one layer outward so every service is protected in one place.

Layers of limits

A real API has several limits, and each catches a different problem. Think of a stadium: outer gates, ticket checkers, section stewards and venue management. Stripe describes running four limiters together for reasons like these.

Layer	The problem it handles	Example
Per IP, at the edge	A botnet floods you from a few IPs	Crude but cheap; blocks floods before they cost anything
Per API key or user	A customer's script retries in a loop	The published limit, e.g. 100 a minute free, 1,000 paid
Per endpoint	Someone keeps calling an expensive route	/export-all-data gets a far tighter limit than a simple read
Concurrency	Slow requests pile up	At most 5 requests in progress per client, however fast they arrive
Load shedding	Total traffic exceeds capacity, e.g. Black Friday	Drop analytics and background syncs first, keep checkout and login

A rate limit controls how many requests arrive per unit of time. A concurrency limit controls how many are being worked on at once. A client sending 10 a minute, each taking 2 minutes, passes the first but piles up 20 open requests.

When one Redis isn't enough

At huge scale, one Redis hits two limits. Volume: millions of requests a second, each making a Redis trip. Distance: a server in Mumbai asking a Redis in Virginia waits roughly 200 ms per round trip. Four patterns handle this.

Sharding. Redis Cluster spreads keys over several Redis servers, like a phone book split into volumes A–H, I–P, Q–Z. A hash of the key picks the server, so customer abc always lands on the same one. Each customer is still counted in one place, so the atomic Lua script still works.

Local plus global. Each gateway keeps a loose token bucket in its own memory, with no network trip. A central Redis-backed service holds the accurate shared limit. Envoy supports both. Floods are rejected locally and instantly, and only the rest pays for the network trip.

Approximate counts. Rate limiting rarely needs to be exact: 102 against a limit of 100 harms no one. Cloudflare uses the sliding window counter from Phase 2 and counts per data center, not worldwide. In their measurement, only a tiny fraction of requests were wrongly allowed or blocked. The counter estimates like this:

> Limit 100 a minute. Last minute: 80 requests. This minute so far: 30. We're 15 seconds in, so 75% of the last 60 seconds overlaps last minute. Estimate = 30 + 80 × 0.75 = 90, so allowed.

Batched syncing. Each server counts in memory and reports its totals every half second or so. Most requests never wait on the network. Between reports, a client can slip a little over its limit.

Approach	Accuracy	Speed	Who uses it
One Redis per request (RateGate)	Exact	Fast enough for most	Most companies
Sharded Redis	Exact	Much more volume	Very high traffic
Local + global	Exact globally, loose locally	Floods stopped instantly	Gateways like Envoy
Approximate, per data center	Nearly exact	Very fast	Global edge networks
Batched syncing	Can overshoot a little	Fastest	Extreme scale

The more traffic, the more accuracy is traded for speed. Systems stay strict only where it matters, such as logins and billing.

When the limiter breaks

Real systems often combine fail-open and fail-closed instead of picking one. A typical sequence when Redis becomes unreachable:

1. Servers fall back to their local in-memory limit, so rough protection remains.
2. Ordinary traffic keeps flowing: fail-open, with that safety net.
3. Sensitive actions such as login attempts stay fail-closed, since even minutes without a limit invite password guessing.
4. An alert wakes an engineer to fix Redis.

Telling clients what's happening

A good limiter tells clients where they stand, so they can slow down before hitting the wall. Most real APIs send these headers on every response:

```
HTTP/1.1 200 OK
X-RateLimit-Limit: 100          ← your limit
X-RateLimit-Remaining: 7        ← requests left
X-RateLimit-Reset: 1762078800   ← when it resets (Unix time)
```

Over the limit, the response is 429 with Retry-After:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12                 ← wait 12 seconds
```

Well-written clients retry with exponential backoff: wait 1 s, then 2, 4, 8. They also add jitter, a small random extra wait. Without it, after an outage thousands of clients retry at the same moments and knock the server over again, a "thundering herd".

How RateGate maps to this

RateGate is a small but faithful version of the core that real systems are built on.

Real-world practice	Where RateGate covers it
Token bucket in shared Redis (Stripe)	Phase 1
Several algorithms, including the sliding window counter (Cloudflare)	Phase 2
Proving atomicity under concurrent load	Phase 3
Fail-open vs fail-closed	Phase 4
Many servers sharing one limit behind a load balancer	Phase 5
Per-plan and per-endpoint limits, X-RateLimit headers	Phase 8 (optional)
Gateways, sharding, edge limiting	Not planned; natural extensions

Glossary

Term	Plain meaning
429 Too Many Requests	The HTTP status code meaning "you're over your limit"
API gateway	A proxy every request passes through first, where limits and other shared rules are checked
Atomic	Runs as one step that nothing else can interrupt or split
async / await	Lets one Python process serve other requests while it waits on something slow, like Redis
CI (continuous integration)	Automatically running tests on every push
Concurrency limit	A cap on how many requests a client can have in progress at once
Container	A small, isolated environment running one program, such as Redis, with everything it needs
Dependency	A library your project uses
Edge network (CDN)	Servers in data centers worldwide that block bad traffic close to its source
Environment variable	A named setting given to a program from outside its code, e.g. `RATEGATE_CAPACITY=5`
EVALSHA	Running a Lua script Redis already stores, by its short hash, instead of resending its text
Exponential backoff	Retrying after waits that double each time: 1 s, 2 s, 4 s, 8 s
Fail-open / fail-closed	When the limiter breaks: allow everything (open) or reject everything (closed)
Horizontal scaling	Handling more traffic by adding more copies of a server
Jitter	A small random extra wait so many clients don't retry at the same moment
Load balancer	Spreads incoming requests across several servers (nginx here)
Load shedding	Dropping low-priority traffic on purpose when the whole system is overloaded
Lockfile (`uv.lock`)	The exact version of every library, so installs are identical everywhere
Lua script	A small program Redis runs internally and atomically
Middleware	Code that runs before (and after) every request
Race condition	A bug where the result depends on which of two simultaneous operations finishes first
Redis	A very fast in-memory database, used here as the shared counter
Retry-After	A response header telling the client how many seconds to wait
Round robin	A load balancer sending requests to each server in turn: 1, 2, 3, 1, 2, 3
Sharding	Splitting data over several servers, each owning a share of the keys
Stateless	A server that keeps nothing between requests, so any copy can handle any request
Thundering herd	Many clients retrying at the same moment and overloading a server again
Token bucket	A limit where tokens refill at a steady rate and each request spends one
Virtual environment (`.venv`)	A private folder of libraries for one project
Abstract base class (ABC)	A class that defines methods its subclasses must provide, and can't be used directly itself

Term	Plain meaning
Classmethod	A method called on the class rather than an object, often to build an object another way
Factory function	A function whose job is to create the right object, such as create_limiter
Fixed window	A limiter that counts per calendar slice and resets at each boundary, which allows double bursts across it
Fixture	Reusable test setup that pytest passes to any test naming it as a parameter
Open/closed principle	Code should be open for extension but closed for modification: add behaviour without editing working code
Parametrize	Running one test several times with different inputs
Refactoring	Improving code's structure without changing what it does
Regression check	Re-running an existing test after a change to prove nothing that used to work broke
Sliding window	A time window that moves with the clock, such as "the last 10 seconds", rather than a fixed slice
Sorted set	A Redis type holding unique members, each with a numeric score, kept in score order
Strategy pattern	Interchangeable implementations of one job behind a shared interface
Type alias	A name given to a type, such as Algorithm = Literal[...]
asyncio.gather	Starts many coroutines at once and waits for all of them, returning their results in order
Boundary test	A test aimed at the edge where behaviour changes, where bugs cluster
Connection pool	A set of open connections reused by many requests; a blocking pool waits when all are busy, the default one raises an error
Coroutine	The paused piece of work created by calling an async function without await
Flaky test	A test that sometimes fails without any real bug, usually from tight timing
Negative control	Running a test on something known to be broken, to prove the test can detect it
Test-driven development (TDD)	Writing a failing test first, then the code that makes it pass (red, then green)
Fail-open	When a check can't run, let the request through unchecked: keeps the service available
Fail-closed	When a check can't run, refuse the request: keeps the service protected
503 Service Unavailable	The HTTP status code meaning "the server can't handle this right now"
Timeout	The longest the app waits for something before giving up with an error
Connect timeout / socket timeout	The limit for opening a connection / for waiting for a reply on an open one
Retry	Sending a failed command again in case the failure was brief
Exponential backoff	Retrying with a longer pause before each attempt
Stale connection	A pooled connection whose other end has gone, for example after Redis restarted
Exception	An object Python raises when something goes wrong; it travels up until some code catches it
try / except	Run some code, and handle a named kind of exception if it is raised
Log level	How serious a log line is: warning means degraded, error means users are affected
Circuit breaker	After repeated failures, stop calling a dependency for a while and fail fast, then test it again

Term	Plain meaning
Horizontal scaling	Handling more traffic by adding identical copies of a server, rather than a bigger machine
Stateless	Keeping no important data in a server's own memory, so any copy can answer any request
Image	A packaged, read-only snapshot of a system and app, built from a Dockerfile
Dockerfile	The list of instructions that builds an image
Layer cache	Docker reusing build steps whose inputs haven't changed
Docker Compose	A tool that starts several containers, their network and settings, from one file
Service / replica	One kind of container in a Compose file / one of several identical containers of a service
Health check	A command Docker runs repeatedly to ask a container whether it is ready
DNS	The system that turns names (like redis or api) into network addresses
Reverse proxy	A server that receives clients' requests and forwards them to backend servers the clients never see
Load balancer / round-robin	Something that spreads requests across servers / giving each server a request in turn
Upstream	nginx's name for the group of backend servers it forwards to
Worker process	One of several nginx processes handling connections side by side, usually one per CPU core
Passive health check	Noticing a dead server from failed real requests, then skipping it for a while
X-Forwarded-For	A header in which a proxy passes on the original client's IP address; clients can fake it
Port forwarding	Passing connections from a port on your PC into a container; on Windows, Docker Desktop's wslrelay does it
monkeypatch	A pytest fixture that temporarily replaces a function or method during a test
.env file	A file of settings for one machine, read by the app and kept out of git
GitHub Actions	GitHub's built-in CI service, driven by YAML files in .github/workflows/
Workflow / run	A YAML file saying what to run and when / one execution of it for one commit
Job / step	A group of steps on its own fresh machine / one command or action inside a job
Runner	The temporary machine a CI job runs on
Action (GitHub)	A reusable published step, referenced as owner/name@version
Service container	A container CI starts beside a job for it to use, such as Redis
Exit code	The number a program hands back when it ends: 0 means success, anything else failure
Git tag	A name attached to one commit, used for versions such as v7 or v10.3.3
Branch	A separate line of commits, so changes can be tried without touching main
Status badge	A small live image in the README showing whether CI passes on main
REST API	Web addresses that return data instead of pages; GitHub's is public for public repositories
Load test	Sending many requests from many simulated users for a sustained time, to see how a system copes
Throughput (RPS)	Requests handled per second
Latency	How long one request takes, from sending to the full answer

Term	Plain meaning
Percentile (p50, p95, p99)	The value below which that share of measurements fall: p95 = 95% of requests were faster
Locust	A load-testing tool in which simulated users are Python classes
Spawn rate	How many simulated users a load test starts per second
Thundering herd	Many clients acting at the same instant, so their requests pile up together
Bottleneck	The part of a system that runs out first and so limits the whole
Capacity test	A load test with no pauses, to find the most a system can serve
Compose profile	A label that keeps a service out of a plain docker compose up until asked for
Queueing	Requests waiting behind others; under full load it dominates latency
Tier	A class of customer with its own limits, such as free or paid
Metric	A number a service keeps updating so it can be watched and graphed
Prometheus	A monitoring system that regularly reads services' /metrics pages and stores the numbers
Scrape	Prometheus fetching a service's /metrics page
Counter / histogram / gauge	A number that only goes up / counts of values in ranges / a number that goes up and down
Label (metrics)	A name=value pair that splits one metric into series, e.g. outcome="limited"
Bucket (histogram)	A range upper edge; its count includes every value at or below it
Mermaid	A text language for diagrams that GitHub draws in Markdown

Progress log and decisions

Add a row each time a phase finishes or a choice is made, newest first.

Progress log

Date	Phase	What happened
2026-11-03	Git	Removed Claude as a GitHub contributor: the Co-Authored-By line deleted from 8 commits with git filter-branch, code and dates unchanged, force-pushed with lease; backup branch backup-before-rewrite kept locally. New commit IDs updated throughout this journal; CI green
2026-11-03	8	Tiers (paid keys get 10x limits, X-RateLimit-Tier), Prometheus metrics (decisions by outcome and tier, Redis-check timer, per copy, hidden behind nginx), architecture diagram and design decisions in the README. 47 tests. Summed counters matched client results exactly; latency unchanged with metrics on. Diagram rendering couldn't be checked here (permissions). Committed as 474465f, CI green; all roadmap phases done
2026-11-03	7	Locust load tests inside Docker: 230 req/s, p50 4 ms, p95 6–7 ms; polite clients never limited; greedy clients get 0.88–1.02/s of the configured 1/s. Capacity 740 / 1,920 / 2,730 served/s with 1 / 3 / 6 copies; the Python copies are the bottleneck, Redis at 28%. Three traps caught: lockstep clients, docker compose run recreating the copies (fixed with --no-deps), a saturated load generator. Committed as 176397f, CI green
2026-11-03	6	GitHub Actions CI: lint, format and 40 tests against a Redis service container with the production timeouts, plus the Phase 5 stack (burst through nginx: exactly 100 across 3 copies) on every push. 7 runs: 1 red from a missing setup-uv@v10 tag, 4 green in a row, 1 red on purpose on a trial branch, then green on main. Badge reads CI passing. Committed as 08ec5bd and pushed
2026-11-02	5	3 API copies behind nginx share one Redis: 200 at once, limit 100 → exactly 100 for every algorithm (about 33 per copy); separate stores per copy would allow 300 of 400. A dying copy costs at most 1 s. 40 tests. Fixed a Phase 4 bug (a retry after a timeout counted a request twice), nginx favouring one copy (zone) and a 38.8 s wait (connect timeout). This PC's tests got their own timeouts in .env. Committed as 5569b27 and pushed
2026-11-02	4	Fail-open (200 + warning) or fail-closed (503 + Retry-After: 5) when Redis is unreachable; hung Redis noticed in about 0.5 s instead of a 5 s freeze; recovers with no restart; 36 tests. Three mistakes caught while building: a too-low single timeout, a test passing for the wrong reason, a non-existent method. Committed as 8bd2123 and pushed
2026-11-02	3	31 tests; 200 concurrent requests allow exactly 100 for every algorithm; a non-atomic control lets 200/200 through. Tests found 2 bugs: pool crash under bursts (HTTP 500s) and a too-short counter Retry-After; both fixed. Committed as d7d9c57 and pushed
2026-11-02	2	Sliding window log and counter added behind RateLimitStrategy; algorithm chosen by RATEGATE_ALGORITHM; middleware untouched; 16 tests pass; all 7 checks passed; committed as 2630d51 and pushed
2026-11-02	Docs	Phase 1 section rewritten in depth: theory, full code with line-by-line notes, build order, the 7 checks, troubleshooting, review questions

Date	Phase	What happened
2026-11-02	1	Token bucket working: burst of 10, refill 1/s, separate buckets per client, state and expiry visible in Redis. Committed as d7687fa
2026-11-02	Docs	Added Why build it, How the copies stay in sync, Real-world context and Sources
2026-11-02	0	Setup done: uv, Python 3.12, libraries installed, Redis running in Docker
2026-11-02	Plan	Tool stack confirmed (all free), 9-phase roadmap and Phase 1 guide written

Decisions

Date	Decision	Reason
2026-11-02	Use Python 3.12, not the system 3.15	Some libraries don't yet have builds for 3.15
2026-11-02	Flat `app/` folder instead of uv's `src/` package layout	A web app run from its folder doesn't need packaging
2026-11-02	Use Redis's clock (`TIME`) inside scripts	Server clocks drift; one clock keeps refill maths correct across instances
2026-11-02	Test against real Redis, not fakeredis	Fakes only partly support Lua scripts
2026-11-02	Add nginx + 3 replicas in Phase 5	Needed to prove the "horizontally scaled" claim
2026-11-02	Strategy pattern with a factory; the middleware stays untouched	Algorithms switch with one setting, and adding one later changes no working code
2026-11-02	Refactor the token bucket before adding new algorithms	One change at a time, proven by a regression check
2026-11-02	Separate limit and window_seconds settings for the window algorithms	Each setting means one thing; defaults give all three the same 1-per-second rate

Date	Decision	Reason
2026-11-02	Sliding log members are time plus a sequence number	Two requests in the same millisecond must not overwrite each other
2026-11-02	The counter keeps its window number as a hash field, not one key per window	Script keys must be declared in KEYS and can't be computed inside the script
2026-11-02	A random client id (uuid4) per test	Tests never share state and never wait for keys to expire
2026-11-02	Exclude docs/ from ruff	ruff formats Python inside Markdown and would rewrite the guides' examples
2026-11-02	Keep a deliberately broken limiter in the tests as a negative control	Proves the concurrency test can detect a race, so its passes mean something
2026-11-02	BlockingConnectionPool, 50 connections, 5 s wait	The default pool raised past 100 busy connections, turning bursts into HTTP 500s; waiting briefly is correct, and 50 is ample for Redis
2026-11-02	One create_redis() for the app and the tests	Tests must exercise exactly the Redis setup production uses
2026-11-02	No refill during concurrency tests (refill_rate 0.001, 60 s windows)	Makes the expected answer exactly 100 instead of a fuzzy range
2026-11-02	Write each bug's failing test before its fix	Proves both that the test detects the bug and that the fix resolved it
2026-11-02	Fail mode is a setting, default open	Availability vs protection depends on the endpoint; for most APIs a broken limiter shouldn't take the service down
2026-11-02	503 + Retry-After: 5 when failing closed, not 429	The client did nothing wrong; 503 states the server's problem and alerts monitoring
2026-11-02	Separate timeouts: 1.0 s connect, 0.25 s per command	Measured: connects took up to 600 ms under a burst, commands at most 1.25 ms; 0.1 s for both broke the burst tests
2026-11-02	One immediate retry (NoBackoff, retries=1)	Replaces stale connections after a restart; library-default retries took 27 s to fail
2026-11-02	Catch only RedisError, in the middleware	HTTP answers belong in the middleware; catching Exception would hide our own bugs behind fail-open
2026-11-02	Simulate outages in tests with a closed port (6399)	Stopping Docker in tests is slow, disturbs other tests and won't work in CI; real stop and pause are checked by hand

Date	Decision	Reason
2026-11-02	Retry only connection errors, never timeouts (replaces the Phase 4 retry)	After a timeout the script may have run; retrying counted requests twice (20 → 40). Also halves outage detection
2026-11-02	nginx round-robin with a shared zone and a 1 s connect timeout	Without zone, 16 workers all favoured one copy; without the timeout, a dead copy cost one request 38.8 s
2026-11-02	Restart nginx after scaling instead of its resolve option	resolve found new copies but caused a 502 in 2 of 3 stop tests; the plain setup answered 250 of 250
2026-11-02	nginx overwrites X-Forwarded-For; uvicorn trusts it; only nginx is published	Appending let fake addresses create fresh buckets; ignoring it put every client in nginx's bucket
2026-11-02	Per-machine timeouts in .env (1.0 s reply, 1.5 s connect); production defaults unchanged	Windows → Docker Desktop replies reached 269 ms; inside Docker 47 ms. Containers and CI keep 0.25 s / 1 s
2026-11-02	Warm the pools in the 4-client concurrency test	The test checks atomicity, not opening 200 connections in one instant through Windows port forwarding
2026-11-03	Two CI jobs: lint and test, plus the full Docker stack	The stack job turns Phase 5's headline claim into a check on every push; separate jobs run in parallel
2026-11-03	Pin setup-uv to v10.3.3, uv to 0.13.6, and install with uv sync --locked	setup-uv has no v10 tag; pinned versions mean a red run is caused by our change, not a tool update
2026-11-03	No .env in CI: test the production timeouts	The looser values only work around Docker Desktop on Windows; CI proves the real values on a normal machine
2026-11-03	Try CI on a temporary branch from a scratch clone, then delete it	Workflows can only be tested on GitHub; main never gets broken or experimental commits
2026-11-03	Run Locust inside Docker as a Compose service with a profile, not on Windows	Measures RateGate, not Windows' port forwarding (4/7 ms vs 7/20 ms); nothing to install; CI and normal up unaffected
2026-11-03	Always docker compose run --no-deps, and prove each run (setting, copy count, Redis key prefix)	Without it, run recreated the copies with default settings; two algorithm runs had silently tested the token bucket
2026-11-03	Start users 5 per second and reset stats after ramp-up	All at once put clients in lockstep and made polite ones look 8x slower (32 ms vs 6 ms)
2026-11-03	Count 429 as success, in its own rows; keep raw results out of git	A 429 is the limiter working; the README holds the summary, CSVs change every run
2026-11-03	Two tiers from a list of paid keys in a setting, one limiter per tier	Configuration-only and self-contained; the algorithms stay unchanged; a database lookup would replace only client_tier

Date	Decision	Reason
2026-11-03	Metrics labelled only by outcome and tier, never by key or IP	Every label value is a new series; a fixed handful stays cheap
2026-11-03	/metrics per copy, exempt from limiting, 404 through nginx	The Prometheus model: monitoring reads each copy; the public neither sees nor distorts it
2026-11-03	Build tiers before metrics	So the metrics tests were red for metrics reasons, not app.state errors
2026-11-03	No Claude co-author line in commits; history rewritten to remove it	Your request: you're the only contributor on GitHub. Rewritten with a backup branch and a lease-checked force-push

Sources

- Scaling your API with rate limiters (Stripe blog)
- Rate limits (Stripe docs)
- Counting things, a lot of different things (Cloudflare blog)
- Rate limiting concepts (Envoy Gateway)
- Global rate limit (Envoy Gateway)